

Audit complet de RepCore

Application de coaching musculation · Guellec Coaching Pro

5,5/10

16 agents experts · benchmark de 14 concurrents · 129 points d'amélioration, chacun avec son prompt de mise en place

Audit réalisé le 29 septembre 2026 · build 1693

Synthèse

RepCore obtient **5,5/10** en moyenne sur 16 axes. Le produit est plus riche que la plupart des concurrents : charge proposée à chaque série, cycle menstruel, analyse vidéo, gamification côté serveur, nutrition CIQUAL hors ligne. Mais 23 défauts critiques l'empêchent aujourd'hui d'être fiable à grande échelle. Le benchmark place RepCore à 7/10, derrière Everfit (9) et Trainerize (8,5), à égalité avec Hexfit et devant Synergy (6,5).

Méthode. 16 agents spécialisés et un analyste de la concurrence ont fouillé le dépôt en parallèle, avec des preuves `fichier:ligne`. Ils ont lancé les tests (92/92 Worker, 5 745 assertions front) et parcouru l'app dans Chromium. Les doublons ont été fusionnés : un sujet traité par deux agents n'apparaît qu'une fois, avec un renvoi « Fusion ». Au total, 129 points (124 des agents et 5 écarts du benchmark), chacun avec un prompt prêt à coller dans Claude Code.

Agent	Note /10	Critiques	Le point le plus urgent
6 · Sécurité applicative	4	2	Le paywall se contourne en écrivant dans son propre dossier
8 · Performance front	4	2	Aucune minification, ~7 Mo retéléchargés à chaque build
16 · IA et automatisation	4	1	Aucun appel IA : poser le proxy Claude sur le Worker
2 · Préparateur physique	5	2	La formule d'e1RM corrige le RIR à l'envers
4 · Athlète en salle	5	1	Fin de repos muette écran verrouillé
7 · Backend et infra	5	2	Le rappel « série » plafonne à ~180 athlètes, sans alerte
12 · Business	5	2	Double abonnement prélevé en silence
14 · Onboarding	5	2	Aucune première séance prête pour l'athlète autonome
1 · Coach expérimenté	6	1	Les bilans disparaissent du tableau de bord au-delà de 8 lignes
3 · Nutritionniste	6	2	Garde-fous caloriques neutralisés ; « Sèche » sans effet
9 · Design UI	6	0	Thème clair illisible (contrastes à 1,8:1)
11 · Marketing	6	1	L'offre invité passe de 2 mois à 1 mois sous les yeux du visiteur
13 · QA	6	1	Aucun test lancé en CI
5 · Synchronisation	6,5	2	Stockage plein : la séance est perdue partout
10 · Engagement	7	1	« Série en danger » supprimée par le plafond de notifications
15 · Suivi des progrès	7	0	Masse grasse des balances reçue puis jetée

Les 12 chantiers à lancer en premier (fort impact, effort S ou M) :

- Sécurité 1 et 2 : droits calculés par le serveur uniquement, et fermeture de la faille XSS athlète → coach.
- Synchronisation 1 et 2 : ne plus perdre de séance quand le stockage est plein, écriture conditionnelle par ETag.
- Préparateur physique 1 : corriger la formule d'e1RM (une ligne, qui fausse aujourd'hui records, plateaux et décharges).
- Nutrition 1 et 2 : plancher calorique rétabli, objectif « Sèche » réellement appliqué.
- Business 2 : empêcher le double abonnement Essentielle + Ultime.
- Backend 2 et 3 : alerte si le serveur tombe, sauvegarde nocturne de la base.
- QA 1 : les tests bloquent la mise en ligne.
- Séance 1 et 5 : écran allumé et notification de fin de repos ; virgule décimale (82,5 kg saisi devient 825 kg).
- Onboarding 1 et 2 : première séance en 3 questions, code d'invitation conservé après installation.
- Performance 2 : minifier au déploiement (affichage 1,1 à 6 s plus rapide).
- Engagement 1 : priorité des notifications, pour que la « série en danger » parte toujours.
- Marketing 1 : même promesse invité (2 mois) sur toutes les pages.

Benchmark concurrentiel

Méthode : prix et fonctions relevés sur les sites des éditeurs, les fiches App Store et les comparatifs Capterra, GetApp et QuickCoach consultés le 29/09/2026 (voir §5). Prix en USD pour les éditeurs américains, HT, facturation annuelle quand elle existe. La « note/10 globale » est mon arbitrage : couverture fonctionnelle, rapport qualité-prix pour un coach indépendant francophone et notes des stores. RepCore a été relu dans le dépôt (`app/rc-core.1693.js`, `index.html`, `tarifs.json`, `cloudflare/src/`, `android/`).

« **Synergy** » : il s'agit très probablement de **Synergy (éditeur Synergy Performance)**. C'est un logiciel en français pour les coaches force et nutrition, de 18 à 99 €/mois, référencé sur Capterra avec 0 avis en septembre 2026, donc un acteur jeune. « **FitBrowser** » : aucune app de ce nom n'existe, en français comme en anglais (trois recherches, dont une approfondie). Pistes probables : **FitBudd** (app en marque blanche, qui publie des comparatifs contre Exercise.com et Hevy Coach) ou **Fit'Distance / Fitme** (acteurs français). Je n'ai rien inventé à son sujet : à faire préciser par l'utilisateur.

1. Tableau comparatif

App	Cible	Prix coach / athlète	Programmation	Nutrition	Messagerie / bilans	Paiement intégré	Gamification	IA	Intégrations santé	Note /10
ABC Trainerize	Coachs en ligne, salles, studios	Gratuit (1 client) ; Pro 5 à 23 \$; Pro 50 à 122 \$; Studio 248 \$. Options : nutrition 20-45 \$, Stripe 10 \$, app à ta marque 169 \$ + 99 \$/an. Gratuit pour l'athlète	Glisser-déposer, habitudes, programmes livrés automatiquement	Suivi + plans de repas (option payante)	Chat, groupes, bilans, vidéo (option)	Oui (Stripe, option)	Badges, défis	Générateur de séances IA	Apple Health, Garmin, Fitbit, MyFitnessPal	8,5
TrueCoach (Xplor)	Coachs force, en ligne ou hybrides	26 \$ (5 clients), 58 \$ (20), 137 \$ (50) ; pas d'offre gratuite. Gratuit pour l'athlète	Excellent éditeur, plus de 3 000 vidéos	Suivi simple ; avancé sur Standard et au-dessus	Chat + commentaires vidéo	Oui (Stripe, sans commission plateforme)	Faible	Non annoncé	Apple Watch, Garmin, WHOOP (Standard et au-dessus), Zapier	7,5
Everfit	Coachs seuls jusqu'aux grandes structures	Gratuit jusqu'à 5 clients ; Pro à partir de 16 \$; Studio 88 \$; options Autoflow 24 \$, repas 33 \$, paiements 8 \$. Gratuit pour l'athlète	Générateur IA, contenu à la demande	Plans de repas IA, recettes, MacroSnap (photo du repas)	1:1, groupes, forums, messages programmés, formulaires	Oui (option)	Classements (Autoflow), communauté	Oui : séances, repas, photo repas, réponses suggérées	Intégrations tierces	9
Kahunas	Coachs physique et bodybuilding en ligne	35 \$ (25 clients), 69 \$ (50), 99 \$ illimité avec app à ta marque	Plus de 1 000 vidéos, minuteurs	1,6 M d'aliments, code-barres	Bilans personnalisés, comparaison de photos avant/après, contenus au compte-gouttes	Stripe + PayPal, 0 % de commission	Habitudes	Non annoncé	Oui (tiers)	8
Hevy + Hevy Coach	Athlètes autonomes ; coachs « programmation seule »	Coach : 25 \$ (10 clients) jusqu'à 520 \$ (500). Athlète : gratuit, Pro à 2,99 \$/mois ou 74,99 \$ à vie	Très bon journal, plus de 400 exercices	Aucune	Chat client	Non	Fil social, suivi d'amis	Export des séances vers ChatGPT et Claude	Apple Watch, Health	7,5
Strong	Athlètes autonomes	Gratuit ; Strong PRO en abonnement	Journal épuré, supersets, RPE	Non	Non	–	Non	Non	Apple Health, Apple Watch, Siri, export CSV	6,5
Fitbod	Athlètes autonomes	Gratuit, puis 12,99 à 95,99 \$	Séances générées selon la récupération musculaire	Non	Non	–	Séries, faible	Cœur du produit	Apple Health, Strava, Fitbit, Watch	7

App	Cible	Prix coach / athlète	Programmation	Nutrition	Messagerie / bilans	Paiement intégré	Gamification	IA	Intégrations santé	Note /10
Harbiz (ES)	Coachs perso et nutritionnistes (Europe du Sud)	19 € (5 clients), 39 € (15), 69 € (30), 119-199 € (50-100)	Oui	Plans nutrition	Messagerie, évaluations	Oui (factures, paiements) + réservation	Faible	Non	Limitées	6,5
Exercise.com	Salles, multi-sites	Sur devis, environ 239 à 500 \$+/mois	Oui	Faible	Automatisations, bilans	Oui (point de vente, CRM)	Faible	Non	Apple Health, Garmin, Fitbit, Withings	6,5 (trop lourd pour un indépendant)
My PT Hub	Coachs perso	36 \$ (3 clients), 90 \$ illimité, 309 \$ tout compris	Oui	Plans nutrition	Messagerie	Oui	Faible	Non	Oui	6,5
MyCoach Pro (FR, Nice)	Clubs et staffs (foot, sport collectif)	Sur démo ; app gratuite pour l'athlète	Planification d'équipe	Non	Questionnaires wellness et RPE, chat	Non	Non	Non	Données GPS et performance	5 (hors cible)
Synergy (FR)	Coachs force + nutrition, 1 à 50 personnes	18 € (Essentiel) à 99 €/mois (Business)	Glisser-déposer, annotation vidéo	Analyse nutritionnelle, plans, recettes, micronutriments	Bilans et questionnaires automatiques	Non annoncé	Non	« Smart Coach » : alertes de progression	Apple Health, Google Health	6,5 (jeune, aucun avis)
Gymkee (FR)	Coachs indépendants	À partir de 28 €/mois, clients illimités	550 vidéos 4K, 12 formats (AMRAP...)	5 M d'aliments, recettes	Messagerie, habitudes	Gymkee Pay	Faible	Copilote « Dwayne » : programmes, clients à risque	Apple Health, Garmin, Zapier	7,5
Hexfit (QC, en français)	Coachs, kinés, nutritionnistes	Essai gratuit, sur abonnement	Plus de 13 000 exercices, génération IA	Oui	Messagerie, dossier client	Facturation + agenda	Faible	Oui	Polar, Fitbit, Garmin, Strava	7
RepCore	Athlètes musculation FR + coachs FR (surtout Kevin)	Athlète : 9,50 € (Essentielle) / 24,90 €/mois (Ultime), engagement 12 mois. Coach : 0 / 19 / 39 €. Coaching : 99 à 600 €	Très poussée : charge proposée à chaque série, RPE/RIR, 1RM estimé, décharges, blocs, tempo vérifié par vidéo, revue morpho	Ciquel (3 484 aliments) + OpenFoodFacts, code-barres (zxing), OCR d'étiquette (tesseract), plan composé par le coach, point hebdo de la diète	Canal de groupe avec publications programmées, modèles de message, brouillon de réponse au bilan, relances, annotation audio et vidéo	Non pour le coach : PayPal seulement vers le compte du créateur	Très forte : XP, badges, duels, saisons, classement d'amis, parrainage	Aucune (choix assumé)	Health Connect (APK Android), Raccourci iPhone, guides pour 8 marques de montres	7

Preuves pour RepCore : tarifs.json ; charge et RPE (app/rc-core.1693.js, 1 789 occurrences de RPE/RIR) ; app/data/ciquel.json (3 484 entrées) ; OFF_URL_RECHERCHE (rc-core:92622) ; app/vendor/zxing, app/vendor/tesseract, app/vendor/mediapipe ; planActif (rc-core:88873) ; canal programmé (rc-core:22974) ; modèles de message avec la mention « AUCUNE IA » (rc-core:106015-106023) ; brouillon de bilan (rc-core:73270) ; cloudflare/src/{xp,duels,saisons,sante,paypal,prospects}.js ; Health Connect (android/LISEZMOI.md) ; « AUCUN PAIEMENT ICI » (cloudflare/src/prospects.js, en-tête) ; boutique à vendeur unique (rc-core:49-58). Incohérence de prix : tarifs.json indique Essentielle à 9,5 €, mais le JSON-LD de index.html:37 annonce "price": "9.95", comme la note économique.

2. Ce que RepCore doit prendre à chaque concurrent

- **Trainerize** : ses programmes livrés automatiquement (un parcours d'accueil déroule semaines, habitudes et messages sans intervention du coach) et son annuaire Trainerize.me. RepCore a déjà la vitrine /coach/<slug> : il lui manque un annuaire qui envoie des prospects vers les coachs.
- **TrueCoach** : la vitesse de l'éditeur de programme côté coach (copier, coller, dupliquer une semaine en un geste) et les commentaires horodatés sous chaque séance. C'est ce qui fidélise les coachs force.
- **Everfit** : l'IA intégrée au quotidien (brouillons de réponse, repas générés, MacroSnap) et Autoflow (inscription et relances automatiques par groupe). RepCore a les relances (lot C3), mais pas l'IA ni la photo du repas.
- **Kahunas** : les **formulaires de bilan conçus par le coach**, la comparaison de photos côte à côte, et surtout l'encaissement Stripe/PayPal sur le compte du coach, sans commission.
- **Hevy / Hevy Coach** : la fluidité du journal, l'app gratuite pour l'athlète quand le coach paie, et l'export d'une séance vers ChatGPT ou Claude. C'est une IA « apporte ton assistant » qui ne coûte rien.
- **Strong** : la sobriété, l'Apple Watch et l'export CSV. Ses 4,9/5 sur 125 000 avis rappellent que la note sur les stores sert d'argument de vente.
- **Fitbod** : la génération automatique de la séance du jour selon la récupération par muscle. RepCore calcule déjà le volume par muscle : il suffit d'en tirer une séance.
- **Harbiz** : la réservation de créneaux et la facturation intégrées, utiles au coach hybride (présentiel et en ligne).
- **Exercise.com** : les tunnels de vente et le CRM qui transforment la vitrine en revenu. Rien d'autre : le reste est surdimensionné.
- **My PT Hub** : le forfait **clients illimités** à prix fixe, un argument simple qui ferait bien sur l'offre coach Pro à 39 €.
- **MyCoach Pro** : le questionnaire wellness quotidien court (RPE, sommeil, fatigue) tourné vers le staff. RepCore a le check-in, il lui manque une vue « équipe du jour » en un écran.
- **Synergy** : l'annotation vidéo (dessin sur l'image) et les micronutriments. RepCore a déjà l'analyse vidéo avec mediapipe : il faut le dire et le montrer en premier sur la page de vente.
- **Gymkee** : le copilote IA qui repère les clients à risque et le paiement « Gymkee Pay » pour vendre des programmes. C'est la preuve qu'un acteur français à 28 € les propose déjà.
- **Hexfit** : l'agenda, la facturation et le dossier client dans une seule interface en français.

3. Les 5 avantages réels et défendables de RepCore

1. **Une intelligence d'entraînement déterministe, sans IA et vérifiable.** Charge proposée à chaque série à partir de la séance précédente, RPE/RIR, 1RM estimé, décharges, retours par muscle rejoués chaque semaine (appliquerRetoursEnAttente, appelé au rendu de l'accueil), tempo prescrit puis vérifié par la vidéo (lot T3). Aucun concurrent pour coaches ne fait autant côté athlète. Seul Fitbod s'en approche, et c'est une boîte noire.
2. **Des charges ajustées à la phase du cycle menstruel** (activées par l'athlète, annoncées dans la FAQ et le JSON-LD de `index.html`). Aucun des 14 concurrents étudiés ne le propose. C'est un vrai différenciateur pour le public féminin.
3. **L'analyse vidéo dans l'app**, sans service tiers : `app/motion-lab.js` (découpe des répétitions), mediapipe (pose) et annotation audio (`cancelAudioAnnotation`, `rc-core:108039`). TrueCoach et Synergy ne font que commenter ou annoter la vidéo.
4. **Une gamification complète gérée côté serveur** : XP, badges rares, duels, saisons, défis, classement d'amis, parrainage (`cloudflare/src/xp.js`, `duels.js`, `saisons.js`, `retention.js`). Aucun outil pour coaches ne va aussi loin. Seul Hevy a un fil social, sans duels ni saisons.
5. **Une nutrition française et rapide** : table Ciqual de l'Anses, recherche OpenFoodFacts, code-barres et OCR d'étiquette en local, saisie en moins d'une minute (lot N3), assiette qui suit la séance réelle (N7), répartition des protéines (N6), point hebdomadaire de la diète (N1). Le tout marche hors ligne en PWA, pour 9,50 €, face aux options nutrition à 20-45 \$/mois chez Trainerize ou 33 \$ chez Everfit.

4. Les écarts fonctionnels propres au benchmark

Trois écarts relevés par le benchmark (assistant IA, formulaires de bilan conçus par le coach, présence sur les stores) sont traités par les agents IA, Coach et Marketing : ils ne sont pas répétés ici.

1. Encaissement du coach par ses athlètes (paiement direct au coach) — Priorité : Critique · Effort : L

Fusion. à arbitrer avec l'agent Business (point 1, modèle économique) : le paiement direct au coach est la suite logique de l'option « le coach paie ».

Pourquoi. Tous les concurrents pour coaches encaissent pour le coach : Kahunas (Stripe et PayPal à 0 %), TrueCoach, Everfit, Trainerize, Harbiz, Gymkee Pay. Chez RepCore, « AUCUN PAIEMENT ICI » (`cloudflare/src/prospects.js`), la boutique n'a qu'un vendeur (`rc-core:49-58`), et la note économique relève que l'argent des athlètes d'un coach tiers arrive chez le créateur. Un coach tiers ne peut donc pas en faire son outil de travail.

Intérêt. Chaque coach doit aujourd'hui gérer ses paiements à côté, avec un lien PayPal et un tableur, puis relancer les impayés à la main. Si le coach connecte son propre compte **PayPal Business**, avec un abonnement ou une commande créés sur son compte (API « PayPal Commerce Platform » en mode tiers, ou simplement son identifiant marchand), l'argent va directement du client au coach. RepCore n'est alors jamais intermédiaire : on respecte la contrainte DSP2 de la note de décision. L'app peut ensuite passer l'athlète en « suivi » dès le paiement (webhook déjà géré dans `cloudflare/src/paypal.js`) et le couper en cas d'impayé. Pour 20 athlètes à 150 €, cela sécurise 3 000 €/mois et évite environ 2 h de relances par mois. C'est aussi l'argument qui justifie l'offre coach Pro à 39 €.

Prompt de mise en place.

```
Contexte : RepCore (dépôt courant). Aujourd'hui un coach tiers ne peut pas encaisser ses athlètes : cloudflare/src/prospects.js dit « AUCUN PAIEMENT ICI », la boutique (app/rc-core.1693.js, bloc « LA BOUTIQUE » vers la ligne 49) n'a qu'un vendeur, et cloudflare/src/paypal.js n'écoute que les abonnements du créateur. Lis NOTE-DECISION-MODELE-ECONOMIQUE.md §2 : RepCore NE DOIT JAMAIS encaisser pour le compte d'un coach.

Objectif : permettre à un coach (palier « coach » ou « pro ») de se faire payer DIRECTEMENT sur SON compte PayPal pour ses formules de coaching (celles de sa vitrine /coach/<slug>).

À faire :
1. Côté coach (rc-core, écran vitrine / formules) : champ « Mon identifiant marchand PayPal » (merchant id ou e-mail PayPal Business), vérifié par un appel au Worker ; statut « relié / non relié ».
2. Worker : nouveau module cloudflare/src/paiements-coach.js. POST /fn/paiementCoach {formuleId, athlete} crée une commande PayPal (Orders v2) avec payee.merchant_id = celui du coach, custom_id = "<coach>|<athlete>|<formule>". Aucun fonds ne transite par le compte RepCore (pas de platform_fees). Webhook : étendre /paypal pour reconnaître custom_id à trois segments et écrire paiements_coach/<coach>/<id> = {athlete, formule, montant, statut, date}, puis accorder l'accès « suivi » à l'athlète pour la durée de la formule (réutilise la logique d'accessExpiry de paypal.js).
3. Côté athlète (page /coach/<slug> et fiche prospect) : bouton « Payer » visible seulement si le coach est relié ; sinon, le parcours prospect actuel ne change pas.
4. Tableau coach : liste des paiements (reçu, en attente, remboursé) sur la fiche athlète.
5. database.rules.json : paiements_coach lisible par le coach concerné seulement, écriture réservée au Worker.
Cas limites : coach non relié ; merchant id invalide ; webhook reçu avant la liaison (réutilise paypal_orphelins) ; remboursement ; athlète déjà en suivi (prolonger, ne pas écraser).
Tests : cloudflare/test/paiements-coach.test.mjs (création de commande avec payee correct, absence de platform_fees, webhook → accès accordé, orphelin rejoué), ajout de cas dans cloudflare/test/paypal.test.mjs ; assertions dans app/tests.js (bouton caché si non relié). Lance 'cd cloudflare && npm test' et la suite front via chargerTests().
Critère de fin : un coach relié vend une formule depuis sa vitrine, l'argent arrive sur SON compte PayPal (vérifié en sandbox), l'athlète passe en « suivi » automatiquement, et tous les tests passent.
```

5. Nutrition : photo du repas, bibliothèque de recettes et plans de repas réutilisables — Priorité : Haute · Effort : M

Fusion. la photo du repas est traitée par l'agent IA (point 6). Ne réaliser ici que la bibliothèque de recettes et les plans de repas.

Pourquoi. Everfit (MacroSnap, repas générés par IA, livres de recettes), Kahunas (1,6 M d'aliments), Gymkee (5 M d'aliments + recettes) et Synergy (recettes, micronutriments) vont au-delà de la saisie. RepCore a une excellente saisie (Ciqual 3 484 aliments, OpenFoodFacts `rc-core:92622`, code-barres, OCR) et un plan composé par le coach (`planActif`, `rc-core:88873`), mais pas de **bibliothèque de recettes** : « recette » n'apparaît que 17 fois, et jamais pour un objet recette.

Intérêt. Le coach nutrition reconstruit chaque plan à la main. Une bibliothèque de recettes (ingrédients Ciqual ou OFF, portions, macros calculées) réutilisable dans les plans et saisissable en un geste par l'athlète divise par 3 le temps de composition d'un plan. Elle rend aussi le journal plus fidèle : un plat maison saisi en 1 tap au lieu de 6 ingrédients. La photo du repas avec estimation (liée à l'écart n° 2) répond à la première cause d'abandon d'un journal alimentaire. C'est l'occasion de dépasser des options vendues 20 à 45 \$/mois chez les concurrents.

Prompt de mise en place.

```
Contexte : RepCore. Nutrition : app/data/ciqual.json (3484 aliments), recherche OpenFoodFacts (OFF_URL_RECHERCHE vers la ligne 92622 de app/rc-core.1693.js), plan alimentaire du coach (planActif vers la ligne 88873), saisie rapide du lot N3.

Objectif : bibliothèque de RECETTES partagée coach → athlètes, utilisable dans le plan et le journal.
1. Modèle : recettes/<coach>/<id> = {nom, portions, ingredients:[{src:'ciqual'|'off', ref, grammes}], etapes?:string, tags:[], creeLe}. Macros et kcal CALCULÉES à la lecture (fonction PURE recetteMacros(recette, portions)), jamais stockées. L'athlète peut aussi créer ses recettes perso sous recettes_perso/<athlete>.
```

2. Écran coach : créer, dupliquer, ajouter une recette à un repas du plan (planActif) avec un nombre de portions.
 3. Journal athlète : les recettes du coach et les siennes apparaissent dans la recherche (badge « recette ») ; saisie en portions ; hors ligne après la première synchro.
 4. Règles database.rules.json : 50 ingrédients au plus, nom ≤ 80 car., grammes 1-2000.
 Cas limites : ingrédient OFF disparu (garder un instantané des macros par 100 g dans l'ingrédient au moment de l'ajout) ; recette supprimée mais déjà journalisée (le journal garde les valeurs) ; portions décimales (0,5).
 Tests : app/tests.js – recetteMacros (somme exacte, arrondis, portions 0,5), instantané OFF, recette supprimée, apparition dans la recherche, mode hors ligne. chargerTests() vert.
 Critère de fin : un coach crée « Porridge protéiné », l'ajoute au petit-déjeuner du plan, l'athlète le journalise en 1 tap hors ligne ; tests verts.

6. Montres : saisie depuis la montre et Apple Health natif — Priorité : Moyenne · Effort : L

Pourquoi. Strong, Hevy et Fitbod ont une app Apple Watch ; Trainerize et TrueCoach synchronisent Garmin, Fitbit et WHOOP. RepCore lit Health Connect via l'APK et un Raccourci iPhone (cloudflare/src/sante.js). Pour les autres marques, il donne des **instructions de navigation** (« Ouvre Garmin Connect, onglet... », rc-core:113386-113401) : c'est de l'aide à la saisie manuelle, pas une intégration.

Intérêt. Le sommeil, la VFC et la FC au repos alimentent déjà les ajustements de RepCore. Plus ils arrivent automatiquement, plus la charge proposée est juste, et moins l'athlète saisit. Garmin (API Health, gratuite pour les développeurs approuvés) couvre une grande part des pratiquants de force qui portent une montre. En natif iOS (écart n° 4), HealthKit remplace le Raccourci, qui casse à chaque mise à jour d'iOS. Une saisie de série depuis la montre reste un atout secondaire.

Prompt de mise en place.

Contexte : RepCore. La santé synchronisée arrive par POST /sante/i (cloudflare/src/sante.js, format rc-sante-1, BORNES, JOURS_MAX=14) depuis Health Connect (android/) ou un Raccourci iPhone. Les autres marques n'ont que des guides (app/rc-core.1693.js, tableau vers les lignes 113285-113401).

Objectif : intégration Garmin Health API côté Worker (push de Garmin), sans rien changer au format interne.
 1. cloudflare/src/garmin.js : liaison OAuth (routes /garmin/lier, /garmin/retour), stockage du jeton chiffré (secret GARMIN_*), réception des « dailies » et « sleeps » en push sur /garmin/push, conversion PURE garminVersJours(payload) → jours rc-sante-1 (pas, sommeilMin, fcRepos, vfc en RMSSD si fournie avec vfcMethode='rmssd'), puis réutilisation de la logique d'écriture de sante.js (mêmes bornes, mêmes rejets comptés dans 'ignore').
 2. App : dans le guide Garmin (id:'garmin'), bouton « Connecter Garmin » qui remplace les instructions quand la liaison est active ; état et révocation dans les réglages santé.
 3. Plateforme 'garmin' ajoutée à PLATEFORMES / METHODE_DEFAULT.
 Cas limites : doublon avec Health Connect le même jour (priorité à la source la plus récente, et on le note dans 'origines') ; révocation côté Garmin ; valeurs hors bornes ; fuseau (jours de Paris, fonction paris de metier.js).
 Tests : cloudflare/test/garmin.test.mjs (conversion pure, bornes, doublons, signature de push invalide → 401) ; app/tests.js (bouton affiché selon l'état).
 `cd cloudflare && npm test` + chargerTests().
 Critère de fin : un compte Garmin de test pousse une journée et elle apparaît dans la santé de l'athlète comme une journée Health Connect ; tests verts.

7. Personnalisation à la marque du coach (logo et couleur dans l'app) — Priorité : Moyenne · Effort : M

Pourquoi. Kahunas (app à ta marque à 99 \$), Trainerize (169 \$ + 99 \$/an), Everfit (marque blanche), Gymkee et Azeo (marque blanche en français à 49 €) vendent tous la marque du coach. TrueCoach le fait « dans l'app » dès l'offre Standard. RepCore n'a aucune occurrence de « marque blanche » ou « white-label » : l'athlète d'un coach tiers voit RepCore et Kevin, pas son coach.

Intérêt. Un coach indépendant vend sa marque : pouvoir mettre son logo, sa couleur et son nom sur l'accueil de ses athlètes, les notifications et le canal est ce qui justifie l'offre Pro à 39 €. Une marque blanche complète dans les stores est hors de portée (plan gratuit, maintenance), mais un habillage limité à l'app couvre l'essentiel pour un effort M. La vitrine /coach/<slug> existe déjà : la même charte s'y appliquerait. Kahunas et Trainerize facturent ce service environ 100 \$.

Prompt de mise en place.

Contexte : RepCore. Le thème vit dans app/rc-style.1693.css (variables CSS) ; la vitrine publique d'un coach est servie par le Worker (cloudflare/src/pages.js, /coach/<slug>) ; le palier du coach est connu (cherche _palierEstCoach dans app/rc-core.1693.js).

Objectif : un coach de palier « pro » habille l'app de SES athlètes.
 1. Modèle : coachs/<coach>/marque = {nom (≤ 40), couleur (#RRGGBB), imageUrl (image déjà hébergée via le flux média existant, ≤ 200 Ko, carrée)}.
 2. Contraste : fonction PURE couleurAccessible(hex) qui refuse une couleur dont le contraste avec le fond sombre de l'app est < 3:1 et propose la plus proche acceptable.
 3. App athlète : au chargement, si l'athlète a un coach « pro » avec une marque, surcharger --red et --red-light (ou les tokens d'accent réels du CSS), afficher le logo et le nom du coach dans l'en-tête de l'accueil et du canal ; « propulsé par RepCore » en petit dans les réglages.
 4. Vitrine /coach/<slug> : même couleur et même logo.
 5. Retour au thème RepCore si le coach passe sous « pro » ou si le lien est rompu.
 Cas limites : logo absent ; image non chargée (repli sur les initiales) ; mode hors ligne (marque mise en cache) ; athlète autonome (aucun changement).
 Tests : app/tests.js (couleurAccessible, application et retrait des variables, repli sur les initiales) ; cloudflare/test/pages.test.mjs (vitrine avec marque). chargerTests() + `cd cloudflare && npm test`.
 Critère de fin : un coach Pro choisit une couleur et un logo, ses athlètes voient l'app à sa marque au lancement suivant ; tests verts.

8. Agenda : réservation de séances en présentiel ou en visio — Priorité : Basse · Effort : M

Pourquoi. Harbiz, Everfit (booking), Hexfit (agenda), Exercise.com et Trainerize (scheduling) gèrent les rendez-vous. RepCore a un « rendez-vous » hebdomadaire de diète (rc-core:6119) et des créneaux de programme, mais aucune réservation de séance avec le coach.

Intérêt. Une partie des coachs français sont hybrides : séances en salle ou en visio en plus du suivi en ligne. Sans agenda, ils gardent Calendly ou un agenda papier, avec une double saisie et des absences non suivies. Un agenda simple (disponibilités du coach, réservation par l'athlète, rappel push la veille via le Worker déjà en place dans planif.js, lien visio facultatif) garde le coach dans RepCore toute la journée. Il nourrit l'observance (bande du lot N8) et les relances (C3). L'effort reste M parce que les notifications et la planification horaire existent déjà.

Prompt de mise en place.

Contexte : RepCore. Le Worker a une planification à la minute (cloudflare/src/planif.js) et l'envoi de push (cloudflare/src/push.js) ; les relances coach existent (cloudflare/src/relances.js, lot C3).

Objectif : réservation de séances avec le coach.
 1. Modèle : agenda/<coach>/dispos = [{jourSemaine 0-6, debut 'HH:MM', fin 'HH:MM', dureeMin, lieu:'salle'|'visio', lienVisio?}] ; agenda/<coach>/rdv/<id> = {athlete, debut (ISO, heure de Paris), dureeMin, lieu, statut:'reserve'|'annule'|'honore'|'absent'}.
 2. Fonction PURE creneauxLibres(dispos, rdv, du, au) → liste de créneaux ; testée sur les changements d'heure (fin mars, fin octobre).
 3. App athlète : « Réserver une séance » (semaine en cours + suivante), annulation possible jusqu'à 12 h avant. App coach : vue semaine, marquer honoré ou absent, bloquer un créneau.
 4. Worker : dans planif.js, travail « rappels_rdv » → push à l'athlète 24 h et 1 h avant ; un rendez-vous « absent » alimente les règles de relance existantes.

```
5. Règles database.rules.json : l'athlète n'écrit que ses propres rdv, et seulement sur un créneau libre (contrôle final côté Worker via /fn/reserver pour éviter les doubles réservations).
Cas limites : deux athlètes sur le même créneau au même instant (transaction) ; coach qui supprime une dispo avec des rdv futurs (les rdv restent, avec un avertissement) ; changement d'heure ; athlète sans coach (fonction cachée).
Tests : cloudflare/test/agenda.test.mjs (réservation concurrente, rappels à la bonne heure de Paris, changement d'heure) ; app/tests.js (creneauxLibres, annulation < 12 h refusée). `cd cloudflare && npm test` + chargerTests().
Critère de fin : un athlète réserve un créneau, reçoit les deux rappels, le coach le marque honoré ; tests verts.
```

5. Sources (ouvertes le 29/09/2026)

- <https://www.capterra.com/p/10046842/Synergy/>
- <https://www.capterra.co.uk/software/1102190/Synergy>
- <https://www.quickcoach.fit/trainerize-pricing-2026.html>
- <https://www.trainerize.com/>
- <https://www.quickcoach.fit/truecoach-pricing-2026.html>
- <https://everfit.io/pricing>
- <https://everfit.io/>
- <https://kahunas.io/> (<https://kahunas.io/pricing> a renvoyé une 404)
- <https://heavycoach.com/>
- <https://coachway.io/articles/hevy-coach-pricing/>
- <https://apps.apple.com/us/app/hevy-workout-tracker-gym-log/id1458862350>
- <https://www.strong.app/> (la fiche App Store de Strong a renvoyé une 429)
- <https://apps.apple.com/us/app/fitbod-gym-fitness-planner/id1041517543>
- <https://www.capterra.com/p/201790/DudyFit/> (Harbiz)
- <https://www.fitbudd.com/compare/best-exercise-com-alternative> (Exercise.com)
- <https://mycoachpro.io/> (redirigé depuis <https://www.mycoachsport.com/>)
- <https://www.myhexfit.com/fr-fr/application-coach-sportif/>
- <https://www.gymkee.com/fr>
- <https://azeoo.com/blog/les-meilleures-applications-de-coaching-sportif-en-2025>

Pages vues seulement dans les résultats de recherche, sans être ouvertes : Harbiz (grille de prix sur intercom.help), My PT Hub ([quickcoach.fit](https://www.quickcoach.fit), prix), Exercise.com (costbench), notes Capterra de Trainerize, TrueCoach et Everfit, Kahunas sur Capterra (4,9/5, 60 avis), comparatif Azeoo, Gymkee, Fit'Distance et Promealplan (coachway.io/fr, promealplan.com/fr), MyCoach Pro sur l'App Store.

Agent 1 — Coach en ligne expérimenté · 6/10

L'espace coach a de vraies briques de productivité, rares chez un produit de cette taille : un tri par gravité (urgencyScore, rc-core:16759), une file de bilans avec « Athlète suivant → » (_entrerFileBilans 25514, _proposerBilanSuivant), un brouillon de réponse pré-écrit (brouillonBilan 73277), des relances serveur tracées (cloudflare/src/relances.js), des messages de canal programmés, la propagation d'un modèle et une sélection multiple. Mais le flux quotidien casse dès qu'on passe à l'échelle. Le bloc « Mes notifications » est plafonné à 8 lignes et masque les bilans à lire derrière les signaux d'entraînement. Il n'existe aucune messagerie 1:1 : tout part sur WhatsApp, hors du dossier. Le coach ne maîtrise pas la cadence des bilans : l'athlète la choisit, et le tableau de bord la lit mal. Et la réponse aux bilans se fait en texte seulement. Estimation pour un coach à 60 athlètes : 12 à 15 min par athlète et par semaine (triage 2 min, bilan 3 à 4 min, conversation WhatsApp non tracée environ 5 min, vidéos et ajustements 2 à 4 min). Pour gérer 120 athlètes à qualité égale, il faudrait descendre à 6 ou 7 minutes.

1. « Mes notifications » cache les bilans dès 8 lignes, et le × groupé enterre tout un groupe pendant 7 jours — Priorité : Critique · Effort : S

Constat. renderTodoBlock (rc-core:25915) empile d'abord une ligne par athlète pour chaque signal d'entraînement (rows.push.apply(rows, lignesEntrainement(clients)), ligne 25946 : douleur, décrochage, « Progression bloquée »...), puis seulement les lignes « Nouveaux bilans à lire » et « Bilans en retard ». Ensuite, elle coupe à TODO_MAX_LIGNES=8 (ligne 24457). Le reste s'affiche sous la forme « + N autres » (ligne 26093), un simple <div> qui ne s'ouvre pas. **Pour reproduire :** 9 athlètes avec un plateau (sg.plateauMuscle) et 1 athlète avec un bilan sans réponse. La ligne des bilans est la 10e : elle disparaît, et avec elle la seule entrée vers la file de bilans (_entrerFileBilans). Autre problème : le × d'une ligne groupée appelle dismissTodoRow (ligne 12280), qui reporte **tous** les athlètes de la ligne (r.list.forEach(c=>reporterAlerte(r.type,c)), ligne 12288) pendant 7 jours, sans annulation possible. Ce report coupe aussi la relance automatique « bilan » (reporteParCoach, relances.js).

Pourquoi le mettre en place. C'est le point d'entrée de toute la journée du coach. Plus le portefeuille grossit, plus il y a de signaux d'entraînement, et plus les bilans (la promesse du coaching payé) passent sous la ligne de coupe. Un appui malheureux sur × fait disparaître 12 bilans pendant une semaine sans que rien ne le signale.

Intérêt réel. À 60 athlètes, 10 à 20 % ont un signal d'entraînement à un instant donné : le plafond de 8 est atteint en permanence. Un bilan qui reste sans réponse 7 jours est la première cause de résiliation citée par les coaches en ligne (« je paie et personne ne me répond »). Si le bilan reste toujours visible et que le × devient une action par athlète avec « Annuler », le risque d'oubli disparaît. Le coach économise aussi la recherche manuelle dans la liste de 60 lignes (environ 1 à 2 min par session de triage, soit 10 min par semaine). Enfin, le bilan est la tâche la plus facturable : il doit toujours rester en haut.

Prompt de mise en place.

Dépôt RepCore (/home/user/coaching). Fichier principal app/rc-core.1693.js (123k lignes, lis par grep + sed -n).

Problème : renderTodoBlock (= ligne 25915) construit 'rows' dans cet ordre : drapeaux, lignes d'entraînement par athlète (_lignesEntrainement, =25295), puis nostart, bilan, overdue, vidéos, notes, expiring, rite, noprog. Il coupe ensuite à TODO_MAX_LIGNES=8 (=24457) et le surplus devient « + N autres » (=26093), non cliquable. Avec 9 athlètes en « Progression bloquée » et 1 nouveau bilan, la ligne bilan disparaît. En plus, dismissTodoRow (=12280) reporte d'un coup TOUS les athlètes d'une ligne groupée pendant 7 jours, sans annulation.

A faire :

1. Garantir la visibilité des lignes « drapeau », « bilan » (nouveau bilan à lire) et « overdue » : elles ne comptent jamais dans le plafond. Le plafond ne s'applique qu'aux autres lignes. Garde l'ordre de gravité dans les bandes (Urgent/Important/À traiter). Ne change ni window_todoRows ni les index passés à dismissTodoRow, _entrerFileBilans et ouvrirAjustement (ils doivent rester cohérents avec ce qui est affiché).
2. Regrouper les lignes d'entraînement d'un même type au-delà de 3 athlètes : une ligne « N athlètes · Progression bloquée » dont le clic ouvre une file comme _entrerFileBilans (crée _entrerFileSignal(idx) sur le même modèle, avec « Athlète suivant »).
3. Rendre « + N autres » cliquable : il déplie les lignes coupées (état en mémoire, pas en localStorage).
4. dismissTodoRow sur une ligne groupée (r.list.length>1) : ouvre une petite feuille qui liste les athlètes avec une case chacun, tous cochés, et le bouton « Reporter 7 jours ». Après tout report, affiche un toast avec le bouton « Annuler » pendant 6 s, qui restaure currentUser.alertStatus tel qu'il était avant (garde une copie des clés touchées) puis appelle saveUser() et renderTodoBlock(getClients()).
5. Ne touche pas à urgencyScore ni à isAlertSnoozed.

Tests à ajouter dans app/tests.js (style existant : ok('libellé',(()=>{...}))()), à lancer dans la console via chargerTests() :

- 10 clients factices, dont 9 avec un signal d'entraînement actif (stub signauxEntrainement ou données qui le déclenchent) et 1 avec hasNewBilan : après renderTodoBlock, window_todoRows contient une ligne type 'bilan'.
- Une ligne groupée reportée puis « Annuler » : alertStatus revient exactement à son état initial.
- Un drapeau rouge reste non reportable.

Build : suis la convention des lots précédents (voir 'git show 388488d --stat' : renommer rc-core/rc-style au numéro de build suivant, mettre à jour app/index.html et app/sw.js). Critère de fin : suite de tests à 0 échec, et scénario manuel « 9 plateaux + 1 bilan » où la ligne bilan est visible.

2. Pas de messagerie coach ↔ athlète : tout part sur WhatsApp, hors du dossier — Priorité : Haute · Effort : L

Constat. Le code le dit lui-même : « AUCUNE messagerie n'est implémentée : on ouvre le WhatsApp du coach » (rc-core:17199), et « La messagerie coach ↔ athlète n'existe pas dans ce produit » (63159). Le « message groupé » ouvre une conversation WhatsApp par athlète, en un appui chacun (openWaGroupe 23557), avec un plafond TPL_GROUPE_MAX=50 (106025). Un numéro saisi en « 06... » n'a pas de destinataire (_numWa 17211 ; libellé « le contact sera à choisir dans WhatsApp », 23584). Le « mot au coach » de l'athlète ne déclenche aucune notification (MOT_COACH_MENTION, 29009). La carte « Un mot de ton coach » n'offre que « Compris » (25904) : l'athlète ne peut pas répondre.

Pourquoi le mettre en place. La messagerie intégrée est le cœur de Trainerize, TrueCoach, Everfit et Kahunas. Aujourd'hui, l'échange le plus fréquent entre coach et athlète n'est ni daté, ni rattaché au dossier, ni visible dans le tri, ni comptable. Le coach mélange les messages de ses 60 athlètes avec sa vie privée sur WhatsApp.

Intérêt réel. C'est environ 5 min par athlète et par semaine qui échappent à l'app. À 60 athlètes, cela fait 5 h de travail invisible, impossible à trier et impossible à déléguer à un assistant. Avec un fil par athlète dans RepCore : (a) « non lu / sans réponse depuis 24 h » devient un signal du tableau de bord ; (b) les modèles tpl* et le brouillon s'appliquent aux messages ; (c) un assistant coach peut répondre depuis le même compte ; (d) l'historique suit l'athlète même si le coach change de téléphone. Les relances passent aussi d'un « Compris » à une vraie conversation. Gain estimé : 1 à 2 min par athlète et par semaine, en supprimant les allers-retours entre applications et les recherches de contexte, soit 1 à 2 h par semaine à 60 athlètes. C'est aussi un argument de vente direct face aux concurrents.

Prompt de mise en place.

Dépôt RepCore (/home/user/coaching). Front : app/rc-core.1693.js + app/index.html. Base : Firebase RTDB (plan Spark), règles dans database.rules.json. Serveur léger : cloudflare/src (metier.js, planif.js, PUSH_TYPES ligne ~41 de metier.js). Aujourd'hui, aucune messagerie 1:1 n'existe (commentaires rc-core ~17199 et ~63159). Les échanges passent par wa.me.

Objectif : un fil de messages privé par couple coach/athlète, texte seulement pour ce lot.

1. Données : messages/<coachKey>/<athleteKey>/<msgId> = {de:'coach'|'athlete', texte (<1000 car.), at (ms), lu (bool)}. Clés = e-mail avec '→', '←' (même

```

convention que coachEmailKey, voir _relCle ≈25700). NE PAS mettre les messages dans users/<email> : saveUser réécrit le document entier (voir le commentaire MOT_COACH_MAX ≈29000).
2. Règles database.rules.json : lecture et écriture pour le coach si $coachKey == auth.token.email.replace('.', '') (représume le motif exact des règles de canaux ou relances auto), et pour l'athlète si $athleteKey est le sien ET users/$athleteKey/coachEmailKey == $coachKey. Chaque participant ne peut écrire que 'de' = son rôle. Validation : texte string 1..1000, at number, pas d'autres champs. Seul le destinataire peut passer 'lu' à true.
3. Côté coach : un nouvel écran s-coach-messages (ajoute-le dans index.html sur le modèle de s-coach-relances). Liste des fils triés par dernier message, pastille « non lu », puis vue d'un fil. Un bouton « Message » sur la fiche athlète (zone #ccd-wa) ouvre le fil. Les modèles existants (tplInserer, TPL_VARS avec {prénom}) sont utilisables dans la zone de saisie.
4. Côté athlète : une entrée « Mon coach » sur l'accueil, et la carte « Un mot de ton coach » (_renderRelanceAthlete ≈25890) gagne un bouton « Répondre » qui ouvre le fil.
5. Notification : à l'envoi, appelle deposerEvenement({type:'message', dest, ...}) (≈20970). Dans cloudflare/src/metier.js, traite 'message' comme 'reponse_bilan' (≈981) : retire le message en base avant d'envoyer, push de type 'coach' pour l'athlète et nouveau type 'message' pour le coach (ajoute-le à PUSH_TYPES et au réglage de notifications de l'athlète).
6. Tableau de bord : dans renderToDoBlock, une ligne « Message sans réponse » (orange) pour les fils dont le dernier message vient de l'athlète depuis plus de 24 h. Le signal s'éteint quand le coach répond, pas quand il ouvre le fil.
7. Lecture : charge à l'ouverture de l'écran et au retour au premier plan (pas d'écoute permanente : le plan Spark limite les connexions simultanées).
Pagination à 50 messages.
Cas limites : athlète détaché du coach (plus d'accès), texte vide, hors ligne (réutilise la file d'envoi existante si elle convient, sinon refuse avec un toast), XSS (escapeHtml partout).
Tests : app/tests.js (ok(...)), lancés par chargerTests() pour les fonctions pures (tri des fils, calcul du « sans réponse 24 h », validation du texte). cloudflare/test/messages.test.mjs pour le traitement de l'événement (cd cloudflare && npm test). Mets à jour scripts/verif/regles.mjs si besoin. Suis la convention de build (git show 388488d --stat). Critère de fin : un coach et un athlète s'écrivent, chacun reçoit une notification, la ligne « Message sans réponse » apparaît puis s'éteint, et tous les tests passent.

```

3. Le coach ne maîtrise pas la cadence des bilans, et le tableau de bord la lit mal — Priorité : Haute · Effort : M

Constat. C'est l'athlète qui choisit « Chaque semaine / Toutes les 2 sem. » (setBilanFreq, rc-core:70207 ; boutons dans index.html:4584). Côté athlète, l'échéance tombe le samedi selon cette fréquence (getNextBilanSaturday 74126). Côté coach, needsAlert (27525) et le serveur de relances (cloudflare/src/relances.js:82) utilisent un délai fixe de 14 jours. **Pour reproduire** : un athlète hebdomadaire dont le dernier bilan date de 10 jours voit « bilan en retard » sur son accueil, alors que le coach ne le voit ni dans « Bilans en retard » ni dans les relances automatiques. À l'inverse, un athlète vendu en « suivi hebdomadaire » peut passer seul à 2 semaines. Le questionnaire est lui aussi figé (BILAN_QUESTIONS 71285) : le coach ne peut ajouter aucune question.

Pourquoi le mettre en place. Le rythme des bilans fait partie de l'offre vendue : c'est au coach de le fixer, par athlète. Un tableau de bord qui déclare un athlète en retard une semaine plus tard que ce que voit l'athlète perd sa crédibilité, et le coach retourne à son tableur.

Intérêt réel. Si le coach fixe la fréquence et le jour (par exemple tous les bilans le dimanche), les bilans arrivent groupés. Il peut alors les traiter d'une traite le lundi matin via la file existante (« Athlète suivant »). C'est le gain de temps le plus fort connu en coaching en ligne : 30 à 40 % de temps par bilan en moins, parce qu'il n'y a plus de changement de contexte au fil de la semaine. À 60 athlètes, cela représente 1 à 1,5 h par semaine. Les retards deviennent justes, donc les relances automatiques partent au bon moment. Enfin, une ou deux questions propres au coach (« Ta prise de créatine ? », « Ton cardio ? ») évitent un aller-retour WhatsApp par bilan.

Prompt de mise en place.

```

Dépôt RepCore (/home/user/coaching), app/rc-core.1693.js.
Constat : _bilanFreq est choisi par l'athlète (setBilanFreq ≈70207, _renderBilanChoiceUI, boutons index.html ≈4584), son échéance est un samedi (getNextBilanSaturday ≈74126, _bilanAnchorSat). Mais needsAlert (≈27525), qui alimente « Bilans en retard », urgencyScore et agregerPortefeuille, utilise 14 jours fixes, et cloudflare/src/relances.js (signauxRelance, ligne ~82) aussi.

À faire :
1. Nouveau champ posé par le coach sur le dossier de l'athlète : bilanCadence = {freq:1|2|4 (semaines), jour:0..6}. Il se règle sur la fiche athlète (zone du calendrier des bilans #ccd-bil-cal), et aussi en masse depuis la barre de sélection (_selMaj ≈26304, ajoute un bouton « Cadence »). Vérifie que database.rules.json autorise le coach à écrire ce champ (même motif que les autres champs coach sur users/$athlete). Borne les valeurs.
2. Crée une fonction PURE echanceBilan(c, maintenant) → {echance, retardJours}. Si bilanCadence existe, elle l'utilise ; sinon, repli sur _bilanFreq||2 et le samedi (comportement actuel de l'athlète). Branche-la dans needsAlert, dans getNextBilanSaturday côté athlète et dans _htmlBilanRetard, pour qu'il n'y ait plus qu'une seule définition du retard.
3. Côté athlète : si bilanCadence est posée par le coach, masque les boutons de fréquence et affiche « Ton coach a fixé : bilan chaque <jour>, toutes les N semaines ».
4. Serveur : dans cloudflare/src/relances.js, signauxRelance lit d.cadence (freq, jour) et calcule overdue avec la même règle. métier.js doit transmettre bilanCadence et _bilanFreq dans `d`. Mets à jour RELANCE_IGNES (texte « quatorze jours après le dernier bilan » ≈25660) pour dire « le lendemain de l'échéance fixée ».
5. (Optionnel dans le même lot) questionsCoach : jusqu'à 3 questions libres du coach (texte ≤120 car.) ajoutées en fin de bilan de suivi, avec réponses stockées dans le bilan sous 'coach-ql.3' et affichées dans BILAN_QUESTIONS.suivi.
Cas limites : athlète sans bilan (nostart garde sa règle), changement de cadence en cours de cycle (l'échéance repart du dernier bilan), fuseau et heure d'été (avance par setDate comme getNextBilanSaturday).
Tests : app/tests.js — échéance hebdo avec dernier bilan à J-10 donne needsAlert true ; bimensuel à J-10 donne false ; repli sans cadence identique à l'ancien comportement. cloudflare/test/relances.test.mjs — overdue selon la cadence. Lance chargerTests() et `cd cloudflare && npm test`. Convention de build : git show 388488d --stat. Critère de fin : le retard affiché à l'athlète et au coach est le même jour pour les deux fréquences.

```

4. Réponse aux bilans en texte seulement, alors que l'enregistreur vocal existe déjà — Priorité : Haute · Effort : M

Constat. saveReponseBilan (rc-core:73073) n'enregistre que b.reponseCoach=txt.slice(0,2000) (73096). L'enregistrement audio est pourtant déjà écrit pour la correction vidéo (startAudioRec / toggleAudioRec, 107916-107940, MediaRecorder opus), avec le téléversement Storage (CLOUD.uploadVideo, 5462). Les pastilles de réponse rapide du bilan sont figées (QC_BILAN_DEFAULTS 106140, « Elle est FIXE »).

Pourquoi le mettre en place. Kahunas, Everfit et TrueCoach misent tout sur la réponse vocale ou vidéo : un coach parle 3 fois plus vite qu'il n'écrit, et l'athlète perçoit une réponse vocale comme plus personnelle. Ici, le brouillon (C2) prépare le constat, mais le coach doit encore taper son commentaire au clavier, souvent sur téléphone.

Intérêt réel. Une réponse de bilan tapée prend 4 à 6 min. En vocal de 60 à 90 s, avec le brouillon chiffré affiché à côté, elle en prend 2. À 60 athlètes en rythme bimensuel (30 bilans par semaine), cela libère 1 à 1,5 h par semaine. La valeur perçue par l'athlète augmente (il entend son coach), ce qui soutient la rétention sur les formules 99 à 600 €. Réutiliser l'enregistreur existant limite le risque.

Prompt de mise en place.

```

Dépôt RepCore (/home/user/coaching), app/rc-core.1693.js.
Aujourd'hui, saveReponseBilan(email,bilanId,taId) (≈73073) ne stocke que du texte (reponseCoach ≤2000). Un enregistreur audio existe pour la correction vidéo : _audioMediaRecorder, startAudioRec, stopAudioRec, toggleAudioRec (≈107916). Le téléversement passe par CLOUD.uploadVideo (≈5462, Firebase Storage). Le bloc de réponse est rendu par renderReponsesBilans, avec des identifiants dérivés de _idBilan (voir _taIdBilan).

À faire :
1. Extraire de l'enregistreur vidéo une fonction réutilisable enregistreurAudio({maxSec:180, onFin(blob,durée)}) sans casser la correction vidéo (garde les mêmes noms publics, avec des adaptateurs). Plafond de 3 min et arrêt automatique.

```

```

2. Dans le bloc de réponse de chaque bilan : bouton « Répondre en vocal ». Pendant l'enregistrement : minuteur. À la fin : écoute, puis « Envoyer » ou « Recommencer ». Le texte pré-écrit du brouillon (brouillonBilan) reste visible au-dessus comme aide-mémoire.
3. Envoi : téléversement vers Storage sous audio/<athleteKey>/bilan_<bilanId>_<ts>.webm (ou .ogg/.mp4 selon le mime), puis écriture dans le bilan de b.reponseAudio={url,duree,at}. reponseCoach devient facultatif si reponseAudio existe. hasNewBilan (≈27562), bilansSansReponse et relances.js (signal 'bilan', 'lder.reponseCoach') doivent considérer « répondu » si reponseCoach OU reponseAudio. Mets à jour les trois endroits et le serveur (cloudflare/src/relances.js signauxRelance + metier.js relecture avant push à ≈981).
4. Côté athlète : lecteur <audio controls> sous le bilan concerné, et la même notification « Ton coach a répondu à ton bilan ».
5. Enchaînement : après l'envoi vocal, même logique que le texte (proposerBilanSuivant(_fileBilansSuivants(c.id))).
6. Laisser le coach enregistrer ses propres pastilles de réponse rapide de bilan (QC_BILAN_DEFAULTS devient la valeur par défaut, surchargée par currentUser.qcBilan, 12 au maximum, 200 car. chacune).
Cas limites : micro refusé (toast clair), passage en arrière-plan pendant l'enregistrement (déjà géré ≈44500, à réutiliser), hors ligne (refuser l'envoi, garder le blob en mémoire), Safari iOS (mime mp4).
Tests app/tests.js (chargerTests()) : hasNewBilan false quand seul reponseAudio est présent ; bilansSansReponse de même ; la correction vidéo garde ses fonctions. cloudflare/test/relances.test.mjs : pas de signal 'bilan' si reponseAudio. Convention de build : git show 388488d --stat. Critère de fin : réponse vocale envoyée et écoutée côté athlète, signal éteint, tests verts.

```

5. Aucune alerte « programme qui se termine » : le coach doit s'en souvenir seul — Priorité : Moyenne · Effort : S

Constat. Le bloc de programme porte bien début et semaines (programmeDe rc-core:66535), et bilanBloc (34469) sait lire un bloc fini. Pourtant, aucune ligne de « Mes notifications » ne l'annonce. Les types de renderToDoBlock sont drapeau, signaux d'entraînement, nostart, bilan, overdue, videos, notes, expiring, rite et noprog. Le seul signal d'échéance, blocfini, concerne le **bloc de priorité musculaire** (blocAvancement 68303), pas le programme. Un athlète qui finit sa dernière semaine continue son programme en boucle sans que le coach en soit averti.

Pourquoi le mettre en place. Préparer le bloc suivant est un rendez-vous fixe de chaque suivi, et il se prépare en amont (bilan de bloc, puis nouveau modèle, puis propagation C4). Sans rappel, le coach tient un agenda à part, ou il découvre le retard via une plainte.

Intérêt réel. Avec une ligne « Bloc qui se termine dans ≤ 7 jours » qui ouvre directement le « Bilan du bloc » (T7) puis l'assignation de modèle (C4), le coach enchaîne les outils déjà construits en 3 gestes au lieu de les chercher. Il évite 1 à 2 semaines de programme en boucle par athlète et par bloc, soit 4 à 6 cas par mois à 60 athlètes. C'est perçu comme du manque de suivi, et c'est une cause directe de résiliation.

Prompt de mise en place.

```

Dépôt RepCore (/home/user/coaching), app/rc-core.1693.js.
programmeDe(user) (≈66535) rend {debut, semaines, decharges}. bilanBloc (≈34469) et bilanBlocExportHtml (≈34605) produisent le bilan de fin de bloc.
renderToDoBlock (≈25915) ne signale jamais la fin d'un programme (seul 'blocfini' existe, et il concerne blocPriorite).

À faire :
1. Fonction PURE finProgramme(c, maintenant) → null ou {joursRestants, fini:boolean, finPrevue}. Signal levé si joursRestants ≤ 7 (y compris un bloc déjà dépassé de ≤ 21 jours). Au-delà de 21 jours de dépassement, le signal tombe (le coach a choisi de laisser tourner).
2. Dans renderToDoBlock, une ligne type 'progfin', couleur var(--green) (bande « À traiter »), libellé « Bloc qui se termine » ou « Bloc terminé », groupée comme les lignes administratives, reportable via dismissToDoRow. Au clic : file sur le modèle de _entrerFileBilans. Chaque athlète ouvre sa fiche directement sur le bouton « Bilan du bloc », puis propose « Assigner le bloc suivant » (écran s-coach-prog-assign, pré-coché sur l'athlète via SEL_ATHLETES/_selCocherSur).
3. Ajouter 'progfin' à RELANCE_LIGNES (≈25655) en ligne grisée avec RELANCE_TRAVAIL.
4. Ne pas le faire entrer dans urgencyScore (même règle que 'rite' et 'blocfini').
Cas limites : programme sans début ou avec semaines hors bornes (programmeDe rend null, donc rien), athlète _fromCode, nouveau programme assigné (début change, donc le signal tombe).
Tests app/tests.js (chargerTests()) : fin dans 5 jours → signal ; dans 10 jours → rien ; dépassé de 30 jours → rien ; nouveau début → rien. Convention de build : git show 388488d --stat. Critère de fin : ligne visible et chemin « Bilan du bloc » puis « assignation » en 3 gestes.

```

6. Portefeuille sans segments ni « dernier contact » : les athlètes silencieux sont invisibles — Priorité : Moyenne · Effort : M

Constat. Aucun champ d'étiquette ou de groupe d'athlètes n'existe (aucun résultat pour tags, groupe, etiquette côté athlète). La sélection multiple (SEL_ATHLETES, rc-core:26282) est en mémoire et se perd à chaque session. Rien ne date le dernier échange du coach avec un athlète (aucun dernierContact). La liste « inactifs » (14 jours sans signe de vie, listeInactifs 27738) est rendue **sous** la grille de vignettes et toute la liste (#ch-inactifs, index.html:1831, après #ch-clients-list à 1772). À 60 athlètes, elle se trouve plusieurs écrans plus bas.

Pourquoi le mettre en place. Le tri par urgence ne montre que les athlètes qui ont un problème. L'athlète régulier et silencieux, qui ne reçoit plus un mot personnel depuis 3 semaines, est celui qui résilie « parce que je n'avais plus besoin de coach ». Trainerize et Everfit ont des groupes (« Prépa compét », « Offre Transfo », « Présentiel ») pour filtrer et agir en masse.

Intérêt réel. Une étiquette par athlète réutilisée par le filtre, le canal, le message groupé et l'assignation de modèle évite de recocher 20 personnes à chaque action (environ 2 min par action groupée). Un compteur « sans échange personnel depuis 14 jours » force une touche humaine régulière, pour 30 s par athlète. C'est le levier de rétention le moins cher. Remonter les inactifs au-dessus de la liste évite qu'ils soient oubliés. À 120 athlètes, c'est ce qui rend le portefeuille pilotable sans tableur.

Prompt de mise en place.

```

Dépôt RepCore (/home/user/coaching), app/rc-core.1693.js + app/index.html.
Constat : pas d'étiquettes d'athlètes. SEL_ATHLETES (≈26282) n'est qu'en mémoire. Aucun horodatage de dernier contact du coach. #ch-inactifs (index.html ≈1831) est rendu sous #ch-clients-list.

À faire :
1. Étiquettes : currentUser.etiquettes = {<id>:{lib (≤24 car.), couleur}} (20 au maximum) et currentUser.etiquettesAth = {<athleteKey>:{ids}}. Les stocker dans le dossier du COACH, pas celui de l'athlète (comme coachNotes et motsLus), pour ne jamais réécrire le document de l'athlète. Vérifie les règles database.rules.json pour ces champs (bornes de taille).
2. Interface : dans la barre de sélection (_selMaj ≈26304), bouton « Étiqueter ». Sur la fiche athlète, pastilles modifiables. Dans la barre d'état du portefeuille (renderPortefeuille ≈17078), une rangée de puces-filtres par étiquette qui filtre renderClientList (même mécanisme que les filtres existants).
3. Réutilisation : openWgGroupe, l'assignation de modèle (s-coach-prog-assign) et la décharge groupée proposent « Cocher l'étiquette... ».
4. Dernier contact : currentUser.contacts = {<athleteKey>:{ts}}, mis à jour par saveReponseBilan, saveVideoCorrection, l'envoi d'un lien WhatsApp depuis l'app (_wBoutonTodo, openWgGroupe/_wagPreparer) et, si le lot messagerie existe, l'envoi d'un message. Fonction PURE sansContact(c, contacts, maintenant, jours=14). Ligne dans renderToDoBlock « Sans échange depuis 14 j » (bande verte, reportable), limitée aux athlètes actifs (isActive).
5. Déplacer #ch-inactifs et #ch-jamais-demarre juste sous #ch-todo dans index.html, en version repliée (3 noms + « voir tout »).
Cas limites : athlète supprimé (nettoyer ses entrées d'étiquettes), étiquette supprimée (retirée de tous les athlètes), coach sur deux appareils (champ dans le dossier coach, donc synchronisé).
Tests app/tests.js (chargerTests()) : filtrage par étiquette ; sansContact à J-15 vrai, à J-3 faux ; saveReponseBilan met à jour contacts. Convention de build : git show 388488d --stat. Critère de fin : filtrer, étiqueter en masse et voir les athlètes silencieux sans défiler la liste.

```

7. Relances automatiques : textes figés et pas de relance sur l'inactivité d'entraînement — Priorité : Moyenne · Effort : M

Constat. Les 5 textes sont codés en dur et identiques pour tous les coachs (RELANCE_CORPS, cloudflare/src/relances.js:34 ; `_waCorpsGroupe rc-core:17659`, avec un test qui impose l'égalité). Seuls 5 signaux sont automatisables (RELANCE_SIGNAUX). L'**inactivité d'entraînement** (inactif, 14 jours sans signe de vie, rc-core:27728) n'y figure même pas en ligne grisée (RELANCE_LIGNES 25655). C'est pourtant le premier signal de désabonnement. Le texte « overdue » dit « quatorze jours », quelle que soit la cadence réelle (voir point 3).

Pourquoi le mettre en place. Un message automatique écrit par un autre, avec le même emoji 🏃 chez tous les coachs, ne sonne pas comme le coach. Beaucoup de coachs laisseront donc les relances coupées (c'est le réglage par défaut). Or relancer un athlète qui ne s'entraîne plus est précisément la relance que le coach oublie quand il a 60 dossiers.

Intérêt réel. Avec des textes personnalisables (variables {prénom}, {jours}) et un signal « pas de séance depuis N jours » (7 à 14, au choix), le coach automatise 80 % de ses relances d'assiduité. Il garde la trace dans relances_auto et la limite d'un message par semaine. Estimation : 5 à 10 relances manuelles de moins par semaine à 60 athlètes (environ 30 à 45 min), et une détection du décrochage 1 à 2 semaines plus tôt.

Prompt de mise en place.

```
Dépôt RepCore (/home/user/coaching). Serveur : cloudflare/src/relances.js (pur), metier.js (relancesCoachUn), planif.js ; tests
cloudflare/test/relances.test.mjs. App : rc-core.1693.js, bloc « LOT C3 » (≈25625-25915 : RELANCE_SIGNAUX, RELANCE_LIGNES, relancesRegles,
renderRelancesCoach). Un test impose que RELANCE_CORPS === _waCorpsGroupe (≈17659).
```

À faire :

1. Nouveau signal 'inactif' : l'athlète a au moins une séance et aucune séance ni bilan depuis 'delai' jours (7 à 21, défaut 10). Côté serveur, signauxRelance(d,t) reçoit d.derniereSeance (metier.js doit la lire : dernière date de users/<a>/sessions, sans télécharger tout le tableau si possible, par exemple via un champ résumé déjà existant ; vérifie dernierSigneDeVie ≈27700 côté app et réutilise le même critère). Épisode daté par 'depuis' = dernière activité + delai. Ajoute 'inactif' à RELANCE_SIGNAUX des deux côtés, en DERNIÈRE position, et à RELANCE_LIGNES (auto:true, quand:'N jours sans séance').
2. Textes personnalisables : relancesAuto.textes = {<signal>: string ≤ 280}. Variables {prénom} et {jours}. S'ils sont absents, le texte par défaut s'applique (RELANCE_CORPS). texteRelance(signal, prenom, perso, vars) applique les variables, échappe et tronque. Écran : un champ de texte par ligne active, avec aperçu « Salut Léa, ... ». Adapte le test d'égalité RELANCE_CORPS/_waCorpsGroupe pour qu'il porte sur les TEXTES PAR DÉFAUT uniquement.
3. Texte par défaut de 'inactif' : « ça fait {jours} jours qu'on ne t'a pas vu à l'entraînement : tout va bien ? Dis-moi si on doit adapter quelque chose. »
4. Garder toutes les garanties existantes : 1 message par 7 jours, 1 par épisode, pause, exclusion, report par le coach (isAlertSnoozed type 'inactif').

Tests : relances.test.mjs – inactif levé à J+10, pas avant ; texte personnalisé avec variables ; texte vide ou trop long remplacé par le défaut ; ordre de priorité inchangé pour les 5 premiers. app/tests.js – relancesRegles accepte 'inactif'. 'cd cloudflare && npm test' puis chargerTests(). Convention de build : git show 388488d --stat. Critère de fin : un coach écrit sa propre relance d'inactivité, elle part une fois, et elle est visible dans « ce qui est parti cette semaine ».

8. Personne ne mesure les minutes par athlète, et « Devenu athlète » ne fait entrer personne — Priorité : Basse · Effort : S

Constat. Aucun indicateur de temps passé par le coach n'existe (aucun résultat pour tempsCoach ou équivalent). La question centrale « combien de minutes par athlète » n'a donc aucune réponse dans l'app. Côté prospect, prospectStatut(id,'athlete') (rc-core:19458) ne fait que PATCHer un statut : il ne crée ni code ni lien d'invitation. Le coach doit ensuite aller dans l'onglet codes (openAddAthlete) et ressaisir le prénom et le contact déjà connus (prospect.prenom, prospect.contact).

Pourquoi le mettre en place. On ne peut pas doubler un portefeuille sans savoir où partent les heures. Trainerize et Kahunas exposent déjà des indicateurs de charge coach. Côté prospect, la conversion est le moment où l'on perd le plus de monde : chaque ressaisie ajoute un délai.

Intérêt réel. Une mesure passive (temps d'écran sur la fiche, le bilan et la vidéo d'un athlète, agrégé par semaine) montre au coach ses 10 athlètes les plus coûteux. C'est la base d'une décision de tarif ou de format : 20 min par semaine pour un client à 99 €, ce n'est pas rentable. Pour le prospect, un bouton « Inviter » pré-rempli transforme la conversion en un geste (lien envoyé par WhatsApp ou e-mail en 10 s au lieu de 2 à 3 min) et relie la mesure « intéressés → athlètes » à de vrais comptes.

Prompt de mise en place.

```
Dépôt RepCore (/home/user/coaching), app/rc-core.1693.js.
```

1. Mesure du temps coach : module PUR chronoCoach. À l'ouverture de s-coach-client, s-coach-bilan-evo et de l'écran de correction vidéo pour un athlète (currentClientId), démarre un segment. Il se ferme au changement d'écran, au passage en arrière-plan (visibilitychange) ou après 3 min sans interaction (plafonner les segments). Agrège dans currentUser.chrono = {<semaineISO>: {<athleteKey>: secondes}}, en gardant 12 semaines au maximum, et écrit au plus une fois toutes les 2 minutes (saveUser réécrit le document entier : ne pas le faire à chaque seconde). Affichage : dans renderPortefeuille, « X min par athlète cette semaine (médiane) » et un volet « Les 10 dossiers qui te prennent le plus de temps », avec les minutes et la formule de l'athlète (status/offre).
2. Prospect vers athlète : dans renderProspects (≈19420), remplace « Devenu athlète » par « Inviter ». Le bouton ouvre le flux d'invitation existant (openAddAthlete / création de code, cherche la fonction qui crée studentCodes) pré-rempli avec prospect.prenom et prospect.contact (e-mail ou téléphone +33 déjà normalisé par cloudflare/src/prospects.js contactNet). Après création, propose l'envoi du lien par WhatsApp (waLink) ou mailto, puis passe le statut à 'athlete' avec codeId stocké dans le prospect. Garde un bouton discret « Marquer athlète sans inviter ».

Cas limites : plusieurs onglets ouverts (un seul segment actif), horloge qui recule, athlète supprimé, prospect sans contact exploitable.

Tests app/tests.js (chargerTests()) : l'agrégation plafonne un segment inactif à 3 min ; purge au-delà de 12 semaines ; la médiane est correcte ; le prospect invité passe en 'athlete' avec codeId. Convention de build : git show 388488d --stat. Critère de fin : le coach lit ses minutes par athlète et par semaine, et un prospect devient invité en un geste.

Agent 2 — Préparateur physique · 5/10

Le socle est ambitieux et mieux pensé que chez la plupart des concurrents. On y trouve les repères de volume MEV/MAV/MRV d'Israetel, un comptage fractionné des séries (1 pour le muscle principal, 0,5 pour le secondaire, pondéré par le RIR), la table RPE de Tuchscherer (RPE_PCT), un calibrage du RIR, une détection de plateau avec écartement des valeurs aberrantes, une décote après une coupure, et une décharge proposée à partir d'un score de fatigue. Le problème, c'est le cœur du moteur : la formule d'e1RM corrige le RIR dans le mauvais sens. Or cette formule alimente les records, la détection de plateau, le score de fatigue, la note de force et la mise à jour du 1RM. S'y ajoutent trois autres défauts : la charge suggérée ignore les répétitions et le RIR prescrits par le coach (elle pousse vers l'échec), le poids du corps est exclu partout, et les exercices à contrepois sont traités à l'envers dans les records.

1. La formule d'e1RM corrige le RIR à l'envers — Priorité : Critique · Effort : S

Constat. `e1rm()` (`app/rc-core.1693.js:50587-50591`) calcule $w * (1 + r/30) / (1 + rir * 0.025)$: plus il reste de répétitions en réserve, plus l'estimation BAISSÉ, alors qu'elle devrait monter (Epley sur les répétitions potentielles : $w * (1 + (r + rir) / 30)$). Valeurs mesurées pour 100 kg × 8 : RIR 0 → 126,7 (juste) ; RIR 2 → 120,6 au lieu de 133,3 ; RIR 4 → 115,2 au lieu de 140. Pour 100 kg × 5 à RIR 3 : 108,5 au lieu de 126,7 (−14 %). Même inversion dans `recordAPorteeExo` (:82919) et dans `calcSug` (:65511, code mort qui multiplie au lieu de diviser : $100 \times 8 @ 0 \rightarrow \ll 8 @ 0 \gg$ donne 120 kg au lieu de 100). Deux cas limites s'y ajoutent. Une série à 1 répétition renvoie 103,3 % de la charge (défaut connu d'Epley). Une série à 0 répétition compte comme une performance dans `perfExercice` (:50613, aucune garde `r > 0`) : un essai raté à 150 kg devient le meilleur score.

Reproduire : créer un exercice avec 4 séances sur 21 jours ou plus. Première séance : 100 kg × 8 à RIR 0. Les trois suivantes : 100 kg × 8 à RIR 3. `etatExercice` renvoie « regression », car 116,9 est inférieur à 0,95 × 126,7. `scoreFatigue` (:13108) ajoute alors le motif « Recul », ce qui peut déclencher une proposition de décharge, alors que l'athlète est au contraire plus fort.

Pourquoi le mettre en place. Cette fonction est la seule source de l'e1RM. Elle alimente les records (`recordsExercice` :82793, `e1rmRecordsDeSeance` :60849), le plateau (`_calculEtat`), la fatigue, `maxE1rmObserve` (:66460, qui recale le 1RM du coach), `noteAthlete` (:80007), le bilan de bloc (:34523) et le récapitulatif coach (:23928, :63333). Aujourd'hui, elle récompense l'échec et pénalise une programmation à RIR 2-3, que la littérature recommande pourtant (Refalo 2023, Robinson 2024).

Intérêt réel. Le coach qui programme en RIR 2-3 (accumulation, décharge à RIR +1) voit aujourd'hui de faux reculs et de faux plateaux. Les propositions de décharge se déclenchent à tort. Une décharge inutile coûte une semaine de progression sur un bloc de 6 à 8 semaines, soit 12 à 16 % du bloc. Les records d'e1RM, la carte de force et le recalage du 1RM (qui ne se déclenche qu'au-delà de +10 %) sont sous-estimés de 5 à 20 % dès que le RIR est renseigné, donc presque jamais déclenchés. C'est aussi une question de crédibilité : un coach STAPS ou CSCS qui compare avec une calculatrice Epley ou RTS verra l'écart immédiatement. Corriger demande une ligne, plus une migration invisible, puisque tout est recalculé à la lecture.

Prompt de mise en place.

```
Dans /home/user/coaching/app/rc-core.1693.js, corrige la fonction e1rm (vers la ligne 50587). Aujourd'hui elle rend  $w * (1 + r/30) / (1 + rir * 0.025)$ , ce qui fait BAISSER l'e1RM quand le RIR augmente. Remplace-la par le modèle Epley sur répétitions potentielles :
- w = parseFloat(poids), r = parseFloat(reps), i = max(0, parseFloat(rir) || 0) ; si w <= 0 ou r <= 0 → return 0 (une série à 0 rep n'est pas une mesure) ;
- n = r + i (répétitions potentielles) ; si n <= 1 → return w (pas d'inflation de 3,3 % sur un single) ;
- sinon return  $w * (1 + n/30)$ .
Ajoute une fonction inverse unique chargePourReps(e1rm, reps, rir) = e1rm / (1 + (reps + rir) / 30) (et =e1rm si reps + rir <= 1), et utilise-la partout où la conversion inverse est codée à la main : recordAPorteeExo (ligne ~82919, `tr.projection*(1+rirRef*0.025)/(1+r/30)` → chargePourReps(tr.projection, r, rirRef)). Supprime calcSug (ligne ~65511, sans appelant, formule fausse) et adapte tests.js qui la cite (~ligne 20899).
Dans perfExercice (~50613), ignore les séries dont _perfReps(s) <= 0 (comme recordsExercice le fait déjà), pour qu'un essai raté ne devienne pas le meilleur score.
Vérifie tous les appelants de e1rm( grep ) : _riteRecords (~23928), bilanBloc (~34523), e1rmRecordsDeSeance (~60856), 63333, maxE1rmObserve (~66484), noteAthlete (~80020), _rapE1rm, recordsExercice : aucun ne doit re-appliquer un facteur RIR. Mets à jour les commentaires de _perfRir (~50599) : un RIR vide compté 0 SOUS-estime désormais l'e1RM (et non plus le sur-estime) ; garde l'exclusion sansRirDominant.
Tests à ajouter dans app/tests.js (lancés via chargerTests() dans la console) : e1rm(100,8,0)=126.67 ; e1rm(100,8,2)=133.33 ; e1rm(100,5,3)=126.67 ; e1rm(100,1,0)=100 ; e1rm(100,0,0)=0 ; monotonie : e1rm(100,8,3)>e1rm(100,8,0) ; chargePourReps(e1rm(80,10,2),10,2)=80 ; scénario : 4 séances sur 28 jours, 1re 100x8@0 puis 3x 100x8@3 → etatExercice n'est PAS 'regression'. Mets à jour les assertions existantes qui figent l'ancienne valeur (tests.js ~20893-20906, ~21086, ~73411).
Critère de fin : toute la suite chargerTests() est verte, et grep "0.025" dans rc-core ne renvoie plus de conversion e1RM.
```

2. La charge suggérée ignore la prescription et pousse vers l'échec — Priorité : Critique · Effort : M

Constat. `chargeSuivante`(charge, rir, contrepois, decote) (:65683) multiplie la dernière charge par un facteur qui dépend uniquement du RIR constaté (`SUG.MULTIPLICATEURS` :65529 : RIR 3 → ×1,075, RIR 5 → ×1,125, RIR 0 → ×1). Les répétitions faites, les répétitions prescrites et le RIR prescrit (`_rirPrescrit` :64319, le RIR +1 des décharges, les écarts de bloc) n'entrent jamais dans le calcul. Le seul point d'équilibre est donc l'échec (RIR 0). Le même calcul est appliqué d'une série à l'autre (:49066-49085) : une consigne « 4 × 10 à RIR 3 » réussie à 100 kg donne 107,5 → 115 → 123,75 kg. Autre exemple : la dernière série est faite à 60 kg × 4 répétitions au lieu de 10, à RIR 2 ; l'app propose 63,75 kg. Enfin, `getPrevPerf` (:65703) ne saute pas les séances de décharge : après une décharge à 100 kg et RIR 3, la semaine suivante propose 107,5 kg.

Pourquoi le mettre en place. La suggestion de charge est la fonction que l'athlète utilise à chaque série. Aujourd'hui, elle contredit la consigne du coach dès que celui-ci prescrit un RIR supérieur à 0, une fourchette de répétitions ou une ondulation (DUP). La norme en musculation est la double progression (monter les répétitions dans la fourchette, puis la charge) calée sur un RIR cible, ce que font RP Hypertrophy et Hevy.

Intérêt réel. Pour l'athlète : moins d'échecs subis, moins de séries ratées, une charge juste dès la première série, et une progression régulière de 1 à 2,5 % par semaine au lieu de sauts de 7 à 12 % suivis de reculs. Pour le coach : ses RIR, ses décharges et ses écarts de bloc deviennent réellement exécutés, et plus seulement affichés. Pour le produit : « la bonne charge à chaque série » est l'argument central face à Trainerize et TrueCoach (qui n'ont pas d'autorégulation) et à RP Hypertrophy (qui a précisément ce moteur). Moins de séances à l'échec, c'est aussi moins de douleurs signalées et moins d'abandons ; c'est l'argument de rétention le plus solide qui soit.

Prompt de mise en place.

```
Contexte : /home/user/coaching/app/rc-core.1693.js. chargeSuivante (~65683) et multiplicateurRir (~65530) calculent la charge suivante uniquement à partir du RIR de la dernière série. Elles ignorent les reps réalisées, les reps prescrites (ex.reps, fourchetteReps ~38905) et le RIR prescrit (_rirPrescrit ~64319, qui intègre déjà les écarts de bloc et la décharge via getSemaineEffective).
Objectif : un moteur d'autorégulation fondé sur l'e1RM (la fonction e1rm corrigée : Epley sur reps+RIR, et chargePourReps comme inverse ; si elles n'existent pas encore, crée-les comme décrit : e1rm = w * (1 + (r + rir) / 30), chargePourReps(e, reps, rir) = e / (1 + (reps + rir) / 30)).
1. Crée une fonction PURE chargeCible({charge, repsFaites, rirFait, repsCibles, rirCible, contrepois, decote}) :
- e = e1rm(charge, repsFaites, rirFait) ; si RIR non renseigné, on prend rirFait = rirCible (hypothèse neutre) ;
- repsCibles = bas de fourchette si 'a-b' (double progression : si repsFaites >= haut de fourchette au RIR cible, viser le bas de fourchette à la charge supérieure) ; rirCible = _rirPrescrit(ex) ou 2 par défaut ;
- cible = chargePourReps(e, repsCibles, rirCible) × decote ;
- borne le saut à ±10 % de la charge faite (sécurité), et l'échec (rir 'echec') vaut rirFait=0 et repsFaites = reps réalisées ;
- contrepois : raisonner sur la charge nette (poids de corps – assistance) si poidsCorpsActuel(user) est connu ; sinon garder l'inversion actuelle ;
```

```

- arrondi via arrondiCharge125 (ou l'arrondi par matériel s'il existe).
2. Branche-la dans _blocExo (~48163, _rawSug) avec les reps/RIR de la dernière série de référence ET la consigne de l'exercice du jour, et dans le pré-remplissage intra-séance (~49066-49085) : la série suivante vise le RIR prescrit, pas le RIR 0 (si la série i est faite pile au RIR cible et aux reps cibles, la charge ne bouge pas).
3. getPrevPerf (~65703) : ignorer les séances sess.deLoad===true et celles d'une semaine semaineEstDecharge, pour qu'une décharge ne serve jamais de référence de progression.
4. Garde la décote de reprise, le facteur de cycle et le cas « programmation % 1RM » (s.rpeCible) inchangés.
5. Mets à jour le texte sous la suggestion (« Basée sur ... ») pour dire « 60 kg x 12 à RIR 2 → objectif 10 reps à RIR 2 ».
Tests (app/tests.js, via chargerTests()) : 60x12@2, cible 10@2 → 62,5 ; 100x10@3, cible 10@3 → 100 (stable) ; 60x4@2, cible 10@2 → baisse (≤ 50) ; 100x8@échec, cible 8@1 → < 100 ; séance de décharge ignorée par getPrevPerf ; intra-séance « 4x10 RIR 3 » réalisée à RIR 3 → charge constante ; contrepoids : assistance baisse quand RIR > cible. Adaptes les tests existants de SUG_MULTIPLICATEURS.
Critère de fin : suite verte, et aucune suggestion ne dépasse +10 % d'une séance à l'autre.

```

3. Poids du corps, élastiques et contrepoids : absents ou traités à l'envers — Priorité : Haute · Effort : M

Constat. serieEligible (:65891) exige weight>0. Une série de tractions, de dips, de pompes ou de nordic curls saisie sans charge compte donc 0 dans le volume hebdomadaire, dans le plateau, dans l'e1RM et dans la suggestion. La note de force (noteAthlete :80007) l'ignore aussi. isCounterweightEx (:65506) rate les noms mêmes du guide : TRACTIONS MACHINE ASSISTE et DIPS MACHINE GUIDEE renvoient false (vérifié), tout comme TRACTIONS ELASTIQUE. Pour ces exercices, un RIR élevé fait donc AUGMENTER l'assistance. Enfin, les records (recordsDeSeance :60816, recordsExercice :82793, recordAPorteeExo :82884) et le plateau ne tiennent jamais compte du contrepoids. Passer de 40 kg à 20 kg d'assistance est lu comme un recul, et remonter à 45 kg d'assistance déclenche « NOUVEAU RECORD ».

Pourquoi le mettre en place. Les tractions et les dips sont des mouvements de base, programmés dans le modèle Fondation lui-même (DIPS ASSISTE, :31467+). Un volume de dos ou de triceps qui les oublie place l'athlète « sous le MEV » à tort, et le coach ajoute alors des séries inutiles. Les applications concurrentes (Hevy, Strong) gèrent les types « poids du corps », « lesté » et « assisté » de façon native.

Intérêt réel. Pour un athlète en salle qui fait 4 x tractions et 3 x dips par semaine, 7 séries dures manquent au volume des dorsaux et des triceps, soit 30 à 50 % du repère MEV. Pour les athlètes à domicile (le profil « maison » de :42855), c'est presque tout le volume qui disparaît. Les records inversés sur les exercices assistés produisent de fausses célébrations et de faux reculs sur les exercices qui motivent le plus les débutants et les femmes (première traction). Corriger rend l'offre crédible pour le coaching à distance « poids du corps + élastiques », un segment où Everfit et Trainerize sont présents.

Prompt de mise en place.

```

Dans /home/user/coaching/app/rc-core.1693.js :
1. Crée typeCharge(ex) PURE → 'externe' | 'poids_corps' | 'leste' | 'assiste' | 'elastique', déduit de ex.typeCharge (champ optionnel posé par le coach) sinon du nom normalisé : 'assiste' si /\b(assiste|guide)e?s\b/ ET /\b(traction|dips)?\b/ (couvre TRACTIONS MACHINE ASSISTE, DIPS MACHINE GUIDEE) ; 'elastique' si /\belastique\b/ ET mouvement au poids du corps ; 'leste' si /\blest/ ; 'poids_corps' pour TRACTIONS, DIPS (hors machine chargée), POMPES, NORDIC, PISTOL, MUSCLE UP, PIKE/HANDSTAND PUSH UP, RELEVÉ DE GENOUX... (liste à partir d'EX_GUIDE_BRUT et EX_MATERIEL_MOTS 'AUCUN'). Fais de isCounterweightEx(name) un simple typeCharge({name})==='assiste' pour garder les appelants.
2. Crée chargeEffective(set, ex, user) : externe → weight ; poids_corps → poidsCorpsActuel(user) × coefficient (tractions/dips 1,0 ; pompes 0,65 ; pompes inclinées 0,5) ; leste → PDC + weight ; assiste → max(0, PDC - weight) ; élastique → PDC (sans soustraction faute de mesure). Sans poids de corps connu → null (on ne compte pas l'e1RM, mais on compte le VOLUME).
3. serieEligible (~65891) : une série done d'un exercice poids_corps/assiste/elastique est éligible même avec weight vide ou 0.
4. perfExercice, recordsExercice, recordsDeSeance, e1rmRecordsDeSeance, recordAPorteeExo, maxElrmObserve, noteAthlete, _carteSeries : utiliser chargeEffective ; pour 'assiste', un record = une assistance PLUS FAIBLE ; sans PDC, record de reps à assistance égale.
5. chargeSuiivante / suggestion : pour 'poids_corps' sans leste, suggérer des répétitions (+1 rep si RIR > cible) au lieu d'une charge.
Tests app/tests.js (chargerTests()) : isCounterweightEx('TRACTIONS MACHINE ASSISTE')===true, ('DIPS MACHINE GUIDEE')===true, ('DIPS MACHINE')===false ; 4 séries de TRACTIONS weight '' done → volume DORSAUX = 4 ; assistance 40-20 kg = progression, pas régression ; recordsDeSeance ne fête pas une assistance plus lourde ; PDC 80 kg, dips lestés +20 × 5 → e1RM basé sur 100 kg.
Critère de fin : suite verte, et l'écran Volume d'un athlète faisant tractions et dips sans charge montre ces séries.

```

4. Trois modèles charge-répétitions contradictoires, et des records incomplets — Priorité : Haute · Effort : M

Constat. Trois modèles donnent trois résultats pour la même série de 100 kg x 8 à RIR 2 : - e1rm() donne 120,6 ; - la calculatrice « Calculer ma charge » (updateCalcTable :116116, modèle linéaire 100-2.5*RIR-2.5*(reps-1)) donne 129,0, et elle extrapole jusqu'à 20 répétitions ; - la table RPE (RPE_PCT :66356, RPE 8 x 8 = 73,9 %) donne 135,3.

Le commentaire de :50583 reconnaît d'ailleurs l'incohérence. La ligne RPE 6 de la table n'est pas cohérente avec la ligne RPE 10 décalée de 4 (86,5 contre 86,3, 81,0 contre 81,1, 75,9 contre 76,2). Côté records, le « record de charge » est la charge la plus lourde tous volumes de répétitions confondus (recordsDeSeance :60816). Il n'existe pas de record par répétitions (meilleur 5RM, meilleur 10RM, meilleure série, meilleur volume), et aucune convention ne précise si une charge d'haltère est par main ou totale (seul le commentaire de :56381 en parle).

Pourquoi le mettre en place. Un athlète qui compare sa suggestion, la calculatrice et la carte de record voit trois chiffres différents pour le même e1RM, ce qui ruine la confiance dans l'outil. La référence d'un préparateur est un modèle unique, qu'il s'agisse d'Epley/RTS ou d'une table RIR à la Helms/Zourdos. Les records par nombre de répétitions sont standard chez Hevy et Strong.

Intérêt réel. Il n'y aurait plus de ticket « la calculatrice dit 130, l'app dit 120 », ce qui supprime une source récurrente de support pour le coach. Les records par répétitions multiplient les occasions de célébration légitimes : un athlète en hypertrophie (8 à 15 répétitions) bat un 10RM bien plus souvent qu'une charge maximale absolue, et c'est de l'engagement gratuit pour la gamification existante. La convention « par main » rend comparables les records et la note de force sur les exercices aux haltères (la moitié du catalogue).

Prompt de mise en place.

```

Dans /home/user/coaching/app/rc-core.1693.js, unifie le modèle charge/répétitions :
1. Une seule paire e1rm()/chargePourReps() (Epley sur reps+RIR, ≤ 12 reps potentielles fiables ; au-delà, afficher « estimation peu fiable »). Si elles n'existent pas encore sous cette forme, crée-les.
2. updateCalcTable (~116116) : remplace 'pct=100-2.5*prir-2.5*(pr-1)' et 'p2' par e1rm(pw,pr,prir) et chargePourReps(e,r,rir). Limite repsRange à [3..15] et grise les cases > 12 reps potentielles.
3. RPE_PCT (~66356) : recalcule la ligne '6' comme la ligne '10' décalée de 4 répétitions (mêmes valeurs que la table RTS, prolongation cohérente au-delà), et ajoute un test de cohérence : pour tout RPE x et reps n, RPE_PCT[x][n] ≈ RPE_PCT['10'][(n-1)-(10-x)] quand l'indice existe.
4. Records : ajoute recordsParReps(user,nomEx) → meilleur poids pour 1,3,5,8,10,12 reps (séries done, hors décharge, hors aberrants), et fais que recordsDeSeance signale aussi un record de reps (même charge, plus de reps) et un record de répétitions-max à une charge donnée. Garde l'affichage existant ; ajoute le type de record dans l'objet rendu (type:'charge'|'reps'|'e1rm').
5. Convention haltères : ajoute ex.chargeParMain (défaut true pour HALTERE(S) dans le nom, via materielExercice) et affiche « kg/main » dans la saisie, la suggestion et les records.
Tests app/tests.js (chargerTests()) : calculatrice et e1rm donnent la même valeur pour 100x8@2 ; cohérence RPE_PCT ; 80x10 puis 80x11 → record de reps ; haltères affichent « /main ».
Critère de fin : suite verte, grep « 2.5*prir » ne renvoie plus rien.

```

5. Cartographie musculaire : erreurs qui faussent le volume hebdomadaire — Priorité : Haute · Effort : S

Constat. Dans `EX_GUIDE_BRUT` (:31802+), la première clé de chaque signature est le muscle principal et les suivantes sont secondaires, à 0,5 (`_exGuide` :32509). On y trouve les erreurs suivantes : - Les écartés, BUTTERFLY et CROSSOVER créditent les TRICEPS : coude fixe, aucun travail. - Les ÉLEVATIONS FRONTALES créditent les TRICEPS. - Tous les SQUATS et PRESSES créditent les ISCHIOS. Kubo 2019 ne montre aucune hypertrophie des ischios au squat, car ils ne changent quasiment pas de longueur. - DIPS, DIPS SUR BARRE et DIPS LESTE ne créditent que les triceps, sans les pectoraux. - SIDE TRICEPS, qui est une pose de posing, est rangée en QUADRICEPS/FESSIERS/ISCHIOS et n'apparaît pas dans `EX_GUIDE_POSING` (:31904). - JUMPING JACK, MONTE DE GENOUX, POWER RUN et GAINAGE CHAISE comptent comme du volume de quadriceps.

Enfin, les repères `REPERES_VOLUME` (:31677) viennent de RP, qui compte des séries DIRECTES, alors que l'app y ajoute 0,5 par série indirecte. Exemple : 16 séries de dos + 6 curls donnent 14 « séries biceps » (zone de progrès), alors que RP lirait 6, soit sous le MEV de 8.

Pourquoi le mettre en place. Tout l'écran Volume, la priorisation de bloc (`blocArbitrage`), le signal « au-dessus du MRV » et la revue morpho des déséquilibres (`MORPHO_COUPLES` :14340, seuil 2,0) reposent sur cette table. Une erreur de classification se propage ainsi sans bruit dans chaque décision de volume.

Intérêt réel. Sur un programme type à 3 séances de jambes, les ischios sont surestimés de 4 à 6 séries par semaine. Le coach croit être dans le MAV alors que l'athlète est sous le MEV, précisément sur le muscle le plus souvent négligé chez les pratiquants et le plus exposé aux blessures (nordic, SDT roumain). Les triceps sont gonflés de 3 à 5 séries par les écartés et les élévations frontales. Corriger une table de données est rapide (effort S) et améliore immédiatement la justesse perçue par un coach expert, le public payant de l'offre coach.

Prompt de mise en place.

```
Dans /home/user/coaching/app/rc-core.1693.js, table EX_GUIDE_BRUT (~31802) :
1. Déplace BUTTERFLY, BUTTERFLY UNILATERAL, CHEST CROSSOVER DUAL, tous les ECARTE* (hors DEVELOPPE) de 'PECTORAUX,DELT_ANT,TRICEPS' vers une signature 'PECTORAUX,DELT_ANT'.
2. Sors ELEVATION FRONTALE* de 'DELT_ANT,TRICEPS' vers 'DELT_ANT'.
3. Squats et presses ('QUADRICEPS,FESSIERS,ISCHIOS') → 'QUADRICEPS,FESSIERS' (sauf SOULEVE DE TERRE*, GOOD MORNING, qui gardent ISCHIOS) ; retire JUMPING JACK, MONTE DE GENOUX, POWER RUN, QUAD STOMP de cette signature et range-les dans EX_GUIDE_CARDIO ; GAINAGE CHAISE → 'QUADRICEPS' avec technique isométrie (poids 0,5 déjà prévu par POIDS_TECHNIQUE).
4. DIPS, DIPS SUR BARRE, DIPS LESTE, DIPS AUX ANNEAUX, DIPS ASSISSE, DIPS ELASTIQUE → 'TRICEPS,PECTORAUX,DELT_ANT'.
5. SIDE TRICEPS : retire-la de la signature jambes et de la liste de patron 'isolation-coude' (~72233) et ajoute-la à EX_GUIDE_POSING (~31904).
6. Ajoute un réglage coach « Compter le travail indirect » (user.reperesComptage = 'fractionne' par défaut | 'direct') lu par serieDure : en mode 'direct', POIDS_ROLE.SECONDAIRE vaut 0, pour comparer honnêtement aux repères RP ; affiche sur l'écran Volume « repères RP en séries directes ; tu comptes le travail indirect à 0,5 » quand le mode est fractionné.
7. Les athlètes ont un cache exMuscles : vérifie que _autoArevoir (R15) re-résout les entrées auto modifiées (sinon ajoute leurs clés à la liste de revue).
Tests app/tests.js (chargerTests()) : resoudreMusclesLecture('ECARTE POULIE HAUTE').s ne contient pas TRICEPS ; ('SQUAT').s ne contient pas ISCHIOS ; ('DIPS').s contient PECTORAUX ; ('SIDE TRICEPS') == VOL_CARDIO ; mode 'direct' → 16 séries de rowing donnent 0 biceps.
Critère de fin : suite verte.
```

6. Arrondis et unités : pas de livres, pas de pas propre au matériel — Priorité : Moyenne · Effort : M

Constat. Tout est en kilos, avec un pas fixe de 1,25 kg sous 20 kg et 2,5 kg au-delà (`arrondiCharge125` :65549, `roundWeight` :65503, `_arrondirCharge` :66391, `arrondiAuPas` :82846). On compte quatre fonctions d'arrondi aux règles différentes : `roundWeight` arrondi à l'unité supérieure sous 20 kg et au plus proche au-delà ; la charge ajustée pour le cycle (:48179) passe par `roundWeight`, et la suggestion brute par `arrondiCharge125`. Aucune préférence ne permet de passer en livres (aucune occurrence de lb ou 2,2046). La suggestion peut produire 63,75 kg sur des haltères (pas réel : 2 kg) ou sur une machine à plaques de 5 kg ; l'athlète ne peut pas l'appliquer.

Pourquoi le mettre en place. Une charge suggérée impossible à charger fait perdre au moteur sa crédibilité dès la première séance. La gestion des unités et du pas par matériel est la base chez Hevy et Strong, et elle est indispensable pour vendre à des coaches hors de France (Belgique, Suisse, Québec, où les salles en livres sont courantes).

Intérêt réel. Aujourd'hui, l'athlète arrondit lui-même, puis la série suivante repart de ce qu'il a saisi : le calcul décroche de la prescription. Avec un pas par matériel (barre 2,5 kg, haltères 2 kg, machine 5 kg, poulie réglable, microdisques 0,5 kg en option), 100 % des suggestions deviennent réalisables. Le passage en livres ouvre le marché nord-américain (Québec) et les salles équipées en livres, sans rien réécrire, puisque le stockage reste en kilos.

Prompt de mise en place.

```
Dans /home/user/coaching/app/rc-core.1693.js :
1. Crée une seule fonction arrondiCharge(kg, {ex, user, sens}) qui remplace roundWeight (~65503), arrondiCharge125 (~65549), _arrondirCharge (~66391) et arrondiAuPas (~82846) (garde ces noms comme enveloppes pour les appelants et les tests). Pas = user.pasMateriel[materiel] sinon défauts : BARRE/SMITH 2,5 (1,25 sous 20), HALTERES 2, MACHINE/PRESSE/HACK 5, POULIE 2,5, AUCUN 1 ; matériel via materielExercice(ex.name) ; 'sens' = 'haut'|'bas'|'proche' (même logique que arrondiCharge125 : dans le sens du changement).
2. Préférence user.unite = 'kg'|'lb'. Stockage TOUJOURS en kg. Affichage/saisie convertis (1 lb = 0,45359237 kg) ; en lb le pas par défaut est 5 lb barre, 5 lb haltères, 10 lb machine. Ajoute le sélecteur dans les réglages athlète (ouvrirReglagesAthlete ~2403) et convertis dans la saisie des séries, la suggestion, les séries d'approche, la calculatrice, les records.
3. La charge ajustée du cycle (~48179) passe par arrondiCharge, sens 'bas'.
Tests app/tests.js (chargerTests()) : arrondiCharge(63.75,{ex:{name:'DEVELOPPE COUCHE HALTERE'},sens:'haut'}) == 64 ; machine 57 kg sens haut → 60 ; conversion 100 kg → 220,5 lb aller-retour sans dérive après 10 sauvegardes ; unite lb → suggestion affichée multiple de 5 lb.
Critère de fin : suite verte, plus aucun appel direct à Math.round(x/2.5)*2.5 hors arrondiCharge.
```

7. Séries d'approche avec les répétitions de la série de travail — Priorité : Moyenne · Effort : S

Constat. `seriesApproche` (:65588) donne à chaque palier « les répétitions PRESCRITES, à l'identique ». Pour une série de travail de 100 kg × 12 (plus de 100 kg, donc 4 paliers : 40/60/75/90 %), l'échauffement affiché est « 40 · 60 · 75 · 90 kg × 12 ». Cela fait 48 répétitions d'approche, dont 12 à 90 % de la charge de travail, ce qui équivaut à une série quasi effective (RIR 3 ou 4 environ) avant la première série comptée.

Pourquoi le mettre en place. La pratique standard, décrite par Helms et Israetel ainsi que dans les protocoles NSCA, consiste à diminuer les répétitions à mesure que la charge monte (par exemple 8, 5, 3, 1). Le but est de préparer le système nerveux sans créer de fatigue. Des approches à répétitions pleines réduisent la performance des séries de travail, donc le stimulus.

Intérêt réel. Les premières séries de travail sont meilleures, de l'ordre de 1 à 2 répétitions en plus sur la série 1 d'un exercice lourd. La séance est aussi plus courte : environ 30 répétitions d'approche en moins par exercice de base, soit 2 à 4 minutes gagnées par séance. Le RIR déclaré est plus juste, ce qui améliore l'eIRM et la suggestion. Il n'y a pas d'impact sur les données, puisque ce bloc est « affichage seul », et la correction est sans risque.

Prompt de mise en place.

```
Dans /home/user/coaching/app/rc-core.1693.js, seriesApproche (~65588) : garde les pourcentages MONTEE_POURCENTS, mais remplace « reps identiques » par des reps dégradées calculées depuis la rep prescrite R (bas de fourchette via fourchetteReps / premier nombre) : pct ≤ 0,5 → min(R,10) ; ≤ 0,65 → min(R,6) ; ≤ 0,8 → min(R,4) ; > 0,8 → min(R,2) (jamais 0, jamais plus que R). Chaque palier porte donc ses propres reps ; _htmlMonteeCharge (~65606) sait déjà afficher la
```

```
forme longue quand les reps diffèrent. Mets à jour le commentaire d'en-tête.  
Tests app/tests.js (chargerTests()) : seriesApproche(100,'12',ex) → reps [10,6,4,2] ; seriesApproche(15,'8',ex) → 1 palier à 8 reps max ; exercices  
isométrie/cardio/contrepois → [] (inchangé). Adapte les tests existants de seriesApproche qui figent « reps identiques ».  
Critère de fin : suite verte.
```

8. Cycle menstruel et maintien : deux réglages à réaligner sur la littérature — Priorité : Basse · Effort : S

Constat. `getCycleFactor` (:43861) retire automatiquement 5 % à la charge suggérée en phase menstruelle déduite du calendrier, même si l'athlète n'a rien déclaré (`phase==='menstrual' → 0,95`). Cette phase désactive aussi « record à portée » (`recordAPortee`, garde `factor<1`). De son côté, `blocArbitrage` (:68380) ramène un muscle « en maintien » exactement au MEV, avec la justification que « sous le MEV on perd du tissu ».

Pourquoi le mettre en place. Les méta-analyses (McNulty 2020, Colenso-Semple 2023) concluent à des effets du cycle sur la performance « triviaux » et variables d'une personne à l'autre. Elles déconseillent un ajustement systématique fondé sur le calendrier ; seul le ressenti déclaré devrait moduler la charge. Pour le maintien, le volume de maintien d'Israetel est inférieur au MEV : environ un tiers du volume d'entraînement suffit à conserver la masse (Bickel 2011).

Intérêt réel. Une athlète qui ne se sent pas diminuée ne perd plus 5 % de charge et ses opportunités de record plusieurs jours par mois. Sur un an, cela représente environ 60 séances sous-chargées évitées. Le maintien ramené de 8 à environ 4 à 6 séries libère 2 à 4 séries par muscle en maintien, à redistribuer sur les muscles prioritaires du bloc, sans dépasser le plafond de +10 % (`blocPlafond`). C'est une priorisation plus efficace, avec la même fatigue.

Prompt de mise en place.

```
Dans /home/user/coaching/app/rc-core.1693.js :  
1. getCycleFactor (~43861) : quand rien n'est déclaré (currentCycle absent('ignore')), la phase calculée ne modifie PLUS la charge (factor 1) sauf si  
confCycle(u).ajusterAuto===true (nouvelle option, défaut false, réglable dans le paramétrage du cycle) ; les options déclarées j1_difficile (0,80) et  
j1_supportable (0,95) restent. Le libellé de phase reste affiché. Conséquence : recordAPortee n'est plus bloqué par une phase non déclarée.  
2. Ajoute un champ mv (volume de maintien) à REPERES_VOLUME (~31677) : mv = max(0, round(mev/2)) par défaut, surchargeable comme les autres via  
reperesEffectifs. blocArbitrage (~68380) : rôle 'bas' → cible = rep.mv (et non rep.mev) ; corrige le commentaire « sous le MEV on perd du tissu » en « sous  
le volume de maintien ».  
Tests app/tests.js (chargerTests()) : cycle configuré, phase menstruelle, rien déclaré → getCycleFactor().factor===1 ; ajusterAuto:true → 0,95 ; j1_difficile  
→ 0,80 ; muscle en maintien PECTORAUX → cible 4. Adapte les tests existants qui figent 0,95 calendaire. Garde assertNoCycleRuleAltersSeries vert.  
Critère de fin : suite verte.
```

Agent 3 — Diététicien-nutritionniste du sport · 6/10

Le module nutrition est riche et bien documenté : la chaîne du tableau (métabolisme de base, puis NAF, puis sport en kcal, puis coefficient d'objectif) est transparente et ses hypothèses sont affichées. Katch-McArdle prend la main dès qu'une masse maigre est mesurée. La table CIQUAL embarquée (3 484 aliments) est cohérente avec les valeurs officielles vérifiées, le scan Open Food Facts convertit correctement les kJ (/4.184) et l'OCR des étiquettes contrôle la cohérence 4/4/9. En revanche, **tous les garde-fous caloriques ont été neutralisés** (plancher non appliqué, bouton -20 illimité, sèche posée d'un clic sans vérification TCA). L'objectif « Sèche » choisi par l'athlète autonome ne produit aucun déficit. Le moteur applique Harris-Benedict 1918 et des g/kg calculés sur le poids total, ce qui donne des cibles aberrantes chez la personne en obésité (288 g de protéines et 8 g de glucides).

Recalcul à la main, avec les fonctions réelles du dépôt (extraites de `app/rc-core.1693.js` et exécutées sous Node) :

Profil	MB Harris (code)	MB Mifflin	Dépense retenue	Cible	P / L / G (g)	Constat
H 80 kg, 180 cm, 30 ans, bureau (NAF 1,2), 4 h de muscu, sèche ×0,85	1 858	1 780	2 430	2 066 (2 068 après arrondi)	192 / 64 / 181	correct ; +78 kcal dus à Harris ; P = 37 % des kcal
idem avec NAF « Actif » 1,6	1 858	1 780	3 173	2 697	192 / 64 / 338	+743 kcal si l'athlète se déclare « actif » en pensant à son sport
F 60 kg, 165 cm, 25 ans, NAF 1,2, 3 h de muscu, prise de masse ×1,10	1 411	1 345	1 843	2 027	132 / 48 / 267	surplus de 184 kcal, raisonnable ; 350 kcal/h de muscu fixes, surestimées pour 60 kg
H 120 kg, 175 cm, 40 ans, NAF 1,2, 3 h, sèche ×0,85	2 313	2 099	2 926	2 487	288 / 96 / 118	+214 kcal dus à Harris ; P = 46 % des kcal
idem, coefficient 0,70	2 313	2 099	2 926	2 048	288 / 96 / 8	cétogène par accident, aucun avertissement
F 50 kg, 155 cm, 25 ans, sans sport, sèche ×0,70	1 297	1 183	1 556	1 089	120 / 40 / 62	sous le plancher de 1 200 kcal, que plus rien n'applique
idem avec lipides à 1,5 g/kg	—	—	1 556	cible 1 089, servie 1 155	120 / 75 / 0	glucides à 0, et le total servi dépasse la cible sans le dire

1. Rétablir les garde-fous caloriques (plancher, -20 illimité, sèche en un clic) — Priorité : Critique · Effort : M

Constat. `_relevePlancher` renvoie la journée sans y toucher (`app/rc-core.1693.js:95602`) et `cibleTableur` écrit `const kcal=ajuste;` au lieu de `Math.max(ajuste,pl)` (`≈ 1.95915`) : le plancher est calculé mais n'est jamais appliqué. `athDelta` → `appliquerDeltaKcal` (86078, 86341) n'a ni borne ni contrôle aTCA. De même, `athObjectif('seche')` (86316) pose un coefficient de 0,85 sans confirmation ni contrôle TCA, alors que `utiliserBesoinsProposes` (`≈ 87020`) bloque le déficit sous TCA. Pour reproduire : chez un athlète sans coach qui a déclaré un antécédent de TCA, cliquer 50 fois sur « -20 ». Le total descend de 1 000 kcal, sans alerte.

Pourquoi le mettre en place. Une application qui laisse un utilisateur descendre à 800 kcal ou moins, sans réaction, s'expose à un risque de santé et de réputation. Le code sait déjà calculer le bon plancher (`plancherEffectif`, 30 kcal/kg de masse maigre en disponibilité énergétique, 1 200/1 500 kcal). Il ne l'applique simplement plus. Le coach doit pouvoir passer outre avec une confirmation tracée. L'athlète seul, non.

Intérêt réel. Trainerize et MyFitnessPal refusent tous deux de proposer moins de 1 200 kcal. RepCore, qui revendique le sérieux d'un coach diplômé, serait le seul à laisser un athlète (y compris avec un antécédent de TCA) se programmer une restriction sévère. Remettre le plancher côté athlète ne coûte rien au coach, qui garde sa dérogation tracée (`_plConfirme`). On retire un risque de signalement ou de litige, qui pèse beaucoup plus lourd que les 9,50 €/mois d'un abonnement Essentielle. Cela protège aussi Kevin : un athlète qui se blesse en sèche sous 1 000 kcal en suivant « ses » chiffres engage sa responsabilité. Enfin, la cohérence revient : le bouton -20, le choix de phase et le point de départ obéiront à la même règle.

Prompt de mise en place.

```
Contexte : RepCore, app/rc-core.1693.js (123 k lignes, utilise grep -n). Le plancher calorique est calculé par plancherEffectif()/plancherKcal() (~1.109993-110056) mais n'est plus appliqué depuis le 24/09/2026 :
- _relevePlancher(j.user) (~1.95602) renvoie j tel quel ;
- cibleTableur() (~1.95815) fait `const kcal=ajuste;` au lieu de le borner ;
- athDelta()/appliquerDeltaKcal() (~1.86078 et 86341) n'ont aucune borne et ne vérifient pas aTCA ;
- athObjectif('seche') (~1.86316) écrit un coefficient de déficit sans rcConfirm ni aTCA, alors que utiliserBesoinsProposes() (~1.87020) le fait.
Travail :
1. Côté ATHLÈTE uniquement (origine 'athlete', et chemin automatique ciblesAthlete/_athEcrireCibles/besoinsProposes) : borner le total servi à plancherKcal(u). Dans cibleTableur, ajoute une option opts.appliquerPlancher (true par défaut pour l'athlète, false quand le coach a confirmé via _plConfirme). Remonte la journée basse par les glucides, comme l'ancienne logique décrite en commentaire dans besoinsProposes (lift reporté sur la journée haute).
2. appliquerDeltaKcal(u,-1,'athlete') : refuser (toast explicatif) si le nouveau total passe sous plancherKcal(u), ou si aTCA(u) est vrai et que sens<0. Côté coach, garder le comportement actuel, mais afficher l'avertissement sousPlancher.
3. athObjectif('seche') : si aTCA(u) ou grossesseSuspend(u), refuser avec le même texte que utiliserBesoinsProposes. Sinon, demander rcConfirm en annonçant le déficit en kcal/jour.
4. Sans plancher lisible (poids/sexe inconnus), appliquer l'absolu 1 200 (F) / 1 500 (H).
Tests à ajouter dans app/tests.js (dans la suite nutrition existante, même style d'assertions) :
- une athlète de 50 kg, 155 cm, 25 ans, sans sport, coefficient 0,70 : ciblesAthlete(u).kcal >= 1200 ;
- 60 clics athDelta(-1) sur un athlète : le total ne passe jamais sous plancherKcal ;
- aTCA vrai : athDelta(-1) et athObjectif('seche') n'écrivent rien ;
- côté coach avec _plConfirme sur les mêmes chiffres : la valeur sous plancher est acceptée.
Lance la suite (scripts/verif/README.md : serveur local puis `node scripts/verif/suite.mjs <url>`, ou chargerTests() en console). Aucune régression. Respecte la convention de build du dépôt (nouveau numéro de build pour rc-core/rc-style, mise à jour d'index.html et sw.js, comme dans les commits « Lot N7/N8 »). Critère de fin : aucun chemin athlète ne peut produire des cibles sous plancherKcal(u), et la suite est verte.
```

2. L'objectif « Sèche » ou « Prise de masse » de l'athlète autonome n'a aucun effet — Priorité : Critique · Effort : S

Constat. `ciblesAthlete` passe `opts.coef=objCoefDefaut('seche')` (0,85) (`≈ 86233`). Or `cibleTableur` lit la phase dans `phaseCourante(u)` (`user.phase`, 108854), que `athObjectif` ne pose jamais. La phase vaut donc 'maintien', et `coefPourPhase(0.85,'maintien')` rend 1 (vérifié sous Node, et 1 aussi pour la masse à 1,10). Pour reproduire : un athlète sans coach choisit « Sèche ». La carte affiche un coefficient de 0,85 (`coef:per.objectif?objCoefDefaut(...)`), mais les kcal écrites correspondent à 100 % de la dépense, et les protéines restent à 1,8 g/kg au lieu de 2,4. `tableur.objectifAthlete` est écrit mais n'est lu nulle part.

Pourquoi le mettre en place. C'est la fonction première du module pour l'athlète autonome (offre Essentielle). Il choisit sèche, ne perd rien, et conclut que l'app ne marche pas. L'écran contredit le calcul.

Intérêt réel. L'athlète en autonomie est le cœur de l'offre payante à 9,50 €/mois. Une sèche sans déficit, c'est 0 g perdu en 4 semaines au lieu d'environ 2 kg (−360 kcal/j pour le profil H 80 kg). Cela se paie en désabonnements, au pire moment : le premier mois. La correction tient en quelques lignes, et elle remet aussi les protéines à 2,4 g/kg pour préserver la masse maigre. Les concurrents (MacroFactor, Trainerize) appliquent l'objectif choisi immédiatement. L'écran cessera d'afficher « 85 % » au-dessus d'un calcul fait à 100 %.

Prompt de mise en place.

```
Bug dans app/rc-core.1693.js : athObjectif(k) (~1.86316) écrit nutrition.perso.objectif et nutrition.tableur.coef, mais cibleTableur() (~1.95815) prend la phase dans phaseCourante(u), qui lit user.phase. Un athlète sans phase est donc en 'maintien', et coefPourPhase(0.85, 'maintien') renvoie 1.00 : aucun déficit ni surplus, et protSuggeree reste à 1,8 g/kg.
Correction :
1. Dans cibleTableur, si opts.objectif (ou nutrition.perso.objectif quand origine==='athlete' et qu'aucune phase coach n'existe) est l'une des clés d'ATH_OBJECTIFS, utiliser cette clé comme 'phase' pour coefPourPhase ET pour protSuggeree. ciblesAthlete doit passer opts.objectif=per.objectif.
2. Ne pas écraser une phase posée par le coach (user.phase existante) : dans ce cas, le choix athlète est ignoré, comme aujourd'hui avec _athVerrouille.
3. Supprimer tableau.objectifAthlete s'il reste sans lecteur, ou le lire au point 1.
4. Le coef renvoyé par ciblesAthlete doit être celui RÉELLEMENT appliqué (t.coef), pas objCoefDefaut(per.objectif).
Tests dans app/tests.js :
- athlète H 80 kg / 180 cm / 30 ans, sans user.phase, perso.objectif='seche' : ciblesAthlete(u).kcal = round(dépense*0,85) (±2 kcal) et protGkg === 2.4 ;
- 'masse' : *1,10 ; 'maintien' : *1,00 ;
- avec user.phase={type:'masse',debut:...} posée par le coach et perso.objectif='seche' : c'est la phase coach qui gagne.
Lance la suite (scripts/verif/suite.mjs ou chargerTests()). Bump de build selon la convention du dépôt.
Critère de fin : l'objectif choisi par l'athlète change réellement les kcal et le g/kg, et le coefficient affiché est celui du calcul.
```

3. Métabolisme de base : Harris-Benedict 1918 par défaut, et formule mal nommée à l'athlète — Priorité : Haute · Effort : S

Constat. MB_FORMULE='harris' (95630) applique Harris-Benedict dans sa version d'origine (95655). Écart mesuré par rapport à Mifflin-St Jeor : +78 kcal (H 80 kg), +66 (F 60 kg), **+214 (H 120 kg)**, +114 (F 50 kg). Mifflin est la formule que l'Academy of Nutrition and Dietetics recommande, et Harris surestime surtout chez la personne en obésité. En plus, la carte « Point de départ proposé » affiche b.source==='katch'?'Katch-McArdle':'Mifflin-St Jeor' (86990) : quand le calcul a été fait en Harris, l'athlète lit « Mifflin-St Jeor ».

Pourquoi le mettre en place. Le code lui-même reconnaît une surestimation de 5 à 15 % (commentaire 95623). Afficher le nom d'une formule qui n'a pas servi est une erreur factuelle, qu'un athlète ou un confrère diététicien relève tout de suite.

Intérêt réel. Sur un profil de 120 kg en sèche, 214 kcal de trop au métabolisme deviennent environ 260 kcal de trop au total une fois multipliées par le NAF. Cela fait environ 1,5 kg de perte en moins sur 6 semaines, et la sèche « ne marche pas ». Garder Harris comme option pour Kevin (son tableur) tout en passant Mifflin par défaut, ou au moins Mifflin quand l'IMC est ≥ 30, sert à la fois le coach et le public large. Corriger le libellé est trivial et fait disparaître une incohérence visible.

Prompt de mise en place.

```
Dans app/rc-core.1693.js :
1. Rendre MB_FORMULE réglable par dossier : nutrition.tableur.formuleMB ∈ {'harris','mifflin'}, avec un défaut global. Crée mbFormuleDe(u) : le réglage du coach, sinon 'mifflin' si IMC (poidsNutritionnel / taille²) ≥ 30, sinon la constante MB_FORMULE. Utilise mbFormuleDe(u) dans mbEstime (lui passer u), dans cibleTableur (~1.95837), dans besoinsProposes (~1.95964) et dans _depensePourPlafond (~1.109960), pour garder un seul chemin.
2. Ajouter un sélecteur « Formule du métabolisme » dans le tableau du coach (là où NAF/coef sont réglés, grep _tb0optsDe), avec l'écart en kcal entre les deux formules affiché à côté.
3. Corriger _htmlDepartAthlete (~1.86990) : le nom affiché doit venir de b.source ('katch'→Katch-McArdle, 'harris'→Harris-Benedict, 'mifflin'→Mifflin-St Jeor). Même vérification pour tout autre libellé construit sur mbSource (grep "Mifflin-St Jeor").
Tests dans app/tests.js :
- mbMifflin(80,180,30,'H')===1780 ; mbHarrisBenedict(80,180,30,'H')===1858 ; mbMifflin(60,165,25,'F')===1345 ; mbHarrisBenedict(120,175,40,'H')===2313 ;
- dossier de 120 kg / 175 cm sans réglage : cibleTableur(u).mbSource==='mifflin' ;
- un dossier en Harris : le HTML de _htmlDepartAthlete contient « Harris-Benedict » et pas « Mifflin ».
Lancer la suite et bumper le build.
Critère de fin : la formule est choisie de façon explicite, identique sur tous les chemins, et toujours nommée correctement.
```

4. Macros en g/kg de poids total : protéines démesurées et glucides à zéro chez la personne en obésité — Priorité : Haute · Effort : M

Constat. _repartition calcule p=Math.round(gParKg*poids) et l=lkg*poids sur le poids total (95571), avec 2,4 g/kg en sèche (PROT_DEFAUT, 95350). Pour un homme de 120 kg, cela donne 288 g de protéines (46 % des kcal) et 96 g de lipides. Avec un coefficient de 0,70, il reste 8 g de glucides. Aucune notion de poids ajusté, de masse maigre pour les macros, ni de glucides minimaux n'existe (grep « poids ajusté », « obes » : rien). Quand P + L dépassent la cible (lipides à 1,5 g/kg), les glucides sont ramenés à 0 et le total servi dépasse la cible (1 155 contre 1 089) sans alerte.

Pourquoi le mettre en place. Les recommandations (ISSN, EFSA) expriment les protéines en g/kg de masse maigre ou de poids ajusté dès que la masse grasse est élevée. 288 g/j n'est ni utile ni tenable. Et un plan à 8 g de glucides pour un pratiquant de musculation compromet la performance, sans qu'aucun écran ne le signale.

Intérêt réel. Les personnes en surpoids sont une part majeure de la clientèle de perte de poids d'un coach. Des cibles réalistes (environ 180 g de protéines, 140 g ou plus de glucides pour le profil de 120 kg) se suivent, et c'est l'adhérence qui fait le résultat, donc le réabonnement. On évite aussi les coûts inutiles en whey et un plan que le coach doit refaire à la main. Katch-McArdle et la masse maigre sont déjà calculés (masseMaigreDuBilan), la brique existe. Aucun concurrent grand public ne le fait bien : c'est un argument de différenciation pour Kevin.

Prompt de mise en place.

```
Dans app/rc-core.1693.js, fonction _repartition(kcal,poids,gParKg,lipGparKg) (~1.95571) et ses appelants cibleTableur (~1.95815) et besoinsProposes (~1.95935) :
1. Introduire poidsMacros(u) : si masseMaigreDuBilan(u) est connue ET que le % de gras dépasse 25 % (H) ou 32 % (F), protéines = gParKg × masse maigre × 1,15 (équivalent g/kg de poids « sec »). Sinon, si IMC ≥ 30 : poids ajusté = poids idéal (IMC 25 × taille²) + 0,25 × (poids - poids idéal). Sinon, le poids total. Les lipides suivent le même poids de référence, avec le plancher LIP_PLANCHER_G_KG appliqué au poids de référence.
2. Glucides minimaux : si le reste donne moins de 2 g/kg de poids de référence (ou moins de 100 g/j), renvoyer un drapeau 'glucidesBas:true', et l'afficher dans le tableau du coach et sur la carte de l'athlète (« glucides très bas : performance en séance compromise »).
3. Si 4P + 9L > kcal (glucides forcés à 0), renvoyer `depasse:kcalServi-kcal` et l'afficher : le total servi ne doit plus dépasser la cible en silence.
4. Afficher dans les hypothèses le poids de référence utilisé (« protéines sur 96 kg de poids ajusté »).
Tests dans app/tests.js :
- H 120 kg, 175 cm, sans tours de mesure, sèche à 2,4 : p <= 230 g et p >= 170 g ;
- H 80 kg, IMC 24,7 : p === 192 (inchangé) ;
- F 50 kg, lipides à 1,5 et coefficient 0,70 : r.depasse > 0 et le flag est rendu dans le HTML de la carte.
Lancer la suite et bumper le build.
```

Critère de fin : aucune cible protéique au-dessus de 2,4 g/kg de poids de référence, et tout cas où les glucides sont bas ou le total dépassé est dit à l'écran.

5. Journal : 143 aliments CIQUAL comptés 0 kcal, base vide après un échec réseau, pas de produits « sport » — Priorité : Moyenne · Effort : S

Constat. 143 fiches de `app/data/ciqual.json` n'ont pas de `k` (62 d'entre elles ont pourtant `P`, `G` et `L` complets, par exemple « Croque-monsieur, jambon, fromage et beurre, fait maison » : `p 14,3, l 18,4, k null`). L'entrée est créée avec `kcal:null` (91870, 94315) et additionnée en `(e.kcal||0)` (87640) : 200 g de croque-monsieur comptent donc 0 kcal. `_loadCiqual` (91001) pose `_ciqualDB=[]` dès le premier échec de fetch et ne réessaie plus pendant toute la session. Aucune whey, aucun skyr, aucune boisson isotonique ni aucun gel dans CIQUAL (grep : aucun résultat), donc sans réseau (OFF) l'athlète ne peut pas saisir sa whey. Les valeurs contrôlées sont justes : œuf cru 140 kcal / 12,8 g P, poulet filet cru 110 / 23,4, huile d'olive 899, lait demi-écrémé 47,7.

Pourquoi le mettre en place. Un journal qui sous-compte fausse l'observance, la « cible tenue », les volts et la décision du coach. La whey est l'aliment le plus saisi par la cible de RepCore.

Intérêt réel. Un repas de 450 kcal compté 0, c'est une journée « tenue » à tort et un coach qui baisse les calories d'un athlète qui mange en réalité plus. La correction Atwater (4/4/9, plus 7 pour l'alcool, plus 2 pour les fibres) est une ligne de code. Ajouter une vingtaine d'aliments « sport » génériques (whey, caséine, skyr, clear whey, gel, boisson isotonique, barre protéinée) met le journal au niveau de MyFitnessPal ou Yazio pour ce public, même hors ligne en salle, là où le réseau est souvent mauvais. Réessayer le chargement de CIQUAL évite une recherche vide, qui ressemble à une panne.

Prompt de mise en place.

```
Fichiers : app/rc-core.1693.js, app/data/ciqual.json, scripts/ciqual_convert.py.
1. Dans scripts/ciqual_convert.py : si l'énergie est absente mais que p, c et l sont présents, calculer k = 4p + 4c + 9l + 2f (f = fibres, 0 si absent), arrondi au dixième, et poser "k_calc":true. Régénérer ciqual.json. Documenter dans le script.
2. Dans rc-core, côté client (défense en profondeur) : dans _eqConstruireEntree (~l.91866), saveFoodEntry et l'entrée OCR/scan (~l.94314), si f.k==null mais que p, c et l sont présents, poser kcal = round((4p+4c+9l+2f)*r) et un marqueur kcalEstimee:true. Afficher « kcal estimées » dans la ligne du journal.
3. Ajouter à ciqual.json (via le script, avec un bloc d'aliments « RepCore génériques », ids négatifs, groupe 'produits pour sportifs') : whey concentrée, whey isolat, caséine, protéine végétale, clear whey, skyr nature, gel énergétique, boisson isotonique, barre protéinée, maltodextrine, crème de riz. Valeurs moyennes pour 100 g, source indiquée en commentaire.
4. _loadCiqual (~l.91001) : en cas d'échec, ne pas mémoriser [] définitivement. Garder un horodatage et réessayer au prochain appel après 30 s, puis afficher « base d'aliments indisponible hors ligne » au lieu d'une liste vide.
Tests dans app/tests.js :
- aliment {k:null,p:14.3,c:20,l:18.4,f:1} à 200 g : kcal > 0 et kcalEstimee===true ;
- chercher 'whey' dans la base locale : au moins 1 résultat ;
- _loadCiqual avec fetch simulé en échec puis en succès : le second appel renvoie la base.
Lancer la suite et bumper le build (sw.js recopie déjà Ciqual d'un cache à l'autre, vérifier que l'URL ne change pas).
Critère de fin : plus aucune entrée comptée 0 kcal quand ses macros sont connues, et la whey se trouve hors ligne.
```

6. Bande d'observance : la journée en cours est comptée comme un « écart » — Priorité : Moyenne · Effort : S

Constat. `html0bservanceCoach` appelle `observance14(..., fin)` avec `fin=localISODate(new Date())` (99518). `observance14` (99459) classe « écart » tout jour qui a au moins une saisie et qui n'est pas à $\pm 7\%$ (cibleTenue, 79142). Pour reproduire : l'athlète note son petit-déjeuner (500 kcal) à 8 h, le coach ouvre la fiche à 10 h. La case la plus récente est rouge, « -1 500 kcal ». Même effet pour une journée saisie à moitié : elle devient un écart, pas un jour incomplet.

Pourquoi le mettre en place. Le lot N8 promet une lecture de motif fiable (« les écarts tombent en fin de semaine »). Une case rouge fabriquée chaque jour fausse la phrase et pousse le coach à relancer à tort.

Intérêt réel. Le coach consulte la fiche surtout en journée, donc la dernière case est fautive presque à chaque fois. Un faux écart par jour, c'est 1 sur 14 dans la bande, soit environ 7 % de bruit, et le motif « répartis sur la semaine » masque le vrai motif du week-end. Une relance injustifiée (« tu n'as pas tenu aujourd'hui ») abîme la relation, et l'athlète arrête de saisir. Exclure le jour courant (ou le marquer « en cours ») et distinguer « journée incomplète » (moins de 50 % de la cible et un seul repas) rend l'outil fiable, ce que Trainerize n'offre pas.

Prompt de mise en place.

```
Dans app/rc-core.1693.js, observance14(log,cibles,finISO) (~l.99459) et html0bservanceCoach (~l.99511) :
1. Le jour === localISODate(new Date()) prend l'état 'en_cours' (case hachurée, légende « aujourd'hui »). Il n'entre ni dans tenus/ecarts ni dans les moyennes, et motifEcarts l'ignore.
2. Nouvel état 'incomplet' : jour saisi dont le total kcal est < 50 % de la cible ET qui ne porte qu'un seul repas distinct (entries[].repas). Il n'est pas compté comme écart. Il apparaît dans la légende (« saisie partielle »), et phraseObservance le mentionne quand il y en a.
3. Ajouter la classe CSS .obs-en_cours / .obs-incomplet dans le fichier rc-style (convention de build).
4. cibleTenueJour (~l.79155) et la série de l'assiette (lot N4) ne doivent pas changer de comportement : ne toucher qu'à la bande.
Tests dans app/tests.js :
- log avec 1 entrée de 500 kcal aujourd'hui, cible 2 000 : le dernier jour a etat==='en_cours', et ecarts===0 ;
- hier, 1 seul repas à 600/2 000 : 'incomplet' ;
- hier, 3 repas à 1 400/2 000 : 'ecart'.
Lancer la suite et bumper le build.
Critère de fin : la bande ne montre plus d'écart pour la journée en cours ni pour une saisie manifestement partielle.
```

7. Dépense sportive en kcal/h fixes, NAF hors sport élevé, cycle ON/OFF non pondéré — Priorité : Moyenne · Effort : M

Constat. `SPORTS_KCAL_H` (94920) donne des kcal/h absolues, identiques pour 50 kg et pour 120 kg (le commentaire l'assume : « Elles ne varient donc pas avec le poids »). Or la dépense suit le poids ($MET \times kg$). L'échelle `NAF_ECHELLE` hors sport va jusqu'à 1,6-1,7 (95678). Si l'athlète se déclare « Actif », la dépense passe de 2 430 à 3 173 kcal pour le profil H 80 kg, et beaucoup d'athlètes répondent en pensant à leur sport. Le cycle ON/OFF applique $\pm 15\%$ de glucides (`CYCLE_GLCU`, `_tbJournées` 97583) sans pondérer par le nombre de jours ON : avec 6 séances, la moyenne hebdomadaire est de +79 kcal/j au-dessus de la cible (calcul : $(6 \times 2\,176 + 1\,960)/7 = 2\,145$ contre 2 066).

Pourquoi le mettre en place. Ce sont trois biais systématiques qui s'additionnent, dans le même sens pour les petits gabarits très actifs : surestimation. Les `MET` du Compendium (Ainsworth 2011) sont la référence, simple à brancher.

Intérêt réel. Pour une femme de 55 kg, 5 h de muscu comptent 250 kcal/j avec la table, contre environ 170 avec des `MET` nets : +80 kcal/j, soit environ 1 kg de moins perdu par trimestre. Pondérer le cycle ON/OFF garantit que la moyenne de la semaine égale la cible, et donc que le déficit annoncé est réel. Le coach gagne en crédibilité quand les résultats suivent la prévision, et c'est ce qui justifie un coaching de 99 à 600 €.

Prompt de mise en place.

```
Dans app/rc-core.1693.js :
1. Remplacer SPORTS_KCAL_H par une table de MET par sport × intensité (Compendium 2011, par ex. Musculation 3,5/5,0/6,0 ; Course à pied 7,9/8,11/5...).
kcalHeureSport(sport,intensite,poids) = (MET - 1) × poids (MET nets, pour ne pas recompter le métabolisme déjà inclus dans le NAF). Garder un repli : sans
```

```
poids, utiliser 75 kg. Mettre à jour depenseSportsParJour, kcalSportParJour et _sportsCoach pour passer poidsNutritionnel(u).kg.
2. Contrôle de cohérence du NAF : si deb-naf ∈ {actif, très actif} ET que le métier reconnu est 'sedentaire', afficher une hypothèse d'alerte au coach (« niveau déclaré supérieur à celui du métier : vérifier qu'il ne compte pas le sport ») dans besoinsProposes et dans le tableau.
3. _tbJournees et le bloc cycle de besoinsProposes : calculer n0n = nombre de créneaux actifs (sessions_config), et n0ff = 7 - n0n. Choisir g0n =  $g \times (1+x)$  et  $g0ff = g \times (1 - x \times n0n / n0ff)$ , avec  $x = \text{CYCLE\_GLUC}$ , pour que  $(n0n \times g0n + n0ff \times g0ff) / 7 === g$  à l'arrondi près. Si n0n vaut 0 ou 7, pas de cycle.
Tests dans app/tests.js :
- kcalHeureSport('Musculatation', 'moderee', 55) < kcalHeureSport('Musculatation', 'moderee', 100) ;
- 6 créneaux : moyenne hebdo des kcal ON/OFF à ±5 kcal de t.kcal ;
- 3 créneaux : idem.
Adapter les assertions existantes qui figent les anciennes valeurs de SPORTS_KCAL_H (grep dans tests.js) en expliquant le changement. Lancer la suite et bumper le build.
Critère de fin : la dépense sportive dépend du poids, et la moyenne de la semaine égale la cible.
```

8. Hydratation : repère sur poids total et aucun suivi de ce qui est bu — Priorité : Basse · Effort : S

Constat. `besoinEau` = 35 ml/kg de poids total + 500 ml par jour de séance (61779). Pour 120 kg, cela donne 4,7 L/j, un chiffre excessif et difficile à tenir. Aucun journal d'eau n'existe : l'hydratation n'est qu'un curseur de ressenti de fin de séance (`hydratationMoyenne`, 61755). grep `waterLog|eauLog` : aucun résultat.

Pourquoi le mettre en place. Le repère affiché doit rester plausible pour tous les gabarits, et un compteur de verres est une fonction de base de toutes les apps concurrentes (Yazio, MyFitnessPal, Trainerize).

Intérêt réel. Un compteur « +250 ml » coûte peu à développer. C'est une interaction quotidienne de plus, qui fait revenir sur l'app et nourrit la série et les volts. Borne le repère (poids de référence, plafond de 4 L hors chaleur) évite un conseil que l'athlète juge absurde et qui décrédibilise le reste des chiffres. Côté coach, voir l'eau réellement bue lors d'une stagnation de la balance (rétention d'eau) aide à expliquer une semaine sans perte.

Prompt de mise en place.

```
Dans app/rc-core.1693.js :
1. besoinEau(user, jourEst0n) (~l.61779) : utiliser le poids de référence (poids ajusté si IMC ≥ 30, même fonction que le point 4 de l'audit si elle existe, sinon la créer localement : poids idéal  $IMC 25 + 0,25 \times \text{excédent}$ ), plafonner à 4,0 L (+0,5 L les jours ON), plancher à 1,5 L.
2. Ajouter nutrition.eau[dateISO] = millilitres. Sur l'écran nutrition, à côté de _htmlHydratationNut (~l.16740) : boutons +250 ml / +500 ml / annuler, et une barre de progression vers besoinEau. Côté coach (_htmlHydratationCoach) : moyenne des 7 derniers jours.
3. Déclarer le nouveau champ dans database.rules.json (même validation que les autres sous-nœuds de nutrition : nombre de 0 à 10 000) et dans app/globaux.d.ts si les types l'exigent.
Tests dans app/tests.js :
- besoinEau pour 120 kg / 175 cm, jour OFF,  $\leq 4,0$  ; pour 60 kg  $\approx 2,1$  ;
- deux ajouts de 250 ml puis une annulation : nutrition.eau[aujourd'hui]===250.
Lancer la suite, et vérifier les règles avec scripts/verif/regles.mjs. Bumper le build.
Critère de fin : le repère est plausible pour tous les gabarits et l'athlète peut noter ce qu'il boit.
```

Agent 4 — Athlète en salle et UX de séance · 5/10

Le socle de la séance est sérieux. La séance est sauvegardée en continu (`woPersist`, reprise via `woResumeAndGo`), le service worker a un délai de garde pour jouer hors ligne, les supersets s'affichent sur un seul écran (`_groupesEx`), le cadran de repos a ± 15 s et « Passer », la charge proposée monte selon le RIR, et on trouve un calculateur de disques et le suivi de la gêne par série. En revanche, le geste qui compte le plus (saisir puis valider une série, puis être prévenu de la fin du repos) coûte environ 3 fois plus d'interactions que dans Hevy ou Strong. La fin du repos reste muette dès que l'écran se verrouille. Le minuteur et le bouton « Suivant » sortent de l'écran. Enfin, deux défauts peuvent fausser les données : « Terminer » enregistre sans rien demander, et une virgule décimale donne 825 kg.

1. La fin du repos est muette dès que l'écran se verrouille — Priorité : Critique · Effort : M

Constat. Le minuteur ne repose que sur un `setInterval` de la page (`rc-core.1693.js:38050`, `_lancerTickRepos`). La vibration et le bip ne partent que dans `_peindreRepos` (`:38308`), donc seulement quand la page tourne au premier plan. Aucun `navigator.wakeLock` dans tout le front (grep vide dans `rc-core`, `index.html` et `sw.js`), et aucune notification locale n'est programmée pour la fin du repos. L'APK est une TWA Bubblewrap (`android/twa-manifest.json`), donc sans moteur natif. Autre manque : aucun repos après la dernière série d'un exercice, puisque `_reposApresSerie` rend `null` quand `i>=d.sets.length-1` (`:38885`).

Pourquoi le mettre en place. Un téléphone se verrouille en 30 s à 1 min, et un repos dure de 90 s à 3 min. Dans la vraie vie, l'athlète pose son téléphone sur le banc, l'écran s'éteint, et le signal de fin de repos ne lui parvient jamais. Hevy et Strong notifient la fin du repos sur l'écran verrouillé (et affichent une Live Activity sur iOS). C'est la fonction pour laquelle on garde une app de suivi ouverte en salle.

Intérêt réel. Sur une séance de 5 × 4, on compte environ 19 repos. Sans signal, chacun dérive en moyenne de 30 à 60 s (on rallume le téléphone pour vérifier, ou on oublie). Cela fait 10 à 19 minutes perdues par séance, soit 15 à 25 % d'une séance d'une heure. Le repos réel (`reposReel`, déjà mesuré dans `toggleSet`) s'écarte alors de la prescription, et le coach lit des données de densité faussées. Pour la rétention, un minuteur peu fiable pousse l'athlète à ouvrir en parallèle le chrono de son téléphone ou Hevy : c'est le premier pas vers l'abandon de l'app pour le suivi. Garder l'écran allumé (Wake Lock) règle le problème pour environ 90 % des cas Android et iOS 18.4+ en PWA installée, pour un coût serveur nul.

Prompt de mise en place.

```
Contexte : RepCore, PWA + APK TWA. Front dans app/rc-core.1693.js (123k lignes, utilise grep -n), app/index.html, app/sw.js. Le minuteur de repos :
demarrerRepos (~l.38017), annulerRepos, ajusterRepos, _lancerTickRepos, _peindreRepos (~l.38297), _reposApresSerie (~l.38880), toggleSet (~l.49604),
launchWorkout (~l.44823), woResumeAndGo, _quitterEcranSeance, finishWorkout.

Objectif : que la fin du repos soit perçue même écran éteint ou app en arrière-plan.

1. Wake Lock pendant la séance :
- Crée woVerrouEcran() / woLibererEcran(). Demande navigator.wakeLock.request('screen') au lancement (launchWorkout, startWorkout, woResumeAndGo) et le
relâche dans _quitterEcranSeance, finishWorkout, cancelWorkout et quitterSeanceSansEnregistrer.
- Réacquiesce-le sur visibilitychange quand !document.hidden et qu'une séance est en cours (le navigateur le relâche automatiquement en arrière-plan).
- try/catch partout : l'API peut être absente ou refusée, et rien ne doit casser.
- Réglage utilisateur currentUser.ecranAllume (vrai par défaut), commutable dans les réglages à côté du son de repos (_majSonReglages).

2. Notification de repos quand la page passe en arrière-plan :
- Sur visibilitychange → hidden, si woState.reposFin est dans le futur et que Notification.permission==='granted', appelle reg.showNotification('Repos en
cours', {body:'Fin à HH:MM:SS · <exercice>', tag:'rc-repos', notify:false, silent:true, data:{url:'./index.html#seance'}}).
- Au retour visible, ferme cette notification (reg.getNotifications({tag:'rc-repos'})).
- Si la page revient après l'échance, déclenche tout de suite arcHaptique('avertir') + bip (si son actif), une seule fois (drapeau reposVibre déjà
présent).
- N'utilise PAS de push serveur (le cron du Worker tourne à la minute, trop grossier).

3. Repos entre deux exercices : modifie _reposApresSerie pour lancer aussi le repos après la dernière série d'un exercice qui n'est pas le dernier groupe de
la séance (durée = repos de l'exercice), sauf si c'est la dernière série de la séance.

4. Tests dans app/tests.js (testExercices, près des assertions existantes sur demarrerRepos vers la l.57297) :
- _reposApresSerie rend une durée pour la dernière série d'un exercice non final, et null pour la dernière série de la séance.
- Avec un navigator.wakeLock simulé, launchWorkout appelle request une fois et finishWorkout relâche le verrou.
- Si wakeLock est absent, launchWorkout ne lève pas d'exception.
- Le handler visibilitychange n'appelle pas showNotification quand reposFin est null.

5. Suis la convention de build du dépôt (voir git log : numéro de build, nom rc-core.<build>.js, CACHE de sw.js).
Critère de fin : chargerTests() dans la console, 0 échec. Vérification manuelle sur Android : un repos de 90 s, écran non touché, l'écran reste allumé et
vibre à l'échance.
```

2. Le minuteur de repos et le bouton « Suivant » sortent de l'écran — Priorité : Haute · Effort : S

Constat. `.screen` n'a qu'un `min-height:100dvh` (`rc-style.1693.css:848`), pas de hauteur fixe. Le cadran `#wo-repos` et la barre de navigation sont dans le flux, sous la zone qui défile (`index.html:4377-4385`). Mesuré avec Playwright en 375×740 : - une fois le repos lancé, l'écran fait 892 px et le haut de « Suivant » est à 879 px, donc hors écran ; - sur un superset, après validation, le cadran commence à 979 px, entièrement invisible ; - en 375×667 (iPhone SE/8), « Suivant » déborde déjà avant tout repos (bas à 715 px).

Pourquoi le mettre en place. L'athlète valide sa série en bas du tableau et ne voit pas le décompte partir. Il doit faire défiler la page pour trouver le minuteur, puis encore pour « Suivant ». Dans Hevy ou Strong, la barre du minuteur et les actions restent fixées en bas d'écran, sous le pouce.

Intérêt réel. Chaque série validée ajoute 1 à 2 défilements, soit 20 à 40 gestes inutiles par séance, faits à une main, souvent avec de la magnésie ou de la sueur. Les supersets (un format très courant chez les coachs, déjà géré par `_groupesEx`) sont les plus touchés, puisque le minuteur y est totalement invisible. La correction est purement CSS/HTML, sans risque pour les données, et rend immédiatement lisible la fonction phare de l'écran de séance. C'est aussi la condition pour que le point 1 serve à quelque chose : un minuteur fiable mais hors écran ne se voit pas.

Prompt de mise en place.

```
Contexte : écran de séance #s-workout dans app/index.html (~l.4362-4390) : topbar, .scroll-area avec #wo-content, #wo-repos (cadran de repos monté par
_monterRepos dans app/rc-core.1693.js ~l.38220), puis la barre de navigation (wo-prev-btn, accueil, wo-next-btn, finishWorkoutEarly). CSS dans app/rc-
style.1693.css : .screen{min-height:100dvh} (l.848), .scroll-area{flex:1;overflow-y:auto} (l.6900).
```

Bug mesuré en 375×740 : pendant un repos, #s-workout fait 892 px et wo-next-btn est hors écran. En superset, #wo-repos commence à 979 px. En 375×667, la barre déborde même sans repos.

À faire :

```
1. #s-workout.active : height:100dvh (repli 100vh) et overflow:hidden. .scroll-area de cet écran : flex:1 1 auto; min-height:0, pour qu'elle seule défile. La
topbar, #wo-repos et la barre du bas en flex-shrink:0.
2. Crée une variante compacte de #wo-repos (hauteur ≤ 88 px : décompte, -15/+15, Passer, son sur une ligne), utilisée quand innerHeight < 800 ou en superset.
Le grand cadran reste disponible au toucher du décompte (bascule). Aucune assertion existante sur la structure (#rep-arc, #rep-num, #rep-lbl, #rep-son) ne
doit tomber : garde ces id.
```

```

3. Respecte env(safe-area-inset-bottom) sur la barre du bas.
4. Vérifie qu'aucun autre écran ne dépend de .screen{min-height} (limite la règle à #s-workout).
5. Tests dans app/tests.js (testExercices) : après launchWorkout sur une séance de 5 exercices et toggleSet(0,0), getBoundingClientRect de #wo-next-btn et de #wo-repos a bottom ≤ innerHeight. Même chose pour un superset (exercice 2 avec ss:true, toggleSet sur les deux) et pour une fenêtre de 375x667 si la suite permet de redimensionner, sinon via un conteneur simulé.
6. Suis la convention de build du dépôt (voir git log).
Critère de fin : chargerTests() à 0 échec. Captures Playwright (Chromium /opt/pw-browsers/chromium-1194, viewport 375x667 et 375x740) où le minuteur et « Suivant » sont visibles sans défilement pendant un repos.

```

3. Une série coûte 3 à 8 interactions au lieu d'une — Priorité : Haute · Effort : M

Constat. - La série 1 ne reçoit jamais la charge suggérée. `_blocExo` crée les séries avec `weight:''` (:48103), alors que la suggestion (ex. « 82.5kg, basée sur ton dernier lundi ») n'est qu'affichée (:48241). Constaté en Playwright à la 2e séance : les 4 charges sont vides. - Les séries suivantes ne se remplissent que si un RIR a été choisi (:49079). Sans RIR, la série 2 reste vide (vérifié). - Aucune colonne « Précédent » par série. `getPrevPerf` ne retient que la dernière série du même créneau (:65703). - Le RIR demande 2 taps (menu, puis option).

Pourquoi le mettre en place. Décompte pour 5 exercices × 4 séries, 2e séance : - RepCore avec RIR : série 1 = champ + 4 frappes + 2 taps RIR + valider = 8 ; séries 2 à 4 = 3 chacune ; plus « Suivant ». Soit 18 par exercice et environ **94 interactions** avec lancement et fin. - RepCore sans RIR : 6 par série, soit environ **129**. - Hevy/Strong (valeurs précédentes reprises au ✓) : 1 tap par série identique, soit environ **25 à 35** avec quelques modifications.

Intérêt réel. On passe de 94-129 à environ 35 interactions : 60 à 95 gestes de moins par séance, et environ 15 000 par an pour un athlète à 4 séances/semaine. Moins de frappes, c'est aussi moins d'erreurs de saisie (voir point 5) et plus de séances saisies en entier, donc des données plus justes pour la surcharge progressive et pour le coach. C'est l'argument n°1 des utilisateurs de Hevy/Strong. Sans cette parité, un athlète exigeant refuse de migrer, et le coach perd le suivi. Le RIR en rangée de pastilles (0 1 2 3 4 5 Échec) fait gagner 20 taps de plus par séance.

Prompt de mise en place.

```

Contexte : app/rc-core.1693.js. Création des séries dans _blocExo (~L.48100, sessionData[idx] avec weight:'). Suggestion calculée dans _blocExo (variables sug, _sugBrut, prev = getPrevPerf ~L.65703). Remplissage automatique dans renderSets (~L.49070-49085 : ne remplit la série i+1 que si s.rir est renseigné). Menu RIR : rirMenuOuvrir, RIR_CHOIX. Validation : toggleSet (~L.49604). Consigne coach : consigneProgEx (s.rpeCible, _cons.kg).

```

À faire :

1. Pré-remplissage de la série 1 : si `sug>0` et que la série n'est ni `userEdited` ni `done`, pose `weight=sug` et `isAuto=true` (même règle que la consigne coach). Ne touche pas aux exercices programmés dont le coach fixe kg (déjà gérés) ni aux dégressives (P1/P2).
2. Propagation sans RIR : si la série `i` est validée avec une charge et sans RIR, la série `i+1` non éditée reçoit la même charge (`isAuto`). Avec RIR, garde chargeSuivante.
3. Colonne ou sous-ligne « Préc. » : pour chaque série `i`, affiche « 80x8 » tiré de la même série `i` de la dernière séance du même créneau (nouvelle fonction `prevSeries(name,slot,progName) → tableau`). Un tap sur cette valeur la copie dans la série (charge + `repsDone` si fourchette). Reste sous 359 px de large (règle déjà testée) : mets-la sous le numéro de série plutôt qu'en 6e colonne si besoin.
4. Validation rapide : `toggleSet` sur une série sans charge mais avec une valeur proposée (`isAuto`) ou précédente la valide avec cette valeur. Sans aucune valeur, laisse le comportement actuel.
5. RIR en un tap : remplace le menu par une rangée de pastilles (Échec,0,1,2,3,4,5) qui s'ouvre sous la ligne au premier tap sur la case, et se referme au choix. Ou mieux : propose le RIR juste après le ✓ dans une bande non bloquante de 4 s. Les valeurs de `RIR_CHOIX` ne changent pas.
6. Tests dans `app/tests.js` (`testExercices`) :
 - avec un historique (`currentUser.sessions` contenant la séance précédente du même slot), la série 1 a `weight===sug` et `isAuto` ;
 - valider la série 1 sans RIR copie la charge sur la série 2 ;
 - `prevSeries` renvoie les valeurs par index ;
 - `toggleSet` sur une série `isAuto` garde la charge proposée ;
 - aucun pré-remplissage sur une série `userEdited`.
7. Suis la convention de build du dépôt.
Critère de fin : `chargerTests()` à 0 échec. Script Playwright (Chromium /opt/pw-browsers/chromium-1194, 375x740) qui joue une 2e séance de 5x4 identique à la 1re et compte ≤ 30 clics/frappes.

4. « Terminer la séance » enregistre sans rien demander et perd les séries non cochées — Priorité : Haute · Effort : S

Constat. Sur le dernier groupe, le gros bouton rouge (« Terminer la séance », `woNav(1)`) appelle directement `finishWorkout(false)` (:49747), sans confirmation. Reproduit : lancer une séance de 5 exercices, valider 1 série, appuyer 5 fois sur « Suivant ». La séance est enregistrée avec `complete:true`, `sets:1`, `setsPlanned:20`. Une série dont la charge est saisie mais sans ✓ n'est pas comptée (`if(!s.done) return`). Le compteur « 0/20 séries » ne se met pas à jour quand on valide une série (constaté : il reste à 0/20), car `woMajProgression` n'est appelé que dans `renderWoEx` (:48002).

Pourquoi le mettre en place. Le même bouton sert à « Suivant » quatre fois, puis à « Terminer » : un double tap ou un tap machinal clôt la séance. Oublier de cocher la dernière série est l'erreur n°1 en salle. Hevy demande « Finir avec des séries non validées ? » et propose de les valider ou de les jeter.

Intérêt réel. Une séance marquée complète à tort fausse l'observance que lit le coach, les séries prévues et réalisées, le tonnage, et la surcharge progressive de la séance suivante (la charge suggérée repart de la dernière série validée). Un clic de trop suffit, et on ne peut pas revenir en arrière (`woState.termine`, snapshot effacé). Une confirmation qui liste les séries saisies mais non cochées (« 3 séries notées non validées : les valider ? ») récupère des données déjà tapées et évite un ticket de support. Le compteur de séries juste redonne aussi à l'athlète un repère fiable de sa progression dans la séance.

Prompt de mise en place.

```

Contexte : app/rc-core.1693.js. woNav (~L.49740) appelle finishWorkout(false) quand nxt>=grs.length. finishWorkout (~L.49796) ne compte que s.done. finishWorkoutEarly (~L.44427) utilise déjà rcConfirm3. Compteurs : woMajProgression (~L.47845), #wo-set-count, #wo-ex-count, appelés seulement depuis renderWoEx.

```

À faire :

1. Nouvelle fonction pure `seriesEnSuspens(woState) → {nonValideesSaisies:[{idx,i}], restantes:n}`. Une série est « saisie » si `weight` ou `repsDone` est renseigné et que `done` vaut `false`.
2. Dans `woNav`, au lieu d'appeler `finishWorkout` directement :
 - s'il y a des séries saisies non validées : `rcConfirm3('Terminer la séance ?', 'N séries notées ne sont pas validées.', 'Les valider et terminer', 'Terminer sans elles', 'Continuer')`. « Valider » les passe en `done` (sans lancer de repos ni d'animation) puis `finishWorkout(false)`.
 - sinon, s'il reste des séries non faites : `rcConfirm('Terminer la séance ? X/Y séries faites')`. Envoie `complete:false` si moins de la moitié des séries est faite (réutilise `finishWorkout(true)`).
 - sinon, termine directement.
3. Anti double tap : ignore un second `woNav(1)` dans les 400 ms.
4. Mets à jour `#wo-set-count` (séries validées / prévues de la séance) à chaque `toggleSet`, `ajouterSerie` et `retirerSerie`, via une fonction dédiée, sans animer la barre d'exercice.
5. Tests dans `app/tests.js` (`testExercices`) :
 - `seriesEnSuspens` sur des cas construits ;
 - `woNav(1)` sur le dernier groupe avec une série saisie non validée n'appelle pas `finishWorkout` avant la réponse (`rcConfirm3` simulé) ;
 - « Continuer » laisse `woState.termine` faux ;
 - le compteur `#wo-set-count` passe de 0/20 à 1/20 après `toggleSet(0,0)`.

6. Suis la convention de build du dépôt.
Critère de fin : chargerTests() à 0 échec. Rejouer le scénario (5 appuis sur Suivant) doit afficher une confirmation.

5. Une virgule décimale transforme 82,5 kg en 825 kg — Priorité : Haute · Effort : S

Constat. Les champs de charge sont des `<input type="number" inputmode="decimal">` (renderSets, vers :49150). `_woChargeSaisie` fait `String(el.value).replace(',', '.')` (:48360), mais pour un `type=number`, `el.value` est déjà nettoyé par le navigateur. Reproduit en Playwright (Chromium, locale fr-FR) : taper « 82,5 » donne `value='825'`, stocké `weight:825`. Aucune borne haute ni aucun contrôle par rapport à l'historique : 825 kg est accepté, marqué « RECORD » et sert de base à la suggestion suivante.

Pourquoi le mettre en place. Les claviers français (Samsung, Gboard FR, iOS FR) affichent une virgule comme séparateur décimal. Les charges en ,25 et ,5 sont quotidiennes (haltères de 22,5 kg, microdisques). Le comportement exact varie selon l'appareil, et c'est justement ce qui rend le bug difficile à diagnostiquer pour le support.

Intérêt réel. Une seule erreur de ce type fausse la charge proposée de la séance suivante (`chargeSuivante` part de la dernière série), les records, l'eIRM et le tonnage que voit le coach. Un tonnage de séance $\times 10$ sur un exercice rend la courbe de volume illisible pendant des semaines. Le correctif coûte quelques lignes : passer en `type="text" inputmode="decimal"` avec un filtre, et demander confirmation au-delà de $1,5 \times$ le meilleur historique. Il supprime une classe entière d'erreurs de saisie, une par une invisibles pour l'athlète.

Prompt de mise en place.

Contexte : app/rc-core.1693.js. Les champs de charge et de répétitions de séance sont générés dans `renderSets` (~L.49037-49400, attributs `type="number" inputmode="decimal" / "numeric"`) et lus par `_woChargeSaisie` (~L.48350), `_woRepsSaisie` et l'écouteur `Enter` de `_woBrancherEnchainement` (~L.48430). Bug : en Chromium fr-FR, taper « 82,5 » dans un `type=number` donne `value '825'`.

À faire :

1. Passe les champs de charge (`weight`, `weight2`) en `type="text" inputmode="decimal" autocomplete="off" pattern="[0-9]*[.,]?[0-9]*"`. Garde `data-serie`, `data-champ` et `enterkeyhint`.
2. Nouvelle fonction pure `lireCharge(brut)` : accepte « 82,5 », « 82.5 » et « 82 ». Refuse le reste (renvoie `null`), arrondit à 0,25 près, borne à 0.500.
3. `_woChargeSaisie` utilise `lireCharge`. Si le résultat est `null`, remets la valeur précédente et affiche un toast. Si la valeur dépasse $1,5 \times$ la meilleure charge historique (`recordsExercice`) et 20 kg, demande `rcConfirm('825 kg ?', ...)` avant d'écrire.
4. Même vérification pour `weight2` (dégressive) et pour tout autre champ de charge de l'écran de séance (vérifie avec `grep type="number"` dans `renderSets` et `_blocExo`).
5. Tests dans `app/tests.js` (`testExercices`) :
 - `lireCharge('82,5')===82.5`, `lireCharge('82.5')===82.5`, `lireCharge('8a')===null`, `lireCharge('-5')===null`, `lireCharge('22,3')===22.25` ;
 - un rendu de `renderSets` ne contient plus de `type="number"` sur les champs de charge ;
 - `_woChargeSaisie` avec « 82,5 » stocke '82.5'.
6. Suis la convention de build du dépôt.
Critère de fin : `chargerTests()` à 0 échec. En Playwright (locale fr-FR), `keyboard.type('82,5')` stocke 82.5.

6. La vidéo en séance fait sortir de la séance — Priorité : Moyenne · Effort : M

Constat. La caméra par série (`filmerSerie`, :105516) envoie la vidéo tout de suite vers Cloudinary, sur le réseau de la salle. Une fois l'envoi réussi, `uploadVideoFile` appelle `loadVideos()` (:105373), qui fait `go('s-videos')` : l'athlète est sorti de l'écran de séance en plein exercice. La vidéo de démonstration est un lien YouTube `target="_blank"` (`_htmlVideoTechniqueExo`, :48068), qui ouvre YouTube hors de l'app et ne marche pas sans réseau. Pourtant, une file d'envoi locale existe déjà (`fileEnvoiPoser`), mais elle ne sert qu'en cas de quota dégradé.

Pourquoi le mettre en place. En salle, le réseau est faible (sous-sol) et le forfait data limité. Un envoi de 30 à 100 Mo entre deux séries bloque et consomme la batterie. Être éjecté vers « Mes corrections » oblige à revenir, retrouver la séance et le minuteur. Regarder la technique ne devrait pas faire quitter l'app.

Intérêt réel. La boucle de correction vidéo est le différenciateur « coaching » face à Hevy (qui n'en a pas) et un argument de vente de l'offre Ultime. Si filmer une série est pénible, l'athlète filme moins, et le coach perd sa matière première. Mettre l'envoi en file d'attente (en fin de séance ou en Wi-Fi) garde l'athlète dans sa séance et fiabilise l'envoi, avec reprise via la file existante. Un lecteur YouTube intégré dans une feuille basse garde l'athlète dans l'app et fait gagner environ 3 gestes (sortie, retour, retrouver la séance) à chaque consultation.

Prompt de mise en place.

Contexte : app/rc-core.1693.js. `filmerSerie` (~L.105516), `_videoSerieEnvoyer` (~L.105538), `uploadVideoFile` (~L.105028, appelle `loadVideos()` après succès ~L.105373 $\rightarrow$ `go('s-videos')`), file locale `fileEnvoiPoser` / `fileEnvoiRetirer` (IndexedDB), carte de fin `renderCarteAFilmer` (~L.103832), lien vidéo de démonstration `_htmlVideoTechniqueExo` (~L.48065), `normaliserUrlVideo`, `videosExo`.

À faire :

1. En séance (`woState` existe et `!woState.termine`), `_videoSerieEnvoyer` ne lance plus l'envoi. Il met la vidéo dans la file (`fileEnvoiPoser` avec lien, nom, emailCible) et affiche un toast « Vidéo gardée, envoi à la fin de la séance ». Il ajoute ensuite un badge « 📺 1 » sur la série (champ `s.video=true` dans `sessionData`).
2. En fin de séance (`finishWorkout`), la carte « À filmer / Vidéos à envoyer » liste les vidéos en file avec un bouton « Envoyer maintenant » (et l'indication « Wi-Fi recommandé » si `navigator.connection.type` ou `effectiveType` est connu). L'envoi réutilise `uploadVideoFile` avec `options.fileId`.
3. `uploadVideoFile` prend une option `{resterIci:true}`. Avec elle, il n'appelle pas `loadVideos()` et n'appelle aucun `go()`. Utilise-la depuis la séance et depuis l'écran de fin.
4. La vidéo de démonstration s'ouvre dans une feuille basse de l'app avec une `iframe youtube-nocookie.com/embed/<id>?playsinline=1`. Hors ligne (`navigator.onLine===false`), affiche « Vidéo indisponible hors connexion » et garde l'illustration.
5. Tests dans `app/tests.js` (`testExercices`) :
 - `_videoSerieEnvoyer` avec un `woState` actif appelle `fileEnvoiPoser` et n'appelle pas `go` (espions) ;
 - `uploadVideoFile` avec `resterIci` ne change pas l'écran actif ;
 - le rendu de `_htmlVideoTechniqueExo` ne contient plus `target="_blank"`.
6. Suis la convention de build du dépôt.
Critère de fin : `chargerTests()` à 0 échec. En Playwright, filmer une série (input simulé) laisse l'écran #s-workout actif.

7. Ni historique de l'exercice ni notes pendant la séance — Priorité : Moyenne · Effort : M

Constat. Le seul repère historique affiché en séance est une phrase : « Basée sur ton dernier lundi (80kg à RIR 1) » (:48241). Aucun accès depuis l'écran de séance à l'historique complet de l'exercice ni à sa courbe. `historiqueExercice` (:68103) n'est appelé que par la substitution. Aucune note libre pour l'athlète, ni par exercice ni par séance : `savePostSession` (:62942) n'enregistre que des curseurs, et `sess.notes` ne sert qu'aux consignes du coach (:44790). Seul le coach peut écrire un réglage (`reglageCoach`).

Pourquoi le mettre en place. « Siège cran 4, poulie haute trou 7, prise serrée » : ces réglages personnels se relisent à chaque séance. Hevy propose des notes par exercice, épinglées d'une séance à l'autre, et l'historique de l'exercice à un tap. Pour le coach, un « épaule qui tire sur la 3e » écrit à chaud vaut plus qu'un curseur de gène.

Intérêt réel. L'athlète ne rouvre plus son carnet ou sa mémoire pour retrouver ses réglages : c'est 1 à 2 minutes de gagnées par séance, et moins de séries d'ajustement ratées. Le coach reçoit un contexte qualitatif lié à la série (machine prise, douleur localisée, fatigue), ce qui rend ses ajustements plus justes et nourrit le brouillon de réponse au bilan déjà développé (Lot C2). L'historique à un tap (5 dernières séances de l'exercice, meilleure série, e1RM) aide l'athlète à choisir sa charge sans quitter la séance. Tout cela existe déjà côté données (`recordsExercice`, `_dataDeSeance`) : il ne manque que l'affichage.

Prompt de mise en place.

```
Contexte : app/rc-core.1693.js. Carte d'exercice en séance : _blocExo (~l.48100). Données : _dataDeSeance, recordsExercice, getPrevPerf (~l.65703),
historiqueExercice (~l.68103). Fin : finishWorkout (~l.49796) construit sess.data. Fiche coach : les séances de l'athlète sont rendues côté coach par
_buildSessionCard (~l.28923).

À faire :
1. Bouton « Historique » (icône horloge, classe hit44) à côté de « Remplacer » dans .wo-cmd. Il ouvre une feuille basse avec :
  - les 5 dernières séances contenant cet exercice (date, séries charge×reps@RIR) ;
  - la meilleure charge et le meilleur e1RM (recordsExercice).
  Lecture seule, fermeture par le voile ou le retour Android (même patron que rc-lexique).
2. Note d'exercice épinglée : currentUser.notesExo[exKey(nom)] = {texte, maj}. Affichée sous le nom de l'exercice en séance (1 ligne, gris), modifiable par
un tap (textarea 140 caractères max). Visible par le coach dans le détail de séance.
3. Note de séance : une zone « Un mot pour ton coach ? » (facultative, 280 caractères) dans l'écran de fin (#wd-ressenti), enregistrée dans sess.noteAthlete
par savePostSession, affichée dans _buildSessionCard côté coach.
4. Échappe tout avec escapeHtml. Vérifie dans database.rules.json que les nouveaux champs (notesExo, sessions/*/noteAthlete) sont autorisés en écriture par
l'athlète et bornés en longueur (ajoute une règle .validate si le fichier en impose).
5. Tests dans app/tests.js (testExercices) :
  - la feuille d'historique liste les séances dans l'ordre antichronologique ;
  - une note d'exercice survit à un nouveau launchWorkout ;
  - savePostSession écrit noteAthlete tronquée à 280 caractères ;
  - le rendu coach échappe « <script> ».
6. Suis la convention de build du dépôt.
Critère de fin : chargerTests() à 0 échec.
```

8. Remplacer un exercice oblige l'athlète à taper un nom — Priorité : Moyenne · Effort : M

Constat. Pour un athlète, `candidatsRemplacement` rend [] (:72647, garde de rôle voulue : la banque appartient au coach). La feuille « Remplacer pour aujourd'hui » ne propose donc qu'un champ texte libre (`ouvrirRemplacement`, :72688). `substitutsSalle` sait pourtant déjà classer des équivalents par muscle, patron et matériel de la salle, mais seul le coach en profite. Un nom tapé à la main (« dev incliné halt ») ne retrouve ni historique, ni charge proposée, ni illustration, ni vidéo : la charge n'est reprise que si ce nom exact a déjà été fait (`_appliquerSubstitut`).

Pourquoi le mettre en place. « Machine prise » arrive presque à chaque séance en salle commerciale. Taper un nom au clavier entre deux séries est lent, et l'orthographe varie d'une fois à l'autre. L'historique se fragmente alors en noms différents, que ni le coach ni la surcharge progressive ne peuvent relier.

Intérêt réel. On peut respecter la règle métier (la banque reste au coach) avec une liste courte validée par le coach. Le coach choisit 1 à 3 remplaçants par exercice dans son éditeur (ou les accepte depuis `substitutsSalle`), et l'athlète les voit en boutons. Remplacer prend 1 tap au lieu d'environ 25 frappes. Les noms restent canoniques, donc l'historique, la charge proposée et la vidéo suivent. Côté coach, le journal des écarts (`journaliserEcart`) devient exploitable (« remplacé 3 fois par X : mettre X au programme ? »). C'est un avantage face à TrueCoach et Trainerize, dont les substitutions restent manuelles.

Prompt de mise en place.

```
Contexte : app/rc-core.1693.js. Remplacement en séance : _htmlBoutonRemplacer, ouvrirRemplacement (~l.72688), candidatsRemplacement (~l.72647, rend [] hors
coach : règle métier à conserver), validerRemplacementManuel, _appliquerSubstitut, _demanderMotifEcart. Équivalents : substitutsSalle (~l.68129). Éditeur
coach des exercices d'une séance : openCoachSessionExercices / validerProgEx (modale prog-ex-modal dans app/index.html).

À faire :
1. Nouveau champ d'exercice `alternatives` (tableau de 0 à 3 noms de la banque) dans sessions_config. Dans la modale d'édition d'exercice du coach, ajoute «
Remplaçants autorisés » avec un bouton « Suggérer » qui préremplit depuis substitutsSalle(client, nom, salleActive(client)), modifiable.
2. Assure la conservation du champ dans _assainirExercices, _normaliserSessionsConfig, _copierSeance / dupliquerSeance, et dans la propagation de modèle
(grep « exSlug » pour trouver les chemins qui recopient les champs d'exercice).
3. Côté athlète, dans ouvrirRemplacement : si ex.alternatives n'est pas vide, affiche-les en boutons hit44 au-dessus du champ libre (qui reste en « Autre...
»). Un tap appelle _appliquerSubstitut(idx, nom) puis _demanderMotifEcart(idx), exactement comme le parcours coach.
4. Ne change pas la garde de candidatsRemplacement : l'athlète ne fouille toujours pas la banque.
5. Tests dans app/tests.js (testExercices) :
  - alternatives survit à _assainirExercices et dupliquerSeance ;
  - pour un athlète, ouvrirRemplacement rend les boutons des alternatives et candidatsRemplacement rend toujours [] ;
  - _appliquerSubstitut sur une alternative déjà faite préremplit la charge.
6. Suis la convention de build du dépôt.
Critère de fin : chargerTests() à 0 échec.
```

Agent 5 — Synchronisation des données · 6,5/10

La couche de synchro est bien plus mûre que la moyenne : fusion à trois voies par empreintes de Merkle (`syncFusion`, `app/rc-core.1693.js:3169`), lignée `_syncMaj` contre les copies périmées, file par dossier (`_aTonTour`, :3321), pierres tombales, file de renvoi persistante (`rc_sync_queue`, :3339), PUT borné à 45 s, instantané de séance `rc_wo_state` et service worker qui retient la bascule pendant une séance (`app/sw.js:210-228`). Mais tout repose sur un **PUT du dossier entier, précédé d'une lecture, sans écriture conditionnelle** : entre deux appareils, rien n'empêche qu'un envoi écrase un autre. S'y ajoutent trois autres pertes silencieuses : le stockage local plein fait envoyer une copie périmée, les envois du coach ne sont pas journalisés avant de partir, et un appareil sans base écrase le serveur à l'aveugle. Parcours tracé : séance finie hors ligne → `finishWorkout` (:49796) → `saveUser` (:115417) → échec → `rc_sync_queue` → événement `online` (:6730) → `_doPushOne` (lecture, fusion, PUT) → le coach la voit à sa prochaine relève, sous 5 à 30 min (:642-643). Ce parcours fonctionne dans le cas nominal. Les défauts sont dans les cas limites ci-dessous.

1. Stockage local plein : l'envoi part avec la copie périmée et la séance est perdue — Priorité : Critique · Effort : S

Constat. `saveUser` (`app/rc-core.1693.js:115434`) appelle `DB.set('users')`. Quand le quota déborde, `setLocal` avale l'erreur et affiche « tes données sont envoyées au cloud » (:5695). Or `_doPush` relit `DB.get('users')` (:4329-4331), c'est-à-dire l'ancienne copie, et n'utilise `currentUser`, seul à porter la séance, que si le dossier est absent. Reproduction : remplir le `localStorage` (photos base64, ou coach avec 30 athlètes dans `rc_users`), terminer une séance, puis tuer l'app. Le serveur reçoit le dossier sans la séance. Au redémarrage, la descente (`syncUser`, :5335) remplace `currentUser.sessions` par la version fusionnée, qui ne contient pas la séance : elle est perdue partout.

Pourquoi le mettre en place. C'est le pire scénario, une perte totale avec un message qui affirme le contraire. Il frappe justement les utilisateurs les plus engagés, qui ont le plus d'historique, de photos ou d'athlètes. L'app sait déjà que 81 % de stockage saturé existe en production (commentaire :44086).

Intérêt réel. Une séance perdue est la première cause de désinstallation sur une app de musculation : l'athlète refait sa saisie une fois, pas deux. Côté coach, un dossier incomplet fausse la progression, les charges suggérées et les relances. La correction tient en une dizaine de lignes et ferme un chemin de perte certain, pas seulement probable. Elle rend aussi le toast de :5695 enfin vrai. C'est le correctif au meilleur rapport gain/effort de ce périmètre.

Prompt de mise en place.

```
Dépôt RepCore (/home/user/coaching). Bug de perte de données dans app/rc-core.1693.js.
Contexte : DB.setLocal (≈l.5687) avale QuotaExceededError et renvoie false ; saveUser (≈l.115417) appelle DB.set('users',users) puis CLOUD.push(users) ;
CLOUD._doPush (≈l.4319) relit DB.get('users') et pousse _frais[currentUser.email][currentUser]. Si l'écriture locale a échoué, DB.get rend l'ANCIENNE copie :
la séance qui vient d'être ajoutée à currentUser ne part jamais, alors que le toast dit « envoyées au cloud ».
À faire :
1. Dans DB, mémoriser l'échec : DB._echecLocal = {users:true/false, session:...} mis à jour à chaque setLocal (true si quota, false si succès).
2. Dans CLOUD._doPush : si DB._echecLocal.users est vrai (ou si currentUser.updatedAt > (_frais[email][{}]).updatedAt), pousser currentUser (copie profonde
via JSON) et non _frais[email]. Conserver la lecture fraîche de DB.get('users') dans les autres cas (un test l.~43736 de app/tests.js exige la présence de
DB.get('users') dans _doPush : garder cette chaîne).
3. Dans CLOUD.syncUser, quand DB.set('users',merged) renvoie false : ne pas faire Object.assign(currentUser, merged[email]) en retirant des clefs qui
n'existent que dans currentUser. Fusionner plutôt syncFusion(base, currentUser, merged[email]) pour que la séance en mémoire survive.
4. Au boot, si rc_session porte des sessions (par id) absentes de rc_users[email], les réinjecter avant le premier saveUser.
5. Toast de :5695 : ne dire « envoyées au cloud » qu'après succès réel du PUT. Sinon : « Stockage plein : séance gardée en mémoire, NE FERME PAS l'app avant
le ✓ ».
Tests (app/tests.js, même style ok()/echec) : simuler un setItem qui lève QuotaExceededError pour 'rc_users', appeler saveUser après ajout d'une séance,
intercepter fetch et vérifier que le corps PUT contient la séance. Second test : syncUser avec quota plein ne retire pas la séance de currentUser.
Critère de fin : les deux tests passent, les tests existants autour de _doPush/DB.set restent verts (lancer la suite via scripts/verif/suite.mjs ou en
ouvrant l'app avec ?tests).
```

2. Course entre appareils : lecture puis PUT sans ETag, le second envoi efface le premier — Priorité : Critique · Effort : M

Constat. `_doPushOne` lit le dossier (:4034), fusionne (:4075), écrit un bloc santé privée, puis fait un PUT du document entier (:4205). Aucun `X-Firebase-ETag` ni `if-match` n'existe dans le fichier (grep vide). La file `_aTonTour` ne sérialise qu'à l'intérieur d'un même appareil. Reproduction : l'athlète (téléphone) et le coach (ordinateur) envoient le même dossier dans la même seconde. A pose la séance S. B, qui a lu avant le PUT de A, réécrit sans S. A enregistre alors comme base sa propre version, qui contient S (:4270). À sa descente suivante, `syncFusion` voit « local = base, distant sans S » et conclut que l'autre appareil a supprimé S (:3183-3184) : S disparaît aussi du téléphone.

Pourquoi le mettre en place. Le commentaire de :4021 promet qu'« une perte de données devient impossible ». En réalité la fenêtre existe à chaque envoi, et le coach et l'athlète écrivent précisément dans le même dossier. Firebase REST propose nativement l'écriture conditionnelle : un 412 renvoie la valeur courante, prête à refusionner.

Intérêt réel. C'est la dernière classe de perte inter-appareils que la fusion à trois voies ne couvre pas. Téléphone, APK et navigateur du même athlète, plus le coach, font jusqu'à quatre écrivains par dossier. Même à 0,5 % des envois concurrents, un coach à 30 athlètes qui publie des programmes chaque semaine verra des séances ou réponses de bilan disparaître plusieurs fois par an, sans trace. Aucune panne n'est plus coûteuse en confiance : « l'app a mangé ma séance ». Trainerize et TrueCoach ont une base serveur transactionnelle, et RepCore doit atteindre le même niveau de garantie sans serveur. L'ETag le permet pour une requête de plus, seulement en cas de conflit.

Prompt de mise en place.

```
Dépôt RepCore, app/rc-core.1693.js, objet CLOUD. Rendre le PUT du dossier conditionnel (optimistic concurrency) via l'ETag de Firebase RTDB REST.
1. pullUser(email) (≈l.4625) : ajouter un paramètre optionnel {etag:true} qui envoie l'en-tête 'X-Firebase-ETag: true' et rend {doc, etag}
(r.headers.get('ETag')). Conserver la signature actuelle pour les autres appelants (retour du doc seul par défaut).
2. _doPushOne (≈l.4034) : lire distant+etag. Au PUT (≈l.4205), ajouter l'en-tête 'if-match': etag. Si r.status===412 : la réponse porte l'ETag courant (en-
tête) et la valeur courante (corps). Refaire syncFusion(tour.base.h, safeAvantFusion, nouveauDistant), rejouer _retirerPrives, recalculer updatedAt=max(...,
distant.updatedAt+1), puis retenter au plus 3 fois avec le nouvel etag. Au-delà, lever une erreur rejouable (file rc_sync_queue).
3. Garder la copie de 'safe' d'avant fusion pour pouvoir refusionner proprement.
4. Le PUT santé privée (≈l.4160-4180) doit rester hors de la fenêtre lecture-écriture : le déplacer APRES le PUT du dossier (il est indépendant).
5. Vérifier que keepalive + en-tête if-match fonctionnent (keepalive accepte les en-têtes).
Tests app/tests.js : fetch simulé. (a) GET rend etag E1, PUT avec if-match E1 → 412 avec corps contenant une séance S2 → second PUT doit contenir S2 ET la
modification locale. (b) Trois 412 de suite → erreur rejouable, clef enfilée dans rc_sync_queue. (c) Scénario deux appareils : banc qui fait tourner deux
instances de syncFusion/_doPushOne sur une fausse base en mémoire avec entrelacement GET-A, GET-B, PUT-A, PUT-B : aucune séance perdue.
Critère de fin : plus aucun PUT sur users/<clé> sans if-match dans _doPushOne, tests verts.
```

3. Sans base de fusion, l'envoi écrase le serveur à l'aveugle (la défense de volume ne se déclenche jamais) — Priorité : Haute · Effort : M

Constat. Sans base (`_tour.base null`), `_doPushOne` n'appelle pas `syncFusion` (:4074) et fait un PUT de la copie locale brute. Le seul garde-fou, « distant plus récent et plus riche » (:4082), exige `distant.updatedAt > safe.updatedAt`. Or `pushOne` (:3754) et `saveUser` (:115418) posent `updatedAt = Date.now()` juste avant : la condition est fautive sauf horloge en retard, et le bloc de rattrapage qu'il contient ne s'exécute donc pas non plus. On se retrouve sans base après `_poserBase` en quota plein (qui la supprime volontairement, :3258), sur un appareil neuf avant la première descente (inscription :11117, `DB.set sans base`), ou pour un athlète que le coach vient d'ajouter. Reproduction : téléphone du coach au quota plein. L'athlète pousse une séance, le coach modifie son programme avant que la relève ne descende : le PUT du coach efface la séance du serveur, puis du téléphone de l'athlète (même mécanisme qu'au point 2).

Pourquoi le mettre en place. Le code documente ce retour à « la version locale l'emporte » comme acceptable, mais la défense censée le couvrir est inerte. Le cas le plus exposé est le téléphone du coach, qui garde tous ses athlètes dans `rc_users` : c'est précisément lui qui sature en premier.

Intérêt réel. Le coach est l'utilisateur payant (19 à 39 €/mois) et le multiplicateur : une perte chez lui touche N athlètes à la fois. Si le moindre envoi sans base passe par une fusion et non plus par un écrasement, le mode dégradé ne détruit plus rien. Les données de l'athlète (séances, bilans, vidéos) sont alors protégées même quand l'appareil du coach est en mauvais état. Le coût est une lecture déjà faite (`distant` est lu dans tous les cas).

Prompt de mise en place.

```
Dépôt RepCore, app/rc-core.1693.js, CLOUD._doPushOne (≈1.4060-4150).
Problème : sans base (_tour.base null), aucun syncFusion, PUT de la copie locale brute ; et le garde-fou `if(distant&&(distant.updatedAt||0)>(safe.updatedAt||0))` ne se déclenche jamais car pushOne/saveUser ont posé updatedAt=Date.now() juste avant.
À faire :
1. Sans base mais avec distant : fusion d'UNION conservatrice : safe = syncFusion(syncEmpreintes({}) /* base vide */, safe, distant) ne convient pas (local gagne). Écrire une fonction pure syncUnionSansBase(local, distant) qui, pour les tableaux de SYNC_PAR_CLEF (sessions, vidéos, bilans), fait l'union par clef (pierres tombales appliquées). Pour les autres champs, garder le distant si le local est vide ou absent, sinon le local. L'appliquer quand !_base && distant.
2. Remplacer la condition du garde-fou par une comparaison de CONTENU indépendante des horodatages : si, après fusion, volume(safe)<volume(distant) ou _cptContenu(safe.sessions_config)<_cptContenu(distant.sessions_config), déclencher le rattrapage existant (syncUser puis nouvel envoi, drapeau 'rattrapage').
3. Quand _poserBase échoue pour quota, libérer d'abord de la place (purgerPhotosAnciennes, suppression des bases d'athlètes non ouverts depuis 30 j) avant de retirer la base.
4. À l'inscription/connexion (≈1.1117-11150), après users[em]=cloudUser, appeler CLOUD._poserBase(em, cloudUser) et poser _syncMaj.
Tests app/tests.js : (a) sans base, distant avec séance S absente du local : le corps PUT contient S. (b) local plus pauvre avec updatedAt plus récent : le rattrapage est appelé (espionner syncUser). (c) connexion sur appareil neuf : la base existe après le login.
Critère de fin : aucun chemin de _doPushOne n'envoie un dossier contenant moins de séances/bilans/vidéos que le distant sans pierre tombale correspondante.
```

4. Les envois du coach vers un athlète ne sont pas journalisés avant de partir — Priorité : Haute · Effort : S

Constat. La clef n'entre dans `rc_sync_queue` qu'au `catch` (:4279). Si l'app est tuée ou gelée pendant la lecture ou le PUT (verrouillage juste après « Publier »), aucun `catch` ne s'exécute. Au redémarrage, `_doPush` ne renvoie que le dossier de `currentUser` (:4319-4331, :6940), et `viderFile` n'a rien. Reproduction : le coach modifie le programme d'un athlète, verrouille le téléphone dans la seconde, iOS gèle la page, puis Android ou iOS la tue. Au retour, la descente fusionne la modification dans le local du coach (elle y est, et le serveur n'a pas bougé), mais elle ne repart jamais : le coach voit son programme, l'athlète jamais. De plus, `keepalive` n'est posé que sous 60 Ko (:4193), et le GET qui précède le PUT n'en bénéficie pas.

Pourquoi le mettre en place. Une file de renvoi ne protège que ce qu'elle contient. Journaliser avant d'envoyer (write-ahead) est la règle de base de l'offline-first, et le correctif se limite à déplacer deux appels.

Intérêt réel. Verrouiller son téléphone juste après avoir validé est le geste le plus courant, et le coach le fait des dizaines de fois par semaine. Un programme publié que l'athlète ne reçoit pas donne une semaine d'entraînement sur l'ancien plan, un « tu n'as pas vu ma modif ? », une perte de crédibilité, et à terme du churn coach. L'athlète en profite aussi : ses propres envois sont déjà couverts au boot par `saveUser` + `_doPush`, mais uniquement s'il rouvre l'app.

Prompt de mise en place.

```
Dépôt RepCore, app/rc-core.1693.js, objet CLOUD.
1. Dans CLOUD.pushOne (≈1.3750) et au début de _doPushOne quand !_tour : appeler this._enfiler(email) AVANT tout réseau (journal write-ahead). _defiler(email) reste appelé seulement après r.ok (≈1.4225, déjà présent). Pour les refus _nonRejouable, défiler explicitement dans le catch.
2. Conserver la déduplication de _enfiler. Relever la borne de _ecrireFile de 50 à 500 clefs (coachs à plus de 50 athlètes), et ne jamais évincer la clef de currentUser.
3. Au retour au premier plan (visibilitychange visible, ≈1.7133) et à l'événement online, appeler CLOUD.viderFile() si enAttenteDeSync()>0.
4. Enregistrer une Background Sync (registration.sync.register('rc-sync')) quand la file n'est pas vide. Dans app/sw.js, sur 'sync', poster un message aux clients ouverts {rc:'vider-file'} (le SW ne peut pas faire l'envoi lui-même : jeton et fusion vivent dans la page). Côté page, écouter ce message et appeler viderFile.
5. Rétablir un petit indicateur athlète « n envois en attente » quand enAttenteDeSync()>0 (le commentaire ≈1.6725 reconnaît que plus rien ne le dit).
Tests app/tests.js : fetch qui ne résout jamais, appeler pushOne('x@t.fr',u), puis vérifier que rc_sync_queue contient 'x@t.fr' AVANT toute résolution. Après succès simulé, la clef est retirée. Un refus _nonRejouable retire la clef.
Mettre à jour le test existant ≈1.8578 qui cherche '_enfiler(email)' dans le catch si la structure change.
Critère de fin : tuer l'app (simulé en abandonnant la promesse) après pushOne laisse la clef en file, et viderFile au démarrage renvoie le dossier athlète.
```

5. Journaux en tableau fusionnés en bloc : une pesée ou une nuit saisie sur un second appareil se perd — Priorité : Moyenne · Effort : S

Constat. Seuls `sessions`, `videos` et `bilans` se fusionnent élément par élément (`SYNC_PAR_CLEF`, :3062). `_syncEstObjet` exclut les tableaux, qui sont donc des feuilles : `weightLog` (Clé date, :108648), `sleepLog`, `stepsLog`, `energieLog`, `journalDouleur`, `fcReposLog`, `historiqueDrapeaux`, et même `sessions_config` sont comparés en bloc, et « celui qui écrit maintenant l'emporte » (:3196). Reproduction : pesée hors ligne sur le téléphone le matin, pesée d'hier ajoutée sur l'ordinateur. Les deux touchent `weightLog`, le dernier envoi gagne et l'autre pesée disparaît. De même, si l'athlète renomme une séance (`renameSession`, :42815) ou change sa photo (:43530) pendant que le coach édite le programme, tout `sessions_config` d'un des deux côtés est écrasé.

Pourquoi le mettre en place. L'usage multi-appareil (téléphone + APK + navigateur) est explicitement visé, et ces journaux sont les données les plus fréquemment saisies. Le mécanisme existe déjà : il suffit d'ajouter des clefs.

Intérêt réel. La courbe de poids et le sommeil servent au coach pour ajuster les calories et la charge. Un point manquant fausse la tendance hebdomadaire. Fusionner `sessions_config` par créneau (index 0-6) évite en plus que le renommage d'une séance par l'athlète annule une semaine de travail de programmation du coach. C'est une extension de 5 lignes, déjà testée par les tests de `syncFusion` existants (app/tests.js:43572-43780).

Prompt de mise en place.

```
Dépôt RepCore, app/rc-core.1693.js, ≈1.3062 SYNC_PAR_CLEF.
Ajouter des clefs d'identité pour les journaux en tableau, pour que syncFusion les fusionne élément par élément :
weightLog: e=>e&&e.date?('w':'+e.date'):null,
```

```
sleepLog / stepsLog / energieLog / fcReposLog / vfclLog / journalDouleur : même patron sur `date` (vérifier la forme réelle de chaque entrée par grep de leur .push et adapter : date, ou date+heure).
historiqueDrapeaux: e=>e&&e.leve?('d:'+e.leve):null.
Pour sessions_config (tableau de 7 créneaux, jamais d'id) : clef par index, 'c':+i. Adapter _syncVersArbre/_syncDepuisArbre pour qu'un champ déclaré « par index » soit reconstruit en tableau ORDONNÉ par index (et non trié par date comme sessions). Ajouter une table SYNC_PAR_INDEX=['sessions_config'].
Vérifier _syncDepuisArbre : le tri par `date` numérique ne doit pas casser les journaux dont date est une chaîne 'YYYY-MM-DD' (trier par String pour ceux-là).
Pierres tombales : si une pesée peut être supprimée dans l'UI, passer par marquerSupprime avec un type 'weightLog' (étendre la liste autorisée ≈L.5720 et le filtre en fin de syncFusion).
Tests app/tests.js (à côté des tests syncFusion ≈L.43572) : base {weightLog:[A]}, local [A,B], distant [A,C] → résultat [A,B,C] trié. sessions_config : local renomme le créneau 2, distant change les exercices du créneau 4 → les deux modifications présentes. Ordre des 7 créneaux conservé.
Critère de fin : tests verts, aucun test syncFusion existant cassé.
```

6. Champs écrits par le serveur léger dans users/ sans horodatage : invisibles aux appareils et exposés à l'écrasement — Priorité : Moyenne · Effort : S

Constat. Le Worker écrit dans le document que le client réécrit en entier, sans toucher `updatedAt : origine/payeLe` (cloudflare/src/metier.js:945, par transaction), `origine/payeLe:null`, `annuleLe` (metier.js:793), `abonnement/statutPaypal` (cloudflare/src/paypal.js:362). Comme `syncUser` ne descend que si `updatedAt` a changé (:5351), ces valeurs n'arrivent sur les appareils qu'à la modification suivante. Un envoi sans base (point 3) les efface. Scénario : `payeLe` effacé, puis nouveau paiement, et la transaction « une seule fois » de `metier.js:945` compte l'attribution payante une seconde fois.

Pourquoi le mettre en place. Deux écrivains sur un même document, dont l'un fait des PUT entiers, doivent respecter un contrat : le serveur horodate, ou mieux, le serveur écrit hors de `users/`. `paypal.js:239` et `:603` le font déjà (`updatedAt: t`), les trois autres l'oublient.

Intérêt réel. Il y a un impact business direct : statistiques d'attribution et de parrainage justes, et commissions d'ambassadeurs non dupliquées. L'athlète voit aussi son statut d'abonnement à jour sans attendre une autre modification. La correction se résume à trois lignes côté Worker et un test par cas dans `cloudflare/test`.

Prompt de mise en place.

```
Dépôt RepCore, dossier cloudflare/.
1. cloudflare/src/metier.js ≈L.793 (annulerAttribution) : ajouter ['users/'+cle+'updatedAt']: t dans le même update multi-chemins.
2. cloudflare/src/metier.js ≈L.945 (attributionPaiement) : après la transaction committée, db.ref().update({'users/'+cle+'updatedAt': t}). Mieux : dupliquer la vérité « déjà payé » dans un nœud serveur hors /users (ex. paypal_premiers/<cle> existe déjà, cf. migration.js) et faire porter la transaction dessus, pour qu'un PUT client ne puisse plus la remettre à zéro.
3. cloudflare/src/paypal.js ≈L.362 : ajouter ['users/'+cle+'updatedAt']: Date.now() (ou le t local de la fonction).
4. Grep tout `users/` + ` en écriture dans cloudflare/src/*.js et appliquer la même règle. Documenter en tête de cloudflare/src/base.js : « toute écriture dans users/<clé> pose updatedAt ».
Tests cloudflare/test (fausse-base.mjs) : pour chacun des trois chemins, vérifier que users/<clé>/updatedAt a été avancé. Pour attributionPaiement : vérifier qu'une remise à null de users/<clé>/origine/payeLe par un client ne permet pas un second comptage (si le point 2 est fait).
Lancer : cd cloudflare && node --test test/metier.test.mjs test/paypal.test.mjs.
Critère de fin : tests verts, aucune écriture serveur dans users/ sans updatedAt.
```

7. Le coach voit la séance 5 à 30 minutes plus tard, et seulement app ouverte — Priorité : Moyenne · Effort : M

Constat. La descente est une relève périodique : `SYNC_PERIODE_MS=300000`, soit 1 800 000 ms (30 min) en mode dégradé (:642-643), plus une relève au retour au premier plan (:7133). Il n'y a ni flux temps réel (`text/event-stream` absent) ni Background Sync. Pendant une séance en direct, un coach qui garde l'écran ouvert attend jusqu'à 5 min, et l'athlète hors ligne n'a plus aucun indicateur d'envoi en attente (retiré, :6720-6726).

Pourquoi le mettre en place. Le parcours « séance hors ligne → réseau → coach » est correct, mais sa latence et son opacité se voient face à Hevy Coach ou Trainerize, où la séance apparaît chez le coach en quelques secondes. Firebase REST permet d'écouter un seul petit nœud en SSE, sans SDK.

Intérêt réel. Le feedback du coach dans l'heure est l'argument de vente du coaching premium (99 à 600 €). Voir la séance au moment où elle se termine permet de féliciter ou corriger à chaud, ce qui augmente la rétention. Un seul flux SSE par coach, sur un nœud « boîte d'arrivée », tient dans la limite des 100 connexions simultanées du plan Spark. Un flux par athlète ne tiendrait pas. Côté athlète, revoir « 1 séance en attente d'envoi » supprime l'angoisse et les doubles saisies.

Prompt de mise en place.

```
Dépôt RepCore. Objectif : le coach reçoit la notification de mise à jour d'un dossier athlète en quelques secondes quand l'app est ouverte, sans multiplier les connexions (plan Spark : 100 connexions simultanées max).
1. Nouveau nœud /boite_coach/<coachKey>/<athleteKey> = updatedAt (nombre). Dans CLOUD._doPushOne (app/rc-core.1693.js ≈L.4225, après r.ok), si safe.coachEmailKey : PUT de safe.updatedAt sur boite_coach/<coachEmailKey>/<safeKey>.json (petit, non bloquant, erreurs avalées).
2. Côté coach, à l'ouverture de s-coach-home : EventSource sur ../boite_coach/<maClé>.json?auth=<token> (Firebase REST streaming). Sur 'put'/'patch' : pour chaque athleteKey dont la valeur ≠ base.maj connue, appeler CLOUD.syncUser(email). Fermer le flux quand document.hidden, le rouvrir au retour. Gérer 'auth_revoked' en rafraîchissant le jeton, et reconnexion avec délai croissant.
3. Garder la relève de 5 min comme filet.
4. Règles : ajouter dans database.rules.json une entrée boite_coach où l'athlète écrit seulement sa propre clé sous le coach dont il porte coachEmailKey, et où le coach lit sa boîte (coordonner avec l'audit sécurité, ne pas élargir /users). Mettre à jour scripts/verif/regles.mjs si nécessaire.
5. Athlète : afficher un discret « v n envoi(s) en attente » quand CLOUD.enAttenteDeSync()>0, masqué sinon.
Tests : app/tests.js, simuler EventSource (classe factice) et vérifier que syncUser est appelé pour l'athlète notifié uniquement. cloudflare/test/regles-*.emu.mjs : athlète A ne peut pas écrire dans la boîte d'un autre coach ni sous la clé d'un autre athlète.
Critère de fin : sur deux navigateurs, une séance terminée côté athlète apparaît chez le coach en moins de 10 s.
```

8. Aucun banc de synchronisation à deux appareils dans la CI — Priorité : Basse · Effort : M

Fusion. le lancement de la suite navigateur en CI (étape 5) est couvert par l'agent QA (point 1). Ne réaliser ici que le banc de synchronisation.

Constat. Les tests de fusion existent (`app/tests.js:43572-43780`, `syncFusion` pur), mais la suite navigateur n'est lancée par aucun workflow : `.github/workflows/firebase.yml` n'exécute que `scripts/verif/syntaxe|actifs|regles|tarifs...`, pas `suite.mjs`. Plusieurs tests vérifient la présence d'une chaîne dans le source (`String(CLOUD._doPush).indexOf("DB.get('users')")`, :43736) plutôt que le comportement. Les courses « mesurées au banc » citées dans les commentaires (:3310, :4064) ne sont rejouées nulle part automatiquement.

Pourquoi le mettre en place. Les points 1 à 5 sont tous des entrelacements (deux appareils, quota, mort de la page). Un test de source n'en attrape aucun, et une régression de synchro ne se voit qu'en production, sous forme de données disparues.

Intérêt réel. Un simulateur déterministe (fausse base RTDB en mémoire, deux instances de CLOUD, ordonnanceur qui entrelace GET/PUT) couvre en quelques secondes les scénarios qui ont déjà coûté les lots B1, N3 et le correctif du 21/09. Il protège la donnée la plus précieuse du produit à chaque build, et permet de refactorer `_doPushOne` (plus de 400 lignes) sans peur.

Prompt de mise en place.

```
Dépôt RepCore. Créer scripts/verif/banc-sync.mjs (Node 22, sans dépendance) et l'ajouter à .github/workflows/firebase.yml comme les autres scripts/verif.
1. Extraire de app/rc-core.1693.js, par lecture de source (voir comment scripts/verif/*.mjs le font déjà, ex. source-prod.mjs), les fonctions pures _syncFnmv,
_syncEstObjet, _syncCanon, _syncVersArbre, _syncDepuisArbre, _syncCarte, syncEmpreintes, syncFusion, SYNC_*.
2. Écrire un mini-modèle de deux appareils : chacun a {local, base}, une fausse RTDB {doc, etag}. Opérations : edit(appareil, f), push(appareil) découpé en
GET puis PUT séparables, pull(appareil).
3. Générer aléatoirement (graine fixe, 2000 itérations) des entrelacements d'ajouts de séances/bilans/pesées sur les deux appareils. Invariant : après
convergence (pull des deux, puis push, puis pull), toute séance jamais supprimée volontairement est présente sur le serveur ET sur les deux appareils.
4. Le banc doit d'abord ÉCHOUER sur le code actuel pour l'entrelacement GET-A, GET-B, PUT-A, PUT-B (point « course sans ETag ») : marquer ce cas comme
attendu-rouge tant que l'ETag n'est pas implémenté, puis passer vert.
5. Ajouter au workflow le lancement de la suite navigateur (scripts/verif/suite.mjs avec Chrome headless) si le runner le permet. Sinon, au minimum banc-
sync.mjs.
Critère de fin : `node scripts/verif/banc-sync.mjs` sort 0, tourne en CI en moins de 30 s, et rougit si l'on retire la fusion de _doPushOne.
```

Agent 6 — Sécurité applicative · 4/10

Plusieurs briques serveur sont faites avec soin. Le webhook PayPal vérifie la signature chez PayPal sur le corps brut et gère l'idempotence (cloudflare/src/paypal.js:99-113, :660-692). Le JWT Firebase est vérifié en RS256, avec contrôle de aud, iss et exp (cloudflare/src/appels.js:39-62), et cloudinaryDestroy contrôle le propriétaire et le coach déclaré (cloudflare/src/medias.js). Aucun secret n'est commité, et escapeHtml/safeUrl servent presque partout. Le payroll reste pourtant **entièrement contournable** par une seule écriture dans son propre dossier. Deux contenus payants sont lisibles par tout compte connecté. Enfin, une XSS stockée athlète → coach permet de prendre la session de Kevin, qui est l'**administrateur** (écriture de droits/, de la boutique et des vitrines).

1. Le payroll se contourne en une écriture : status, role et coachPlan sont écrits par l'utilisateur — Priorité : Critique · Effort : L

Fusion. la fermeture de l'essai et de l'achat côté client rejoint le point 7 de l'agent Backend (ouvrirEssai et verifierAchatProgramme portés dans le Worker) et le point 1 de l'agent Business (quota coach) : un seul chantier « droits serveur ».

Constat. users/\$emailKey donne au titulaire l'écriture de tout son nœud, sans liste blanche sur status, role, paymentStatus, accessExpiry, coachPlan, coachSubActive ni programmesAchetees (database.rules.json:22). Côté client, palierDe rend 'suivi' (le palier le plus haut, PALIERS_ORDRE en rc-core.1693.js:7507) dès que u.role==='coach' (:7588). _palierHerite rend 'suivi' sur status==='COACHING_SUIVI' **avant** le contrôle payeCru (:7627) : c'est vrai même après la bascule droitsServeur, tant que droits/ n'a pas été posé à la main. Le quota coach (19/39 €) se lit dans coachPlan/coachSubActive du dossier (:1442-1461, ce que le commentaire reconnaît). De plus, n'importe quel compte peut créer /rc_codes/RC-XXXX-XXXX avec le contenu de son choix, y compris type:'coach' ou months:120 (database.rules.json:903, clause !data.exists()). Reproduction : connecté en athlète, fetch(DB+ '/users/<ma_clé>/status.json?auth='+jeton,{method:'PUT',body:'COACHING_SUIVI'}), puis recharger : tout Ultime est ouvert, sans fin.

Pourquoi le mettre en place. Toute la marge vient de trois leviers (Essentielle 9,50 €, Ultime 24,90 €, coach 19/39 €), et chacun se désactive depuis la console du navigateur. L'astuce circule vite dans une communauté fitness (Reddit, Discord, TikTok « app premium gratuite »). Le Worker écrit déjà droits/ avec l'Admin SDK (metier.js:238 majDroits) : l'infrastructure pour faire décider le serveur existe.

Intérêt réel. Chaque fraudeur, c'est 114 à 298,80 €/an de perdus, et un coach Pro fraudeur coûte 468 €/an. À 1 000 utilisateurs payants, une fuite de 10 % représente déjà 12 à 30 k€/an. La fraude fausse aussi les métriques de conversion sur lesquelles repose la note de décision économique. Elle rend enfin le parrainage absurde : pourquoi payer quand on peut s'offrir le palier « suivi ». Une fois corrigé, un accès payé se prouve (PayPal → Worker → droits/). Kevin peut alors vendre avec la même garantie que Trainerize ou Everfit, dont les droits sont décidés côté serveur. L'Admin SDK du Worker contourne les règles et reste le seul à pouvoir écrire ces champs.

Prompt de mise en place.

```
Contexte : RepCore (dépôt courant). Le palier d'accès se décide côté client à partir de champs que l'utilisateur écrit lui-même dans users/<clé> (database.rules.json ligne ~22 : .write du titulaire sur tout le nœud). palierDe() (app/rc-core.1693.js ~7585) rend 'suivi' si u.role==='coach', et _palierHerite (~7623) rend 'suivi' si status==='COACHING_SUIVI' avant tout contrôle. coachPlanDe/coachSubActif (~1442-1461) lisent coachPlan/coachSubActive dans le dossier. /rc_codes/$code peut être créé par n'importe quel compte (clause !data.exists(), ligne ~903).
```

Objectif : droits/<clé> (écrit uniquement par le Worker via compte de service, ou par le créateur) devient la SEULE source de vérité pour le palier athlète, le rôle coach et le plan coach.

```
1. database.rules.json, sous users/$emailKey, ajouter des .validate de gel pour les champs de droits :
"status", "paymentStatus", "accessExpiry", "coachPlan", "coachSubActive", "programmesAchetees", "role" :
".validate": "newData.val() === data.val() || auth.token.email === 'guellec.coachingpro@gmail.com'".
Exception pour "role" : autoriser la première pose (!data.exists()) à 'athlète' uniquement. Le rôle 'coach' est posé par le Worker.
Même gel pour abonnement/formule, abonnement/statutPaypal, abonnement/finAccesPaypal et abonnement/dernierPaieementLe.
Attention : l'app fait un PUT du dossier entier ; une valeur identique passe, mais le client ne doit plus MODIFIER ces champs (voir 3).

2. Worker (cloudflare/src/appels.js, table APPELS dans cloudflare/src/index.js) : ajouter des fonctions /fn/ authentifiées :
- redeemCode({code}) : lit rc_codes/<code>. Vérifie que le coach émetteur est enregistré (voir coaches_registre ci-dessous), que le code est actif, non expiré, non consommé, et plafonne les mois (12 pour un affilié, comme moisAutorises). Il pose redeemed, puis droits/<athlète> {palier:'suivi', echeance, source:'code_coach'} et users/<athlète>/coachEmailKey, coachId et status.
- ouvrirEssai() : une fois par compte (droits/<k>/essaiOuvertle absent), palier 'ultime' pendant TARIFS.essai.jours.
- devenirCoach({invitation}) : consume un code type 'coach' émis par le créateur, puis écrit coaches_registre/<k> = {plan:'libre', le} et users/<k>/role = 'coach'.
Dans paypal.js, paiementAbonnement et fermerAlaFin écrivent aussi coaches_registre/<k>/plan et actifJusqu pour les plans coach.

3. Règles : nouveau nœud "coachs_registre": {"$k": {"$.read": "auth != null && auth.token.email.replace('.', '') === $k", "$.write": false}}. Dans rc_codes/$code/.write, remplacer !data.exists() par "!data.exists() && root.child('coachs_registre').child(auth.token.email.replace('.', ''))!.exists() && newData.child('coachEmailKey').val() === auth.token.email.replace('.', '')". Retirer la clause qui permet à un tiers de passer redeemed à true : c'est le Worker qui consomme.

4. Client (app/rc-core.1693.js) :
- palierDe : supprimer le raccourci u.role==='coach'. Pour un athlète, lire UNIQUEMENT droitsDe(u). Le repli _palierHerite ne sert que si droits/ n'a JAMAIS pu être lu (état 'inconnu'), et jamais pour 'suivi' : COACHING_SUIVI n'ouvre plus rien sans droits/.
- coachPlanDe/coachSubActif : lire coaches_registre/<clé> (nouvelle méthode CLOUD.pullCoachRegistre, mise en cache comme _droitsLus).
- appliquerPayloadCode et appliquerCodeAprèsInscription : appeler CLOUD.callFn('redeemCode') au lieu du PATCH direct, puis rafraichirDroits(currentUser,true).
- onApprove (~121880) : ne plus écrire status, paymentStatus ni coachPlan. Afficher « paiement reçu, activation... » et interroger droits/ toutes les 5 s pendant 2 min (le webhook écrit). Garder paiementRecent comme simple affichage.

5. Migration : script cloudflare/scripts/remplir-droits.mjs qui parcourt users (compte de service), écrit droits/ pour les abonnés PayPal ACTIVE vérifiés chez PayPal et pour les athlètes suivis dont le code existe, puis pose reglages_publics/droitsServeur.

6. Tests :
- Nouveau cloudflare/test/regles-droits.emu.mjs, sur le modèle de regles-amis.emu.mjs (émulateur, jetons alg none). Un athlète NE PEUT PAS écrire status='COACHING_SUIVI', role='coach', coachPlan='pro' ni accessExpiry=0 dans son dossier. Il PEUT réécrire la même valeur. Il ne peut pas créer /rc_codes sans être dans coaches_registre. Le créateur peut tout écrire.
- cloudflare/test/appels.test.mjs : cas redeemCode (code forgé par un non-coach refusé, mois plafonnés, double consommation refusée).
- app/tests.js : palierDe({role:'coach'}) avec droits absents rend 'aucun' pour un athlète. palierDe({status:'COACHING_SUIVI'}) avec droits 'absent' rend 'aucun'.

Critère de fin : 'node scripts/verif/regles.mjs', 'node scripts/verif/suite.mjs', les tests cloudflare (npm test dans cloudflare/) et le nouvel emu passent.
La reproduction PUT status=COACHING_SUIVI n'ouvre plus aucun écran payant.
```

2. XSS stockée athlète → coach, qui mène à la prise du compte administrateur — Priorité : Critique · Effort : M

Constat. _videoEmbed(url) insère l'URL de la vidéo de l'athlète **sans échappement ni safeUrl** : <video src="\$url"> et (app/rc-core.1693.js:103821-103826). Elle est appelée dans l'écran coach « Corriger la vidéo » (_vcCorpsHtml, :106958, rendu :107016). Même défaut sur les photos de bilan côté coach, dans renderBilanEvolution (:31357), et dans htmlRapport (:63804-63805). users/<athlète>/videos et bilans n'ont aucune validation. Le jeton de rafraichissement Firebase est stocké dans localStorage.rc_fb_auth (:3222-3225), et aucune Content-Security-Policy n'est posée (ni firebase.json ni <meta http-equiv>). Scénario : l'attaquant s'inscrit, pose coachEmailKey et coachId sur ceux de Kevin (coachId est lisible dans tout /rc_codes/<code>, en lecture publique, et dans les liens coachpkg). Il écrit annuaire_coach/<kevin>/<lui>, puis ajoute une vidéo

{id:'v1',name:'Squat',url:'https://x/a.mp4?'}'. Kevin ouvre la correction et l'attaquant récupère son refresh token. En sens inverse (coach → athlète) : value="{s.name|''}" (:42673), ainsi que t.ts et t.audioUrl dans les annotations (:106920-106923).

Pourquoi le mettre en place. L'adresse de Kevin est codée en dur comme administrateur dans les règles : droits, boutique, saisons, ambassadeurs, reglages_publics et defi_mois. Voler sa session revient à contrôler tout le business : s'offrir des accès, changer les prix, repointer les liens d'achat de sa vitrine vers un autre PayPal, lire paypal_journal. Tout athlète peut viser tout coach, sans rien acheter.

Intérêt réel. La correction vidéo est la fonctionnalité phare, celle qui fait la différence face à Hevy Coach ou TrueCoach. Kevin l'ouvre donc plusieurs fois par jour, et l'exploitation est quasi certaine dès qu'un attaquant la tente. Un seul incident (prix modifiés, liens d'achat détournés, accès offerts en masse) coûte bien plus que la correction, qui tient en quelques heures. Pour les coaches affiliés, c'est aussi une garantie commerciale : « vos athlètes ne peuvent pas pirater votre compte ». Une CSP stricte ferme en plus toute la classe de failles pour les 728 innerHTML restants.

Prompt de mise en place.

Contexte : RepCore, app/rc-core.1693.js. Des données écrites par l'athlète (users/athlète/videos[], url, bilans[*] photos) sont injectées sans échappement dans des écrans du coach. Le coach administrateur (guellec.coachingpro@gmail.com) a des droits d'écriture globaux dans database.rules.json. Le refresh token est dans localStorage 'rc_fb_auth'.

Corriger, sans rien changer au rendu visuel :

1. _videoEmbed (≈ligne 103821) : url = normaliserUrlVideo(url). N'accepter que https://res.cloudinary.com/, https://firebasestorage.googleapis.com/ et les formes YouTube. Utiliser escapeHtml(url) dans src et safeUrl(url) dans href. Si l'URL est refusée, rendre un texte « Lien vidéo invalide ».
2. renderBilanEvolution (≈31357) : src="{escapeHtml(img|'')}" , et n'accepter que data:image/(jpeg|png|webp);base64,..., https://res.cloudinary.com/ ou blob: (fonction utilitaire srcImageSure(s), à placer à côté de safeUrl ≈115400).
3. htmlRapport (≈63804-63805) : même srcImageSure + escapeHtml.
4. _renderSessionManager (≈42673) : value="{escapeHtml(s.name|'')}" .
5. _renderTsAnnotations (≈106920) : retirer l'expression inline tsToSecs('\${t.ts}') et utiliser data-sec="{Number(sec)|0}" plus un gestionnaire délégué. title/texte : escapeHtml(t.ts). audio src : srcImageSure-like limité à https://res.cloudinary.com/.
6. Chercher et corriger toutes les autres interpolations d'URL non échappées : grep -nE 'src="\\${[src="\x27+|href="\\${{' app/rc-core.1693.js. Pour chaque cas dont la donnée vient d'un dossier (users, coach_public, canaux), appliquer escapeHtml/safeUrl.
7. Défense serveur, database.rules.json sous users/\$emailKey :
"videos": {"\$i": {"url": {"\$.validate": "newData.isString() && newData.val().length < 600 && (newData.val().startsWith('https://res.cloudinary.com/') || newData.val().startsWith('https://firebasestorage.googleapis.com/') || newData.val().startsWith('https://youtu.be/')) || newData.val().startsWith('https://www.youtube.com/'))"}}}.

Vérifier avant avec scripts/mesure_photos_bilan.py ou un export qu'aucune donnée existante n'est rejetée (sinon le PUT entier du dossier échoue : voir les commentaires de database.rules.json sur ce piège).

8. CSP : ajouter dans firebase.json (headers, source "/"app/**" et "/"app") un Content-Security-Policy qui n'autorise que 'self', les domaines réellement utilisés (firebaseio.com, identitytoolkit.googleapis.com, securetoken.googleapis.com, api.cloudinary.com, res.cloudinary.com, *.paypal.com, repcore-serveur.repcore.workers.dev, youtube.com pour frame-src), object-src 'none' et base-uri 'self'. Les onclick inline imposent 'unsafe-inline' pour script-src à court terme : l'indiquer en commentaire et ajouter une tâche de migration. Même sans cela, img-src, media-src et connect-src bornés empêchent l'exfiltration par fetch vers un domaine tiers.
9. Tests app/tests.js : _videoEmbed('https://x/a.mp4?') ne contient ni '' ni 'onerror'. _videoEmbed('javascript:alert(1)') ne contient pas 'javascript:'. renderBilanEvolution avec une photo '<svg onload=1>' ne produit aucun élément svg (monter dans un div hors DOM et compter querySelectorAll('svg,[onerror],[onload]'). Ajouter aussi un test « balayage » : pour chaque fonction de rendu coach listée, injecter un dossier piégé dans chaque champ texte et vérifier l'absence d'attribut on*.

Critère de fin : node scripts/verif/suite.mjs vert, les nouveaux tests passent, et aucune des charges ci-dessus ne s'exécute dans l'écran « Corriger la vidéo » ouvert sur un athlète piégé.

3. Contenus payants lisibles par tout compte connecté : programmes de la boutique et bibliothèque d'exercices — Priorité : Haute · Effort : M

Constat. boutique a ".read": "auth != null" et porte le champ seances (jusqu'à 240 000 caractères, le programme complet) (database.rules.json:333-342). L'app l'y publie (rc-core.1693.js:33829, :41989). Un simple GET /boutique.json?auth=<jeton d'un compte gratuit> rend tous les programmes vendus 14,90 €. exercices s'ouvre à users/<moi>/role === 'coach' (database.rules.json:1913), champ que l'utilisateur écrit lui-même, ce qui ouvre la bibliothèque Ultime en une écriture.

Pourquoi le mettre en place. C'est le seul contenu réellement protégé côté serveur, et il ne l'est pas. Une fois un programme publié, n'importe quel compte gratuit peut le télécharger en masse et le repartager. La vente de programmes est justement la voie choisie pour monétiser sans intermédiation (note de décision, §2).

Intérêt réel. Chaque programme vaut 14,90 € et ouvre trois mois d'Ultime (OFFRES.boutique_prog). Un seul export revendu ou partagé sur un forum tue la vente de ce programme pour toujours. Protéger ces contenus rend le catalogue durable : Kevin peut investir dans « Prise de masse », « Sèche » et « Force » (emplacements aCompleter en :150-185) sans craindre qu'ils fuient le jour même. La fiche produit (nom, prix, accroche) reste publique, donc aucune perte de conversion.

Prompt de mise en place.

Contexte : RepCore, database.rules.json : "boutique" (≈ligne 331) est lisible par tout compte connecté et contient "seances" (le contenu vendu). "exercices" (≈ligne 1913) s'ouvre à users/<moi>/role === 'coach', champ que l'utilisateur écrit lui-même.

À faire :

1. Séparer la fiche publique du contenu : boutique/<id> ne garde que {nom, prixCts, accroche, image, semaines, niveau} (lecture auth != null) ; le contenu part dans boutique_contenu/<id>/seances, lisible seulement si root.child('droits/' + cléDeAuth + '/programmes/' + \$id).exists() OU si l'e-mail vérifié est celui du créateur. Écriture réservée au créateur.
2. App (rc-core, publication ~L.33829 et ~L.41989) : écrire la fiche et le contenu séparément ; la lecture du contenu après achat passe par boutique_contenu. Migration : script scripts/migrer_boutique.mjs qui déplace seances des fiches existantes (mode --simulation par défaut).
3. "exercices" : remplacer la condition role==='coach' par une donnée que l'utilisateur ne peut pas écrire (registre coaches/<clé>/actif écrit par le Worker ou le créateur, ou droits/<clé>.palier in ['ultime','suivi']). Tests : cloudflare/test/regles-*.emu.mjs (nouveau cas) – un compte gratuit ne lit pas boutique_contenu/<id>, un acheteur le lit, un compte qui s'écrit role:'coach' ne lit pas exercices. node scripts/verif/regles.mjs vert.

Critère de fin : GET /boutique.json avec un jeton gratuit ne renvoie aucun champ seances.

4. Le parrainage se retourne en fabrique de mois gratuits — Priorité : Haute · Effort : M

Constat. Un mois est crédité au parrain quand un filleul atteint 4 séances (cloudflare/src/metier.js:615, crediterMoisOffert :648). Ces séances sont déclarées par le filleul et comptées sans contrôle de dates distinctes (cloudflare/src/xp.js:146). L'e-mail n'est pas vérifié à l'inscription (accounts:signUp, rc-core:3611), l'« appareil » est auto-déclaré, et il n'existe aucun plafond de mois par parrain.

Pourquoi le mettre en place. Un parrain peut créer des comptes jetables et y saisir 4 séances en quelques minutes pour cumuler des mois gratuits sans limite. Le mécanisme de viralité le plus rentable devient alors une fuite de revenu, et il fausse aussi les statistiques d'attribution et les commissions d'ambassadeurs.

Intérêt réel. Le parrainage doit rester généreux pour les vrais filleuls, car c'est le canal d'acquisition le moins cher. Conditionner le crédit au premier paiement du filleul (ou à 4 séances réparties sur au moins 10 jours, avec e-mail vérifié) supprime l'intérêt de la fraude sans gêner un utilisateur honnête. Un plafond (par exemple 6 mois offerts par an) borne le risque même si une faille subsiste. Les ambassadeurs, eux, touchent déjà au paiement : les deux mécaniques deviennent cohérentes.

Prompt de mise en place.

```
Contexte : RepCore, cloudflare/src/metier.js (parrainage : seuil 4 séances ~l.615, crediterMoisOffert ~l.648), cloudflare/src/xp.js (comptage des séances ~l.146), fonctions/parrainage-calcul.js (règles pures), app/rc-core.1693.js (inscription accounts:signUp ~l.3611).
À faire (défensif, sans rien exposer) :
1. Le filleul n'est « qualifié » que si : son e-mail est vérifié (auth.token.email_verified, à lire dans le jeton transmis au Worker ou via l'API Identity Toolkit), ET ses 4 séances comptées ont chacune ≥ 1 série validée, tombent sur 4 jours calendaires distincts, et s'étalent sur au moins 10 jours. Écrire cette règle comme fonction PURE filleulQualifie(seances, emailVerifie, maintenant) dans fonctions/parrainage-calcul.js (partagée par le Worker).
2. Option recommandée (constante PARRAINAGE_AU_PAIEMENT=true) : créditer le parrain au premier paiement du filleul (le chemin paypal.js → attributionPaiement existe déjà), et non plus aux 4 séances.
3. Plafond : au plus PARRAIN_MOIS_MAX_AN = 6 mois offerts par parrain sur 12 mois glissants ; au-delà, journaliser parrainage_plafond<clé> et prévenir le créateur par push.
4. App : après inscription, envoyer l'e-mail de vérification (accounts:send0obCode, requestType VERIFY_EMAIL) et afficher un rappel discret tant que ce n'est pas fait.
Tests : fonctions/test (filleulQualifie : 4 séances le même jour → faux ; 4 jours distincts sur 12 jours avec e-mail vérifié → vrai ; e-mail non vérifié → faux) ; cloudflare/test/metier.test.mjs (plafond atteint → pas de crédit, journal écrit ; mode au paiement → crédit seulement après le webhook de premier paiement). cd cloudflare && npm test ; node --test fonctions/test/.
Critère de fin : aucun mois offert sans filleul qualifié, et jamais plus de 6 par parrain et par an.
```

5. Le préérage d'envoi Cloudinary est ouvert à tous — Priorité : Moyenne · Effort : M

Constat. Le préérage non signé `repcore_videos` du compte `dnut57ml` est appelé directement depuis le navigateur (`rc-core:64560-64569`, `:105126`). Toute personne qui lit le code de la page peut envoyer des fichiers sur ce compte, sans être connectée.

Pourquoi le mettre en place. Le quota Cloudinary gratuit (environ 25 crédits par mois) peut être épuisé par des envois étrangers : la correction vidéo, argument de l'offre Ultime, cesse alors de fonctionner pour tous. Un hébergement détourné peut aussi faire suspendre le compte.

Intérêt réel. Passer par des envois signés par le Worker (qui vérifie le jeton Firebase, comme `cloudinaryDestroy` le fait déjà) protège le quota et garantit que chaque média appartient à un compte identifié, ce qui simplifie aussi la purge des orphelins (`scripts/purge_cloudinary_orphelins.py`). Le coût reste nul et le parcours de l'athlète ne change pas.

Prompt de mise en place.

```
Contexte : RepCore. Envois Cloudinary non signés depuis le front (app/rc-core.1693.js ~l.64560-64569 et ~l.105126, preset repcore_videos, cloud dnut57ml). Le Worker sait déjà vérifier un jeton Firebase (cloudflare/src/appels.js) et parler à Cloudinary (cloudflare/src/medias.js, cloudinaryDestroy).
À faire :
1. Worker : nouvel appel /fn/cloudinarySigner({dossier, type}) qui vérifie le jeton, impose le dossier « u/<cléDuCompte> » (ou celui d'un athlète dont l'appelant est le coach, même contrôle que medias.js), et renvoie {timestamp, signature, api_key, folder, public_id?} signés avec CLOUDINARY_API_SECRET (déjà en secret pour la suppression). Limite : 30 signatures par heure et par compte.
2. App : avant chaque envoi, appeler cloudinarySigner puis poster vers l'API Cloudinary avec ces champs (envoi SIGNÉ), en gardant la file locale existante (fileEnvoiPoser) en cas d'échec.
3. Console Cloudinary (à faire par Kevin, à écrire dans cloudflare/README.md) : passer repcore_videos en « signed », limiter formats (mp4, mov, webm, jpg, png, webp) et taille (100 Mo vidéo, 10 Mo image).
Tests : cloudflare/test (signature correcte pour des paramètres connus ; refus sans jeton ; refus d'un dossier d'un autre compte ; limite horaire) ; app/tests.js (l'envoi appelle d'abord la signature, repli en file si elle échoue). cd cloudflare && npm test.
Critère de fin : plus aucun upload_preset non signé dans le front, et un envoi sans jeton est refusé par Cloudinary.
```

6. Saturation des quotas : écritures anonymes et route /veille non protégées — Priorité : Moyenne · Effort : S

Constat. Le dossier utilisateur n'a pas de borne de taille globale. `/metrics` et `/attribution` acceptent des écritures anonymes. `/veille` n'est ni authentifié ni limité, alors que le quota gratuit du Worker (100 000 appels par jour) est partagé avec le webhook PayPal. Les limites Spark (1 Go stocké, 100 connexions simultanées) s'ajoutent.

Pourquoi le mettre en place. Une boucle de requêtes suffit à épuiser le quota du jour : webhooks PayPal, notifications et défis s'arrêtent alors jusqu'à minuit UTC. La correction est une affaire de règles et de limiteurs déjà présents ailleurs dans le Worker (`/arrivee`).

Intérêt réel. Protège le chemin qui encaisse l'argent et la disponibilité du service pour tous les coaches, pour un effort de quelques heures. Les compteurs `/metrics` restent anonymes mais bornés, donc les tableaux de bord restent justes.

Prompt de mise en place.

```
Contexte : RepCore. database.rules.json (/metrics ~l.1010, /attribution, users$emailKey), cloudflare/src/index.js (routes /veille, /arrivee avec son limiteur).
À faire :
1. /veille : POST seulement, limiteur identique à /arrivee (6 appels / 60 s par IP), 429 au-delà. (Si le point 8 de l'agent Backend est déjà fait, ne rien dupliquer.)
2. /metrics<date><evenement> : n'accepter qu'un incrément de +1 (newData.val() === data.val()+1 ou 1 si absent), date = aujourd'hui ± 1 jour, événement dans la liste blanche existante.
3. /attribution : mêmes bornes (incrément +1, clés en liste blanche, profondeur fixe).
4. users$emailKey : ajouter des .validate de longueur sur les champs texte libres les plus gros (notes, messages, bilans : 4 000 caractères) et $autre:false sous les nœuds dont le schéma est fermé. Pas de borne globale impossible à exprimer : documenter dans scripts/verif/regles.mjs ce qui est borné.
5. Alerte de quota : dans le poulx du Worker (voir agent Backend, point 2), compter les 429 servis et prévenir le créateur au-delà de 500 par heure.
Tests : cloudflare/test/index.test.mjs (/veille en GET → 405 ; 7e appel en 60 s → 429) ; tests émulateur des règles (incrément de +5 sur /metrics refusé ; clé inconnue refusée ; note de 10 000 caractères refusée). cd cloudflare && npm test ; node scripts/verif/regles.mjs.
Critère de fin : aucune route publique sans limite, et aucune écriture anonyme non bornée.
```

7. Les droits administrateur reposent sur l'e-mail, sans vérification — Priorité : Moyenne · Effort : S

Constat. Les règles comparent `auth.token.email === 'guellec.coachingpro@gmail.com'` sans exiger `email_verified`, et le Worker ne contrôle pas non plus `email_verified`. `/sante` indique en outre publiquement si l'ancien secret de la base sert encore (fuite d'information mineure).

Pourquoi le mettre en place. Toute la boutique, les saisons, les ambassadeurs et `droits/` dépendent de ce compte. Lier les droits à un identifiant immuable (l'UID Firebase de Kevin) plutôt qu'à une chaîne d'e-mail ferme une classe entière de risques pour un effort minime.

Intérêt réel. C'est la clé de voûte des points 1 à 3 : tant que l'administrateur est défini par un e-mail, chaque règle « créateur seulement » hérite de cette fragilité. Le changement se fait en une passe de recherche et remplacement, plus un test.

Prompt de mise en place.

```
Contexte : RepCore. database.rules.json compare auth.token.email à l'e-mail du créateur dans de nombreuses règles ; cloudflare/src (appels.js, index.js)
identifie aussi le créateur par e-mail. Route /sante (cloudflare/src/index.js -l.186).
À faire :
1. Récupérer l'UID Firebase du compte créateur (console Authentication) et le poser dans une constante unique : dans les règles, remplacer chaque
'auth.token.email === '<e-mail du créateur>'' par '(auth.uid === '<UID>' && auth.token.email_verified === true)' ; dans le Worker, constante CREATEUR_UID et
vérification de email_verified dans le jeton pour toute route sensible.
2. Ajouter à scripts/verif/regles.mjs un contrôle qui échoue si l'e-mail du créateur apparaît encore comme condition d'accès dans database.rules.json.
3. /sante : ne plus exposer le mode d'accès à la base (retirer le champ « acces ») ; garder ok/base/vapid.
Tests : tests émulateur (un compte dont l'e-mail est celui du créateur mais l'UID différent ne peut pas écrire /boutique ; le vrai UID vérifié le peut) ;
cloudflare/test/index.test.mjs (/sante ne contient plus « acces »). node scripts/verif/regles.mjs ; cd cloudflare && npm test.
Critère de fin : plus aucune règle d'administration fondée sur l'e-mail seul.
```

Agent 7 — Backend et infrastructure · 5/10

Le serveur léger est bien conçu pour ce qu'il est : verrou en transaction (planif.js:144), reprises avec `evenements_ko` au 5e échec, idempotence PayPal (`en_cours` / fait), budget de sous-requêtes vérifié avant chaque tour, et une vraie suite de tests (22 fichiers, `npm test` : 92 sous-tests `node:test` + les compteurs internes, **tout passe**, 2,6 s ; `node --test test/` échoue en revanche, il faut garder la glob du `package.json`). Mais la capacité réelle est très basse : j'ai mesuré sur la base simulée que le rappel « série » du jeudi **plafonne à 180 athlètes** et que plus de la moitié du budget de chaque minute part en tenue de livre. Aucune alerte si le cron tombe, aucune sauvegarde de la base, et le Worker se déploie à la main sans CI : l'app ne peut pas encore tenir 1 000 athlètes de façon fiable.

1. Le cron plafonne à ~180 athlètes (rappel série) et coupe ses envois sans rien dire — Priorité : Critique · Effort : M

Constat. Chaque minute relit l'état de chaque travail par un GET séparé (planif.js:179-180, une requête par travail actif, jusqu'à 18), plus 4 requêtes de verrou (deux `transaction`, `base.js`:132-145). Mesuré sur base vide pendant 1 440 minutes : 23 098 sous-requêtes par jour de pure tenue de livre, jusqu'à 32 sur 38 vers 19 h. Il reste 6 à 17 requêtes utiles sur `BUDGET = 38` (planif.js:24), soit ~1 push par minute (`COUT_PUSH = 8`, `metier.js`:145). `serie` (planif.js:50) et `retour` (planif.js:69) n'ont pas de garde `p.heure < 21`. Après 21 h, `envoyerPush(..., {attendre:false})` renvoie `calme` et jette le message (`metier.js`:270-275, `metier.js`:323), mais le curseur avance et le travail finit « fini ». Pour le reproduire : simuler un jeudi (fausse base, tous les travaux actifs) avec N abonnés à série vivante. On obtient 100 → 100 servis, 200 → 180, 300 → 180, 500 → 180, et le travail est quand même marqué `fini`.

Pourquoi le mettre en place. C'est la limite qui casse en premier, bien avant les 100 000 appels par jour (~12 000 athlètes à ~8 appels par jour chacun). Les pertes ne se voient pas : aucun `evenements_ko`, aucun compteur. Plus il y a d'événements dans la file, plus le débit des travaux baisse, parce que la file passe en premier.

Intérêt réel. Lire l'état des travaux en un seul GET (`/worker/jobs`) libère ~15 requêtes par minute, ce qui fait passer le débit de ~1 à ~3 push par minute. Un index compact par athlète (`worker/profils/<uid>`) : `streak`, `streakWeek`, `lastSession`, `fname`, `accessExpiry`, `status`, `pushPrefs`), lu par pages de 200 en une requête, ramène le coût d'un athlète qu'on ne relance pas de 3 à 5 lectures à ~0. Le plafond devient alors `MAX_CHIFFREMENTS = 5` par exécution, soit ~900 séries sur 18 h-21 h. Le rappel série est le levier de rétention principal du produit : chez `Hevy Coach` ou `Trainerize`, il part à tout le monde. Aujourd'hui, un coach qui a 250 athlètes en perd 70 chaque jeudi sans le savoir. Garder le curseur au lieu de marquer « fini » après 21 h permet de reprendre le lendemain ou de compter les sautés.

Prompt de mise en place.

```
Contexte : cloudflare/src/planif.js (minute(), travaux()) et cloudflare/src/metier.js (planifies.*, envoyerPush). Plan gratuit Cloudflare : 50 sous-requêtes par exécution, BUDGET=38. Mesure actuelle : 23 098 sous-requêtes/jour sur base vide, rappel série plafonné à 180 athlètes (jeudi 18h-21h).
À faire :
1. planif.js : lire TOUT /worker/jobs en UN GET au début de l'étape 2 (db.ref('worker/jobs').get()), puis n'écrire que les travaux dont l'état a changé, en UN update multi-chemins à la fin (db.ref().update({'worker/jobs/<nom>': etat, ...})). Un travail déjà 'fini' pour la période ne coûte plus aucune requête.
2. Ajouter 'p.heure < 21' au 'quand' de 'serie', 'retour', 'bilan', 'wrapped', 'badge' et 'duels' (ceux qui envoient avec attendre:false). Si la fenêtre se ferme avant la fin de la liste : NE PAS mettre fini=true ; écrire etat.sautes = nombre de clés restantes, et consigner une ligne dans evenements_ko ({type:'travail_incomplet', nom, sautes, le}).
3. Réduire le coût des athlètes qu'on ne relance pas : dans metier.js, 'planifies.serie', 'acces' et 'retourUn' lisent 3 à 5 champs par athlète. Créer 'worker/profils/<uid>' (streak, streakWeek, lastSession, fname, accessExpiry, status, statut de suspension), mis à jour par le Worker quand il traite les événements de séance, OU lire 'users/<uid>' en ?shallow=false limité aux champs via une requête groupée. Solution retenue : un nœud 'worker/profils' lu PAR PAGES (orderByKey, startAt, limitToFirst 200) en UNE requête, filtré en mémoire, et UNE requête d'écriture par push seulement.
4. Faire commencer 'serie' à 17h00 au lieu de 18h00 si les tests métier l'acceptent (documenter dans README « Charge »).
Tests : étendre cloudflare/test/charge.test.mjs avec (a) un scénario « tous les travaux actifs, jeudi 18h, N=300 » SANS pré-marquer les travaux finis, qui doit servir 300/300 avant 21h ; (b) un test qui vérifie qu'après 21h aucun athlète n'est marqué traité sans push et que 'sautes' est écrit ; (c) un test de tenue de livre : base vide, 1 440 minutes, total des sous-requêtes < 8 000/jour. Lancer 'cd cloudflare && npm test' et 'CHARGE=10,100,1000 node test/charge.test.mjs', puis mettre à jour le tableau « Charge » du README.
Finis quand : ces trois tests passent, aucune exécution ne dépasse 50 requêtes, et le README donne les nouveaux plafonds.
```

2. Personne n'est prévenu si le cron tombe : /sante ne regarde que la configuration — Priorité : Critique · Effort : S

Constat. `/sante` (index.js:186-195) ne fait que vérifier que les secrets sont posés : il répond `ok:true` même si aucune minute n'a tourné depuis une semaine. `scheduled()` (index.js:201-203) ne journalise rien (le bilan de `minute()` est jeté) et `wrangler.toml` n'a pas de bloc `[observability]`. `evenements_ko` ne s'affiche que dans l'écran Ambassadeurs de l'app, ce qui suppose que Kevin l'ouvre. Aucun workflow ne sonde le Worker (`.github/workflows/` n'a qu'un seul cron, `rapport-payeur.yml`, mensuel).

Pourquoi le mettre en place. Un secret expiré (compte de service révoqué, VAPID changé), un quota épuisé ou une erreur 500 de la base, et toutes les notifications, les défis, les parrainages et la file PayPal s'arrêtent en silence. Tout ce qu'il faut existe déjà : `scripts/envoyer_mail.mjs`, le secret `MAIL_MOT_DE_PASSE`, et `envoyerPush(..., {urgent:true})`.

Intérêt réel. On passe de « panne découverte quand un athlète se plaint » à une alerte en 15 minutes. La transaction de fin de bail (planif.js:227) écrit déjà dans `worker/verrou` : y ajouter `derniere`, `requetes`, `file`, `echecs` ne coûte **aucune requête de plus**. Un courriel quand `evenements_ko` grossit évite que des paiements, parrainages ou défis restent en rade des jours entiers. C'est la condition pour vendre du coaching à 99-600 € : un coach payant ne tolère pas un silence de 48 h sur les réponses aux bilans. Coût : 0 € (GitHub Actions et Workers Logs sont gratuits).

Prompt de mise en place.

```
Contexte : cloudflare/src/planif.js (minute, bloc finally), cloudflare/src/index.js (route /sante, scheduled), cloudflare/wrangler.toml, .github/workflows/, scripts/envoyer_mail.mjs (déjà utilisé par rapport-payeur.yml avec MAIL_MOT_DE_PASSE).
À faire :
1. planif.js : dans la transaction du finally qui rend le bail, ajouter au nœud worker/verrou : poulx = { t: horloge(), requetes: compteur(), evenements: bilan.evenements, echecs: bilan.echecs, erreur: bilan.erreur || null, source: source || 'cron' }. Aucune requête supplémentaire.
2. index.js scheduled : 'ctx.waitUntil(minute(outils(env)).then((b) => console.log(JSON.stringify(b))).catch((e) => console.error('minute', e && e.message)))'. wrangler.toml : ajouter '[observability]{nenabled = true\nhead_sampling_rate = 0.1}'.
3. /sante : lire worker/verrou (1 GET) et evenements_ko?shallow=true (1 GET), puis renvoyer { ok, derniereMinuteILYA_s, file: taille approximative via /evenements?shallow=true&limitToFirst... (sinon omettre), ko: nombre }. Statut 503 si le poulx a plus de 5 minutes ou si l'accès à la base échoue. Mettre en cache 30 s (caches.default) pour que /sante ne devienne pas une porte vers le quota.
4. Nouveau workflow .github/workflows/veille-serveur.yml : cron '*/*15 * * * *' + workflow_dispatch ; 'curl -fsS https://repcore-serveur.repcore.workers.dev/sante' ; en cas d'échec, ou si ko > 0 et en hausse par rapport au dernier passage (utiliser actions/cache pour garder la dernière valeur), envoyer un courriel avec scripts/envoyer_mail.mjs (mêmes variables MAIL_* que rapport-payeur.yml), une fois par heure au plus.
5. metier.js / planif.js : quand echec() range un événement dans evenements_ko, envoyer aussi un push urgent au créateur (envoyerPush(emailKey(CREATOR_EMAIL), {type:'coach', title:'Serveur : un événement en échec', body: type+ ' : '+erreur, {urgent:true}})), au plus un par heure (garde worker/alerte_ko = heure).
Tests : cloudflare/test/index.test.mjs : /sante renvoie 503 si poulx absent ou vieux de plus de 5 min, 200 sinon ; planif.test.mjs : le poulx est écrit sans requête supplémentaire (compteur identique avant/après). 'cd cloudflare && npm test' tout vert.
Finis quand : couper le cron (ou fausser le secret de la base) fait échouer le workflow et envoie un courriel en moins de 20 minutes.
```

3. Aucune sauvegarde de la Realtime Database — Priorité : Haute · Effort : S

Constat. Rien dans `scripts/`, `.github/workflows/` ni `cloudflare/src/` n'exporte la base. Le plan Spark n'a pas de sauvegardes automatiques (réservées à Blaze). Toute la vérité métier est dans une seule base : `droits/`, `paypal_transactions`, `paypal_journal`, `parrainage/`, les séances, les bilans. Le Worker y écrit avec un accès administrateur total (compte de service, `google.js:174`). Une écriture multi-chemins fautive (`db.ref().update(maj)`) à la racine, `planif.js:123`, `metier.js:163`) ou une erreur de manipulation dans la console ne se rattrape pas.

Pourquoi le mettre en place. Une seule mauvaise mise à jour à la racine peut effacer des milliers de dossiers. Et `droits/` + `paypal_*` sont la preuve de ce qui a été payé. Le secret `FIREBASE_SERVICE_ACCOUNT` existe déjà dans GitHub (`firebase.yml`).

Intérêt réel. Sans sauvegarde, une perte de données veut dire perdre les coachs : l'historique de séances est ce qui les retient. Avec un export chiffré chaque nuit des nœuds critiques et un export complet chaque semaine, on peut revenir en arrière de 24 h, pour 0 €. Attention au quota Spark de **10**

Go/mois téléchargés : un export complet chaque nuit d'une base de 300 Mo le mangerait presque entier. D'où le rythme proposé : nœuds critiques chaque nuit, `users/` le dimanche. Une fois en Blaze, les sauvegardes quotidiennes intégrées prennent le relais.

Prompt de mise en place.

```
Contexte : base Firebase RTDB repcore-sync (https://repcore-sync-default-rtdb.firebaseio.com), secret GitHub FIREBASE_SERVICE_ACCOUNT déjà utilisé par .github/workflows/firebase.yml. Quota Spark : 10 Go/mois téléchargés, réponse REST max 256 Mo.
À faire :
1. scripts/sauvegarde_base.mjs (Node 22, sans dépendance) : obtenir un jeton OAuth du compte de service (reprendre la logique de cloudflare/src/google.js, portée https://www.googleapis.com/auth/firebase.database), lire /.json?shallow=true, puis exporter nœud par nœud (GET /<nœud>.json?format=export). Pour un nœud trop gros (réponse > 200 Mo ou erreur 413), descendre d'un niveau (shallow puis enfants par lots). Écrire un .json.gz par nœud et un manifeste (taille, sha256, date).
2. Mode '--critiques' : seulement droits, paypal_evenements, paypal_transactions, paypal_journal, paypal_abonnes, paypal_premiers, paypal_orphelins, parrainage, ambassadeurs, ambassadeurs_publics, evenements_ko, worker. Mode '--complet' : tout.
3. Chiffrer l'archive (openssl enc -aes-256-cbc -pbkdf2, avec un nouveau secret SAUVEGARDE_CLE) et la téléverser en artefact GitHub (retention-days: 35) ; pour le complet, aussi en release privée 'sauvegarde-AAAA-MM-JJ' en ne gardant que les 8 dernières (supprimer les plus anciennes via gh).
4. Workflow .github/workflows/sauvegarde.yml : cron '17 2 * * *' (--critiques) et '37 2 * * 0' (--complet) + workflow_dispatch. En cas d'échec, courriel via scripts/envoyer_mail.mjs (mêmes MAIL_* que rapport-payeur.yml). Afficher la taille exportée dans le résumé du travail, pour suivre la consommation du quota de 10 Go.
5. scripts/restaurer_noeud.mjs : restaure UN nœud depuis une archive (PUT), avec --simulation par défaut qui affiche le diff de clés.
6. Documenter dans docs/sauvegarde.md : la fréquence, le coût en bande passante, la procédure de restauration testée.
Tests : un test Node (scripts/verif/sauvegarde.test.mjs) qui fait tourner l'export contre cloudflare/test/fausse-base.mjs (fetch injecté), vérifie la descente d'un niveau sur 413 et l'aller-retour export → restauration à l'identique.
Finir quand : un export --critiques manuel via workflow_dispatch produit une archive chiffrée, et la restauration simulée d'un nœud affiche 0 différence.
```

4. Le Worker se déploie à la main, et ses tests ne tournent nulle part — Priorité : Haute · Effort : S

Fusion. le job « test » du Worker est aussi demandé par l'agent QA (point 1). Un seul workflow : celui-ci teste ET déploie le Worker, QA branche les autres suites.

Constat. Aucun workflow ne lance `cloudflare/npm test` ni `wrangler deploy`. Le déploiement se fait depuis le PC Windows (`README`, « Installer », étape 5, `installer.ps1`). On ne peut donc pas savoir si le code en production est celui de `main`. Autres constats : `firebase.yml:194` et `deploye.sh` (« == fonctions == ») tentent `firebase deploy --only functions` à chaque livraison. `pages.yml:24` publie **tout le dépôt** (path: `.`) sur GitHub Pages à chaque poussée sur `main`, y compris `cloudflare/`, `functions/`, `database.rules.json`, `*.ps1` et les notes internes.

Pourquoi le mettre en place. Les lots C3 à C6 ont tous touché `cloudflare/src/` (`git log -- cloudflare/`). Un oubli de `wrangler deploy` laisse l'app appeler une route qui n'existe pas encore (`/prospect`, `/vitrine-vue`), et un déploiement d'une branche locale non poussée met en ligne du code que personne n'a testé.

Intérêt réel. Chaque livraison devient reproductible : tests verts, puis déploiement, puis sonde `/sante`, puis retour arrière possible (`wrangler rollback`). C'est 0 € (un jeton d'API Cloudflare limité à « Workers Scripts:Edit »). On supprime aussi deux sources de bruit : l'avertissement « Fonctions NON déployées » à chaque livraison, qui habitue à ignorer les avertissements, et la publication Pages du dépôt entier, qui expose l'architecture et les scripts d'installation.

Prompt de mise en place.

```
Contexte : cloudflare/ (wrangler.toml, package.json "test": node --test "test/*.test.mjs"), .github/workflows/firebase.yml, .github/workflows/pages.yml, deploye.sh.
À faire :
1. Nouveau .github/workflows/cloudflare.yml : on push sur main (paths: cloudflare/**, functions/*-calcul.js, .github/workflows/cloudflare.yml) + pull_request + workflow_dispatch. Job test : setup-node 22, 'cd cloudflare && npm test'. Job deploy (needs: test, uniquement sur main) : cloudflare/wrangler-action@v3 avec apiToken {{ secrets.CLOUDFLARE_API_TOKEN }}, accountId {{ secrets.CLOUDFLARE_ACCOUNT_ID }}, workingDirectory cloudflare, command 'deploy'. Puis une étape de fumée : 5 essais de 'curl -fsS https://repcore-serveur.repcore.workers.dev/sante | jq -e '.ok and .vapid and .secret''. En cas d'échec, 'npm wrangler@4 rollback' et le travail échoue. Si les secrets sont absents : ::warning:: et sortie propre, comme apk.yml.
2. firebase.yml : retirer le bloc 'firebase deploy --only functions', remplacé par un commentaire qui renvoie à functions/README.md (déploiement manuel, décidé une fois). deploye.sh : même chose, et corriger le message « aucun paiement PayPal n'ouvre de droit serveur », faux depuis que le Worker traite /paypal et écrit droits/.
3. pages.yml : ne publier que vault/ (le seul contenu servi par Pages d'après firebase.yml) : ajouter une étape qui copie vault/ dans _pages/ et 'path: _pages'. Vérifier qu'aucune autre page n'est servie par Pages (grep github.io dans le dépôt).
4. cloudflare/README.md : section « Déployer » qui renvoie au workflow, avec la création du jeton (Cloudflare > My Profile > API Tokens > modèle « Edit Cloudflare Workers ») et les deux secrets GitHub.
Tests : une PR de test fait tourner le job test sans déployer ; un workflow_dispatch sur main déploie et la sonde passe.
Finir quand : 'main' et le Worker en ligne ne peuvent plus diverger sans qu'un travail rouge le montre.
```

5. Le webhook PayPal tourne sans budget, à 45 sous-requêtes sur 50 — Priorité : Haute · Effort : M

Constat. Hors d'un réveil, `_reste = () => Infinity` (`metier.js:175`) : recevoirWebhook (`index.js:107-110`) n'a pas de plafond. Mesuré sur `paypal.test.mjs` instrumenté : 45 sous-requêtes pour « premier paiement d'un filleul », 41 pour un achat de programme, 39 pour un orphelin rejoué. `rejouerOrphelins` (`paypal.js:180-190`) boucle sans budget, et chaque orphelin de plus coûte ~20 requêtes. Il faut ajouter en production le jeton Google (isolat froid, +1), le jeton PayPal (+1) et un push par appareil du parrain. Au-delà de 50, Cloudflare lève « Too many subrequests », la réponse est 500, l'état passe à `erreur`, et PayPal renvoie le même événement, qui retombe sur le même mur.

Pourquoi le mettre en place. C'est le chemin qui encaisse l'argent. Un abonnement dont l'`ACTIVATED` et le premier `SALE.COMPLETED` arrivent avant que l'app l'ait signalé (cas documenté, `paypal.js:171`) donne deux orphelins. Le lien suivant les rejoue dans la même exécution : 60 à 80 sous-requêtes.

Intérêt réel. Sans ça, un paiement bien reçu n'ouvre pas l'accès, ne crédite pas le parrain et n'est pas compté comme « payant ». Le client paie et se voit refuser l'accès, ce qui produit une demande de remboursement ou un litige, lui-même traité par ce chemin. Rendre ce chemin borné (rejouer au plus 1 orphelin par requête, envoyer le reste en tâche dans la file, pousser les push en différé) coûte ~50 lignes et supprime le seul scénario de blocage durable des paiements.

Prompt de mise en place.

```
Contexte : cloudflare/src/index.js (route /paypal), cloudflare/src/paypal.js (recevoirWebhook, traiter, indexer, rejouerOrphelins), cloudflare/src/metier.js (fixerBudget, pousserA, differer, tache). Plafond Cloudflare gratuit : 50 sous-requêtes par exécution. Mesuré : jusqu'à 45 dans un seul webhook de test.
À faire :
1. index.js : pour /paypal (et /fn/*, /prospect), créer un compteur comme outils() et appeler M.fixerBudget() => 44 - o.compteur() avant recevoirWebhook.
2. paypal.js rejouerOrphelins : rejouer au plus UN orphelin dans une exécution de webhook. Pour les suivants, les déposer en sous-tâches {type:'tache', quoi:'orphelin_paypal', abo, k} via M.differer (une écriture), à traiter par planif.js/traiter (nouvelle branche qui appelle paypal.rejouerUnOrphelin(abo, k)), dans l'ordre de create_time.
3. Tous les envoyerPush du chemin PayPal (parrain, filleul, litige) : passer par M.pousserA / differer quand M.peutPousser() est faux, pour que le push parte à la minute suivante au lieu de faire tomber le webhook.
4. Si reste() < 8 avant une étape d'écriture critique, lever une erreur « budget » AVANT d'écrire quoi que ce soit (l'état reste 'en_cours', puis 503, puis PayPal renvoie) plutôt que d'échouer au milieu.
Tests : dans cloudflare/test/paypal.test.mjs, envelopper fetchImpl d'un compteur et affirmer `<= 46` pour CHAQUE appel de w.envoyer. Ajouter un test « trois orphelins puis le lien » : le webhook qui fait le lien reste sous 46, les deux autres orphelins passent en sous-tâches, et après 2 appels à minute() tout est joué dans l'ordre (paypal_premiers posé une seule fois, parrain crédité une seule fois). `cd cloudflare && npm test` tout vert.
Finis quand : aucun test PayPal ni remboursement ne dépasse 46 sous-requêtes par appel.
```

6. Notifications : messages de la nuit écrasés, pannes passagères perdues, heure de Paris pour tous — Priorité : Moyenne · Effort : M

Fusion. la priorité des notifications est définie par l'agent Engagement (point 1, table PUSH_PRIORITE). Réutiliser cette table pour push_attente au lieu d'en créer une seconde.

Constat. En heures calmes, envoyerPush fait push_attente<uid>.set(message) (metier.js:273) : il n'y a qu'une place. Pour le reproduire : réponse du coach au bilan à 22 h, puis « Nouveau défi » à 23 h. À 8 h 05, seul le défi part, et la réponse du coach est perdue (vérifié en simulation). Si un service de push répond 429 ou 5xx, le journal du jour est effacé (metier.js:298), mais le message n'est ni remis en file ni compté. Il n'y a pas de fuseau par utilisateur : paris() (metier.js:46) et heuresCalmes (metier.js:75) valent pour tous, alors que l'app ne stocke aucun fuseau (seul sess.tz, rc-core:49907). Le changement d'heure est bien géré (Intl, testé dans lignes/xp).

Pourquoi le mettre en place. La règle « une notification par jour » rend chaque notification précieuse : la perdre ou envoyer la mauvaise se voit tout de suite. La réponse du coach est la notification la plus utile au produit : c'est la preuve du suivi humain que vendent TrueCoach et Everfit.

Intérêt réel. Garder une file de priorités pour la nuit (coach > défi > reste) et réessayer une fois sur 429/5xx rattrape les pertes silencieuses. Le fuseau par utilisateur ouvre les athlètes des DOM, du Québec, de Suisse ou de Dubaï, clientèle naturelle d'un coach francophone en ligne : un athlète à La Réunion reçoit aujourd'hui « série en danger » à 20 h-23 h locales, et rien de 23 h à 10 h. Coût serveur nul : une lecture de champ déjà prévue dans l'index du point 1.

Prompt de mise en place.

```
Contexte : cloudflare/src/metier.js (paris, heuresCalmes, pushAutorise, envoyerPush, apresHeuresCalmes), cloudflare/src/planif.js, app/rc-core.1693.js (sauvegarde du dossier utilisateur).
À faire :
1. push_attente : remplacer le 'set' par une fusion par priorité. Stocker push_attente<uid> = {message, at, prio} et, dans une transaction, ne remplacer que si la nouvelle priorité est >= l'actuelle. PRIO = {coach:5, acces:4, prospect:4, filleul:3, defi:2, relance:2, serie:2, bilan:2, wrapped:1, badge:1, retour:1, sante:1}. Ajouter 'cumul' (le nombre de messages fusionnés) et, s'il vaut plus de 1, un corps « + N autres nouvelles » à l'envoi de 8 h 05.
2. envoyerPush : sur 429 ou 5xx de TOUS les appareils, ne pas perdre le message. S'il vient d'une tâche ou d'un événement, lever une erreur « push_transitoire » pour que planif.js le remette en file (essais+1, ESSAIS_MAX existant). Garder 404/410 = abonnement supprimé.
3. Fuseau : l'app écrit users<uid>/tz = Intl.DateTimeFormat().resolvedOptions().timeZone (validé par une regex IANA, repli 'Europe/Paris') à chaque démarrage si la valeur a changé. Côté Worker, paris(t) devient heureLocale(t, tz), avec un cache des Intl.DateTimeFormat par fuseau (aujourd'hui un formateur est construit à chaque appel, ~0,07 ms). heuresCalmes, pushAutorise et le jour du plafond utilisent le fuseau de l'athlète. Les travaux planifiés (serie, bilan, badge...) gardent leur heure de déclenchement en Paris mais n'envoient que si l'heure LOCALE de l'athlète est entre 8h et 21h ; sinon ils déposent dans push_attente.
4. Ne pas toucher aux règles de sécurité RTDB au-delà de l'ajout du champ tz (format).
Tests : cloudflare/test/push.test.mjs : deux messages de nuit, le coach survit au défi ; 503 de l'unique appareil, la tâche repart en file avec essais=1 et push_log non consommé. metier.test.mjs : tz='Indian/Reunion' à 19h Paris donne heures calmes ; 'America/Montreal' à 14h Paris donne 8h locales, envoi autorisé ; le changement d'heure du 25/10/2026 ne décale pas le jour du plafond. app/tests.js : l'écriture de tz, avec le repli sur une valeur invalide. `cd cloudflare && npm test` tout vert.
Finis quand : les tests passent et le README (« Règles d'envoi ») décrit la priorité et le fuseau.
```

7. Cloud Functions : ne pas passer Blaze pour elles, porter les deux qui manquent dans le Worker — Priorité : Moyenne · Effort : M

Constat. functions/index.js déclare ~30 fonctions, dont les planifiées pushSerieEnDanger, defisQuotidien, etc. (index.js:1106-1837) et les déclencheurs parrainageDemande, defiApresSeance, qui **doublent** le Worker. Seules deux n'ont pas d'équivalent et sont coupées côté app : ouvrirEssai et verifierAchatProgramme (FONCTIONS_SERVEUR=false, rc-core:102, :1666, :42257). L'essai et l'achat de programme vivent donc dans des champs que le client écrit. Le jour où quelq'un active Blaze, firebase.yml:194 déploie **tout** automatiquement : série, défis et parrainages traités deux fois, en parallèle du Worker (les fonctions lisent push_log sans transaction, functions/index.js:1202). Le README du Worker affirme aussi « Le Worker n'écrit jamais dans droits/ » (cloudflare/README.md:68), alors que majDroits y écrit (metier.js:238, :677, :717).

Pourquoi le mettre en place. Blaze pour les fonctions coûterait peu (voir chiffrage), mais créerait deux serveurs qui font la même chose : c'est une source de doubles notifications et de doubles crédits de parrainage. Le Worker sait déjà répondre au protocole onCall (/fn/<nom>, appels.js).

Intérêt réel. Chiffrage Blaze pour les **fonctions seules** : à 1 000 athlètes, ~0,5-1 €/mois (Cloud Scheduler : 8 tâches – 3 gratuites = 0,50 \$, Artifact Registry ~0,10-0,30 \$; invocations et calcul dans la franchise de 2 M/mois). À 10 000 athlètes, ~2-6 €/mois (defiApresSeance sur chaque écriture de sessions : ~3 M invocations, soit 1 M payant à 0,40 \$). Le vrai coût de Blaze serait la **base** (5 \$/Go stocké au-delà de 1 Go, 1 \$/Go téléchargé au-delà de 10 Go/mois), pas les fonctions. Porter ouvrirEssai et verifierAchatProgramme dans /fn/ coûte 0 €. Ça ferme la dernière porte « je m'ouvre un essai ou un programme depuis la console », qui touche directement le chiffre d'affaires de la boutique de programmes.

Prompt de mise en place.

```
Contexte : functions/index.js (ouvrirEssai l.981, verifierAchatProgramme l.917), cloudflare/src/appels.js (protocole onCall, jeton Firebase vérifié), cloudflare/src/index.js (APPELS = { clouidinaryDestroy, santeJeton }, cloudflare/src/metier.js (majDroits), cloudflare/src/paypal.js (lecture de commande /v2/checkout/orders, prix boutique déjà vérifié pour le webhook), app/rc-core.1693.js (FONCTIONS_SERVEUR l.102, appels l.1666 et l.42257).
À faire :
1. cloudflare/src/essai.js : ouvrirEssai({auth, data:{jours}}) : jours borné à ESSAI_JOURS (30, depuis tarifs.json), refus si droits<cle> a déjà un essai ou si dejaPaye(cle) ; écriture par majDroits(palier:'ultime', echeance, source:'essai', essaiFinis). Idempotent (un second appel renvoie l'échéance existante).
2. verifierAchatProgramme({auth, data:{orderId, programmeId}}) : relire la commande chez PayPal (réutiliser le client de paypal.js), vérifier statut COMPLETED, custom_id '<cle>|<programme>', montant = prix boutique ; écrire droits<cle>/programmes/<id> = {jusqua}. Réutiliser la logique du webhook pour ne pas la dupliquer.
3. Les ajouter à APPELS dans index.js. Dans rc-core, remplacer FONCTIONS_SERVEUR par une liste FONCTIONS_WORKER =
```

```
[ 'ouvrirEssai', 'verifierAchatProgramme', 'cloudinaryDestroy', 'santeJeton' ] et appeler si SERVEUR_LEGER. Ne pas modifier les règles RTDB (autre chantier).
4. functions/README.md : en tête, « NE PAS DÉPLOYER : doublerait le Worker ». Liste de ce qui a été porté et de ce qui est abandonné. firebase.json : retirer le codebase functions (ou "ignore": ["**"]) pour qu'un passage en Blaze ne déploie rien par accident.
5. cloudflare/README.md l.68 : corriger (« le Worker écrit droits/ par majDroits, en transaction, et ne réécrit jamais un accès posé à la main »).
Tests : cloudflare/test/droits.test.mjs : essai ouvert une fois, refusé à un payant, idempotent ; achat vérifié ouvert, montant faux refusé, commande d'un autre compte refusée. app/tests.js : l'app appelle bien /fn/ouvrirEssai quand SERVEUR_LEGER est vrai. `cd cloudflare && npm test` puis tests front.
Finis quand : les deux fonctions répondent sur /fn/ et aucun chemin de déploiement ne publie functions/.
```

8. Cible d'architecture pour 1 000 puis 10 000 athlètes — Priorité : Moyenne · Effort : L

Constat. Les plafonds actuels, du plus bas au plus haut : ~180 athlètes (rappel série), ~2 000 abonnés (accès + retour), ~6 événements par minute dans la file (cron + /veille, 2-3 événements par exécution à cout 12), ~12 000 athlètes pour les 100 000 appels par jour. /veille n'a aucune limite de débit (index.js:115-119, contrairement à /arrivee) : une boucle de requêtes suffit à épuiser le quota quotidien et à couper la suite, webhooks PayPal compris. Côté base, les limites Spark (1 Go stocké, 10 Go/mois téléchargés, 100 connexions simultanées) arriveront avant 10 000 athlètes.

Pourquoi le mettre en place. Il faut décider maintenant de la trajectoire pour que chaque lot aille dans le même sens, et ne pas découvrir les plafonds en production quand un coach amène 300 athlètes d'un coup.

Intérêt réel. Mon avis : - **Jusqu'à ~1 000 athlètes** : rester sur Cloudflare Free + Spark, avec les points 1, 2, 3 et 5 faits, et une limite de débit sur /veille. Coût : 0 €. - **Dès ~800 athlètes actifs ou au premier coach à plus de 150 athlètes : Workers Paid à 5 \$/mois.** Il donne 10 M d'appels, 30 s de calcul et ~1 000 sous-requêtes par exécution (20 fois plus), plus **Cloudflare Queues** : chaque push devient un message de file traité dans sa propre exécution, ce qui supprime la gymnastique tâche/budget. Un rappel série à 10 000 athlètes part en ~15 minutes. Total estimé : 5-6 \$/mois à 10 000 athlètes (Queues : ~2 M opérations par mois). - **Vers 3 000-10 000 athlètes : Blaze pour la base seulement,** pour les sauvegardes quotidiennes intégrées et la levée des plafonds, avec une alerte de budget à 20 €. Estimation : 10-60 €/mois, dominés par la bande passante de synchronisation (à mesurer dans Console > Usage). On garde la RTDB comme source de vérité et le Worker comme seul serveur. On ne déploie pas les Cloud Functions. Au total, ~50 €/mois à 10 000 athlètes, contre 1 à 2 abonnements Essentielle pour 100 athlètes : marge intacte.

Prompt de mise en place.

```
Contexte : cloudflare/ (Worker gratuit, cron chaque minute, file /evenements + sous-tâches `tache`, BUDGET=38, MAX_CHIFFREMENTS=5). Objectif : préparer le passage Workers Paid + Queues sans casser le fonctionnement gratuit actuel.
À faire, en deux temps :
A. Tout de suite (gratuit) :
  1. index.js /veille : appliquer LIMITE_ARRIVEES (ou un binding LIMITE_VEIL : 6 appels / 60 s par IP) et n'accepter que POST ; 429 au-delà.
  2. Rendre les plafonds configurables par variables : BUDGET, MAX_CHIFFREMENTS et COUT_PUSH lus depuis env (défauts = valeurs actuelles), passés par outils(env) à minute() et creerMetier(). Plan payant : BUDGET=900, MAX_CHIFFREMENTS=200.
  3. Écrire docs/capacite.md avec les plafonds mesurés (série 180, accès/retour ~2 000, file ~6 évt/s/min, quota 100k/j ≈ 12 000 athlètes) et la règle de bascule (≥ 800 athlètes actifs, ou un travail `sautes` > 0 deux jeudis de suite).
B. Préparé derrière un drapeau (env.FILE_PUSH === 'queue') :
  4. wrangler.toml : [[queues.producers]] binding = "PUSHES", queue = "repcore-push" ; [[queues.consumers]] queue = "repcore-push", max_batch_size = 20, max_retries = 5, dead_letter_queue = "repcore-push-ko".
  5. metier.js differer()/pousserA() : si env.PUSHES existe, `env.PUSHES.sendBatch(taches.map(body => ({body})))` au lieu d'écrire dans /evenements. Handler `queue(batch, env)` dans index.js qui appelle envoyerPush pour chaque message, `msg.retry()` sur 429/5xx, `msg.ack()` sinon.
  6. Les travaux planifiés (serie, bilan, acces, retour...) : lire les athlètes par pages de 500 et ENQUEUEUR un message par athlète au lieu de les traiter dans le cron.
Tests : cloudflare/test/charge.test.mjs, un scénario « queue » avec une fausse file en mémoire : 10 000 abonnés, série servie à 100 % en moins de 30 réveils, aucun doublon ; les tests existants passent sans drapeau (mode gratuit inchangé). `cd cloudflare && npm test`.
Finis quand : le mode gratuit est inchangé et vert, le mode queue passe le test des 10 000, et docs/capacite.md dit quand et comment basculer (5 $/mois).
```

Agent 8 — Performance et architecture front · 4/10

Les bases sont bonnes : actifs versionnés et immutable (firebase.json, build 1417), bibliothèques lourdes chargées seulement quand on s'en sert (tesseract rc-core:112893, pdf.js :116709, zxing :92938, motion-lab :107857, ciqual :91004), écouteurs globaux protégés contre l'empilement et createObjectURL / revokeObjectURL appariés (17/17). Mais **rien n'est minifié** (7,25 Mo de JS dont 2,5 M caractères de commentaires, soit 35 %). Le **service worker retélécharge environ 7 Mo par appareil à chaque build** (21 builds le 29/09), l'écran reste **noir jusqu'à l'exécution de rc-core** (13,2 s en 4G lente), et aucun contrôle de code ne bloque la mise en ligne. Mesurées ici, trois vraies erreurs (ReferenceError ou fonction écrasée) sont en production.

Mesures (Chromium 140 Playwright, serveur local brotli q5, viewport 393×851, CPU ×4, service worker bloqué pour la mesure à froid, 3 passages) :

Scénario	Octets transférés	FCP = fin de l'exécution de rc-core
Actuel, 4G médiane (9 Mb/s, RTT 170 ms)	2,57 Mo	2,8 s
Minifié (esbuild + commentaires HTML retirés), 4G médiane	1,35 Mo	1,7 s
Actuel, 4G lente (1,6 Mb/s, RTT 150 ms)	2,57 Mo	13,2 s
Minifié, 4G lente	1,35 Mo	7,1 s
Réouverture avec service worker, CPU ×4	110 Ko (index) + 3 × sw.js	0,25 à 0,45 s

Poids mesurés en octets (brut → gzip -9 → brotli 11) : - rc-core : 7 247 931 → 2 441 099 → 1 760 453. Minifié : 3 878 652 → 1 211 035 → 893 688. - rc-style : 1 618 065 → 287 372 → 213 908. Minifié : 1 248 268 → 144 759 → 105 693. - index.html : 473 747 → 121 837 → 96 092, dont 139 540 de commentaires HTML. - motion-lab : 657 173 → 199 651 → 160 072. Minifié : 361 828 → 92 033 en brotli.

Le coût CPU mesuré par trace reste modeste : analyse JS en arrière-plan 120 ms, EvaluateScript 55 ms, ParseAuthorStyleSheet 64 ms. **Le goulot est le réseau, pas le processeur.**

1. Le service worker retélécharge tout à chaque build — Priorité : Critique · Effort : M

Constat. À l'installation, app/sw.js:222 fait `c.add(new Request(a,{cache:'reload'}))` sur les 141 entrées d'ASSETS (sw.js:170). Cela fait 11,9 Mo bruts, soit environ 4,8 Mo transférés, et l'opération contourne le cache HTTP. L'install recommence donc à chaque build, même pour les images qui n'ont pas changé. J'ai mesuré 153 requêtes et 7,41 Mo au premier passage, dont rc-core.1693.js **téléchargé deux fois** (2 × 2,08 Mo) : une fois par la page, une fois par l'install. Le report de cache dans activate (sw.js:270-440) arrive trop tard, les octets sont déjà partis. `git log` compte 21 builds le 29/09/2026 et 8 le 28/09. Enfin, la réponse réseau d'index.html est mise en cache sans vérifier `r.ok` (sw.js:516-520) : une page 5xx ou 404 écrase la copie hors ligne.

Pourquoi le mettre en place. Le plan Spark de Firebase Hosting plafonne à 360 Mo par jour, et la limite est déjà citée dans firebase.json et dans scripts/versionner_actifs.py. Avec 7 Mo par appareil et par build, environ 50 mises à jour d'appareils suffisent à atteindre le plafond, et le site entier tombe en 429 pour tout le monde. Chaque athlète paie aussi ces 7 Mo sur son forfait mobile.

Intérêt réel. Une mise à jour ne devrait coûter que ce qui change : index.html (96 Ko), rc-core (0,9 Mo une fois minifié) et rc-style (0,1 Mo), soit environ 1,1 Mo au lieu de 7 Mo (-85 %). Le quota passe d'environ 50 à plus de 300 appareils mis à jour par jour, sans passer au plan Blaze. Le premier passage tombe de 7,4 Mo à environ 3,5 Mo, parce que rc-core ne part plus deux fois. C'est surtout ce qui permet de livrer 20 builds par jour sans « casser » l'app pour les derniers arrivés de la journée. Un coach avec 30 athlètes, qui ouvrent l'app deux fois par jour après chaque livraison, ne consomme plus le quota à lui seul. Côté athlète, on évite 150 Mo de données mobiles un jour de livraisons en rafale.

Prompt de mise en place.

```
Contexte : app/sw.js (service worker de RepCore, PWA). À l'install, il fait c.add(new Request(a,{cache:'reload'})) sur les 141 entrées d'ASSETS (sw.js -l.222). Tout est donc retéléchargé à chaque build (environ 4,8 Mo transférés), y compris des images inchangées. rc-core.<build>.js part deux fois au premier passage : la page le demande, puis l'install le redemande. Le quota Firebase Hosting Spark est de 360 Mo par jour.
```

À faire dans app/sw.js :

- Dans le handler install, pour chaque entrée d'ASSETS, commence par chercher une réponse existante dans les anciens caches (caches.keys() filtré sur `/^repcore-v\d+$/`, du plus récent au plus ancien) :
 - Actif versionné par son nom (`/rc-(core|style)\.d+\.(js|css)$/`) : même URL = même contenu. Recopie-le s'il existe (la page vient souvent de le télécharger dans l'ancien cache). Sinon, fetch sans cache : 'reload' : le cache HTTP immutable suffit.
 - Images, polices et vendor/ (tout ce qui n'est ni index.html ni un actif rc-*) : recopie la réponse existante si elle a moins de 30 jours (lis l'en-tête 'date'). Sinon, fetch normal.
 - index.html : garde cache : 'reload' (pour lui, c'est voulu, voir le commentaire du 27/09).Conserve Promise.allSettled : un échec ne doit pas bloquer l'install.
- Dans le handler fetch, branche index.html : ne fais `c.put('./index.html', clone)` que si `r.ok && r.status===200 && (r.headers.get('content-type')||'').includes('text/html')`.
- Ajoute en tête de sw.js la constante `STATIC_TTL_MS` (30 jours) et commente la règle.
- Ne touche pas au mécanisme séance-en-cours (`SEANCE_CLE`), ni à `PURGE_EXERCICES`, ni à la purge des versions (`_gardees`).

Tests :

- Étends scripts/verif/worker-actifs.mjs (le banc qui simule caches et activate) : sème un cache repcore-v1000 qui contient rc-core.1001.js et les images, installe un worker `CACHE='repcore-v1001'`, et vérifie que fetch n'a été appelé que pour index.html et pour les actifs absents. Ajoute un cas où une réponse 503 sur index.html ne remplace pas la copie en cache.
 - Ajoute dans app/tests.js une assertion de source : le handler install ne contient plus "cache : 'reload'" hors de la branche index.html.
 - Lance `node scripts/verif/worker-actifs.mjs` et `node scripts/verif/syntaxe.mjs`.
- Critère de fin : sur un Chromium local (python3 -m http.server à la racine du dépôt), un changement de build (incrémenter `RC_BUILD` et `CACHE` avec scripts/versionner_actifs.py sur une copie) ne retélécharge que index.html, rc-core, rc-style et sw.js. Vérifie-le dans l'onglet Réseau ou avec un compteur de requêtes côté serveur.

2. Aucune minification au déploiement — Priorité : Critique · Effort : S

Constat. .github/workflows/firebase.yml copie app/ tel quel (`cp -a app ... _site/`) : il n'y a aucune étape de build. rc-core.1693.js contient 2,5 M caractères de commentaires sur 7,09 M, et index.html 139 Ko de commentaires HTML. Avec esbuild (`--minify`, 0,96 s), rc-core passe de 1,76 Mo à 0,89 Mo en brotli. Mesuré en 4G médiane et CPU ×4, le FCP passe de 2,8 s à 1,7 s, et de 13,2 s à 7,1 s en 4G lente. Point d'attention : 1 210 assertions de app/tests.js lisent le source avec `String(fonction)`, et le code applicatif en fait autant (`assertNoCycleRuleAltersSeries`, rc-core:43914). Il faut donc minifier **uniquement** _site, jamais le dépôt.

Pourquoi le mettre en place. C'est le gain le plus fort pour l'effort le plus faible : aucune ligne de code applicatif ne change, et les commentaires qui font la mémoire du projet restent dans le dépôt. Tout nouvel athlète, qui ouvre l'app depuis le lien Instagram en 4G, gagne 1,1 s en conditions médianes et 6 s en réseau faible.

Intérêt réel. Le premier contact est le moment où un prospect décide de rester ou non. Un écran noir de 13 s en 4G lente (salle de sport en sous-sol) équivaut à un abandon. La minification en retire 6 s. Elle réduit aussi de moitié la bande passante Hosting consommée par chaque build (rc-core : -0,87 Mo par appareil) et se cumule avec le point 1. Le tas JS au démarrage passe de 19 Mo à 13 Mo (mesuré), ce qui compte sur un Android à 3 ou 4 Go de RAM où Chrome tue les onglets en arrière-plan. Face à Trainerize ou Hevy Coach, apps natives qui s'ouvrent instantanément, c'est la condition pour que la PWA ou la TWA ne paraisse pas « lente ».

Prompt de mise en place.

Contexte : le dépôt RepCore sert app/ sans aucun build. .github/workflows/firebase.yml assemble _site par cp -a puis déploie. app/rc-core.<build>.js fait 7,2 Mo, dont 35 % de commentaires, et app/rc-style.<build>.css 1,6 Mo. app/tests.js et assertNoCycleRuleAltersSeries (rc-core) lisent le SOURCE des fonctions avec String(fn) : on ne minifie donc QUE la copie _site, jamais les fichiers du dépôt.

A faire :

1. Crée scripts/minifier_site.mjs (Node 22, sans dépendance installée dans le dépôt : npx -y esbuild@0.25 depuis le workflow). Il :
 - minifie en place _site/app/rc-core.*.js et _site/app/motion-lab.js avec esbuild : --minify-whitespace --minify-syntax, SANS --minify-identifiers (les noms globaux sont appelés depuis des onclick="..." dans le HTML et dans des chaînes innerHTML), --target=es2020, --legal-comments=none ;
 - minifie _site/app/rc-style.*.css (esbuild --loader=css --minify) ;
 - retire de _site/app/index.html les commentaires HTML <!-- ... --> (sauf les conditionnels <!--[]), et les commentaires // en début de ligne dans les <script> en ligne SEULEMENT si tu peux le faire sans casser les chaînes. Sinon, laisse les scripts en ligne tels quels ;
 - affiche avant/après en octets bruts, gzip et brotli (node:zlib), et échoue (exit 1) si un fichier minifié ne se parse plus : new Function() pour le JS, comme scripts/verif/syntaxe.mjs.
 2. Dans .github/workflows/firebase.yml, ajoute une étape « Minifier _site » APRÈS « Assembler _site » et AVANT « Vérifier que le code se parse », puis fais tourner node scripts/verif/syntaxe.mjs _site/app/index.html (le script accepte un chemin en argument) sur la version minifiée.
 3. Exclue aussi app/tests.js de _site (rm -f _site/app/tests.js après la copie) : 5,3 Mo publics qui ne servent qu'en local. Vérifie d'abord que scripts/verif/suite.mjs sert bien le dépôt local et non la production.
 4. Vérifie que l'assertion d'app/tests.js qui compare RC_BUILD et CACHE ne lit pas le fichier déployé.
- Cas limites : assertNoCycleRuleAltersSeries applique String(getCycleFactor).match(/seriesReduce\s*:\s*[0-9]+\s*/). Garder --minify-syntax conserve les propriétés : vérifie que la fonction rend encore {ok:true} sur la version minifiée, en chargeant _site/app/index.html dans Chromium (/opt/pw-browsers ou le Chrome du runner) et en évaluant assertNoCycleRuleAltersSeries().
- Critère de fin : le workflow passe, et _site/app/rc-core.*.js pèse moins de 4 Mo bruts et moins de 1 Mo en brotli. Un chargement local de _site/app/index.html n'affiche aucune erreur en console et atteint l'écran s-install ou s-welcome.

3. Écran noir jusqu'à l'exécution des 7 Mo de rc-core — Priorité : Haute · Effort : M

Constat. L'écran de démarrage #s-splash est bien dans le HTML (index.html:565), mais le logo est en opacity:0 (splash-logo-anim, splash-halo, g[data-bolt]), et seul rc-core le révèle par ses animations (rc-core:6820-6836). Mesuré : first-paint à 3,35 s (4G lente), un fond carbone noir sans rien dessus, puis FCP à 13,2 s. Le FCP est égal au DCL, qui est égal à la fin du téléchargement de rc-core plus 120 ms. En plus, rc-style (214 Ko en brotli) bloque le rendu, et vendor/qr.js est un script synchrone dans le <head> (index.html:418).

Pourquoi le mettre en place. L'utilisateur doit voir la marque tout de suite, même si le code n'est pas encore là. Un logo visible en moins d'une seconde change la perception (« ça charge ») par rapport à « c'est cassé ». Une animation en CSS pur ne dépend d'aucun script.

Intérêt réel. En 4G médiane, le premier affichage utile passerait de 2,8 s (1,7 s après minification) à environ 0,5 s : il suffit du HTML et d'un petit CSS critique en ligne. En 4G lente, on passe de 13 s d'écran noir à environ 1,5 s avec le logo animé, pendant que le code arrive. Le parcours d'installation du point de vente (/app/#install, lien du QR et d'Instagram) est le plus sensible, car ces visiteurs n'ont encore rien en cache. C'est gratuit en quota et sans risque fonctionnel, puisque rc-core reprend la main exactement comme aujourd'hui.

Prompt de mise en place.

Contexte : app/index.html contient #s-splash (vers la l.565). Son logo (#splash-logo-anim), son halo (#splash-halo) et l'éclair (g[data-bolt]) sont en style="opacity:0", et ne deviennent visibles que quand app/rc-core.<build>.js leur pose des animations (fonction autour de rc-core l.6820 : rcStrike, rcHalo, boltFlash, splashLineIn, splashTextIn). La feuille rc-style (1,6 Mo) bloque le rendu, et rc-core est un <script> synchrone en fin de <body>. Conséquence mesurée : écran noir jusqu'à la fin de l'exécution de rc-core (2,8 s en 4G, 13 s en 4G lente).

A faire :

1. Dans le <style> en ligne de index.html (celui des @font-face, ~l.423), ajoute un CSS critique minimal : fond du splash (reprenant les valeurs de .carbon-bg et .screen.active de rc-style), mise en page centrée, et les @keyframes rcStrike et rcHalo copiées depuis rc-style. Applique-les en CSS pur, sans JS, sur #splash-logo-anim et #splash-halo (animation: rcStrike 420ms ... both). Retire le style="opacity:0" en ligne de ces deux éléments.
 2. Dans rc-core, garde la pose des animations, mais rends-la idempotente : si l'élément a déjà une animation CSS (getComputedStyle(el).animationName !== 'none'), ne la relance pas, pour éviter un double flash.
 3. Passe <script src="/vendor/qr.js"> en defer. Vérifie avant que rien ne l'appelle de façon synchrone au chargement : grep RepCoreQR dans app/rc-core.*.js. Les appels sont protégés par window.RepCoreQR&&...
 4. N'ajoute PAS defer à rc-core : les scripts en ligne et l'ordre d'initialisation (DOMContentLoaded dans rc-core l.118157+) en dépendent. Ce sera une étape ultérieure.
 5. Le thème clair (data-theme="clair" posé par le premier script) doit rester correct : le CSS critique utilise var(--bg) avec un repli.
- Tests : ajoute dans app/tests.js une assertion qui vérifie que #splash-logo-anim n'a plus d'opacity:0 en ligne dans le HTML servi (fetch('./index.html')), et que la règle @keyframes rcStrike existe dans un <style> en ligne. Mesure avant/après avec Playwright (Chromium dans /opt/pw-browsers), réseau limité par CDP (Network.emulateNetworkConditions 1,6 Mb/s, 150 ms) et service worker bloqué : relève performance.getEntriesByType('paint').
- Critère de fin : first-contentful-paint < 1,5 s en 4G lente simulée (contre 13 s aujourd'hui), capture d'écran à 2 s montrant le logo, et suite node scripts/verif/suite.mjs sans nouvel échec par rapport à la base.

4. Feuille de style : 44 % de sélecteurs [style*=...] pour le thème clair — Priorité : Haute · Effort : M

Fusion. complémentaire du point 1 de l'agent Design (contrastes du thème clair) : corriger d'abord le générateur, puis sortir le bloc clair dans son propre fichier.

Constat. rc-style.1693.css fait 1,6 Mo et 15 444 lignes. On y trouve 4 878 occurrences de :root[data-theme="clair"] et 10 446 sélecteurs d'attribut [style*=...]/[style^=...], pour 707 Ko de règles, soit 44 % du fichier. Exemple de ligne de 5 913 caractères : :root[data-theme="clair"] [style^="color:#000"]:not([style="background:var(--red)"]):not(...). Ils rattrapent les 1 415 style="..." en ligne d'index.html et les quelque 4 268 de rc-core. Or le thème par défaut est sombre (index.html:49-53) : ces 707 Ko sont analysés à chaque ouverture pour rien.

Pourquoi le mettre en place. Les sélecteurs d'attribut en sous-chaîne ne peuvent pas être filtrés par Blink et s'évaluent sur chaque élément à chaque recalcul de style. Chaque nouveau style en ligne ajouté dans rc-core exige une nouvelle exception ici : c'est une dette qui grossit à chaque lot.

Intérêt réel. Charger rc-theme-clair.css seulement si le thème clair est actif économise environ 100 Ko en brotli et environ 40 % de l'analyse CSS (64 ms mesurés à CPU x4) pour la majorité en thème sombre. Côté recalcul, j'ai mesuré en thème clair 20 navigations go() en 324 ms avec ces règles contre 279 ms sans (-14 %, CPU x4). À moyen terme, remplacer les styles en ligne par des classes et des jetons (var(--text)) supprime la cause : le thème clair devient un simple jeu de variables, comme chez Hevy ou Everfit. Cela évite aussi les régressions visuelles silencieuses quand un style en ligne ne correspond plus au motif d'exception.

Prompt de mise en place.

Contexte : app/rc-style.<build>.css (1,6 Mo) contient environ 707 Ko de règles préfixées par :root[data-theme="clair"], dont la plupart ciblent des attributs [style="..."]. Le thème par défaut est sombre. data-theme="clair" est posé très tôt par le premier <script> d'app/index.html (clé localStorage rc_theme), et themeAppliquer (rc-core) le change à chaud. scripts/versionner_actifs.py renomme rc-style.<build>.css à chaque build et réécrit index.html et la liste ASSETS de app/sw.js.

Étape 1 (ce prompt) : sortir le thème clair dans un fichier à part.

1. Écris scripts/extraire_theme_clair.py : il lit rc-style.<build>.css, déplace chaque règle (et chaque bloc @media qui en contient) dont TOUS les sélecteurs commencent par :root[data-theme="clair"] vers app/rc-theme.<build>.css, et laisse le reste en place. Les règles mixtes restent dans rc-style. Il affiche les tailles avant/après. Il est idempotent.

2. Dans index.html, charge rc-theme.<build>.css avec <link rel="stylesheet" id="rc-theme-clair" href="/rc-theme.<build>.css" media="not all">, et dans le premier script (celui qui pose data-theme), passe media à 'all' quand le thème clair est actif. Dans themeAppliquer (rc-core), fais la même bascule.

3. Mets à jour scripts/versionner_actifs.py pour renommer aussi rc-theme.<build>.css, firebase.json (la règle /app/rc-*.@(js|css) le couvre déjà : vérifie-le), la liste ASSETS de sw.js, et scripts/verif/actifs.mjs.

4. Garde l'ordre de cascade : rc-theme doit venir APRÈS rc-style.

Tests : dans app/tests.js, bascule le thème clair et vérifie que getComputedStyle(document.body).backgroundColor change, et qu'un élément témoin (par exemple .card) prend bien la couleur claire. Lance node scripts/verif/suite.mjs et compare le nombre d'échecs à la base (git stash). Contrôle visuel : capture de s-welcome et s-client-home en clair avant/après avec scripts/verif/capture.mjs, les images doivent être identiques.

Critère de fin : rc-style < 950 Ko bruts, aucune différence visuelle en clair ni en sombre, et le fichier thème n'est pas appliqué (media="not all") en thème sombre.

Étape suivante (à noter en commentaire, pas à faire ici) : remplacer progressivement les style="color:#000;background:..." de rc-core par des classes et des jetons, écran par écran, en supprimant l'exception correspondante dans rc-theme.

5. Pas d'analyse statique ni de suite de tests en CI : trois bugs réels en production — Priorité : Haute · Effort : S

Fusion. l'étape 5 (suite front en CI) est couverte par l'agent QA (point 1). Ne réaliser ici que les étapes 1 à 4 (trois bugs, lint, types).

Constat. Aucun ESLint dans le dépôt. types.yml ne vérifie que motion-lab, **il est rouge depuis le build 1682** (4 erreurs : tempoLu, exKey absents de app/globaux.d.ts, _ml possiblement null) et il est non bloquant par choix. La suite app/tests.js (5,3 Mo, environ 3 700 assertions) ne se lance qu'à la main, depuis la console (chargerTests(), rc-core:123100) ou par scripts/verif/suite.mjs : aucun workflow ne l'appelle. J'ai lancé eslint (no-undef, no-redeclare) sur l'ensemble des scripts ; il trouve : - **offRecherche** (rc-core:93309) : impossibles.push(...) vise une variable jamais déclarée (la variable locale s'appelle rejets). Dès qu'une fiche Open Food Facts est invalide, on a une ReferenceError, le catch l'attrape, et l'athlète lit « Recherche impossible : impossibles is not defined. » Pour reproduire : Nutrition → rechercher « chocolat » → « Chercher un produit de marque ». - **echOuvrirDialogue** et **fermerEcheance** (rc-core:66990, :67060) appellent renderClientDetail(), qui n'existe pas. L'erreur est avalée par try{}catch{}, et la fiche coach n'est pas rafraîchie après l'ouverture d'une échéance. - **motion-lab.js** déclare deux fois mlAnnulerTrace (:7451 et :9619). La seconde, qui réinitialise la trajectoire, écrase la première. Le bouton « Annuler » d'un tracé en cours (:7356, :7360) ne vide donc pas _ml.trace.

Pourquoi le mettre en place. Dans un fichier de 123 000 lignes où tout est global (5 093 fonctions, 1 811 constantes au niveau racine), une faute de nom ne se voit qu'à l'exécution, et les try{}catch(e){} omniprésents la rendent muette. Une règle no-undef bloquante coûte 30 secondes par build et aurait arrêté ces trois bugs.

Intérêt réel. La recherche de produits de marque est une fonction clé face à MyFitnessPal et Trainerize, et elle échoue aujourd'hui dès qu'une seule fiche est incomplète, ce qui arrive sur la plupart des recherches Open Food Facts. Le coach voit une fiche périmée après avoir posé une échéance. Lancer la suite existante en CI rentabilise 5 Mo de tests déjà écrits : 21 builds par jour sans filet, c'est 21 occasions de régression. Coût : environ 2 minutes de runner GitHub par poussée, sur un quota gratuit largement suffisant.

Prompt de mise en place.

Contexte : RepCore n'a aucun lint. Tous les scripts de la page (scripts en ligne d'app/index.html, app/vendor/qv.js, app/vendor/rc-video.js, app/rc-core.<build>.js, app/motion-lab.js) partagent la même portée globale. La suite app/tests.js n'est lancée qu'à la main (scripts/verif/suite.mjs via le protocole DevTools). .github/workflows/types.yml échoue depuis le build 1682.

1. Corrige les trois bugs trouvés :

a. app/rc-core.<build>.js, fonction offRecherche (~l.93309) : impossibles.push({nom:a.n,raison:v.raison}) → rejets.push(...) (la variable déclarée juste au-dessus est 'const l=[],rejets=[]'). Test dans app/tests.js : remplace temporairement _offFetch par un faux qui rend un hit valide et un hit sans nutriments, puis vérifie que offRecherche('x') rend {ok:true, liste.length===1, rejets===[]}. Restaure _offFetch.

b. rc-core echOuvrirDialogue (~l.66990) et fermerEcheance (~l.67060) : renderClientDetail n'existe pas. Cherche la fonction qui repeint la fiche athlète côté coach (openClientDetail(cid,refresh,force) vers l.30230) et appelle-la avec l'identifiant du client ouvert et _refresh=true. Test : après ouvrirEcheance simulé, la fonction de rendu a bien été appelée (espion).

c. app/motion-lab.js : il y a deux fonction mlAnnulerTrace (l.7451 : annule le tracé d'annotation, l.9619 : sort du mode trajectoire). Renomme la seconde en mlAnnulerTrajectoire et mets à jour ses appelants (grep sur le HTML généré dans motion-lab.js). Vérifie que les boutons « Annuler » du tracé en cours (l.7356, l.7360) appellent bien la version qui fait _mlxAnnulerTrace().

2. Crée scripts/verif/lint.mjs : il concatène, dans l'ordre de la page, les scripts en ligne d'index.html, vendor/qv.js, vendor/rc-video.js, rc-core.<build>.js et motion-lab.js dans un fichier temporaire, puis lance npx -y eslint@9 avec une config plate en ligne : sourceType 'script', globals navigateur (npx -y -p globals) plus Tesseract, pdfjsLib, Pose, MP4Box, Mp4Muxer, paypal, RepCoreQR, et les règles no-undef, no-redeclare et no-dupe-keys en 'error'. Il ramène chaque erreur à fichier:ligne d'origine et sort en 1 s'il en trouve.

3. Ajoute l'étape « Lint » dans .github/workflows/firebase.yml, BLOQUANTE, juste après « Verifier que le code se parse ».

4. Répare types.yml : ajoute tempoLu(txt:string):any et exKey(nom:string):string dans app/globaux.d.ts, et protège _ml à motion-lab.js:l639. npx -y -p typescript@5.9 tsc -p tsconfig.json doit rendre 0 erreur.

5. Crée .github/workflows/suite.yml : sur push et pull_request, lance python3 -m http.server 8799 à la RACINE du dépôt, démarre le Chrome du runner (google-chrome --headless-new --remote-debugging-port=9223 ; si Page.captureScreenshot bloque, bascule en --headless=old comme le décrit scripts/verif/README.md), puis node scripts/verif/suite.mjs "http://127.0.0.1:8799/app/index.html". Enregistre la liste des échecs attendus hors navigateur réel dans scripts/verif/echecs-attendus.json et fais échouer le job seulement si un échec n'y figure pas.

Critère de fin : lint à 0 erreur, tsc à 0 erreur, et le job suite vert avec la liste d'échecs attendus figée.

6. DB.get('users') relit et reparse tout le stockage local à chaque appel — Priorité : Moyenne · Effort : M

Constat. DB.get (rc-core:5661) fait JSON.parse(localStorage.getItem('rc'+k)) et aplattit sessions_config à chaque appel. On compte 217 appels DB.get(), dont 212 sur 'users'. saveUser (rc-core:115417) enchaîne une lecture complète, deux JSON.stringify (users et session) et un CLOUD.push, et elle est appelée 311 fois dans la code. Mesuré dans la page, CPU x4, avec un rc_users synthétique de 3 Mo (20 athlètes × 120 séances) : 67 ms par lecture, 181 ms par lecture plus écriture. À 60 athlètes (9 Mo), setItem dépasse le quota.

Pourquoi le mettre en place. C'est le coût processeur principal une fois l'app chargée : chaque validation de série ou chaque ouverture de fiche bloque le fil principal de 100 à 250 ms sur un Android moyen de gamme chez un coach bien rempli. Ça se ressent au toucher sous forme d'à-coups.

Intérêt réel. Un cache mémoire (objet parsé gardé tant que la chaîne en stockage n'a pas changé) ramène la lecture à moins de 1 ms. La saisie en séance (renderSets, finishWorkout) devient fluide, et l'ouverture d'une fiche coach ne se fige plus. Plus un coach a d'athlètes, plus le gain est grand : c'est précisément le client qui paie 19 à 39 € par mois. C'est aussi un préalable pour sortir users du localStorage vers IndexedDB sans réécrire les 217 appels.

Prompt de mise en place.

Contexte : app/rc-core.<build>.js, objet DB (~l.5661). DB.get(k) fait JSON.parse(localStorage.getItem('rc'+k)) à CHAQUE appel, puis _aplatirTousSessionsConfig ou _aplatirSessionsConfig. Il y a 217 appels, dont 212 sur 'users', et saveUser (~l.115417) relit et réécrit tout. Mesuré : 67 ms

par `DB.get('users')` pour 3 Mo à CPU x4. Ne touche PAS à la logique de synchronisation (CLOUD.*) : on optimise seulement la lecture locale.

À faire :

1. Ajoute un cache mémoire dans DB : `_memo = new Map(clé → {brut:string, val:any})`. Dans `get(k)`, lis `brut = localStorage.getItem('rc'+k)`. Si `_memo` a la même chaîne brute (comparaison `===` sur la chaîne, 0(1) si c'est la même référence renvoyée), rends un CLONE PROFOND de `val` (`structuredClone`) et non la référence : de nombreux appelants modifient l'objet rendu puis appellent `DB.set`, ou pas. Mesure `structuredClone` : s'il coûte plus de 30 % du `JSON.parse`, garde le `parse` mais évite quand même l'aplatissement répété.
 2. Dans `setLocal(k,v)` et `del(k)`, mets à jour ou invalide `_memo`. Écoute `window 'storage'` (autre onglet) pour invalider la clé concernée.
 3. Dans `saveUser`, le `CLOUD.push(users)` final double celui de `DB.set('users')` (qui pousse déjà) : ne le supprime PAS, signale-le seulement en commentaire à l'équipe synchro.
 4. Instrumente : en mode `?perf=1`, `console.log` du temps cumulé de `DB.get` par seconde.
- Tests dans `app/tests.js` :
- `DB.get('users')` renvoyé deux fois donne deux objets distincts (`a!==b`) mais `deepEqual` ;
 - modifier l'objet rendu ne change pas le suivant `DB.get` ;
 - `DB.set` puis `DB.get` rend la nouvelle valeur ;
 - `localStorage.setItem('rc_users', ...)` direct (hors DB) est bien vu au `get` suivant.
- Lance `node scripts/verif/suite.mjs` et `node scripts/verif/idempotence.mjs` (3 passes identiques).
- Critère de fin : avec un `rc_users` synthétique de 3 Mo, 10 `DB.get('users')` consécutifs coûtent au moins 5 fois moins qu'avant (mesure `performance.now()` consignée en commentaire), sans nouvel échec dans la suite.

7. Trajectoire de découpage de rc-core, sans réécriture — Priorité : Moyenne · Effort : L

Constat. Tout le code est dans un seul fichier de 123 204 lignes, en portée globale : 5 093 fonctions, 1 811 constantes, 387 `onclick=` dans `index.html` et des centaines d'autres dans les chaînes `innerHTML` (713 affectations). Les plus grosses masses sont des **données** : `CLOUD` (123 Ko, 1.3214), `WO_ZONES` (90 Ko, 1.47698), `BPOSE/BPOSE_F` (57 Ko, 1.70659), `MORPHO_PROFILS` (24 Ko), `RC_PHRASES` (23 Ko), `EX_VIDEOS` (19 Ko). Par leur nom, les fonctions `coach` en pèsent environ 11 %, et l'anatomie ou la morphologie environ 10 %. On trouve aussi 53 fonctions jamais référencées (17 Ko, par exemple `startWorkout` 1.47528, `qcEditRow` 1.106411, `recordsRecentsPublics` 1.18998), 60 fonctions appelées seulement par `tests.js` (40 Ko, par exemple `analyzePhotoWithClaude` 1.116287, `importVideoLinksFromPdf`) et un doublon `_lundiDe` (1.26753 et 1.62987). Le bon modèle existe déjà : `chargerMotionLab()` (1.107851).

Pourquoi le mettre en place. À chaque build, un athlète retélécharge le code `coach`, l'anatomie et les tables qu'il n'ouvre pas. Aucune revue humaine ni aucun outil ne tient sur 123 000 lignes, et un changement de nom global casse loin sans prévenir. Découper par *concaténation ordonnée* garde exactement la même sémantique de script global, donc aucun risque sur les `onclick`.

Intérêt réel. Étape par étape : - (a) Découper le source en `src/core/*.js`, concaténés au build : les revues et les diffs git deviennent lisibles, les conflits entre lots disparaissent, et le code ne change pas d'un octet une fois concaténé. - (b) Sortir les tables de données en JSON chargé à la demande : environ 330 Ko bruts de moins au démarrage. - (c) Sortir un paquet « `coach` » paresseux, comme `motion-lab` : environ -25 % de JS pour un athlète, et pour lui un build qui ne touche que le `coach` ne coûte plus rien. - (d) Supprimer le code mort : -57 Ko.

Au bout du compte, un athlète télécharge environ 600 Ko en brotli au lieu de 1,76 Mo, et chaque lot touche un fichier de 3 000 lignes au lieu de 123 000.

Prompt de mise en place.

Contexte : `app/rc-core.<build>.js` fait 123 204 lignes en portée globale (script classique, pas un module). Des centaines de `onclick="nomDeFonction()"` dans `app/index.html` et dans des chaînes `innerHTML` exigent que les fonctions restent globales. `scripts/versionner_actifs.py` renomme `rc-core` à chaque build. `app/tests.js` lit le source avec `String(fn)` et `window_RC_SRC_PROD`. Modèle de chargement paresseux existant : `chargerMotionLab()` (~1.107851).

ÉTAPE (a) SEULEMENT dans ce prompt : découper sans changer le code produit.

1. Écris `scripts/decouper_core.mjs` (acorn via `npx -y`) : il lit `rc-core.<build>.js` et coupe aux frontières des bandeaux de section existants (les lignes « // ===== TITRE ===== ») en fichiers `src/core/NNN-titre-slug.js`, avec `NNN` = ordre à 3 chiffres. Ne coupe JAMAIS au milieu d'une instruction de niveau racine : vérifie avec l'AST que chaque morceau finit sur la fin d'un nœud du Program. Vise 40 à 80 fichiers de 1 000 à 4 000 lignes.
 2. Écris `scripts/assembler_core.mjs` : il concatène `src/core/*.js` dans l'ordre, séparés par un saut de ligne, et écrit `app/rc-core.<build>.js`. Garantie : `assembler(decouper(f)) === f` octet pour octet. Le script le vérifie et sort en 1 sinon.
 3. Branche `assembler_core.mjs` au début de `scripts/versionner_actifs.py` (subprocess `node`) et comme première étape de `.github/workflows/firebase.yml`, avec un contrôle qui échoue si le fichier généré diffère de celui commité. Pendant la transition, on committe les deux.
 4. Documente dans `src/core/LISEZMOI.md` l'ordre de chargement et la règle : un fichier ne peut utiliser à son niveau racine (hors fonctions) que des déclarations de fichiers de numéro inférieur.
 5. Ajoute dans `scripts/verif/lint.mjs` (s'il existe) le contrôle `no-redeclare`, qui signale par exemple le doublon `_lundiDe` (1.26753 / 1.62987) : supprime la seconde déclaration, elle est identique.
- Tests : `node scripts/assembler_core.mjs --verifier` ; `node scripts/verif/syntaxe.mjs` ; `node scripts/verif/suite.mjs` (même nombre de tests joués et d'échecs qu'avant).
- Critère de fin : `src/core/` existe, l'assemblage reproduit `rc-core` à l'identique, et la CI le contrôle.
- Étapes suivantes, à écrire comme TODO numérotés dans `LISEZMOI.md`, sans les faire :
- (b) déplacer `WO_ZONES`, `BPOSE`, `BPOSE_F`, `MORPHO_PROFILS`, `EX_VIDEOS` vers `app/data/*.json`, chargés à la demande avec le même motif que `_loadCical` ;
 - (c) produire `app/rc-coach.<build>.js` (fonctions `openClientDetail`, `renderClientList`, `_htmlTableauxTableur`, `canal*`, `relance*`, `ccd*`...) chargé par un `chargerCoach()` au premier `go('s-coach*')`, avec des stubs globaux qui attendent le chargement ;
 - (d) supprimer les 53 fonctions sans référence (liste à régénérer par script : nom présent une seule fois dans `rc-core`, `index.html`, `motion-lab.js`, `sw.js` et `vendor/rc-video.js`).

8. Petites fuites de bande passante récurrentes — Priorité : Basse · Effort : S

Constat. À chaque ouverture, `sw.js` (52 Ko, 18 Ko en brotli) est demandé trois fois : l'enregistrement, `reg.update()` (`index.html` ~1.410) et la sonde de version `sw.js?v=...`. Mesuré : 9 requêtes `sw.js` sur 3 rechargements. `motion-lab.js?v=<build>` (160 Ko en brotli) est exclu du report entre caches (`sw.js`, `_exclu`) et donc retéléchargé à chaque build par tout `coach` qui ouvre le labo. `app/tests.js` (5,3 Mo) est publié en production par `cp -a app`. L'écouteur `scroll` de `_ccdArmerAncres` (`rc-core:30106`) reste actif sur tous les écrans et appelle `getBoundingClientRect` sur 3 sections tant que `_ccdVue=== 'donnees'`, même quand on a quitté la fiche.

Pourquoi le mettre en place. Ce sont des coûts faibles mais payés à chaque ouverture et par chaque appareil, et ils s'ajoutent au quota Hosting. L'écouteur force un calcul de mise en page pendant le défilement d'écrans qui ne le concernent pas.

Intérêt réel. Environ 36 Ko de moins par ouverture : sur 3 316 ouvertures par jour (le chiffre cité par `versionner_actifs.py`), cela fait environ 120 Mo par jour, soit un tiers du quota Spark. Pour un `coach` qui utilise la vidéo, c'est 160 Ko de moins par build. Le défilement de l'accueil `coach` devient plus fluide sur un Android moyen de gamme (plus de mise en page forcée par image). Publier `tests.js` expose aussi 5 Mo inutiles.

Prompt de mise en place.

Contexte : RepCore PWA. (1) À chaque ouverture, `app/index.html` enregistre `./sw.js`, appelle `reg.update()`, puis une sonde de version refait un `fetch` de `sw.js?v=...` : trois téléchargements de 18 Ko en brotli. (2) `app/sw.js` exclut `/motion-lab.js` du report entre caches (fonction `_exclu` dans `activate`), alors que l'URL porte déjà `?v=<build>`. (3) `.github/workflows/firebase.yml` publie `app/tests.js` (5,3 Mo). (4) `rc-core _ccdArmerAncres` (~1.30106) pose un écouteur `window 'scroll'` permanent qui lit `getBoundingClientRect` de sections `#ccd-et*`, tant que `_ccdVue=== 'donnees'`, même hors de l'écran `s-coach-client`.

À faire :

1. Remplace la sonde de version qui lit `sw.js` par un `fetch` de `./apk-version.json` ou d'un nouveau `./version.json` (`{ "build": "1693" }`), écrit par `scripts/versionner_actifs.py` et servi en no-cache dans `firebase.json`). Supprime le `reg.update()` redondant si la sonde s'en charge : garde un seul chemin qui

```
déclenche update() quand la version publiée est supérieure à RC_BUILD.  
2. Dans sw.js, retire /motion-lab\./ de _exclu, mais purge les motion-lab.js?v=X dont X ne fait pas partie de _gardees (même logique que les actifs rc-*).  
3. Dans firebase.yml, après la copie : rm -f _site/app/tests.js. Vérifie que chargerTests() affiche un message clair en production (« suite non disponible ici »).  
4. Dans l'écouteur scroll de _ccdArmerAncres, sors tout de suite si !document.getElementById('s-coach-client')?.classList.contains('active').  
Tests : dans app/tests.js, vérifie que le source de l'écouteur contient le garde s-coach-client et que _exclu ne contient plus motion-lab. Étends scripts/verif/worker-actifs.mjs pour qu'un motion-lab.js?v=ancien soit purgé et que ?v=courant soit reporté.  
Critère de fin : dans Chromium local, une réouverture ne fait plus qu'une seule requête réseau de version (moins de 1 Ko) en plus d'index.html, et node scripts/verif/suite.mjs ne montre aucun nouvel échec.
```

Agent 9 — Design UI et identité visuelle · 6/10

RepCore a une vraie identité (noir carbone, rouge #E02020, médaillons métal-rouge cohérents, landing soignée) et, depuis l'audit L0 du 10/08, un vrai chantier de tokens a été mené : échelle typo à 8 crans (3 517 appels `var(--fs-*)` contre 513 tailles en dur), faux gras Bebas corrigé (1 règle restante sur 138), z-index en 7 crans, durées `--t-0-4` (230 usages), `--text-faint` remonté à AA en sombre, 36 blocs `prefers-reduced-motion`. Mais la couleur n'a jamais été gouvernée : 235 hex en dur au L0, **523 aujourd'hui** dans l'app (74 rouges vifs distincts, 194 gris). Le thème clair généré casse les contrastes. Les pages publiques i/p/c sont hors charte (police système, CTA vert citron). Il manque un système que l'on fait respecter automatiquement pour atteindre le niveau Whoop/Future.

Méthode. Captures Playwright 390×844 (landing, écrans d'entrée de l'app en sombre et en clair, i/ en mode navigateur Instagram, p/ et c/ vides et remplis avec des données simulées, réseau externe coupé), planche de toutes les illustrations, mesure automatique des contrastes WCAG sur 5 écrans × 2 thèmes, comptage par script sur `rc-style.1693.css` (hors bloc clair généré, lignes 12047-15062), `rc-core.1693.js` et `app/index.html`.

Suivi de l'audit L0 (PDF). - Fait : échelle typographique, graisses Bebas, z-index, durées, élévations tokenisées (mais en 3 familles), rayons `--r-1-4`, échelle d'espacement plus `scripts/espacements.py`, thème clair, contraste de `--text-faint` et `--red-text`. - Pas fait : couleurs (voir point 3) ; les 7 classes mortes relevées au L0 (`.txt-title`, `.txt-stat-sm`, `.brand-font`, `.card-accent`, `.card-warn`, `.card-ok`, `.accent-line`) ne sont toujours utilisées nulle part (`rc-style.1693.css:1094-1098`) ; `--red2` a encore 0 usage. - En recul : @keyframes passés de 24 à 77, box-shadow de 73 à 285 valeurs distinctes (app entière).

1. Thème clair : contrastes cassés et voiles noirs — Priorité : Haute · Effort : M

Constat. `scripts/theme_clair.py:99` (`clair()`) inverse la luminosité sans vérifier le contraste. `--sub` y devient #787878, soit 4,0:1 sur #f4f4f4 et 3,7:1 sur `--surface-1` #ebebeb. `--text-faint` devient #808080 (3,5:1). `--green` #22c55e reste tel quel, alors qu'il sert de couleur de texte : 1,79:1 mesuré sur « Ton premier mois est complet » (`s-athlete-entry`). Mesure automatique en clair : 6 échecs AA sur `s-athlete-entry`, 7 sur `s-login`, 13 sur `s-register`, contre 0 en sombre. Les voiles `radial-gradient(100% 100% from black, to transparent)` écrits en ligne (`app/index.html:930` et `:1026`) restent noirs. Pour le voir : Réglages > Apparence > Clair, puis « Espace athlète » : le fond devient un dégradé gris sale et le sous-titre disparaît à gauche.

Pourquoi le mettre en place. Le clair est proposé dans les réglages et s'active tout seul en « Auto » (`rc-core.1693.js:2331`). Un athlète qui s'entraîne en salle éclairée ou dehors choisit le clair justement pour mieux lire. Aujourd'hui, c'est le mode où il lit le moins bien. Le script est déterministe : un garde-fou de contraste le corrige d'un coup, sur les 2 734 règles générées.

Intérêt réel. Le thème clair ferait passer RepCore devant Trainerize ou TrueCoach, qui n'en ont pas de complet, mais seulement s'il est lisible. Aujourd'hui, un utilisateur iPhone en mode clair système qui choisit « Auto » voit des libellés gris pâles sur les champs de connexion (EMAIL, MOT DE PASSE à 4,0:1 en 11 px) et le prix de l'essai en vert illisible (1,79:1), au moment où il décide de s'inscrire. Corriger les 4 à 5 jetons fautifs et les 2 voiles règle environ 30 échecs mesurés sur les seuls écrans d'entrée, et probablement des centaines dans l'app connectée, qui applique la même transformation. Le travail tient dans un seul script, sans rien toucher à la main dans les 700 Ko générés.

Prompt de mise en place.

Dans ce dépôt (RepCore), le thème clair est GÉNÉRÉ par `scripts/theme_clair.py` dans `app/rc-style.1693.css`, entre les marqueurs « THEME CLAIR – GENERE » et « FIN DU THEME CLAIR GENERE ». Ne modifie jamais ce bloc à la main : corrige le générateur, puis relance-le.

Problèmes mesurés :

1. La fonction `clair()` (`scripts/theme_clair.py`, vers la ligne 99) inverse la luminosité ($L2 = 0.985 - L \cdot 0.95$) sans contrôler le contraste. Résultat : `--sub` et `--text-dim` valent #787878 (4,0:1 sur #f4f4f4, 3,7:1 sur #ebebeb) et `--text-faint` vaut #808080 (3,5:1). Ils servent à 10-12 px, donc il faut au moins 4,5:1.
2. Les propriétés personnalisées (`--green`, `--orange`, `--info`, `--warning-gold`, `--gluc`, `--pub-f`) ne sont pas traitées comme du TEXTE : `--green` #22c55e reste inchangé et donne 1,79:1 en texte sur fond clair (écran `s-athlete-entry`, « Ton premier mois est complet »).
3. Les voiles noirs en ligne de `app/index.html:930` et `:1026` (`radial-gradient(ellipse 90% 78% at 50% 58%, transparent 22%, rgba(0,0,0,.48) 62%, rgba(0,0,0,.88) 100%)`) restent noirs en clair et salissent les écrans `s-coach-entry` et `s-athlete-entry`.

À faire :

- Ajoute au script une fonction `contraste(c1, c2)` (luminance relative WCAG 2.1). Pour chaque couleur au rôle 'texte', et pour chaque jeton de `:root` dont le nom commence par `--text`, `--sub`, `--red-text`, `--green`, `--orange`, `--info`, `--warning`, `--amber`, `--gluc`, `--pub`, `--link`, assombris la couleur (même teinte, luminosité réduite par pas de 0,01) jusqu'à obtenir $\geq 4,5:1$ contre #f4f4f4 ET contre #ebebeb. En clair, génère en plus des variantes texte `--green-text`, `--orange-text`, et remplace `color:var(--green)` par `color:var(--green-text)` dans les règles copiées. Les fonds et les bordures gardent la couleur vive.
 - Pour les voiles : ajoute une classe `.vignette-entree` dans la partie sombre de la feuille (déplace-y le `radial-gradient`), remplace les deux `style=""` de `app/index.html:930` et `:1026` par cette classe, et dans le générateur, fais en sorte qu'un background dont tous les arrêts sont `rgba(0,0,0,a)` sur un élément `position:absolute;inset:0` devienne transparent en clair. Le plus simple : une règle explicite `:root[data-theme="clair"] .vignette-entree{background:none}`.
 - Ajoute un mode `--verifier` au script : il ne modifie rien, calcule le contraste de chaque paire (`color`, `background-color` résolue) des jetons, et sort en code 1 s'il reste un texte sous 4,5:1.
 - Relance `python3 scripts/theme_clair.py`.
 - Tests dans `app/tests.js` (suis le style des tests existants autour de `themeEffectif`) : vérifie que la feuille contient, dans le bloc clair, `--sub` et `--text-faint` à $\geq 4,5:1$ sur #f4f4f4, et qu'aucune règle du bloc clair ne contient « `rgba(0,0,0,.88)` » dans un `radial-gradient`. Lance la suite de tests comme le fait le dépôt (voir le haut de `app/tests.js` ou `.github/workflows`).
 - Vérification visuelle : `python3 -m http.server` à la racine, Playwright 390×844, `localStorage rc_theme='clair'`, `go('s-athlete-entry')`, `go('s-login')`, `go('s-register')` : aucun texte sous 4,5:1 (script de contraste sur les éléments feuilles).
- Critère de fin : le mode `--verifier` sort en 0, les trois écrans passent AA en clair, le rendu sombre est identique au pixel près.

2. Pages publiques i/p/c hors charte — Priorité : Haute · Effort : M

Constat. Ces trois pages sont les premières que voit un prospect, et aucune n'utilise les polices de la marque : `p/index.html:24` et `c/index.html` sont en `font:15px -apple-system`, `i/index.html` en pile système. Il y a 5 jeux de tokens différents : app `--bg` #080808/`--border` #242424, landing `--border` #1e1e1e/`--red-light` #e96363, p `--fond` #08080a/`--gris` #a1a1aa, c `--fond` #0b0b0c/`--carte` #151518, i `--fond` #080808/`--carte` #141414. Pire, `i/index.html:39` déclare `--vif:#c9f24d` (vert citron) comme couleur des boutons principaux (`.bouton`, l. 50). C'est le premier écran de l'athlète invité depuis Instagram (capture 06d-i-instagram : « Copier le lien » en vert fluo). La vitrine coach `c/` n'a ni logo ni titre en Bebas (capture 08b-c-demo). Les états vides de p/ et c/ se réduisent à une phrase et un bouton, sans marque, et la trame de fond commence à mi-écran (capture 07-p-vide).

Pourquoi le mettre en place. La landing a une vraie direction artistique (titres Bebas en capitales, rouge, halos). La page qui vend le coaching de Kevin (`c/`), elle, ressemble à une page générique en Arial. Un prospect qui passe de la landing à la vitrine, puis à l'invitation, traverse trois marques différentes, dont une verte.

Intérêt réel. `c/` porte les formules de coaching de 150 à 350 €, visibles sur la capture : c'est la page au panier moyen le plus élevé du produit. Kahunas et Everfit vendent des vitrines « à la marque du coach » ; ici, la vitrine est moins soignée que la landing à 9,50 €. Une charte commune (Bebas pour les titres, Montserrat pour le texte, un seul jeu de tokens) donne une perception premium dès le lien partagé en story. `i/` est l'écran de conversion de l'invité d'un coach : un CTA rouge de marque au lieu du vert citron évite l'effet « site tiers » au moment où l'athlète doit faire confiance. Coût : 3 fichiers HTML de 15 à 20 Ko, sans dépendance nouvelle, car les woff2 sont déjà dans `app/fonts/`.

Prompt de mise en place.

Dépôt RepCore. Les pages publiques `i/index.html`, `p/index.html` et `c/index.html` ont chacune leurs propres variables CSS et utilisent la police système. Il faut les aligner sur la charte de la landing (`index.html` à la racine) et de l'app.

1. Crée un fichier partagé `charte.css` à la racine (servi par firebase hosting, vérifie `firebase.json` : pas de `rewrite` qui l'intercepte) avec :
 - `@font-face` Montserrat (`app/fonts/montserrat-var-latin.woff2`, 100 900) et Bebas Neue (`app/fonts/bebasneue-400-latin.woff2`, 400 seulement), `font-display:swap`, `unicode-range latin` : reprends exactement les déclarations de `app/index.html` (cherche `"@font-face"`).
 - Un seul jeu de tokens repris de `app/rc-style.1693.css` : `root` : `--bg #080808`, `--surface-1 #111111`, `--surface-2 #181818`, `--border #242424`, `--text #efefef`, `--sub #8a8a8a`, `--red #E02020`, `--red-text #FF5A4A`, `--green #22c55e`, `--r-1 4px`, `--r-2 8px`, `--r-3 12px`, `--r-4 18px`, `--fs-* (10/11/12/14/16/20/26/44)`, `--pile-titre "Bebas Neue"`, ...
 - Composants : `.cta` (bouton principal rouge plat, rayon `var(--r-3)`, hauteur mini 48 px, Montserrat 800 en capitales, `letter-spacing 1.5px`, comme « ESSAYER GRATUITEMENT » de la landing), `.cta-sec` (contour), `h1` et `h2` en Bebas capitales, `carte`.
2. Dans `i/`, `p/` et `c/` : ajoute `<link rel="stylesheet" href="..charte.css">` (attention : `p/` et `c/` sont aussi servies en `/pseudo` et `/coach/slug`, donc utilise un chemin ABSOLU `/charte.css` ; vérifie les `rewrites` dans `firebase.json`). Supprime les `:root` locaux (`--fond`, `--carte`, `--bord`, `--gris`, `--txt`, `--vif`, `--rouge`) et remplace leurs usages par les tokens communs. Remplace `--vif:#c9f24d` (`i/index.html:39`, `.bouton 1.50`, `.promesse strong 1.59`) par le rouge de marque. Passe `body` en Montserrat, les `h1` en Bebas.
3. Ajoute en tête de `c/` et de `p/` un bandeau de marque discret (logo `/logo.png` 28 px + « REPCORE » en Bebas, lien vers `/`), comme l'en-tête de la landing.
4. État vide (fonction `vide()` dans `p/` et `c/`) : logo, titre Bebas « PAGE INTROUVABLE », texte en `--sub`, `.cta`. Corrige la trame de fond de `p/` pour qu'elle couvre tout l'écran (`background-attachment` ou `min-height:100dvh` sur le conteneur).
5. Ne change aucune logique JS (lecture Firebase, formulaires, suivi `/arrivee`).

Vérification : `python3 -m http.server 8765` à la racine, Playwright 390x844, en interceptant les appels `firebaseio` (`profils_publics`, `volts_publics`, `vitrites`) avec des données factices, et en coupant les appels `workers.dev` pour ne pas compter de fausses visites. Capture `i/?c=ABC123` avec un `userAgent` contenant "Instagram", `p/?u=julie`, `c/?s=kevin-guellec`, `p/` et `c/` vides. Critère de fin : `font-family` calculée = Montserrat sur `body` et Bebas Neue sur `h1` dans les 3 pages, plus aucune occurrence de `#c9f24d` dans le dépôt (hors `vault/`), captures comparées côte à côte avec la landing.

3. Couleurs non gouvernées : 523 hex, 74 rouges, 194 gris — Priorité : Haute • Effort : L

Constat. Comptage sans commentaires ni bloc clair : 423 hex distincts dans `rc-style.1693.css`, 149 dans `rc-core.1693.js`, 70 dans `app/index.html`, soit **523 au total dans l'app** (235 au L0) et 477 `rgba()`. On compte **74 rouges vifs distincts** (`#c00000`, `#c10000`, `#c10b0b`, `#c20c0c`, `#d40d0d`, `#e01818`, `#e1dd1d`, `#e21414`...). Le rouge de marque `#E02020` est écrit en dur 223 fois au lieu de `var(--red)` (112 dans la CSS, 100 dans `rc-core`, 11 dans `index.html`, par exemple `.wel-titre-r`, `rc-style.1693.css:10016`). Deux familles de gris coexistent : 83 gris neutres (`#111`, `#222`...) et 111 gris teintés « zinc » (`#b4b4bb` x34, `#8a8a92` x32, `#26262b` x36). `scripts/theme_clair.py` le reconnaît lui-même dans son en-tête : « environ 1 700 couleurs sont écrites en dur ».

Pourquoi le mettre en place. Tout le reste en dépend : thème clair, cohérence, retouches de marque. Tant qu'un rouge existe en 74 versions, un changement de teinte ou une correction de contraste reste un chantier de plusieurs semaines, et le générateur clair doit deviner l'intention de chaque littéral. Le dépôt a déjà le modèle à suivre : `scripts/espacements.py --verifier`.

Intérêt réel. Les apps premium (Whoop, Future) tiennent sur 10 à 15 couleurs nommées ; c'est ce qui donne l'impression de netteté. Ramener 74 rouges à 4 crans (`--red`, `--red-text`, `--red-deep`, `--red-bg`) et 194 gris à environ 8 surfaces et paliers de texte : - rend le rendu homogène d'un écran à l'autre ; - divise la taille du bloc clair (700 Ko générés sur 1,6 Mo de CSS), car il suffit de redéfinir les jetons ; - rend possible une vraie charte coach en marque blanche (couleur d'accent du coach), argument de vente Kahunas et Everfit, impossible aujourd'hui. Un garde-fou en CI empêche la dérive : les lots récents (N, C, T) ont réintroduit des littéraux après la tokenisation.

Prompt de mise en place.

Dépôt RepCore. Objectif : faire gouverner les couleurs de l'app par des jetons, en suivant le modèle de `scripts/espacements.py` (échelle + mode `--verifier`).

1. Crée `scripts/couleurs.py` (même structure qu'`espacements.py`) :
 - Il lit `app/rc-style.<build>.css` (en EXCLUANT le bloc entre « THEME CLAIR – GENERE » et « FIN DU THEME CLAIR GENERE »), `app/rc-core.<build>.js` et `app/index.html`.
 - Il extrait tous les littéraux `hex` et `rgba()` des propriétés de couleur, hors commentaires.
 - Mode `--rapport` : liste par famille (rouges, gris neutres, gris teintés, verts, oranges, bleus, violets) avec le nombre d'occurrences et le jeton le plus proche (distance `deltaE CIE76` dans l'espace Lab).
 - Mode `--appliquer` : remplace chaque littéral dont le `deltaE` au jeton le plus proche est `< 6` par `var(--jeton)`. Les alpha restent : un `rgba(224,32,32,.35)` devient `color-mix(in srgb,var(--red) 35%,transparent)`.
 - Mode `--verifier` : sort en code 1 si le nombre de littéraux dépasse un plafond stocké dans `scripts/couleurs-plafond.txt` (valeur de départ : le nombre actuel). Le plafond ne doit que baisser.
2. Complète le `:root` de `app/rc-style.1693.css` (début du fichier) avec une palette FERMÉE : `--red #E02020`, `--red-deep #8d0000` (dégradés de bouton), `--red-bg`, `--red-bg-2`, `--red-text` ; gris neutres uniquement : `--bg`, `--surface-0.3`, `--border`, `--border-strong`, `--text`, `--text-strong`, `--text-mid`, `--sub`, `--text-faint` ; états : `--green`, `--orange`, `--amber`, `--info`, `--link`. Supprime la famille zinc (`#b4b4bb`, `#d4d4d8`, `#8a8a92`, `#a0a0a8`, `#26262b`, `#2a2a2e`, `#33333a`) en la rabattant sur les gris neutres équivalents. Supprime `--red2` (0 usage).
3. Lance `--appliquer`, puis relance `python3 scripts/theme_clair.py`.
4. Branche `--verifier` dans le workflow CI existant qui lance les tests (`.github/workflows/*.yml`, celui qui exécute `app/tests.js`) ainsi que `python3 scripts/espacements.py --verifier`. Ce dernier sort aujourd'hui en 1 avec 52 valeurs hors échelle : corrige-les d'abord avec `python3 scripts/espacements.py`.
5. Tests : ajoute dans `app/tests.js` un test qui compte les occurrences de « `#E02020` »/« `#E02020` » hors `:root` dans la feuille et dans `rc-core`, et attend 0. Vérification visuelle : captures Playwright 390x844 avant et après de `s-welcome`, `s-login`, `s-athlete-entry` (sombre et clair). Différence tolérée : `deltaE < 6`. Critère de fin : 74 rouges ramenés à 5 jetons au plus, plus aucun gris teinté, plafond de littéraux réduit d'au moins 70 %, CI verte.

4. Typographie : échelle adoptée à 87 %, mais des tailles sous le plancher et deux voix de titre — Priorité : Moyenne • Effort : M

Constat. Le commentaire des tokens (`rc-style.1693.css:140-155`) annonce un plancher `--fs-2xs` de 10 px et « toute nouvelle taille passe par ces variables ». Pourtant la CSS garde **475 tailles en dur sur 42 valeurs**, dont **83 déclarations sous 10 px** (6,5 / 7 / 7,5 / 8 / 8,5 / 9 / 9,5 px, par exemple `.rcf-exp 9 px` à la l. 1943 et 8,5 px à la l. 2259, `.ccdx .ccd-v-s 8,5 px` à la l. 2685). Elles se concentrent sur les lignes 6000-11000, écrites après la tokenisation. L'écran `s-welcome` affiche **11 tailles différentes** (10, 11, 11,5, 12, 12,5, 13, 13,5, 15, 16, 31,2, 32 px), valeurs relevées dans le navigateur. Autre problème : les titres des écrans d'entrée de l'app sont en Montserrat 900 en bas de casse (« La bonne charge, à chaque série. », `.wel-titre l. 10014` ; « Installe RepCore. », `index.html:681`), alors que la landing affiche la même phrase en Bebas capitales. Trois styles de titre coexistent donc sur les écrans d'entrée : « ESPACE ATHLÈTE » en Montserrat capitales rouge, « Connexion » en Montserrat bas de casse blanc, et `.wel-titre` en Montserrat noir.

Pourquoi le mettre en place. 8,5 px sur un téléphone tenu à bout de bras entre deux séries n'est pas lisible, et le fichier lui-même fixe la limite à 10. La phrase-signature de la marque ne devrait pas changer de police entre la landing et l'app : c'est la première chose que voit l'utilisateur après avoir cliqué sur « Commencer ».

Intérêt réel. Le passage landing → app est le moment où l'on juge si c'est « le même produit ». La même accroche en Bebas des deux côtés renforce la reconnaissance, comme Future qui garde sa typo de titre de la pub jusqu'au premier écran. Supprimer les 83 tailles sous le plancher règle le principal problème de lisibilité des cartes denses (fiche charges, tableau coach). Ramener les demi-crans à l'échelle réduit le « bruit » perçu sans changer la mise en page (±0,5 px).

Prompt de mise en place.

Dépôt RepCore. L'échelle typographique est déclarée dans le `:root` de `app/rc-style.1693.css` (`--fs-2xs 10px ... --fs-3xl 44px`, avec des commentaires qui fixent un plancher de 10 px). Il reste 475 font-size en px dans la feuille (hors bloc clair généré), dont 83 sous 10 px, et quelques-unes dans `rc-core.1693.js` et `app/index.html`.

1. Crée `scripts/typo.py` sur le modèle de `scripts/espacements.py` : il cale chaque font-size:Npx de la feuille (hors bloc clair), de `rc-core.<build>.js` et de `app/index.html` sur le jeton le plus proche (`10--fs-2xs`, `11--fs-xs`, `12--fs-sm`, `14--fs-md`, `16--fs-lg`, `20--fs-xl`, `26--fs-2xl`, `44--fs-3xl`). Toute valeur < 10 px passe à `var(--fs-2xs)`. N'y touche pas : les `clamp()`, `calc()`, `em`, `cqw`, et la règle Safari `max(16px,1em)`. Il a un mode `--verifier` qui sort en 1 s'il reste un font-size en px hors de ces exceptions.
2. Lance-le, puis relance `python3 scripts/theme_clair.py`.
3. Contrôle visuel des zones denses qui utilisaient 8-9 px (`.rcf-*`, `.ccdx*`, `.rc-obj-b`, lignes ~1900-2700 de la feuille) : si un libellé déborde, réduis le `letter-spacing` (`--ls-1`) plutôt que la taille.
4. Titres des écrans d'entrée : passe `.wel-titre` (feuille, ~L.1014) et le titre « Installe RepCore. » (`app/index.html:681`, style en ligne à déplacer dans une classe `.inst-titre`) en `var(--pile-titre)`, `font-weight:400`, `text-transform:uppercase`, `letter-spacing var(--ls-2)`, taille `clamp(40px,11vw,56px)`, comme le hero de `index.html` (racine). Remplace `color:#E02020` par `var(--red)`. Alignes aussi « Connexion » (`s-login`) et « ESPACE ATHLÈTE » / « ESPACE COACH » sur une seule classe `.titre-entree` (Bebas capitales, blanc, accent rouge sur un mot seulement).
5. Tests dans `app/tests.js` : la feuille ne contient plus de font-size:[0-9.]px hors exceptions ; `.wel-titre` utilise `var(--pile-titre)`. Vérification : Playwright 390x844, `go('s-welcome')`, relève les tailles calculées des éléments de texte : au plus 6 valeurs distinctes. Captures `s-welcome`, `s-login`, `s-athlete-entry` à comparer avec la landing.

Critère de fin : `typo.py --verifier` sort en 0, aucun texte calculé sous 10 px sur les écrans publics, même police de titre sur la landing et l'accueil de l'app.

5. Rayons, ombres, élévations : trois systèmes concurrents — Priorité : Moyenne · Effort : M

Constat. Les rayons déclarés sont `--r-1 4 / --r-2 8 / --r-3 12 / --r-4 18`, plus des alias `--r-sm 8 / --r-lg 16`. Mais les valeurs en dur les plus fréquentes sont **hors échelle** : 10px (77 fois), 14px (43), 9px (29), 11px, 13px... soit 67 valeurs distinctes dans l'app. Le bouton de base `.btn` a un rayon de 10 px en dur (`rc-style.1693.css:861`) et `.btn-sm 4 px` (l. 921) : deux boutons voisins n'ont pas la même forme. Pour les ombres, trois familles d'élévation cohabitent (`--e0...e4`, `--elev-0...3`, `--el-1/2`, le commentaire de la l. 101 l'admet), avec 55 usages cumulés contre **285 box-shadow distincts** en dur. Dans le même esprit, le bouton « SE CONNECTER » est en dégradé à 135° (capture 04-s-login), alors que la landing utilise un rouge plat.

Pourquoi le mettre en place. La forme des boutons et des cartes est ce que l'œil compare en premier d'un écran à l'autre. Un rayon de 10 px à côté d'un rayon de 4 px, et des cartes à 9, 10, 12 et 14 px, donnent l'impression de bricolage que `scripts/espacements.py` voulait justement supprimer pour les marges.

Intérêt réel. Unifier sur 3 rayons (carte, contrôle, pastille) et 3 élévations (repos, levée, appel) rend l'interface immédiatement plus « dessinée ». C'est la signature de Whoop : un seul rayon, une seule ombre. Le coût est mécanique (script de calage comme pour les espacements) et le gain se voit sur les 56 écrans. Cela simplifie aussi le thème clair, qui n'aura plus que 3 ombres à redéfinir au lieu de 285.

Prompt de mise en place.

Dépôt RepCore, `app/rc-style.1693.css` (hors bloc clair généré), `app/rc-core.1693.js`, `app/index.html`.

1. Rayons : fixe l'échelle à `--r-1 4px` (pastilles, badges), `--r-2 8px` (champs, petits boutons), `--r-3 12px` (cartes, boutons), `--r-4 18px` (feuilles modales), `--r-full`. Supprime `--r-sm` et `--r-lg` en les rabattant sur `--r-2` et `--r-3`. Crée `scripts/rayons.py` (modèle : `scripts/espacements.py`, avec `--verifier`) qui cale chaque `border-radius` en px : 1-5 px → `--r-1`, 6-9 → `--r-2`, 10-15 → `--r-3`, 16-24 → `--r-4`, 50% et 999px → `--r-full`. Ne touche pas aux valeurs composées de plusieurs coins si elles sont asymétriques : liste-les dans le rapport.
2. `.btn` (feuille ~L.861, `border-radius:10px`) et `.btn-sm` (~L.921, `var(--r-1)`) passent tous deux à `var(--r-3)`.
3. Élévations : garde UNE famille `--e0...e3` plus `--e-inset` et `--glow-red`. Remplace les usages de `--elev-*` et `--el-*` par l'équivalent le plus proche (`--elev-1` → `--e1`, `--elev-2/-el-1` → `--e2`, `--elev-3/-el-2` → `--e3`) et supprime leurs déclarations. Ajoute à `scripts/rayons.py` (ou à un `scripts/ombres.py`) un rapport des `box-shadow` en dur : chaque ombre noire simple (0 Npx Mpx rgba(0,0,0,a)) est calée sur `--e1/-e2/-e3` selon son flou (≤3, ≤10, >10 px).
4. Bouton principal : une seule recette pour `.btn-red` (ou la classe du bouton « SE CONNECTER » de `s-login`, à retrouver dans `app/index.html` vers la l.1130), rouge plat `var(--red)` avec `--glow-red`, identique au CTA de la landing.
5. Relance `scripts/theme_clair.py`. Tests dans `app/tests.js` : aucune déclaration `--elev-` ni `--el-` restante, `.btn` et `.btn-sm` ont le même rayon.

Critère de fin : `rayons.py --verifier` sort en 0, 5 rayons distincts au plus dans la feuille, captures Playwright 390x844 de `s-welcome`, `s-login` et `s-athlete-entry` sans régression visible.

6. Cibles tactiles sous 44 px sur des actions courantes — Priorité : Moyenne · Effort : S

Constat. Mesures dans le navigateur à 390 px : `.btn-sm` fait environ 34 px de haut (`padding:10px 16px` et 11 px de texte, `rc-style.1693.css:921`), et il est utilisé **379 fois** dans `rc-core` et `index.html` ; - sur `s-welcome`, les bascules « Annuel/Mensuel » font 83x34 et 91x34, « J'ai déjà un compte » 151x37, « Espace coach » 114x37, la flèche 38x38 ; - sur `s-login`, « Mot de passe oublié ? » fait 18 px de haut et « Créer un compte » 18 px ; - sur `s-register`, la case CGU fait 16x16 et les liens CGV et mentions 14 px ; - sur la landing, « Se connecter » fait 91x16. `.back-btn` respecte bien 44x44 (l. 7341) : le bon modèle existe déjà dans la feuille.

Pourquoi le mettre en place. En salle, on tape avec des mains moites ou gantées, souvent entre deux séries. 44 px (Apple) ou 48 px (Material) est le minimum. Le bouton secondaire le plus utilisé de l'app, à 34 px, fait rater des taps sur l'écran de séance comme sur la fiche coach.

Intérêt réel. Moins de taps ratés, c'est moins de frustration pendant la séance, le moment où l'app est la plus utilisée. Hevy et Ladder mettent toutes les actions de séance à 48 px ou plus. La correction tient en une règle (`min-height:44px` sur `.btn-sm`, plus une zone de tap élargie par `::after` sur les liens textuels), sans changer la taille visuelle du texte.

Prompt de mise en place.

Dépôt RepCore. Objectif : toute cible tactile fait au moins 44x44 px, sans changer la taille visuelle du texte.

1. `app/rc-style.1693.css` : `.btn-sm` (~L.921) reçoit `min-height:44px` et `display:inline-flex; align-items:center; justify-content:center`. Vérifie les listes horizontales qui l'utilisent (`grep "btn-sm" dans rc-core.1693.js`) : si une rangée déborde à 390 px, passe-la en `flex-wrap`.
2. Crée une classe utilitaire `.tap-zone(position:relative) .tap-zone::after{content:'';position:absolute;inset:-12px -8px}` et applique-la aux liens textuels d'action : « Mot de passe oublié ? » et « Créer un compte » (`app/index.html` ~L.1134-1135), liens CGV et mentions de `s-register`, lien « Se connecter » de l'en-tête de `index.html` (racine).
3. La case CGU de `s-register` (`#r-cgu`) : enveloppe-la avec son texte dans un `<label>` de 44 px de haut au minimum, la case à 22x22 (`accent-color:var(--red)`).
4. `s-welcome` : bascules Annuel/Mensuel (`.wel-b-an`, `.wel-b-mois`), boutons « J'ai déjà un compte » et « Espace coach », flèche `.wel-fleche` → `min-height:44px` (et 44x44 pour la flèche).
5. Relance `python3 scripts/theme_clair.py`.
6. Ajoute un test dans `app/tests.js` qui vérifie que la règle `.btn-sm` contient `min-height:44px`.

Vérification : Playwright 390x844, sur `index.html`, `app/` puis `go('s-welcome')`, `go('s-login')`, `go('s-register')` : liste tous les `a`, `button`, [`role=button`], `input[type=checkbox]` visibles dont `getBoundingClientRect()` fait moins de 44 de haut ou de large (le `::after` ne compte pas dans le `rect` : pour `.tap-zone`, vérifie avec `elementFromPoint` à 10 px au-dessus du texte). Critère de fin : 0 cible sous 44 px sur ces 4 écrans.

Agent 10 — Engagement, gamification et rétention · 7/10

Le socle de gamification est au-dessus de la concurrence coachée (Trainerize, TrueCoach et Everfit n'ont rien d'approchant) : des volts recalculés par le serveur (`cloudflare/src/xp.js`), 10 rangs avec des sous-niveaux, 50 badges avec leur rareté calculée chaque nuit, une série hebdomadaire avec jokers, des duels, un classement entre amis, un Wrapped, des saisons, un parcours d'accueil en 7 étapes, et une mesure de la rétention J1/J7/J30 qui compare les groupes avec et sans chaque levier. Mais les relances les plus rentables (« série en danger », résultat de duel) sont jetées sans bruit par le plafond d'une notification par jour. Toute la compétition (défis, duels, saisons) repose sur une valeur que le client déclare lui-même. Deux règles différentes décident si une semaine est « validée ». Et il n'existe aucune boucle quotidienne ni récompense variable les jours sans entraînement.

1. Les notifications de jeu perdues à cause du plafond — Priorité : Critique · Effort : M

Fusion. la table `PUSH_PRIORITE` créée ici sert aussi au point 6 de l'agent Backend (messages de la nuit).

Constat. `pushAutorise` (`cloudflare/src/metier.js:82`) et la transaction `push_log` (`metier.js:281`) laissent passer une seule notification par jour, la **première arrivée**, sans priorité. Les tâches passent dans l'ordre de `planif.js:43-72` : défis 48 h à 9 h, santé à 10 h, relances coach à 10 h 30, accès et retour à 11 h, et seulement ensuite la **série en danger le jeudi à 18 h** (`metier.js:311`), le J-2 et la **clôture des duels** à 18 h 30 (`metier.js:1117`, `1103`) et les réactions à 19 h (`metier.js:1155`). Pire : `duels/<id>/rappel` est écrit à true **avant** l'envoi (`metier.js:1128`) et `reactions_push` est effacé avant l'envoi (`metier.js:1157`). Un envoi refusé pour « plafond » est donc perdu pour toujours. Enfin, dès que le push serveur est actif, le rappel local du samedi 10 h (`app/sw.js:676-706`) se tait, alors que le serveur n'envoie que celui du jeudi. Reproduction : un athlète dans un défi du Canal qui finit samedi 9 h reçoit jeudi 9 h « Plus que 48 h ». À 18 h, `serie()` appelle `envoyerPush`, qui répond `{envoye:0, raison:'plafond'}`, et la série de 12 semaines casse sans prévenir. De même, un duel clos un jour où la notification « Ta nuit n'est pas encore arrivée » (santé) est déjà partie : ni « Tu as gagné » ni « Revanche ? » n'arrive.

Pourquoi le mettre en place. L'aversion à la perte (« ta série est en danger ») et le résultat d'un duel (« Revanche ? ») sont les deux déclencheurs qui convertissent le mieux dans le modèle Hook. Aujourd'hui, ils passent **après** des rappels administratifs ou de santé. Un plafond est sain, mais il doit arbitrer par valeur, pas par heure d'arrivée.

Intérêt réel. Chez Duolingo, le rappel « streak en danger » du soir est la notification qui ramène le plus de monde (environ +10 à 20 % de sessions le jour J selon leurs publications). Un jeudi ordinaire, dès qu'un athlète a une autre notification (défi, santé, accès), sa relance de série disparaît. Sur une base de 300 athlètes à la série active, avec un défi ou une synchronisation santé actifs, on peut estimer qu'entre 20 et 40 % des jeudis sont touchés. Rétablir le rappel, plus un « dernier appel » le samedi, peut sauver une série sur cinq qui aurait cassé. Or une série cassée est le premier signe de désengagement (la probabilité de revenir à J30 est divisée par 2 environ après une rupture). La clôture de duel qui arrive vraiment relance aussi des revanches, donc des duels en chaîne.

Prompt de mise en place.

```
Contexte : dépôt RepCore, Worker Cloudflare dans cloudflare/src. Les notifications sont limitées à une par jour et par athlète (pushAutorise dans cloudflare/src/metier.js et la transaction push_log dans envoyerPush), mais c'est la première arrivée qui prend la place. Résultat : la « série en danger » du jeudi 18 h (planifies.serie), le J-2 et le résultat des duels (duelQuotidienUn, duelCloturer, 18 h 30) et les réactions (19 h) sont jetés dès qu'une notification moins importante est partie le matin (défis 48 h 9 h, sante_rappel 10 h, acces/retour 11 h). En plus, duels/<id>/rappel est posé avant l'envoi et reactions_push est effacé avant l'envoi : un refus « plafond » perd ces messages pour toujours.
```

À faire :

```
1. Dans metier.js, ajoute une table PUSH_PRIORITE (nombre, plus grand = plus important) : duel_fin 90, serie 85, duel_j2 80, defi 70 (48 h de défi ou saison), retour 60, wrapped 55, badge 50, reactions 45, bilan 40, acces 40, sante 20, relance 30, filleul 50, parcours 60. Ajoute un champ facultatif 'prio' aux messages (duels.js : pushResultat → 'duel_fin', pushRappel → 'duel_j2' ; metier.js : serie, reactions, etc.). Le TYPE (pour les préférences pushPrefs) ne change pas.
2. Remplace le plafond « 1 par jour » par : au plus 2 notifications par jour, et la 2e seulement si sa priorité est ≥ 80 (série, duel). push_log/<uid> devient {jour, n, at, type, prio}. La transaction refuse si n>=2, ou si n>=1 et prio<80. Garde heuresCalmes tel quel.
3. Réservation du jeudi : dans planifies.acces, sante (rappelSanteUn dans sante.js via envoyerPush) et retourUn, n'envoie rien le jeudi entre 8 h et 18 h si l'athlète a une série > 0 non validée cette semaine (streakWeek !== lundiParis(t)). Reporte au lendemain (acces : le lendemain suffit, il reste 2 jours ; sante : on saute ; retour : garde le comportement actuel qui fait taire la série).
4. N'écris duels/<id>/rappel=true et n'efface reactions_push/<uid> qu'APRES un envoi réussi (r.envoye>0), ou si la raison est définitive ('coupe', 'aucun_abonnement'). Pour 'plafond' ou 'calme', laisse l'état pour que le travail du lendemain réessaie (pour le J-2, seulement si fin - t > 12 h).
5. Ajoute un 2e créneau serveur « dernier appel » le samedi 10 h (planif.js : { nom: 'serie_sam', quand: p => p.joursem === 6 && apres(p,10,0), cles: {} => M.abonnes(), un: M.planifies.serieSamedi, cout: 15, push: true }), même logique que serie() avec le tag 'serie-<lundi>-sam', le titre « Dernier week-end pour ta série de N semaines » et le corps « Une séance d'ici dimanche soir et elle continue. » Ce créneau part seulement si le jeudi n'a pas été ouvert entre-temps (lastSession < jeudi 18 h).
Cas limites : urgent (litige PayPal) toujours hors plafond ; un 2e push de prio ≥ 80 ne passe jamais en heures calmes ; les sous-tâches différées (tachePush) gardent leur prio.
Tests : dans cloudflare/test/push.test.mjs et duels.test.mjs, ajoute (a) défi 48 h envoyé à 9 h puis serie à 18 h → les deux partent ; (b) sante puis reactions → reactions refusé ; (c) duel clôturé alors qu'un push est déjà parti → le résultat part (2e, prio 90) ; (d) J-2 refusé en heures calmes → duels/<id>/rappel reste absent ; (e) 3e push de prio 90 → refusé. Lance 'cd cloudflare && npm test'. C'est fini quand tous les tests passent et que planif.js contient serie_sam.
```

2. Défis, duels et saisons : les scores sont déclarés par le client — Priorité : Haute · Effort : M

Fusion. rejoint le point 1 de l'agent Sécurité (aucun champ de valeur écrit par le client).

Constat. Les volts sont passés côté serveur pour empêcher la triche (`xp.js:4-5`), mais pas la compétition. Pour les duels, la saisie est libre : `duels/$id/progres/$k/valeur` accepte n'importe quel nombre < 1e9 écrit par le joueur lui-même (`database.rules.json:1442-1449`). Les défis du Canal acceptent `participants/$k/valeur` et `metrique` écrits par l'athlète (`database.rules.json:667-669`). Les saisons font de même via `saisons_progres/$id/$k` (`database.rules.json:1494-1502`). Le Worker se contente de comparer (`duels.js:15-17`, `scoresDe`, et `participants()` dans `metier.js:431`). S'y ajoute la mesure `seances` (`functions/defis-calcul.js:65`), qui compte aussi les séances à **zéro série** (voir le point 3). Reproduction : dans la console d'un duel en cours, `firebase.database().ref('duels/<id>/progres/<maClé>').set({valeur:999,maj:Date.now()})`. À la clôture, `gagnantDe` désigne le tricheur, qui reçoit le badge CHAMPION (`resultatPour`, +40 V), et l'adversaire reçoit « X remporte le duel ».

Pourquoi le mettre en place. Un classement truqué une seule fois suffit à tuer la motivation sociale de tout un groupe de coaching : les athlètes honnêtes cessent de participer. Le serveur a déjà tout ce qu'il faut. `xp_etat` relit chaque nouvelle séance (`avancer`, `xp.js:132`) et tient même un cumul hebdomadaire `sem` (`xp.js:173-175`) : il suffit d'y ajouter un cumul **par jour**.

Intérêt réel. Les défis d'équipe et les duels sont le principal levier « relationnel » (Octalysis, drive 5), et ce sont les leviers `duel` et `defi` que `retention.js:29-30` mesure déjà. Un score serveur fiable permet de les mettre en avant sans risque, et de les ouvrir entre coaches (point 5). Le coût est faible : pas une lecture de plus, puisque `avancer` lit déjà les séances. Pour le coach, un défi « 12 séances en novembre » dont il peut garantir le résultat se vend comme une animation premium de sa communauté.

Prompt de mise en place.

Contexte : RepCore. Les volts sont recalculés par le Worker (cloudflare/src/xp.js, avancer ; appelé par xpRecalculer dans cloudflare/src/metier.js) à chaque séance terminée. En revanche, les scores des défis du Canal (participants/<k>/valeur, metrique), des duels (duels/<id>/progres/<k>/valeur) et des saisons (saisons_progres/<id>/<k>) sont écrits par l'app elle-même (defiValeur dans app/rc-core.1693.js) et seulement comparés par le Worker. N'importe qui peut écrire 999 depuis la console.

À faire :

1. xp.js / avancer : en plus de e.sem, tiens e.jr = {AAAA-MM-JJ: {n, ton, pr}} pour les 40 derniers jours. n = séances à ≥1 série validée (seriesValidees>0), ton = somme poids*xreps des séries done (même règle que tonnageSeance dans fonctions/defis-calcul.js), pr = liste compacte {cleExo: charge max du jour}. Purge au-delà de 40 jours comme e.jours. Exporte une fonction PURE valeurServeur(etat, mesure, debut, fin, quota) : seances = Σ n, tonnage = Σ ton, serie = nombre de semaines calendaires (lundiDuJour) où Σ n ≥ quota, progressionPct = moyenne des (max pendant / meilleur avant debut - 1)×100 (avant = e.meilleurs figé à l'ouverture : stocke un instantané e.ref[idDefi] au premier calcul après debut).
2. metier.js / xpRecalculer : après avancer, lis la liste des duels actifs du joueur (index à créer : duels_joueur/<k>/<id>=true, posé par duelEvenement) et des saisons suivies (saisonSuivie). Écris toi-même duels/<id>/progres/<k> = {valeur: valeurServeur(...), maj:t, srv:true} et saisons_progres/<id>/<k>. Pour les défis du Canal, fais de même si users/<k>/coachEmailKey a un défi en cours (defisDuCoach).
3. database.rules.json : retire le .write client sur duels/\$id/progres/\$k, saisons_progres/\$id/\$k et participants/\$k/valeur|metrique (le Worker passe outre les règles). Mets à jour scripts/verif/regles.mjs et les tests émulateur cloudflare/test/regles-evenements.emu.mjs.
4. app/rc-core.1693.js : garde defiValeur pour l'affichage immédiat de la jauge (optimiste), mais n'écris plus en base. Affiche la valeur serveur dès qu'elle arrive.

Cas limites : quota = sessions_config actifs (lis-le une fois par recalcul) ; un duel démarré (démarrage) avant la séance qui le lance compte à partir de debut ; une séance à 0 série ne compte jamais ; budget du Worker : au plus 4 écritures de progression par recalcul, le reste en sous-tâche (différer).
Tests : cloudflare/test/xp.test.mjs (valeurServeur sur des jeux fixes, identiques aux fixtures de fonctions/test/defis.test.js) et cloudflare/test/duels.test.mjs (un client qui écrit 999 est refusé ; le score suit les séances). `cd cloudflare && npm test` au vert, plus `node scripts/verif/regles.mjs`.

3. Semaine validée : deux définitions, et les séances vides comptent — Priorité : Haute · Effort : S

Constat. La série stockée (updateStreak, app/rc-core.1693.js:65486) crédite une semaine dès que le **nombre de séances sur les 7 derniers jours glissants** atteint le quota. Les volts « semaine » (+150), SEMAINE VALIDÉE, SANS FAUTE, PHÉNIX et les duels « série » utilisent au contraire la **semaine calendaire** (_badgesFaits, rc-core:76368-76385, et defis-calcul.js:67-71). Or INARRÊTABLE prend le maximum des deux (f.serie, rc-core:76403). Par ailleurs, « Abandonner » avec 0 série enregistre quand même une séance (finishWorkout, rc-core:49912) : elle compte pour la série, ASSIDU, la 1re et la 2e séance du parcours (+50 V) et la mesure seances, alors que le serveur ne la compte pas (xp.js:146). Reproduction : quota de 3, séances vendredi, samedi et dimanche (semaine validée), puis le lundi, ouvrir une séance et l'abandonner immédiatement. Le compteur passe à +2 semaines, sans les 150 V ni la semaine calendaire, et le push du jeudi se tait (streakWeek === lundi).

Pourquoi le mettre en place. Une série n'a de valeur que si elle est juste et difficile à contourner : c'est ce qui rend la peur de la perdre crédible. Deux règles qui se contredisent font afficher « 9 semaines » à côté d'une vitrine qui en dit 7 et d'un Wrapped qui en dit 6. Cela détruit la confiance dans tous les compteurs.

Intérêt réel. Chez Strava comme chez Duolingo, l'intégrité du streak est protégée parce que c'est le premier prédicteur de la rétention à J30. Une règle unique (semaine calendaire, séances à ≥1 série validée) rend le push « série en danger » exact, supprime la ferme à séances vides (qui fausse aussi ASSIDU et la rareté des badges) et aligne l'app, le Worker (serieDuJour) et les duels. L'effort est faible : une fonction, plus un test.

Prompt de mise en place.

Contexte : app/rc-core.1693.js. updateStreak crédite une semaine quand le nombre de séances des 7 derniers jours GLISSANTS atteint le quota (seancesPrevuesParSemaine) ; _badgesFaits, xpCalcul (volts « semaine »), les duels et defis-calcul.js utilisent la semaine CALENDRAIRE (lundi-dimanche). De plus, finishWorkout pousse une séance même à 0 série validée (« Abandonner » tout de suite), et elle compte pour la série, ASSIDU (f.seances), les étapes premiere_seance/deuxieme_seance du parcours et defiValeur('seances').

À faire :

1. Crée une fonction PURE seanceComptee(s) = au moins une série done===true dans s.data (ou s.sets>0 pour l'ancienne forme). Utilise-la dans : updateStreak (faites), _badgesFaits (f.seances, semaine[cle].n), PARCOURS DEMARRAGE (via f.seances), defiValeur (mesure seances et serie), wrapped (nombre de séances), et fonctions/defis-calcul.js valeurDefi (même règle, les fixtures des deux bancs doivent rester alignées).
2. updateStreak : remplace la fenêtre glissante par la semaine calendaire courante : faites = séances comptées dont _lundiDe(date) === _lundiDe(now). Ne touche ni aux jokers ni à _streakPerime.
3. Laisse l'historique existant tel quel (pas de recalcul rétroactif de u.streak) : la nouvelle règle vaut pour les semaines futures.
4. Côté serveur, vérifie que serieDuJour (cloudflare/src/metier.js) reste cohérent (il lit streakWeek) ; aucun changement attendu.

Cas limites : une séance à cheval sur minuit dimanche compte pour la semaine de sa date de fin ; un athlète à quota 1 ; une suspension active (updateStreak retourne tôt, inchangé) ; une séance importée sans data mais avec sets>0.
Tests (app/tests.js) : (a) ven/sam/dim + lundi abandonné à 0 série → streak +1 seulement ; (b) ven/sam/dim + lundi réel → la semaine du lundi n'est PAS validée avant 3 séances dans la semaine ; (c) ASSIDU I ne compte pas les séances vides ; (d) mêmes fixtures dans fonctions/test/defis.test.js. Lance la suite de tests front (voir l'en-tête de app/tests.js) et `cd fonctions && npm test`. Fini quand tout est vert.

4. Aucune boucle quotidienne ni récompense variable les jours de repos — Priorité : Haute · Effort : M

Constat. Les jours sans entraînement, l'app ne propose que des actions à gain fixe : check-in 10 V, journal 15 V, sommeil 5 V, cible 40 V (XP_ACTIONS, rc-core:79108). Il n'existe ni quête du jour, ni coffre, ni bonus aléatoire (aucune occurrence de « mission du jour », « quête », « coffre » ou « double volts »). La série étant hebdomadaire, rien ne se joue au quotidien. Seule exception, la série d'assiette, réservée à qui a une cible alimentaire (_rendreSerieAssiette, rc-core:79293). Quant au rappel local « Séance du jour », il dépend du réveil périodique du service worker (sw.js:824-857), qui n'est pas fiable sur un téléphone.

Pourquoi le mettre en place. Dans le modèle Hook, la récompense **variable** est ce qui crée l'envie de revenir ; une récompense fixe s'utilise en 2 à 3 semaines. En musculation, 4 jours sur 7 sont des jours sans séance : ce sont eux qui font chuter le taux d'athlètes actifs chaque jour par rapport au mois (DAU/MAU, déjà mesuré dans retention.js:118).

Intérêt réel. Duolingo et Whoop tiennent un DAU/MAU de 40 à 55 % grâce à une action quotidienne de 30 secondes. Une app de musculation sans boucle quotidienne plafonne vers 20 à 25 %. Une « mission du jour » en 3 cases, qui s'adapte au jour (séance ou repos), avec un « coffre » à gain variable, peut raisonnablement viser +10 points de DAU/MAU et +3 à 5 points de J30. Pour le coach, les missions des jours de repos poussent précisément ce dont il a besoin : check-in, sommeil, protéines, mobilité. Ses bilans sont mieux remplis et il repère plus tôt un athlète qui décroche.

Prompt de mise en place.

Contexte : app/rc-core.1693.js (accueil athlète : loadClientHome, en-tête .clh-tete, xpCalcul / XP_ACTIONS / XP_PLAFOND_JOUR ~1.79100-79900, parcours PARCOURS_DEMARRAGE pour le modèle d'étapes). Le Worker borne les volts hors séance dans cloudflare/src/xp.js (totalServeur).

À faire : « LA MISSION DU JOUR ».

1. Fonction PURE missionsDuJour(u, jour) → 3 missions déterministes (graine = hash(clé du compte + jour), aucun aléa côté serveur), prises dans une table MISSIONS selon le type de jour (nutIsOnDay / séance prévue ce jour-là dans sessions_config) :
 - jour de séance : « Termine ta séance », « Bats 1 record ou fais +1 rép sur ton 1er exercice », « Note ton RIR sur toutes les séries » ;
 - jour de repos : « Check-in du matin », « 10 min de mobilité (bouton qui lance un minuteur) », « Atteins tes protéines », « Saisis ta nuit », « Regarde ta prochaine séance et ses records à portée (_recordsAPorteeParJour) », « Réagis à la séance d'un ami ».N'utilise jamais une mission impossible (pas de protéines sans cible, pas d'ami sans carnet, rien de nutritionnel sous atCA).
2. Carte « Mission du jour » sous l'en-tête d'accueil : trois cases, se cochent seules (relues à chaque rendu depuis les faits : u.checkin[jour], sleepLog,

```

_badgesFaits...). Aucune saisie déclarative.
3. LE COFFRE (récompense variable) : les 3 cases faites → bouton « Ouvrir ». Tirage déterministe (même graine) : 60 % +20 V, 25 % +50 V, 10 % un joker de série (si streakJokers < STREAK_JOKERS_MAX, sinon +50 V), 5 % « Volts doublés sur ta prochaine séance » (plafond 400 inchangé). Animation courte (réutilise arcCompteur). Stocke u.missions[jour] = {faites:[...], coffre:{gain, at}} ; nouvelle catégorie xpCalcul cat.mission, SOUS le plafond du jour.
4. Série de missions : 7 jours d'affilée avec coffre → badge unique « SEPT SUR SEPT » (ajoute-le à BADGES_ACQUIS, à database.rules.json et à scripts/verif/regles.mjs, qui compare les listes).
5. Worker : xp.js totalServeur borne cat.mission à jours × 50, et le joker n'est accepté que s'il est daté dans u.missions.
Cas limites : changement de fuseau (jour local), coach (rien), suspension (missions en pause, sans coffre), compte de moins de 2 jours (missions du parcours à la place).
Tests (app/tests.js) : déterminisme (même jour → mêmes missions et même coffre), aucune mission nutritionnelle sous atCA ou sans cible, cases cochées depuis les faits, gain dans le plafond, badge SEPT SUR SEPT au 7e jour ; cloudflare/test/xp.test.mjs pour la borne. Fini quand les tests passent et que la carte s'affiche à l'accueil un jour de repos.

```

5. Ligues hebdomadaires avec montée et descente — Priorité : Haute · Effort : L

Constat. La compétition se limite au carnet d'amis (classement de la semaine classementSemaine, rc-core:21789, qui demande de suivre quelqu'un), aux duels à deux et aux défis que le coach crée à la main. Un athlète sans ami inscrit, soit la majorité au lancement, n'a aucun adversaire. Pourtant, le serveur calcule déjà des volts hebdomadaires infalsifiables (e.sem, xp.js:173-175, publiés dans volts_publics/<pseudo>.sem, metier.js:1268-1271).

Pourquoi le mettre en place. Les ligues de Duolingo (30 joueurs de niveau proche, montée et descente chaque semaine) ont été leur plus gros gain d'engagement mesuré. Elles donnent à **chaque** utilisateur une compétition à sa taille, qui se renouvelle chaque semaine, sans rien demander au coach.

Intérêt réel. Duolingo a publié +17 % de temps d'apprentissage et un triplement des utilisateurs très engagés après l'arrivée des ligues. Ici, les données existent déjà côté serveur, ce qui supprime le risque de triche. Une ligue de 20 athlètes répartis par rang donne une raison de faire la 3e séance le dimanche (« tu es 6e, le top 5 monte ») et fournit au Wrapped et au profil public un nouveau trophée partageable (« Ligue FOUDRE, 1er »). C'est aussi un argument de vente pour l'offre Essentielle face à Hevy, dont le social n'a pas de ligues.

Prompt de mise en place.

```

Contexte : RepCore. Le Worker (cloudflare/src/xp.js avancer) tient pour chaque compte etat.sem = {lundi: {v: volts d'entraînement de la semaine, n: séances}} – infalsifiable – et le publie dans volts_publics/<pseudo>.sem (metier.js, xpRecalculer). Les travaux planifiés sont dans cloudflare/src/planif.js (cursors par lots, budget 50 sous-requêtes).

À faire : LES LIGUES.
1. Nouveau module PUR cloudflare/src/ligues.js : 6 divisions (BRONZE, ACIER, VOLTAGE, FOUDRE, TITAN, LÉGENDE), groupes de 20. repartir(comptes, t) : chaque lundi 00 h 30 (Paris), les comptes actifs la semaine passée (sem.n>=1) ou inscrits depuis moins de 14 jours sont rangés par division, puis mélangés de façon déterministe (graine = lundi) en groupes de 20. cloturer(groupe, semV) : top 5 montent, 5 derniers descendent (sauf BRONZE), les autres restent ; égalité départagée par n puis par l'heure de la dernière séance. Aucun compte inactif 2 semaines n'est reclassé (il sort des ligues et y rentre à sa prochaine séance, dans sa division).
2. metier.js : travail 'ligues' (lundi 00 h 30) qui clôt la semaine passée (écrit ligues_resultats/<k>/<lundi> = {division, place, mouvement}) et crée ligues/<lundi>/<groupe> = {division, membres:{pseudoOuAlias: true}}. Le score se lit dans xp_etat/<k>.sem (lecture serveur). Écris ligues_public/<lundi>/<groupe> = [{nom (pseudo ou « Athlète 7 » si pas de page publique), v, n}] mis à jour dans xpRecalculer (une écriture de plus quand le compte est en ligue).
3. Pushs (via la priorité du plafond, voir point 1) : samedi 11 h « Tu es 6e de ta ligue FOUDRE : le top 5 monte » seulement à la place 6-8 ou 13-16 (zone de bascule) ; lundi 9 h le résultat (« Tu montes en TITAN < »).
4. database.rules.json : ligues_public lisible par les membres du groupe seulement (ligues_membres/<k> = {lundi, groupe}) ; aucune écriture client.
5. app/rc-core.1693.js : carte « Ma ligue » à l'accueil (sous le rang) et écran liste (zones montée/descente colorées), badges uniques « PROMU » (1re montée) et « SOMMET » (1er de LÉGENDE) ajoutés à BADGES_ACQUIS + règles + scripts/verif/regles.mjs.
Cas limites : moins de 8 actifs dans une division → fusion avec la division voisine ; coach exclus ; opt-out (u.liguesOff) ; suspension = hors ligue sans descente.
Tests : cloudflare/test/ligues.test.mjs (répartition déterministe, montée/descente, fusion des petites divisions, inactifs, égalités) ; test front de rendu de la carte. `cd cloudflare && npm test` au vert.

```

6. Rangs mal équilibrés : la nutrition pèse autant que l'entraînement, et la fin de parcours est un désert — Priorité : Moyenne · Effort : S

Constat. Le commentaire de calibration (rc-core:79348-79355) et le test « la courbe tient le calendrier » (app/tests.js:50310) ignorent le journal, alors que les lots N2 et N4 ont ajouté jusqu'à 70 V par jour (journal 15, cible 40, check-in 10, sommeil 5) plus 75 V par semaine d'assiette. Un athlète qui remplit tout gagne donc environ 1 140 V par semaine au lieu d'environ 650, et atteint LÉGENDE en **environ 17 mois au lieu de 30**. Quelqu'un qui ne fait que noter ses repas atteint MACHINE en environ 15 semaines **sans soulever une barre**. Les écarts entre rangs ne sont pas monotones : 6 500 → 6 000 et 9 000 → 8 000 (RANGS, rc-core:79547). Il faut ensuite 32 000 V entre TITAN et LÉGENDE, soit environ un an avec des records qui se raréfient, et rien au-delà de LÉGENDE. Enfin, la relance J+7 promet « Une séance suffit à garder ton rang » (cloudflare/src/retour.js:93), alors qu'un rang ne se perd jamais : la promesse est fausse.

Pourquoi le mettre en place. Un rang de musculation gagné en saisissant des repas perd sa valeur de statut, qui est le moteur de sa mise en avant (profil public, carte de rang). Une fin sans nouveau palier fait décrocher les athlètes les plus engagés, qui sont aussi les meilleurs ambassadeurs.

Intérêt réel. Garder la courbe d'origine (IMPULSION en environ 2 semaines, VOLTAGE en environ 1 mois : des premiers passages rapides, décisifs pour J7/J30) tout en rendant chaque rang crédible. Un système de « prestige » après LÉGENDE (Call of Duty, Clash) garde les athlètes à 2 ans et plus, précisément ceux qui paient 24,90 €/mois depuis longtemps. Corriger la phrase J+7 retire une promesse fausse qui abîme la confiance dès qu'on s'en aperçoit.

Prompt de mise en place.

```

Contexte : app/rc-core.1693.js – XP_ACTIONS (~l.79108), commentaire de calibration + RANGS (~l.79347-79560), xpCalcul (~l.79704) ; test « Volts : la courbe tient le calendrier » dans app/tests.js (~l.50310). Constantes miroir dans cloudflare/src/xp.js (XP, RANGS, un test compare les deux) et RANGS_NOMS dans cloudflare/src/retour.js.

À faire :
1. Plafonne la part hors entraînement : dans xpCalcul, les catégories nutrition + cible + sommeil + checkin + semaineAssiette ne peuvent pas dépasser, sur 7 jours glissants, 60 % des volts d'entraînement (seance+complete+record+semaine) de la même fenêtre + 150 V (plancher pour un athlète blessé ou suspendu). L'excédent est écreté (ajoute-le à 'ecrete'). Même borne dans xp.js totalServeur.
2. Lisse RANGS pour des écarts croissants et une fin atteignable : 0, 1800, 3800, 7500, 13000, 20000, 28500, 38500, 51000, 68000 (ajuste pour que le test calibré tombe dans ±25 % : IMPULSION ~2 sem., VOLTAGE ~1 mois, MACHINE ~2 mois, ÉLITE ~4 mois, SURTENSION ~6-7 mois, MONSTRE ~10 mois, FOUDRE ~14 mois, TITAN ~19 mois, LÉGENDE ~26 mois). Les rangs déjà fêtés (u.xpRang) ne redescendent jamais : si le total tombe sous le seuil du rang fêté, affiche le rang fêté.
3. PRESTIGE après LÉGENDE : chaque tranche de 20 000 V au-delà = « LÉGENDE *N » (étoile affichée, pas d'image nouvelle), fêtée petit comme un sous-niveau (_celebrerSousNiveau).
4. Ajoute au test de calibration un 2e athlète « tout remplir » (journal quotidien à la cible 5 j/7, check-in et nuit quotidiens) : LÉGENDE pas avant 20 mois ; et un 3e « nutrition seule, 0 séance » : jamais au-delà de VOLTAGE en 6 mois.
5. Mets le commentaire de calibration à jour (il dit « +20 V par jour au plus », faux depuis N2/N4).
6. cloudflare/src/retour.js messageRetour palier 7 : remplace « à garder ton rang X » par « pour avancer vers <rang suivant> » (ajoute les seuils, ou lis xp_serveur/<k>.total pour le reste à faire : « plus que 1 240 V avant MACHINE »). Mets à jour cloudflare/test/retour.test.mjs.

```

Garde en phase xp.js (RANGS, borne) et rc-core (le test miroir de xp.test.mjs doit rester vert). Lance les tests front et `cd cloudflare && npm test`. Fini quand les trois athlètes de calibration passent.

7. Relances de réengagement : trop tardives, génériques, sans « la veille » — Priorité : Moyenne · Effort : S

Constat. Le premier message à un inactif part à J+7 (RETOUR_PALIER=[7,14,30], retour.js:20), alors que la série casse dès J+11 pour un quota de 3 (ecartNormalJours+7, rc-core:63026). Rien ne part à J+3 ou J+4, au moment où l'habitude se rompt. Aucune notification serveur ne signale « demain, séance Push A, 2 records à portée » : l'anticipation reste confiée à l'alarme locale du service worker, peu fiable. Le push « badge proche » ne couvre qu'ASSIDU (metier.js:376-385) et compte les séances par shallow(), donc séances vides comprises. Rien non plus pour BRISEUR, TONNAGE ou un palier de série (SERIE_PALIER, rc-core:65385), qui sont pourtant les plus proches pour un athlète régulier. Enfin, la relance J+30 renvoie vers `./?reprise=1` sans rien célébrer une fois la séance faite.

Pourquoi le mettre en place. Dans le modèle Hook, un déclencheur externe est efficace **avant** la rupture, et quand il porte une information personnelle (« ton record à 82,5 kg est à portée ») plutôt qu'un rappel générique. Le « presque atteint » (effet de gradient d'objectif) est l'un des déclencheurs les plus puissants, et l'app a déjà toutes les données pour le calculer.

Intérêt réel. Relancer à J+3 ou J+4, quand l'athlète est encore récupérable, rapporte en général 2 à 3 fois plus que relancer à J+7 (quelqu'un d'absent depuis une semaine ne revient qu'environ une fois sur deux). Les notifications « presque » (badge ou série à 1 semaine) ont un taux d'ouverture bien au-dessus d'un rappel d'agenda. Pour le coach, moins d'athlètes glissent en « décrochage » : moins de relances manuelles, moins de résiliations au 2e mois.

Prompt de mise en place.

Contexte : cloudflare/src/retour.js (paliers 7/14/30, messageRetour), cloudflare/src/metier.js (retourUn, planifies.badge ~1.376), cloudflare/src/planif.js (travaux). L'app publie déjà dans le dossier : streak, streakWeek, lastSession, sessions_config, xpRang, tonnageTotal ; le Worker tient xp_etat/<k> (meilleurs, sem, faites).

À faire :

1. Palier PRÉCOCE : ajoute à RETOUR_PALIER un palier dynamique = ecartNormal+1 jours (ceil(7/créneaux actifs)+2 ; 4 jours pour 3 séances/sem.), avant 7. Message : titre « <Prénom>, ta semaine se joue maintenant », corps « Ta série de N semaines tient si tu fais ta séance d'ici dimanche. » (sans série : « Ta prochaine séance t'attend, elle prend 45 min. »). RETOUR_MAX passe à 4. Garde l'anti-doublon avec la série (retourAutorise / retourRecent).
2. LA VEILLE : nouveau travail 'veille' à 19 h 30 : pour chaque abonné dont DEMAIN est un créneau actif de sessions_config, et qui n'a pas de séance aujourd'hui, push type 'serie' prio 50 : « Demain : <nom du créneau> » + « <exercice> : <charge> kg, ton record est à portée » si xp_etat.meilleurs donne un exercice du créneau dont la dernière charge est ≥ 97,5 % du record (l'app écrit déjà les noms de créneaux dans sessions_config). Au plus 2 fois par semaine par athlète (worker/veille/<k> = [dates]).
3. BADGE PROCHE élargi (dimanche 17 h) : remplace le compte shallow par xp_etat/<k>.faites (séances à ≥1 série) ; ajoute BRISEUR (records : compte depuis xp_etat.s.record/50) à ≤2, et PALIER DE SÉRIE (streak+1 ∈ [4,8,12,26,52]) : « Une semaine de plus et tu passes 12 semaines d'affilée ». Choisis le plus proche, un seul push.
4. LE RETOUR FÊTE : dans l'app (app/rc-core.1693.js, fin de séance), si la séance précédente date de ≥ 14 jours, affiche la carte « Retour au combat » (le médaillon RETURN existe dans BADGES) et +50 V « retour » une fois par période (catégorie xpCalcul bornée côté xp.js).

Tests : cloudflare/test/retour.test.mjs (palier précoce selon le quota, max 4, pas de doublon avec la série), nouveau test veille (demain actif, record à portée, 2/semaine), planif (le travail existe) ; test front du bonus retour. `cd cloudflare && npm test` et tests front au vert.

8. Des saisons qui n'existent que si Kevin les crée — Priorité : Basse · Effort : M

Constat. Le moteur des saisons est complet : compteur collectif, badge Édition, notifications de lancement, de mi-parcours, J-2 et fin (saisons.js, metier.js:1176). Mais une saison n'existe que si Kevin la saisit à la main dans l'écran d'administration (database.rules.json : .write réservé à son e-mail). Sans calendrier, il n'y aura pas d'édition le mois où il n'y pense pas. Et les défis d'équipe dépendent de chaque coach (defisQuotidienCoach).

Pourquoi le mettre en place. Un rendez-vous collectif récurrent et prévisible (« le défi de novembre ») crée de l'attente et une raison de revenir en début de mois, comme les défis mensuels de Strava, qui sont le cœur de leur engagement social. Un badge daté « jamais réédité » (rareté par la date, principe de rareté d'Octalysis) motive fortement les collectionneurs.

Intérêt réel. Strava rapporte que les participants à ses défis mensuels restent nettement plus actifs, et les badges datés sont très partagés (et la vitrine de rareté existe déjà). Un calendrier de 12 éditions préparé une fois assure un pic d'activité chaque mois sans aucun travail de Kevin ni des coaches. Il alimente aussi le Wrapped (« 3 éditions bouclées cette année ») et crée un objectif collectif interclubs (« 10 000 séances RepCore en novembre ») qui rend visible la taille de la communauté.

Prompt de mise en place.

Contexte : cloudflare/src/saisons.js (moteur pur), metier.js saisonsHeure (chaque heure), /saisons écrit à la main par Kevin (database.rules.json, .write réservé à son e-mail). Aucune saison n'existe sans saisie manuelle.

À faire :

1. Nouveau fichier cloudflare/src/calendrier-saisons.js : un tableau de 12 modèles mensuels {mois, nom, mesure, objectifPerso, objectifCollectifParParticipant, couleurAccent, texteAccueil, badgeCle}. Exemples : janvier « Résolution tenue » (serie, 4 semaines validées), mars « Mars en fonte » (tonnage, 40 000 kg), mai « Progression de printemps » (progressionPct, +3 %), juillet « Été sans pause » (seances, 12), novembre « Novembre de fer » (seances, 14), décembre « Finir fort » (serie, 3 semaines). Les objectifs se calibrent sur un athlète à 3 séances/semaine qui y arrive à 80 %.
2. Travail 'saisons_auto' le 25 de chaque mois à 12 h (planif.js) : si /saisons ne contient aucune saison dont debut tombe le mois suivant, crée saisons/<mois>-<annee> depuis le modèle (debut = 1er 00 h Paris, fin = dernier jour 23 h 59, objectifCollectif = participants du mois précédent × objectifCollectifParParticipant, au moins 1). Une saison posée à la main par Kevin pour ce mois l'emporte toujours.
3. App (app/rc-core.1693.js) : bandeau d'accueil « La saison <nom> commence dans N jours » du 25 au dernier jour du mois, avec un bouton « Je participe » (inscription = abonnement aux notifications de la saison, ce qui prépare les destinataires de 'mi' et 'j2').
4. Wrapped (wrappedSlides) : ajoute la ligne « Édition <nom> bouclée » quand saisons_resultats/<moi>-<id> existe pour la période.

Tests : cloudflare/test/saisons.test.mjs (création le 25, pas de doublon, priorité à la saison manuelle, objectifCollectif ≥ 1, changement d'année décembre → janvier), test front du bandeau. `cd cloudflare && npm test` au vert.

Courbe de rétention attendue (estimation, en l'absence de données publiées dans le dépôt : /stats/retention est calculé chaque nuit par retention.js). État actuel pour un athlète autonome en essai : J1 ≈ 35-40 % (le parcours de 14 jours aide), J7 ≈ 20-25 %, J30 ≈ 10-14 %. Pour un athlète suivi par un coach, bien plus haut (J30 ≈ 55-70 %, porté par la relation). Avec les points 1, 3, 4 et 7 (fiabilité des déclencheurs et boucle quotidienne) : J7 +4 à 6 points et J30 +3 à 5 points en autonomie. Avec les ligues (point 5) : J30 +2 à 4 points de plus. À vérifier dans l'écran des leviers de retention.js, qui dispose déjà d'un seuil de 30 personnes par groupe.

Agent 11 — Marketing et acquisition · 6/10

La mécanique d'acquisition est au-dessus de ce qu'on voit d'habitude à ce stade. Le parrainage est double face, avec un anti-fraude sérieux (fonctions/parrainage-calcul.js : e-mail normalisé, délai de 7 jours). Les ambassadeurs sont commissionnés (ambassadeurs-calcul.js). Les liens sont attribués par une fonction unique (lienAttribue, rc-core:19562), les pages /@pseudo et /coach/<slug> ont un aperçu Open Graph personnalisé, et la rétention est suivie par cohortes [1]/7/30 (cloudflare/src/retention.js). La landing a un titre concret (« La bonne charge, à chaque série. »). Mais le tunnel perd ce que la viralité apporte. La promesse faite à un invité change selon la page (1 ou 2 mois). Il n'y a aucune preuve sociale, aucun parcours pour les coachs tiers alors que c'est la moitié de la cible. Le SEO se limite à 3 articles orphelins avec un prix faux dans les données structurées, et l'APK n'est pas sur le Play Store.

1. Invité ou ambassadeur : la promesse change d'une page à l'autre, et la landing l'efface — Priorité : Critique · Effort : S

Constat. L'app, le Worker et la carte d'invitation donnent **2 mois** à l'invité (tarifs.json `essai_parrainage.moisEnPlus: 1` ; invitationDonnees rc-core:20648 mois: `essai.mois + moisEnPlus` ; BONUS_ESSAI_JOURS = 30 dans cloudflare/src/metier.js:42). Sur `/i?ref=...`, la page affiche d'abord « 2 mois d'accès complet » (`/i/index.html:97`), puis `poserNom` remplace ce texte par « Julie t'offre ton premier mois » : l'offre passe de 2 mois à 1 sous les yeux du visiteur. La FAQ de la landing, visible et en JSON-LD (`index.html:83-86`), dit aussi « Son lien t'offre ton premier mois ». Enfin, le lien ambassadeur mène à la landing (`amblienInvitation` → `RC_URL_VITRINE + ?amb=`, rc-core:19974), qui pose `body.invite` (`index.html:966`) mais ne change rien : le CSS vise des éléments `.ami` qui n'existent plus (`index.html:447`, un seul `.cadeau` ligne 736). Les commentaires rc-core:19845 et 20202 (« `essai_parrainage` vaut 0 ») sont périmés. Pour reproduire : ouvrir `/?amb=CODE` ou `/?ref=JULIE7K2`, la page affiche « 1 mois d'essai » sans nom ni avantage ; ouvrir `/i/?ref=<code valide>` et regarder « 2 mois » devenir « ton premier mois ».

Pourquoi le mettre en place. Un invité qui a vu « 2 mois » sur la story de son ami puis lit « 1 mois » sur la page d'arrivée doute de l'offre, et ce doute se paie à l'inscription. Pour un ambassadeur payé à la commission, c'est son propre argument qui disparaît, et il arrêtera de relayer.

Intérêt réel. Le trafic invité est le mieux qualifié de tous : quelqu'un de confiance a recommandé l'app. Sur les SaaS grand public, le taux d'inscription d'un visiteur invité est de 2 à 4 fois celui d'un visiteur froid, à condition que la page confirme ce qu'on lui a promis. Afficher « Julie t'offre 1 mois : 2 mois d'essai au lieu d'1 » en tête de la landing et de `/i`, avec le prénom ou le nom de l'ambassadeur (déjà lisibles dans `/parrainage/codesPublics` et `/ambassadeurs_publics`), c'est reprendre ce que fait déjà `/i`. Sur 100 clics ambassadeurs par mois, gagner 5 points d'inscription donne 5 essais de plus. Avec 20 % de passage à l'abonnement et 9,50 € × 12, cela fait environ 110 € de revenu annuel par ambassadeur et par mois de trafic. Le coût est quasi nul, car les données et les textes existent déjà.

Prompt de mise en place.

Contexte : RepCore, dépôt /home/user/coaching. L'essai parrainé dure `TARIFS.essai.mois + TARIFS.essai_parrainage.moisEnPlus` (= 2 mois, `tarifs.json`). L'app (rc-core:20648 `invitationDonnees`) et le Worker (`cloudflare/src/metier.js` BONUS_ESSAI_JOURS) le donnent déjà, mais les pages publiques le disent mal.

À faire :

- `i/index.html`, fonction `poserNom` : quand `ami=true` (arrivée `?ref=`) et `demi=false`, le texte doit garder le nombre de mois parrainé : « <Prénom> t'offre un mois : 2 mois d'essai, toute l'app ouverte, sans carte bancaire ». Lis le « 2 » dans l'élément `#mois-ami` (`data-nb="essai.moisParraine"`, posé par `scripts/tarifs.mjs`), jamais en dur. Même règle pour la branche ambassadeur hors `ultime_demi`.
- `index.html` (landing) : quand `?ref=` ou `?amb=` est présent (le bloc qui pose `body.invite`, vers la ligne 966), afficher un bandeau d'accueil au-dessus du `h1` : « <Prénom> t'invite sur RepCore » + « 2 mois d'essai au lieu de 1 ». Le prénom est lu par `fetch` sur `https://repcore-sync-default-rtdb.firebaseio.com/parrainage/codesPublics/<CODE>.json` (champ `prenom`), et pour un ambassadeur sur `/ambassadeurs_publics/<CODE>.json` (champ `nom`), comme le fait `i/index.html`. Écris-le avec `textContent` seulement. Si la lecture échoue, afficher le bandeau sans prénom. Avec un code ambassadeur dont l'avantage vaut 'ultime_demi', afficher l'offre demi-prix (reprendre la logique de `i/index.html` ligne ~236) et pas les 2 mois. Rendre visible le second `.cadeau` (classe `.ami`) dans `#tarifs` quand `body.invite` est posé, avec le nombre parrainé.
- `index.html`, FAQ visible ET JSON-LD FAQPage : « Un ami m'a invité : qu'est-ce que ça change ? » → « Son lien double ton essai : 2 mois, toute l'app comprise, sans carte bancaire. Et le jour de ton premier paiement, ton ami gagne un mois. » Le 2 passe par `data-nb="essai.moisParraine"` dans la version visible. Le JSON-LD reste du texte, donc ajoute dans `scripts/tarifs.mjs` une réécriture ou une vérification de cette phrase.
- Corriger les commentaires périmés rc-core:1693.js lignes ~19845 et ~20202 (« vaut 0 ») : l'essai parrainé vaut `moisEnPlus = 1` mois en plus.
- `scripts/verif/tarifs.mjs` : ajouter un contrôle qui échoue si `index.html` ou `i/index.html` contient « ton premier mois » à côté de « `ami` » ou « `invite` » alors que `moisEnPlus > 0`.

Tests : `node scripts/tarifs.mjs` puis `node scripts/verif/tarifs.mjs` doivent passer. Ajouter dans `app/tests.js` (bloc « Parrainage ») un test : `invitationDonnees(u).mois === TARIFS.essai.mois + TARIFS.essai_parrainage.moisEnPlus`. Vérification manuelle en local : `/?ref=ABCDE23` et `/i/?ref=ABCDE23` affichent 2 mois partout, et `?amb=XXX` affiche le nom de l'ambassadeur.

Terminé quand : aucune page publique n'annonce un nombre de mois différent de ce que l'app donne, pour `?ref=`, `?amb=` (mois en plus) et `?amb=` (`ultime_demi`).

2. La moitié B2B n'a pas de porte d'entrée : aucune page pour les coachs tiers — Priorité : Haute · Effort : M

Constat. La landing ne parle qu'à l'athlète autonome : « Application de musculation », tarifs Essentielle et Ultime, coaching par Kevin (`index.html:545-793`). Il n'y a pas un mot sur les offres coach 0/19/39 € (`tarifs.json` `coach`). Le pitch coach n'existe qu'à l'intérieur de l'app, après le choix du rôle (`app/index.html:1007`, « Gratuit pour votre premier client... »). L'article B2B `blog/suivi-athletes-sans-tableur.html:117` renvoie le coach vers `../app/`, l'écran athlète. La vitrine d'un coach (`c/index.html:182`) finit sur un lien vers la landing athlète, sans `src` et sans « Tu es coach ? ».

Pourquoi le mettre en place. Trainerize, TrueCoach, Everfit et Kahunas vivent tous d'une page « pour les coachs » avec les bénéfices, les tarifs par nombre de clients, la migration et une démo. Un coach qui arrive sur RepCore aujourd'hui ne trouve pas l'offre qui le concerne. Chaque coach acquis amène en plus 5 à 30 athlètes : c'est le levier B2B2C le plus rentable du produit.

Intérêt réel. Un coach à 19 € ou 39 € par mois vaut 2 à 4 abonnés Essentielle. Il fait aussi circuler RepCore auprès de tous ses athlètes : visuel « COACHÉ AVEC REPCORE » (rc-core:18278), vitrine, défis. Une page `/coachs` indexable sur les requêtes « logiciel coaching sportif » et « application coach sportif » et « alternative Trainerize » peut capter 50 à 200 visites qualifiées par mois en quelques mois, ce qui est peu de volume mais très rentable. Ajouter « Propulsé par RepCore : tu es coach ? Crée ta vitrine » en pied de chaque vitrine transforme chaque coach en canal d'acquisition de coachs. Le compteur `role_selected_coach` existe déjà (rc-core:10614) pour mesurer l'effet.

Prompt de mise en place.

Contexte : RepCore, dépôt /home/user/coaching. La landing `index.html` vise l'athlète. Le pitch coach n'existe que dans `app/index.html` (~ligne 940-1010) et les offres coach dans `tarifs.json` (clé "coach" : libre 0, coach 19, pro 39 €/mois). Les prix sont TOUJOURS recopiés par `scripts/tarifs.mjs` (attributs `data-tarif` / `data-nb`), jamais écrits en dur.

À faire :

- Créer `coachs.html` à la racine (fichier autonome comme `index.html` : même feuille de style inline, mêmes polices `app/fonts`, même bloc de mesure `landing_view` renommé en `coach_landing_view`). Sections :
 - hero : « Tes athlètes saisissent. Toi, tu coaches. » + sous-titre « Programmes, séances, bilans, nutrition et correction vidéo dans une seule app, gratuite pour ton premier client. » ; CTA « Créer mon espace coach » → `app/?role=coach&src=coachs#install` ;
 - 4 bénéfices avec les vraies captures d'`app/img/vente` : séance saisie par l'athlète, bilans, correction vidéo image par image, tableau de bord qui trie qui décroche ;

```

- « Pourquoi pas Trainerize / TrueCoach / Everfit » : un tableau sobre sur les faits vérifiables de RepCore (en français, hors ligne en salle, charge proposée à chaque série, athlète gratuit avec ton code, pas d'app à installer depuis un store). Aucune affirmation chiffrée sur les concurrents ;
- tarifs coach lus dans tarifs.json via data-tarif="coach.libre|coach.coach|coach.pro" ; ajouter ces clés à scripts/tarifs.mjs et à la liste PAGES de scripts/verif/tarifs.mjs ;
- FAQ (migration depuis un tableur : renvoyer vers blog/numeriser-fiche-programme-papier.html ; que paient mes athlètes ; puis-je mettre mon logo) + JSON-LD FAQPage et SoftwareApplication ;
- le relais des paramètres ?amb/?ref/?src vers app/ et l'appel /arrivee, recopiés d'index.html.
2. app : si l'URL porte ?role=coach, présélectionner le rôle coach sur l'écran de choix (chercher rcmVue('role_selected_coach')) ~ligne 10614 et la fonction qui affiche ce choix). Ne rien changer si le paramètre est absent.
3. index.html : ajouter un lien « Tu es coach ? » dans la nav et le pied, vers coaches.html.
4. c/index.html ligne ~182 : le pied devient « <a href=racine>RepCore</a> · Tu es coach ? <a href=racine+coaches.html?src=vitrine>Crée ta vitrine</a> ».
p/index.html pied : ajouter ?src=profil au lien racine.
5. blog/suivi-athletes-sans-tableur.html : les liens ../app/ et ../ deviennent ../coaches.html?src=blog.
6. sitemap.xml : ajouter coaches.html (priority 0.9).
7. Ajouter 'coach_landing_view' à la liste blanche de /metrics dans database.rules.json (regex $evenement ~ligne 1010) et à RCM_EVENTS / RCM_TUNNEL dans rc-core (ligne 465 et ~118186).
Tests : node scripts/verif/tarifs.mjs ; cd cloudflare && npm test (pages.test.mjs ne doit pas casser sur le pied de c/index.html) ; dans app/tests.js, un test qui vérifie que ?role=coach présélectionne le rôle coach.
Terminé quand : un coach peut aller de Google ou d'une vitrine jusqu'à la création de son espace sans passer par une page qui parle d'autre chose, et le passage se lit dans l'écran tunnel.

```

3. Zéro preuve sociale sur toute la page de vente — Priorité : Haute · Effort : M

Constat. index.html n'a ni témoignage, ni note, ni nombre d'utilisateurs ou de séances, ni avant/après d'athlète réel. Les seules preuves sont les diplômes et la course HYROX de Kevin (index.html:554, 793-810) et des captures d'un « profil de démonstration ». Le plan Instagram V2 met l'idée 19 « Témoignages sur l'accueil » « en attente », et le recueil prévu est un formulaire papier (CONSENTEMENT-TEMOIGNAGE.md) : aucune fonction de l'app ne collecte un avis (grep témoignage dans rc-core ne renvoie rien d'utile).

Pourquoi le mettre en place. À 24,90 € par mois avec 12 mois d'engagement, le visiteur froid a besoin d'une confirmation par des pairs. Chaque concurrent affiche des avis d'utilisateurs et des notes de store. Un formulaire papier ne passera jamais à l'échelle : il faut que l'app demande l'avis au bon moment, après un record ou un bilan réussi.

Intérêt réel. Sur une page de SaaS grand public, ajouter 3 à 6 témoignages crédibles (prénom, photo ou emblème de rang, chiffre réel comme « -4 cm de tour de taille en 10 semaines ») augmente souvent le taux de clic sur le CTA de 10 à 30 %. Des compteurs réels (« 12 400 séances enregistrées ce mois ») rendent le produit vivant sans rien inventer, car /stats et /attribution agrègent déjà l'activité. Pour Kevin, les témoignages servent aussi à vendre le coaching à 99-600 €. Côté athlète, être mis en avant est aussi une récompense qui entretient la viralité.

Prompt de mise en place.

```

Contexte : RepCore, dépôt /home/user/coaching. Aucune preuve sociale sur index.html. Le Worker Cloudflare (cloudflare/src) a un accès administrateur et une planification quotidienne (planif.js). Les athlètes ont déjà des pages publiques (/profils_publics) avec consentement.

À faire :
1. App (rc-core.1693.js) : une feuille « Ton avis sur RepCore », proposée UNE seule fois, au 3e record OU au 4e bilan hebdomadaire complété, jamais pendant l'essai avant J14, et jamais si elle a déjà été refusée (clé u.temoignage.propose). Champs : texte (40 à 400 caractères), note de 1 à 5, et case « J'accepte que ce texte, mon prénom et mon rang soient publiés sur le site de RepCore » (obligatoire pour publier, décochée par défaut), plus une case facultative « ajouter mon évolution (poids/tour de taille) ». Enregistrement dans /temoignages/<emailKey> = {texte, note, prenom, rang, evolution?, consenti:true, le}.
2. database.rules.json : /temoignages/$k écrit par son titulaire seulement, avec validation des tailles, lu par le créateur. /temoignages_publics lisible par tous et écrit par le créateur seul.
3. Écran admin (le créateur, CREATOR_EMAIL) : liste des témoignages avec les boutons « Publier » et « Masquer ». Publier recopie dans /temoignages_publics/<id> {texte, prenom, rang, note, evolution}, sans e-mail ni clé. Kevin peut corriger une faute mais pas le sens : conserver l'original à côté.
4. index.html : une section « Ils s'entraînent avec RepCore » entre #fonctions et #tarifs, remplie par fetch sur /temoignages_publics.json (limitToLast=6). Emblème de rang depuis app/img/rangs/rang_<n>-512.webp, textContent seulement. Si la liste est vide ou si la lecture échoue, la section reste cachée (pas de faux avis de remplacement). Sous le titre, UN compteur réel lu dans un agrégat public /stats/public (séances enregistrées sur 30 jours), que le Worker calcule chaque nuit dans planif.js à partir des agrégats de retention.js, et qu'on n'affiche qu'au-dessus de 1000.
5. Ajouter aggregateRating au JSON-LD SoftwareApplication UNIQUEMENT quand au moins 5 avis sont publiés : le script scripts/tarifs.mjs ou un nouveau scripts/avis.mjs le régénère depuis la base, sinon on ne met rien.
Tests : app/tests.js (déclenchement une seule fois, refus mémorisé, rien avant J14), cloudflare/test (calcul de /stats/public sans aucune clé de compte), et un test des règles avec l'émulateur sur le modèle de regles-evenements.emu.mjs.
Terminé quand : un athlète peut laisser un avis depuis l'app, Kevin le publie en un geste, et il apparaît sur la landing sans redéploiement.

```

4. SEO : prix faux dans les données structurées, blog orphelin, pages publiques hors index — Priorité : Haute · Effort : M

Constat. Le JSON-LD SoftwareApplication annonce Essentielle à "price": "9.95" (index.html:37) alors que tarifs.json dit 9,50. Google peut reprendre ce chiffre dans l'extrait, et scripts/verif/tarifs.mjs ne le voit pas : sa regex ne cherche que des montants suivis de « € ». Les 3 articles (blog/*.html, environ 1 500 mots chacun) n'ont pas de page index, aucun lien depuis la landing, pas de JSON-LD Article ni de datePublished, et leurs CTA ../app/ ne portent pas de src=blog. sitemap.xml n'a pas de <lastmod>. Les vitrines /coach/<slug> et les profils /pseudo, avec titre et description injectés par cloudflare/src/pages.js:117-129, sont servis depuis workers.dev sans canonical et ne figurent pas dans le sitemap.

Pourquoi le mettre en place. Le SEO est le seul canal qui travaille sans budget ni publication quotidienne. Trois articles non reliés ne se positionnent pas, un prix faux dans l'extrait de recherche crée de la méfiance, et les vitrines de coaches sont du contenu local (« coach sportif Lyon ») aujourd'hui perdu.

Intérêt réel. Relier le blog à la landing et publier 2 articles par mois sur les requêtes de la cible (« programme musculation débutant », « calcul charge RIR », « application suivi musculation », « logiciel coaching sportif ») peut viser 1 000 à 3 000 visites organiques par mois sous 6 à 9 mois. Avec 3 % d'inscription et 20 % de conversion, cela fait 6 à 18 abonnés par mois, gratuitement. Les vitrines indexées donnent à chaque coach une raison de plus de rester (il y gagne des prospects, déjà mesurés par le Lot C6) et ramènent des visiteurs à RepCore. La correction du JSON-LD prend 5 minutes et supprime un écart de prix visible dans Google.

Prompt de mise en place.

```

Contexte : RepCore, dépôt /home/user/coaching. Pages statiques servies par GitHub Pages (canonical https://repcore131.github.io/coaching/) et par Firebase (repcore-sync.web.app). Les prix viennent de tarifs.json via scripts/tarifs.mjs.

À faire :
1. index.html JSON-LD SoftwareApplication (lignes ~36-40) : les "price" doivent être écrits par scripts/tarifs.mjs (ajouter un marqueur, par exemple remplacer les valeurs entre "price": " " et " pour Essentielle et Ultime avec essentielle.mois et ultime.mois au format 9.50). Dans scripts/verif/tarifs.mjs, ajouter un contrôle qui parse chaque bloc <script type="application/ld+json"> de index.html et échoue si un "price" ne correspond à aucune valeur de tarifs.json, plus un auto-contrôle (injecter 9.95 et vérifier que c'est vu).
2. Créer blog/index.html : liste des articles (titre, chapô, date), JSON-LD Blog, lien depuis la nav et le pied d'index.html (« Conseils »). Dans chaque article : JSON-LD Article (headline, datePublished, dateModified, author Kevin Guellec, image), og:type article, un bloc « À lire aussi » (déjà présent, à

```

garder) et un encadré CTA final « Essaie RepCore 1 mois » avec href ../app/?src=blog-<slug>#install (l'article coach renvoie vers ../coachs.html?src=blog si la page existe).

3. Ajouter le compteur blog_view dans le même style que landing_view d'index.html (déduplication sessionStorage), avec la liste blanche de database.rules.json (/metrics \$evenement) et RCM_EVENTS dans rc-core.

4. sitemap.xml : ajouter blog/index.html et <lastmod> sur chaque URL. Écrire scripts/sitemap.mjs qui régénère le fichier depuis la liste des pages (lastmod = date du dernier commit git du fichier), et l'appeler depuis deploie.sh ou le workflow de publication.

5. cloudflare/src/pages.js : dans le rendu d'une vitrine coach publiée et d'un profil public, injecter <link rel="canonical" href="https://repcore-sync.web.app/coach/<slug>" (ou /@pseudo), et <meta name="robots" content="noindex"> pour un profil athlète (vie privée) mais PAS pour une vitrine coach. Ajouter un point /sitemap-coachs.xml au Worker qui liste les vitrines publiées (lecture de l'annuaire, bornée à 500), et le référencer dans robots.txt.

Tests : node scripts/verif/tarifs.mjs ; cd cloudflare && npm test (ajouter dans pages.test.mjs : canonical présent pour un coach, noindex pour un athlète, sitemap-coachs valide en XML) ; validation manuelle avec l'outil de test des résultats enrichis de Google sur index.html et un article.

Terminé quand : le JSON-LD dit 9.50 et le contrôle casse si ce n'est plus le cas, chaque article est à un clic de la landing et porte src=blog, et les vitrines coach ont un canonical et un sitemap.

5. Le tunnel de la landing se mesure mal : vues gonflées, aucun clic de CTA, clics d'attribution comptés deux fois — Priorité : Moyenne · Effort : S

Constat. La mesure existe (compteurs /metrics, attribution par src et cohortes de rétention), mais le haut du tunnel est faussé. landing_view s'incrémente à **chaque chargement** (index.html:235-246, sans déduplication, alors que l'app déduplique par session avec rcmVue, rc-core:830). Il s'incrémente aussi pour une PWA installée redirigée par location.replace('app/'), puisque le script continue de s'exécuter. Aucun clic sur « Essayer gratuitement » n'est compté. Sur le chemin /@julie → « Rejoins Julie » → /i, l'arrivée est comptée deux fois : p/index.html:251 puis i/index.html appellent tous deux /arrivee, avec une clé de déduplication en localStorage propre à chaque origine (workers.dev d'un côté, web.app de l'autre). Pour reproduire : ouvrir un lien /@pseudo?src=bio depuis un téléphone, toucher « Rejoins... », et attribution/jours/<date>/src/bio/clic prend +2. Enfin, les CTA de la landing et du blog sans src versent tout le trafic organique dans « direct ».

Pourquoi le mettre en place. Le coefficient viral et le taux vue → inscription affichés dans l'écran Viralité guident les décisions (quel visuel pousser, quel ambassadeur garder). Des clics doublés font paraître les partages deux fois plus efficaces qu'ils ne sont, et des vues gonflées font paraître la landing moins efficace.

Intérêt réel. Ce sont des corrections de quelques lignes. Elles transforment l'écran tunnel en outil de décision fiable : taux de clic sur le CTA de la landing (pour tester un nouveau titre), vues réelles et taux de clic par source. Sans elles, un test A/B du titre ou de l'offre invité (point 1) ne se lit pas. À 500 vues par jour, confondre rechargements et visiteurs peut sous-estimer la conversion de 20 à 40 %.

Prompt de mise en place.

Contexte : RepCore, dépôt /home/user/coaching. Compteurs anonymes par jour dans /metrics/<AAAA-MM-JJ>/<evenement> (liste blanche regex dans database.rules.json ~ligne 1010, RCM_EVENTS rc-core ligne 465, écran RCM_TUNNEL ~ligne 118186). Attribution par src dans /attribution/jours via le Worker /arrivee (cloudflare/src/index.js ~ligne 123, metier.js arrivee()).

À faire :

1. index.html, bloc « Mesure de tunnel » (~ligne 226) : ne rien envoyer si la page est lancée en mode installé (même test que le bloc de redirection juste au-dessus : sortir avant le fetch), et dédupliquer par session avec sessionStorage 'rcm_vue_landing_view', en try/catch comme rcmVue.
 2. index.html : compter landing_cta_click une fois par session au premier clic sur n'importe quel a.cta dont le href commence par app/ (délégation sur document, fetch keepalive). Ajouter l'événement à la liste blanche des règles, à RCM_EVENTS et à RCM_TUNNEL juste après « Page de vente vue ».
 3. Double comptage /arrivee : dans p/index.html et c/index.html, quand l'arrivée a été comptée, ajouter &vu=1 au lien du CTA vers /i (variable cible ~ligne 204 de p, ~180 de c). Dans i/index.html, ne pas appeler /arrivee si vu=1 est présent, puis retirer vu du paramètre transmis à l'app.
 4. Sources par défaut : sur la landing, si aucun src n'est présent et que document.referrer vient d'un moteur (google., bing., duckduckgo., qwant., ecosia.), poser src=seo pour le relais vers app/ et pour /arrivee ; un referrer.instagram.com ou l.instagram.com donne src=ig. Ne jamais écraser un src déjà présent.
 5. fonctions/attribution-calcul.js / écran viralityDonnees (rc-core ~19640) : rien à changer côté calcul, mais ajouter une ligne d'aide dans l'écran : « clics = arrivées dédupliquées par appareil et par jour ».
- Tests : cloudflare && npm test (ajouter dans index.test.mjs un cas /arrivee avec vu=1 non compté si tu le gères côté serveur, sinon un test de i/index.html dans app/tests.js via un faux location). Dans app/tests.js : landing_cta_click présent dans RCM_EVENTS et dans RCM_TUNNEL. Vérification manuelle : 3 rechargements de la landing = 1 landing_view ; le chemin /@pseudo → /i = 1 clic.
- Terminé quand : l'écran tunnel montre vues → clics CTA → app ouverte sans double comptage, et le trafic organique n'est plus rangé dans « direct ».

6. Les visuels partagés ne ramènent pas vers RepCore : pas de compte Instagram tagué, pas d'adresse lisible — Priorité : Moyenne · Effort : S

Constat. Les légendes prêtes à coller devaient taguer le compte officiel, mais RC_COMPTE_INSTAGRAM = '' (rc-core:46499, « À REMPLIR »), donc aucune légende ne le mentionne. Les visuels portent seulement « NOM · REPCORE » (rc-core:46011, 46152), sans adresse, et la règle d'or du plan V4 exclut le QR. Tout dépend donc de l'ajout manuel du sticker Lien. Or l'adresse de la landing, repcore131.github.io/coaching, ne se retient pas et ne se tape pas.

Pourquoi le mettre en place. Sur Instagram, la majorité des stories sont partagées sans sticker. Le spectateur intrigué cherche alors « RepCore » sur Instagram ou sur Google, et ne trouve ni compte tagué ni adresse mémorisable. Chaque visuel partagé qui ne mène nulle part est de la portée perdue.

Intérêt réel. Le produit fabrique beaucoup de contenu partageable : bilan, record, rang, Wrapped, et vidéo de la foudre (docs/video-partage.md). Si 20 % seulement des partages portent le sticker, les 80 % restants ne rapportent aujourd'hui que de la notoriété sans chemin de retour. Taguer @repcore... dans chaque légende de post et afficher une adresse courte sur le visuel peut au moins doubler les arrivées issues d'un partage (compteur partage → clic de l'écran Viralité). Le compte officiel y gagne en plus des abonnés à chaque partage.

Prompt de mise en place.

Contexte : RepCore, dépôt /home/user/coaching, app/rc-core.1693.js. Les légendes de partage (_LEGENDES, ~ligne 46500) ajoutent le compte Instagram officiel seulement si RC_COMPTE_INSTAGRAM (ligne ~46499) n'est pas vide, et il est vide. Les visuels (story et post) signent « NOM · REPCORE » (~46011, ~46152, ~78592) sans adresse.

À faire :

1. Demander à l'utilisateur le @ du compte Instagram officiel et le renseigner dans RC_COMPTE_INSTAGRAM. Vérifier que toutes les légendes le placent en fin de texte, sans dépasser LEGENDE_MAX (220) : si la légende déborde, tronquer le texte et pas le @.
 2. Ajouter une constante RC_ADRESSE_AFFICHEE (par exemple 'repcore-sync.web.app', ou un domaine court si l'utilisateur en fournit un). Si elle n'est pas vide, les fonctions de dessin des visuels (chercher les appels qui écrivent sig+ ' · REPCORE' et écrireEspace('REPCORE', _)) écrivent sous la signature, en petit (= 60 % de la taille de la signature, opacité 0,7), l'adresse sans https://. Pas de QR. Le rendu doit rester lisible sur les 3 fonds (visuelFormat story et post).
 3. index.html : ajouter le lien du compte Instagram officiel dans le pied, et dans le JSON-LD SoftwareApplication un 'sameAs' vers ce compte.
 4. Dans la feuille de partage, quand la story est partagée, garder le toast « Lien copié » et y ajouter « colle-le avec le sticker Lien » (un seul toast).
- Tests : app/tests.js, bloc « Vidéo : » et bloc des visuels : RC_ADRESSE_AFFICHEE vide donne le même rendu qu'avant (comparer les données de dessin ou la hauteur utilisée), non vide donne une ligne de plus ; toute légende contient le @ quand il est renseigné, et aucune ne dépasse LEGENDE_MAX. Refaire la procédure manuelle de docs/video-partage.md pour un record.
- Terminé quand : chaque post partagé tague le compte officiel, et chaque visuel porte une adresse qu'on peut taper.

7. Pas de présence sur le Play Store : l'APK est prête mais invisible — Priorité : Moyenne · Effort : M

Constat. La TWA Android est prête : `android/twa-manifest.json` avec `packageName` `com.repcore.app` et hôte `repcore-sync.web.app`, `android/publier.sh`, `store_icon.png`, et `.well-known/assetlinks.json` publié. Mais elle ne se distribue que par un fichier APK expliqué sur `aide-apk.html`, qui est en `noindex`, et `RC_APK_URL = ''` (`rc-core:941`). Aucune fiche de store n'existe dans le dépôt (pas de titre court, de description ni de captures au format store), et la landing affiche « Android et iPhone » (`index.html:878`) sans badge de store.

Pourquoi le mettre en place. Le Play Store est à la fois un canal de découverte (recherche « carnet musculation », « suivi musculation »), un signal de confiance (note, nombre de téléchargements) et une installation en un geste, là où l'installation d'une PWA reste obscure pour une partie du public. Hevy, Strong, Trainerize et TrueCoach y sont tous.

Intérêt réel. Une fiche Play Store bien travaillée en français sur une niche (« carnet de musculation avec charge proposée ») peut apporter quelques dizaines à quelques centaines d'installations organiques par mois, pour 25 \$ de frais uniques. Les avis du store deviennent une preuve sociale réutilisable sur la landing (point 3). Côté coach, « téléchargez l'app sur le Play Store » est bien plus simple à dire à un client que « ajoutez le site à l'écran d'accueil ». Le chemin reste le même tunnel mesuré, car la TWA ouvre `/app/` avec `?src=play` (utm du referrer d'installation).

Prompt de mise en place.

Contexte : RepCore, dépôt `/home/user/coaching`. La TWA Android existe (`android/twa-manifest.json`, `packageName` `com.repcore.app`, `android/publier.sh`, `android/LISEZMOI.md`) mais n'est pas publiée sur le Play Store. La landing `index.html` dit « Android et iPhone » sans badge.

À faire (code et contenu, la création du compte développeur restant à l'utilisateur) :

1. Créer `android/fiche-store/fr-FR/` au format fastlane (`title.txt` ≤ 30 caractères : « RepCore – Carnet de musculation » ; `short_description.txt` ≤ 80 : promesse « La bonne charge à chaque série » ; `full_description.txt` ≤ 4000, reprenant la landing : 4 en 1, charge proposée, 400+ exercices filmés, nutrition hors ligne, coachs, 1 mois d'essai sans carte, sans prix en dur autre que ceux de `tarifs.json` recopiés par `scripts/tarifs.mjs` ; `changelogs<versionCode>.txt`). Ajouter un script `scripts/verif/fiche-store.mjs` qui vérifie les longueurs et l'absence de prix non lié.
 2. Captures : lister dans `android/fiche-store/LISEZMOI.md` les 6 captures à produire (1080×2340) à partir des images existantes d'`app/img/vente` (`seance-charge`, `nutrition-journal`, `evolution-poids`, `correction`, `exercice-tirage`, un écran coach) et la bannière 1024×500 à partir d'`og-vente.jpg`. Si `scripts/verif/capture.mjs` le permet, générer les captures automatiquement.
 3. `twa-manifest.json` : `startUrl` avec `?src=play` pour que l'attribution range ces installations dans « play ». Vérifier que `attribSrc` accepte 'play'.
 4. `index.html` et `coachs.html` : ajouter une constante `RC_PLAY_URL` en haut du script. Si elle n'est pas vide, afficher un badge « Disponible sur Google Play » à côté du CTA final et dans `.rassure`, et seulement sur Android (test UA). Sinon, rien ne change.
 5. Mettre à jour `android/LISEZMOI.md` avec la procédure de publication (compte, fiche interne, test fermé de 14 jours avec 12 testeurs exigé pour les nouveaux comptes personnels, puis production).
- Tests : `node scripts/verif/fiche-store.mjs` ; `node scripts/verif/tarifs.mjs` ; `app/tests.js` : `attribSrc('play') === 'play'`.
- Terminé quand : la fiche est prête à coller dans la Play Console, la TWA remonte `src=play`, et le badge s'active en renseignant une seule constante.

Agent 12 — Business et monétisation · 5/10

La base de facturation est plus solide que la moyenne d'un produit à ce stade. `tarifs.json` est la source unique des prix et `node scripts/verif/tarifs.mjs` passe (`app`, `index`, `terms`, `aide-apk`, `i/`, `c/` alignés). Le webhook PayPal (`cloudflare/src/paypal.js`) est idempotent, vérifie la signature et relit chaque objet chez PayPal ; il traite les remboursements, rétrofacturations et litiges, et 34 tests passent (`paypal.test.mjs`, `remboursements.test.mjs`). Le modèle économique, lui, n'est toujours pas tranché : un coach tiers en palier « libre » peut offrir l'app complète à un nombre illimité d'athlètes pour 0 €. À cela s'ajoutent plusieurs trous de facturation : double abonnement possible, résiliation jamais transmise à PayPal, engagement de 12 mois promis mais pas appliqué, remboursement d'un renouvellement sans effet sur l'accès. Enfin, la grille (engagement 12 mois, annuel sans remise) est mal placée face au marché.

1. Modèle économique non tranché : l'athlète d'un coach tiers ne rapporte rien — Priorité : Critique · Effort : M

Constat. `linkToCoach` (`app/rc-core.1693.js:11840-11846`) passe l'athlète en `status='COACHING_SUIVI'` sans `accessExpiry`, et `checkAccess` (`l. 8167-8169`) ouvre alors l'accès sans limite de durée, au palier suivi : tout Ultime, plus canal, vidéo et morpho (`CAPACITES`, `l. 1140`). Le quota du coach n'est jamais appliqué : `coachQuotaDepasse` (`l. 1551`) « ALERTE, il ne bloque jamais », et `alertePalier` (`l. 1946`) écrit au coach « Rien ne change tant que tu ne le choisis pas : tes athlètes gardent tout leur accès ». `NOTE-DECISION-MODELE-ECONOMIQUE.md` §6 dit toujours « AUCUNE — statu quo ». Pour reproduire : créer un coach (palier libre, quota 1), rattacher 20 athlètes par code : les 20 ont un accès complet sans limite, et le coach ne voit qu'un bandeau.

Pourquoi le mettre en place. Dans les faits, le code a déjà basculé vers l'option B (le coach paie, l'athlète rattaché est gratuit : `COACH_PALIER`, `l. 1418`), mais sans le verrou qui la rend rentable. Aujourd'hui, chaque coach tiers qui réussit fait perdre de l'argent : il consomme le quota du plan Spark (200 comptes Libres, `l. 1557`) sans rien payer. La note de décision est d'ailleurs périmée : elle parle de 9,95 € et de textes « centralisés » qui ont disparu de `app/index.html`.

Intérêt réel. Chiffrage : un coach tiers qui suit 12 athlètes rapporte 0 €/mois aujourd'hui, contre 19 € TTC (environ 14,90 € nets HT après TVA et frais PayPal) une fois le quota appliqué. Dix coaches ainsi, c'est 190 à 390 € de MRR, avec un coût d'acquisition nul puisque ce sont les coaches qui recrutent. En face : Hevy Coach demande 25 \$ jusqu'à 10 clients, Kahunas 35 \$ jusqu'à 25, et aucun ne donne un accès illimité gratuit. Même au prix fixé, 19 € pour 15 athlètes, soit 1,27 €/athlète, reste le moins cher du marché, donc facile à justifier. Le plus important : sans verrou, aucun coach n'a de raison de passer à Coach ou Pro, et la projection de revenus coaches vaut zéro. Il faut une période de grâce (deux cycles, déjà comptés par `majCyclesPaliers`) pour ne pas pénaliser les athlètes sans prévenir.

Prompt de mise en place.

```
Contexte : RepCore (dépôt courant). Le modèle retenu de fait est « le coach paie, l'athlète rattaché est gratuit » (COACH_PALIER dans app/rc-core.1693.js ~l.1418 : libre 1 athlète / coach 19 € 15 athlètes / pro 39 € illimité, prix dans tarifs.json). Aujourd'hui, le dépassement de quota n'est qu'une alerte (coachQuotaDepasse l.1551, alertePalier l.1920-1955) : un coach libre peut suivre un nombre illimité d'athlètes, qui ont tous le palier 'suivi' gratuit (linkToCoach l.11840, checkAccess l.8141, palierDe l.7588, _palierHerite l.7633).
À faire :
1. Dans app/rc-core.1693.js, ajouter une fonction PURE athleteCouvertParCoach(athlete, coach, users) qui dit si l'athlète occupe une place couverte par le quota de son coach. Les places sont attribuées par ordre de rattachement : champ rattachéLe posé dans linkToCoach et à la consommation d'un code (~l.11905), repli sur l'ordre de coach.clients.
2. Période de grâce : tant que paliersDe(coach).cyclesAuDessus < PALIERS_CYCLES_AVANT_PROPOSITION + 1 (donc 3 mois), tous les athlètes restent couverts. Au-delà, un athlète hors quota passe au palier 'aucun', sauf essai actif ou abonnement personnel. Il voit alors l'écran d'accès (loadAccessGate) avec un message qui lui propose Essentielle et invite son coach à passer au palier supérieur.
3. Le coach ne peut pas écrire droits/, et les athlètes ne lisent pas le dossier du coach. Faire recopier par le serveur léger (cloudflare/src/metier.js, travail quotidien de planif.js) un indicateur couvertParCoach:{jusqu:ms} dans droits/<athlète> à partir du coachPlan écrit par cloudflare/src/paypal.js. palierDe doit le lire quand d.etat==='serveur'.
4. Remplacer le texte d'alertePalier « Rien ne change tant que tu ne le choisis pas » par l'échéance réelle : « Au-delà du <date>, N athlètes perdront leur accès suivi ».
5. Mettre à jour NOTE-DECISION-MODELE-ECONOMIQUE.md §6 : option B appliquée, date.
Cas limites : Kevin (CREATOR_EMAIL) n'est jamais soumis au quota. Un athlète qui a un abonnement AUTONOMIE_PREMIUM garde son palier payé. Un coach qui redescend sous son quota rend l'accès tout de suite. Un cache local vide (countActiveAthletesFiable false) ne coupe jamais personne.
Tests : dans app/tests.js, un coach libre avec 3 athlètes (1 couvert, 2 hors quota, avant et après la grâce), un coach Pro, un cache vide, le créateur. Dans cloudflare/test/metier.test.mjs, la recopie de couvertParCoach. Lancer node app/tests.js (ou la commande de scripts/verif/suite.mjs) et node cloudflare/test/metier.test.mjs.
Critère de fin : tous les tests passent. Un coach libre qui suit 20 athlètes depuis plus de 3 mois n'en couvre plus qu'un seul, et l'écran de chacun dit pourquoi et quoi faire.
```

2. Double abonnement : passer d'Essentielle à Ultime prélève les deux, en silence — Priorité : Critique · Effort : S

Constat. `rcVerrouUltime` puis `accueilChoisir('ultime')` (`l. 8133-8136`) ouvre `s-subscribe` même pour un abonné actif. `loadSubscribePage` (`l. 121701`) ne teste pas l'abonnement existant. `onApprove` (`l. 121890` et suivantes) écrase `paypalSubscriptionId` avec le nouvel ID et n'annule jamais l'ancien. Côté serveur, les paiements de l'ancien abonnement donnent `paiement_sans_ouverture` (`cloudflare/src/paypal.js:284`) et sa résiliation donne `ancien_abonnement` (`l. 226`) : l'argent est encaissé et personne n'est prévenu. Pour reproduire : un abonné Essentielle ouvre le verrou « Volume », clique « Voir Ultime » et paie. PayPal prélève alors 9,50 € + 24,90 € par mois, et `engagementJusqu` report pour 12 mois.

Pourquoi le mettre en place. C'est le chemin de montée en gamme que l'app met en avant sur chaque verrou, donc justement celui des meilleurs clients. Un double prélèvement non signalé mène tout droit au litige PayPal. Le litige coûte des frais, peut geler le compte marchand, et il est déjà traité en « perdu » par `litigeClos`.

Intérêt réel. Avec 20 % d'abonnés Essentielle qui montent en Ultime, sur 100 abonnés, 20 personnes paient 9,50 € de trop chaque mois, soit 190 €/mois à rembourser. En pratique, cela veut dire 20 litiges potentiels : au-delà d'un taux de litige d'environ 1 %, PayPal peut retenir les fonds. Corriger le parcours transforme la montée en gamme en vrai upsell (+15,40 €/mois par client) au lieu d'un risque. Ajouter la révision de plan PayPal (`/v1/billing/subscriptions/{id}/revise`) donne un changement de formule en un clic, sans deuxième abonnement : c'est le standard du marché.

Prompt de mise en place.

```
Contexte : RepCore. app/rc-core.1693.js : accueilChoisir (l.8133), loadSubscribePage (~l.121701), renderPaypalButton/onApprove (~l.121850-121975). Serveur léger : cloudflare/src/paypal.js (jetonPaypal, lireAbonnement, OFFRES_PAYPAL) et cloudflare/src/index.js (routage).
Problème : un abonné actif peut souscrire un 2e abonnement (Essentielle vers Ultime) ; l'ancien continue d'être prélevé et le serveur l'ignore (« ancien_abonnement », « paiement_sans_ouverture »).
À faire :
1. Serveur : ajouter une route POST /abonnement/changer (authenticifiée comme les autres routes du Worker qui lisent un jeton Firebase). Corps : {plan_id}. Elle vérifie que users/<clé>/paypalSubscriptionId est ACTIVE chez PayPal, puis appelle POST /v1/billing/subscriptions/{id}/revise avec le nouveau plan_id, et renvoie le lien d'approbation (links rel=approve). Le webhook BILLING.SUBSCRIPTION.UPDATED doit être traité dans traiter() : relire l'abonnement et appeler droitsOuverts avec OFFRES_PAYPAL[plan_id].
2. Serveur : dans paiementAbonnement, quand le paiement vient d'un abonnement ACTIVE qui n'est pas le courant du dossier, écrire une ligne paypal_journal {quoi:'double_abonnement'} et envoyer un push admin (pousserAdmin existe déjà) : « <client> a deux abonnements actifs ».
3. App : dans accueilChoisir et loadSubscribePage, si aUnAbonnement(currentUser) et que le statut n'est pas résilié, ne pas monter les boutons PayPal de
```

souscription. Afficher « Changer de formule » qui appelle la route du point 1. Pour un passage Ultime vers Essentielle, afficher la date d'effet.

4. App : dans onApprove, si un paypalSubscriptionId différent existait déjà, l'envoyer au serveur (abonnementSignaler) avec {replace:ancien}. Le serveur annule l'ancien (POST /v1/billing/subscriptions/{ancien}/cancel, reason « remplacé ») seulement si le nouveau est ACTIVE.

Cas limites : ancien abonnement déjà CANCELLED ; coach (COACH_PALIER) qui passe de coach à pro (même logique) ; plan demi (P-57P...) ; engagementJusqu ne doit PAS repartir à zéro lors d'un changement de formule.

Tests : cloudflare/test/paypal.test.mjs (révision, double abonnement détecté puis journalisé, annulation de l'ancien) et app/tests.js (abonné Essentielle : accueilChoisir('ultime')) ne rend pas le bouton de souscription). Lancer node cloudflare/test/paypal.test.mjs et la suite app.

Critère de fin : aucun chemin de l'app ne permet d'avoir deux abonnements actifs, et un doublon qui existe déjà est signalé à l'administrateur.

3. Résiliation et engagement de 12 mois : promesse, CGV et code disent trois choses différentes — Priorité : Haute · Effort : M

Constat. demanderResiliation (l. 2219) n'écrit que abonnement.resiliationDemandee dans le dossier ; ni l'app ni le Worker n'appellent /cancel chez PayPal (aucune occurrence dans cloudflare/src). Le commentaire l. 2141 (« il n'y a pas de serveur pour appeler leur API ») est faux depuis que le Worker détient PAYPAL_CLIENT_SECRET. Après la résiliation dans l'app, le prélèvement mensuel continue au-delà des 12 mois, alors que la landing (index.html:773) promet « sans reconduction ». À l'inverse, si l'utilisateur annule dans PayPal au 3e mois, fermerALaFin (paypal.js:232) ferme l'accès à next_billing_time, soit le mois suivant. Pourtant index.html:773 promet que « ton accès reste ouvert jusqu'au bout des 12 mois », et terms.html:138 dit que les échéances restantes « demeurent dues », sans aucun mécanisme pour les encaisser.

Pourquoi le mettre en place. Un engagement de 12 mois porté par un abonnement PayPal mensuel ne peut pas être imposé : le client peut annuler en deux clics dans PayPal. On garde donc le coût commercial de l'engagement (conversion plus faible) sans son bénéfice (revenu garanti). Et la résiliation dans l'app qui ne résilie rien entraîne des prélèvements non voulus, donc des litiges.

Intérêt réel. Il y a deux sorties cohérentes, à arbitrer. (a) Supprimer l'engagement sur le mensuel et garder l'annuel payé d'avance avec une vraie remise (voir point 5). C'est le standard de Hevy, Fitbod ou MacroFactor, et on peut en attendre +20 à 40 % de conversion essai → payant sur une offre mensuelle sans engagement. (b) Garder l'engagement, mais alors le Worker doit annuler chez PayPal au 12e mois quand une résiliation a été demandée, et garder l'accès jusqu'à engagementJusqu. Dans les deux cas, câbler /cancel supprime la principale source de litiges « prélèvement non autorisé », chacun coûtant environ 20 € de frais plus le montant contesté.

Prompt de mise en place.

Contexte : RepCore. Résiliation dans l'app : demanderResiliation (app/rc-core.1693.js l.2219), textes RESIL_PAYPAL (l.2275) et commentaire l.2136-2146. Webhook : cloudflare/src/paypal.js fermerALaFin (l.222-245). Promesses : index.html ~l.773 et ~l.836, terms.html \$4-\$5 (l.113-145). Engagement : tarifs.json engagementMois=12, abonnement.engagementJusqu posé dans onApprove. Décision retenue par défaut (à confirmer avec Kevin avant de coder ; sinon appliquer la variante B) :

Variante A « mensuel sans engagement » : tarifs.json engagementMois=0 pour le mensuel. Les textes de index.html, terms.html et c/, i/ passent par scripts/tarifs.mjs ; scripts/verif/tarifs.mjs doit accepter « sans engagement » quand engagementMois vaut 0.

Variante B « engagement tenu » :

1. Worker : nouvelle route POST /resiliation (jeton Firebase). Elle écrit resiliations/<cl>={ts,motif,abo}. Le travail quotidien (planif.js) appelle POST /v1/billing/subscriptions/{abo}/cancel chez PayPal quand now >= engagementJusqu - 3 jours. Pendant l'engagement, rien n'est annulé chez PayPal.
2. fermerALaFin : pour un athlète dont users/<cl>/abonnement/engagementJusqu > fin calculée ET dont la résiliation n'a pas été demandée dans l'app (annulation directe dans PayPal), NE PAS prolonger l'accès. Écrire paypal_journal {quoi:'rupture_engagement', mois_restantes} et envoyer un push admin, pour que Kevin décide du recouvrement.
3. App : demanderResiliation appelle la route (file hors ligne existante conservée). Retirer le commentaire faux « pas de serveur pour appeler leur API ». L'écran de confirmation affiche la date d'effet réelle (engagementJusqu).

Dans les deux variantes : un abonnement résilié dans l'app ne doit plus JAMAIS être prélevé après sa date d'effet (annulation serveur).

Tests : cloudflare/test/paypal.test.mjs (résiliation demandée, puis annulation PayPal envoyée à la bonne date ; annulation directe PayPal pendant l'engagement, puis ligne de journal) ; node scripts/verif/tarifs.mjs ; suite app pour demanderResiliation.

Critère de fin : la landing, les CGV, l'écran de résiliation et le comportement du Worker décrivent la même chose, et un test le prouve.

4. Le remboursement d'un renouvellement ne ferme pas l'accès — Priorité : Haute · Effort : S

Constat. Dans annuler (cloudflare/src/paypal.js:458-466), l'accès n'est fermé (fermerAcces) que si rec.premier. Pour un renouvellement remboursé en totalité, seule la commission est reprise. Si l'abonnement est ensuite annulé, fermerALaFin repousse la fin à next_billing_time. Pour reproduire : un abonné annuel au 2e renouvellement (114 €) réclame le remboursement du prélèvement non voulu que les CGV \$7 promettent de rembourser (terms.html:157). Kevin rembourse et annule : PAYMENT.SALE.REFUNDED donne « rien à reprendre », puis CANCELLED pose une fin un an plus tard. Le client est remboursé et garde Ultime ou Essentielle douze mois.

Pourquoi le mettre en place. C'est exactement le cas des CGV (prélèvement après le terme), donc le remboursement le plus fréquent. Le module a été écrit pour que « l'accès du remboursé se ferme », et ce cas lui échappe.

Intérêt réel. Chaque remboursement annuel mal traité, c'est 114 à 298,80 € de service donné. Un remboursement mensuel mal traité laisse un mois gratuit par-dessus le remboursement. Avec 5 % des renouvellements remboursés sur 100 abonnés annuels, cela fait 5 comptes par an qui gardent l'accès sans payer. Corriger aligne aussi droits/ sur la réalité encaissée, ce qui fiabilise le MRR (point 7).

Prompt de mise en place.

Contexte : cloudflare/src/paypal.js, fonctions annuler (l.451), fermerAcces (l.431), fermerALaFin (l.222), noterTransaction/origine.

Problème : un remboursement TOTAL d'un paiement qui n'est pas le premier ne ferme rien ; l'accès reste ouvert jusqu'à la prochaine échéance PayPal, voire un an pour un annuel.

À faire :

1. Dans annuler(), pour rec.type==='abonnement' et !rec.premier : calculer la période couverte par la transaction remboursée (rec.le + 1 mois, ou + 12 mois si le plan est un plan annuel d'OFFRES_PAYPAL ; ajouter à OFFRES_PAYPAL un champ periode:'mois'|'an'). Si cette période contient maintenant ou le futur, fermer l'accès à max(rec.le, now) via fermerAcces, et l'inscrire dans actions.
2. Dans fermerALaFin : si paypal.transactions contient, pour cet abonnement, une vente remboursée dont la période couvrirait next_billing_time, prendre comme fin la date du remboursement au lieu de next_billing_time.
3. Ne rien changer pour le remboursement PARTIEL (geste commercial), qui garde l'accès.

Tests dans cloudflare/test/remboursements.test.mjs : renouvellement mensuel remboursé en totalité (accès fermé) ; renouvellement annuel remboursé puis CANCELLED (fin = date du remboursement, pas +1 an) ; ancien renouvellement remboursé longtemps après (période passée, rien ne change) ; remboursement partiel (inchangé). Lancer node cloudflare/test/remboursements.test.mjs et node cloudflare/test/paypal.test.mjs.

Critère de fin : tous les tests passent, et paypal_journal indique « accès fermé au <date> » pour un renouvellement remboursé.

5. Grille tarifaire mal placée face au marché (annuel sans remise, verrou incohérent) — Priorité : Haute · Effort : S

Constat. tarifs.json : essentielle.an = 114 = 12 × 9,5 et ultime.an = 298,8 = 12 × 24,9, donc aucune remise pour l'annuel (commentaire rc-core l. 1087). Le verrou affiche du coup « Ultime : 24,90 € par mois en annuel, ou 24,90 € au mois » (rcVerrouBloc, l. 1261), une phrase qui ne donne aucune raison de choisir l'annuel. Marché : Hevy Pro coûte environ 3 \$/mois ou environ 24 \$/an ; Fitbod environ 16 \$/mois ou environ 96 \$/an (-50 %) ; Kahunas donne -25 % en annuel ; Hevy Coach 25 \$ jusqu'à 10 clients, avec 30 jours d'essai sans carte (coachway.io). Aucun ne demande 12 mois d'engagement sur le mensuel.

Pourquoi le mettre en place. Un abonnement à 9,50 € avec 12 mois d'engagement se retrouve à côté d'apps à 3-5 € sans engagement. La valeur de RepCore (cycle, OCR, vidéo) doit justifier l'écart, pas un contrat. L'annuel est le levier de trésorerie et de rétention le plus rentable, et il est neutralisé.

Intérêt réel. Proposition : mensuel sans engagement à 9,50 € et 24,90 €, annuel à 79 € (-31 %) et 199 € (-33 %). Un taux annuel de 30 % des souscriptions (courant quand la remise dépasse 25 %) fait rentrer 12 mois de trésorerie d'un coup, et le churn annuel est bien plus faible que le churn mensuel (environ 10 %/mois en fitness grand public). LTV actuelle estimée : Essentielle environ 7,25 € nets HT/mois × 10 mois ≈ 72 € ; Ultime environ 19,6 € × 10 ≈ 196 €. Un abonné annuel à 79 € renouvelé à 50 % vaut environ 99 € HT net, soit +37 %. Cela fixe le CAC maximal (LTV/3) à environ 24 € pour Essentielle et environ 65 € pour Ultime. Le verrou redevient vendeur : « 16,58 €/mois en annuel ».

Prompt de mise en place.

```
Contexte : les prix de RepCore vivent dans tarifs.json (racine). Ils sont copiés par node scripts/tarifs.mjs dans app/rc-core.1693.js (bloc TARIFS), index.html, terms.html, aide-apk.html, i/, c/, et contrôlés par node scripts/verif/tarifs.mjs. Les plans PayPal se mettent à jour par node scripts/paypal_plans.mjs --tarifs, et cloudflare/src/paypal.js OFFRES_PAYPAL garde la liste des montants acceptés.
À faire (après validation des montants par Kevin ; valeurs par défaut ci-dessous) :
1. tarifs.json : essentielle.an = 79, ultime.an = 199. Corriger aussi le _lisezmoi (il cite scripts/tarifs.py, qui n'existe pas : c'est scripts/tarifs.mjs).
2. scripts/tarifs.mjs, fonction incoherences : autoriser an < 12 × mois (remise) et refuser an > 12 × mois.
3. Lancer node scripts/tarifs.mjs, puis node scripts/verif/tarifs.mjs.
4. rcVerrouBloc (app/rc-core.1693.js ~l.1258) : n'afficher « X par mois en annuel » que si _economie('ultime').texte n'est pas vide ; sinon afficher seulement « Ultime : 24,90 € par mois ».
5. PayPal : créer de NOUVEAUX plans annuels (ne pas modifier les anciens, pour garder les abonnés existants à leur prix) avec scripts/paypal_plans.mjs. Mettre à jour PAYPAL_PLAN_ID_ANNUEL et PAYPAL_PLAN_ID_ULTIME_ANNUEL dans rc-core. Dans cloudflare/src/paypal.js OFFRES_PAYPAL, ajouter les nouveaux plans et GARDER les anciens avec leurs montants (le test paypal.test.mjs « la table OFFRES du serveur suit les plans et les prix de l'app » doit passer).
6. Si le point 3 de l'audit (engagement) retient la variante A, la bascule mensuel/annuel de index.html doit dire « sans engagement » pour le mensuel.
Tests : node scripts/verif/tarifs.mjs ; node cloudflare/test/paypal.test.mjs ; dans app/tests.js, une assertion : rcVerrouBloc('volume') ne contient jamais deux fois le même montant.
Critère de fin : le verrou et l'écran d'abonnement affichent une économie réelle en annuel, et tous les contrôles de prix passent.
```

6. Boutique quasi vide et achats de programmes qui peuvent ne pas être reconnus — Priorité : Moyenne · Effort : S

Fusion. le port de verifierAchatProgramme dans le Worker est demandé par l'agent Backend (point 7) ; les deux prompts touchent paypal.js : les faire dans la même passe.

Constat. RC_PROGRAMMES (l. 117-190) contient un seul programme vendable (« Fondations », 14,90 €) et quatre emplacements aCompleter. L'écriture dans boutique/ est réservée au créateur (database.rules.json:334) : il n'y a donc pas de marketplace pour les coachs tiers. Le Worker ne valide un achat que si boutique/<id>/prixCts existe en base (paypal.js:302-307). Or « Fondations » et revision-programme (40 €, offreRevision, l. 204) n'existent que dans le JS. Tant qu'ils ne sont pas publiés en base, PayPal encaisse mais le Worker répond achat_non_compte, et après 72 h (PAIEMENT_RECENT_MS) l'acheteur perd l'Ultime promis (_palierHerite renvoie aucun avant programmeOuvreUltime, l. 7640-7652). De plus, une révision validée ouvre 3 mois d'Ultime (PROGRAMME_MS, l. 34) alors que tarifs.json dit revision_prog.mois: 1.

Pourquoi le mettre en place. C'est le seul revenu ponctuel dans l'app, et son acheteur est le plus engagé. Une boutique à un seul article ne vend rien, et un achat non reconnu, c'est un client payant qui voit un cadenas.

Intérêt réel. 4 programmes à 14,90 € achetés par 10 % des utilisateurs gratuits ou en essai (200 comptes) représentent environ 300 € de revenu ponctuel. Chaque achat ouvre aussi 3 mois d'Ultime, ce qui prépare la conversion vers l'abonnement. Plus tard, une marketplace coachs avec commission (modèle TrueCoach +5 %) donnerait un revenu par coach au-delà de l'abonnement ; mais comme le montre la note de décision, elle suppose un prestataire de paiement tiers, donc ce n'est pas pour maintenant. Dans l'immédiat : rendre fiable ce qui existe.

Prompt de mise en place.

```
Contexte : boutique RepCore. Catalogue embarqué RC_PROGRAMMES et offreRevision dans app/rc-core.1693.js (l.117-220). Achat : ouvrirAchatProgramme / _enregistrerAchat (~l.42140-42270), custom_id « <clé>|<id> ». Validation serveur : cloudflare/src/paypal.js achat() (l.291-320), qui exige boutique/<id>/prixCts en base.
À faire :
1. cloudflare/src/paypal.js achat() : si boutique/<prog>/prixCts est absent, se replier sur une table PRIX_EMBARQUES exportée par le Worker ({fondations:1490,'revision-programme':Math.round(TARIFS.coaching.revision_prog.prix*100)}). Ajouter un test de parité (comme celui d'OFFRES) qui relit rc-core pour éviter tout écart.
2. Durée ouverte : remplacer PROGRAMME_MS fixe par la durée de l'offre (boutique_prog.mois = 3, revision_prog.mois = 1), passée dans la même table.
3. App : _palierHerite (l.7633) doit tester programmeOuvreUltime AVANT le 'if(!payeCru) return 'aucun'' uniquement pendant PAIEMENT_RECENT_MS. Au-delà, afficher sur l'écran boutique « Achat en cours de vérification, écris à <CREATOR_EMAIL> » plutôt qu'un cadenas muet, si programmesAchete[id] existe sans droits.ultimeJusqu.
4. Remplir les 4 emplacements aCompleter uniquement avec les contenus que Kevin fournira (ne rien inventer) ; en attendant, ne rien changer à programmeVendable.
Tests : cloudflare/test/paypal.test.mjs (achat de fondations sans nœud boutique : validé, ultimeJusqu = +3 mois ; révision : +1 mois ; mauvais montant : refusé) ; app/tests.js pour le message de vérification. Lancer les deux suites.
Critère de fin : un achat de Fondations ou d'une révision payé chez PayPal écrit toujours droits/ avec la bonne durée.
```

7. Pilotage aveugle : MRR de l'app faux, rapport payeur en 403, aucun LTV/churn — Priorité : Moyenne · Effort : M

Constat. Le MRR affiché (pp-mrr, l. 120115-120131) se calcule depuis le cache local des dossiers (status==='AUTONOMIE_PREMIUM' && paymentStatus==='active'). Or le Worker ne passe jamais paymentStatus à cancelled : il n'écrit que 'active' (paypal.js:261). Les résiliés restent donc comptés, et « pp-cancelled » vaut toujours 0. Les abonnements coach (19/39 €) sont exclus, et l'Ultime « demi » est compté 24,90 €. rapport_payeur.mjs, lancé chaque mois par .github/workflows/rapport-payeur.yml, dépend d'une API que PayPal refuse sur ce compte (403 documenté en tête de scripts/rapport_abonnements.mjs). La note de décision et les commentaires de rc-core (l. 1002, 1006) citent encore 9,95 € et 99 €.

Pourquoi le mettre en place. Sans MRR, churn et conversion essai → payant fiables, impossible de fixer un CAC, d'arbitrer A/B dans la note de décision ou de valider une hausse de prix. La donnée existe déjà côté serveur (droits/, paypal_transactions, paypal_fins).

Intérêt réel. Projection à 12 mois, scénario réaliste : 60 Essentielle, 25 Ultime, 6 Coach, 2 Pro, soit 570 + 622,50 + 114 + 78 = 1 384,50 € de MRR TTC, environ 1 080 € nets HT. Pour la tenir, il faut suivre chaque mois : conversion essai → payant (objectif 8 à 15 % sans carte), churn mensuel (≤ 8 %), part d'annuel, revenu par coach. Un tableau calculé par le Worker à partir de droits/ et paypal_transactions le donne sans rien saisir. Il détecte aussi tout de suite les anomalies des points 2 et 4 (doubles abonnements, remboursements).

Prompt de mise en place.

```
Contexte : RepCore. MRR affiché dans l'app : app/rc-core.1693.js ~l.120115-120131 (ids pp-total-subs, pp-mrr, pp-cancelled). Données serveur : droits/<clé> (palier, echeance, source, abo), paypal_transactions/<id> (cle, abo, type, montant, premier, rembourse, annuleLe), paypal_fins, paypal_premiers. Serveur léger : cloudflare/src/ (planif.js pour le travail quotidien, metier.js, index.js pour les routes).
À faire :
1. cloudflare/src/metier.js : une fonction PURE indicateurs(droits, transactions, users, maintenant) qui renvoie {mrrTTC, mrrParFormule: {essentielle,ultime,coach,pro}, abonesActifs, resiliesEnCours, churnMois, conversionEssai, partAnnuel, revenuParCoach, rembourseMois}. Les annuels comptent
```

pour an/12, les plans demi pour le montant réellement prélevé, les coachs via users/<clé>/coachPlan + coachSubActive.

2. planif.js : une fois par jour, écrire indicateurs/<AAAA-MM-JJ> (lecture réservée au créateur dans database.rules.json ; ajouter la règle et la tester dans scripts/verif/regles.mjs).

3. App : l'onglet monétisation du créateur lit indicateurs/ (le dernier jour et 12 mois de courbe) au lieu de recalculer depuis le cache. Supprimer le calcul local. Pour un coach tiers, afficher seulement son palier et son quota.

4. cloudflare/src/paypal.js fermerALaFin : écrire aussi users/<clé>/paymentStatus='cancelled' quand la fin est atteinte (dans finUne), pour que les écrans existants redeviennent justes.

5. .github/workflows/rapport-payeur.yml : lancer scripts/rapport_abonnements.mjs (qui fonctionne) à la place de rapport_payeur.mjs, ou le désactiver avec un commentaire.

6. Nettoyer les mentions obsolètes 9,95 € / 99 € dans les commentaires de rc-core (~l.1000-1015) et mettre à jour NOTE-DECISION-MODELE-ECONOMIQUE.md.

Tests : cloudflare/test/metier.test.mjs (jeu de données inventé : 3 mensuels, 1 annuel, 1 demi, 1 résilié, 1 remboursé, 2 coachs, avec des valeurs attendues calculées à la main) ; node scripts/verif/regles.mjs ; suite app.

Critère de fin : le MRR affiché au créateur est égal, au centime près, à la somme calculée à la main sur le jeu de test, et les résiliés en sont exclus.

Sources marché : [Hevy Coach pricing](#), [Kahunas pricing](#), [Hevy pricing](#) ; Trainerize, Everfit et TrueCoach d'après le tableau sourcé de NOTE-DECISION-MODELE-ECONOMIQUE.md §4.

Agent 13 — QA et chasse aux bugs · 6/10

Le filet de tests est énorme et presque entièrement vert : 92/92 tests Node du Worker (cd cloudflare && node --test "test/*.test.mjs", 3,4 s), 6/6 tests de fonctions/test, 11 scripts scripts/verif hors réseau au vert, et la suite intégrée app/tests.js joue 5 745 assertions pour 2 échecs une fois bien lancée. Au chargement de /, /app/, /i/, /p/, /c/, /a/ et /maj/, aucune erreur JS (Chromium, Europe/Paris et America/Montreal). Aucun NaN, undefined ou Infinity affiché sur 11 onglets athlète, que le dossier soit vide ou rempli avec des valeurs piégées (« 62,5 », 0, vides). Mais ce filet ne protège rien. **Aucun de ces tests n'est lancé en CI** (.github/workflows/firebase.yml ne lance que des contrôles statiques). La suite front s'**interrompt en silence** selon la taille de la fenêtre : 1 760 assertions sautées avec la fenêtre par défaut de Chrome sans tête (800×600). Enfin, les dates « AAAA-MM-JJ » affichées sont décalées d'un jour hors fuseaux positifs.

Résultats bruts constatés (29/09/2026, build 1693) : - cloudflare/test/*.test.mjs : **92 pass / 0 fail**. Les 11 fichiers sans node:test (droits, paypal, sante, metier, planif...) passent aussi lancés seuls (11+23+32+17+12... « tests passés »). Les 3 regles*.emu.mjs n'ont **pas été lancés** : il leur faut l'émulateur Firebase en Java. - fonctions/test/*.test.js : 6/6. - scripts/verif : syntaxe, actifs, worker-actifs, suppression-distante, illustrations, regles, tarifs, references et source-prod sont OK (exit 0). banniere-geometrie, video-allege et banc-sante demandent Chrome en mode débogage distant (CDP) et ne sont pas en CI. - scripts/verif/suite.mjs (Chrome sans tête, CDP, fenêtre par défaut) : « **SUITE INTERROMPUE — la liste de gène ne s'ouvre pas — arrêt après 3985 assertion(s)** », 3 échecs. Même suite avec --window-size=412,4000 : **total 5745, 2 échecs**. - scripts/verif/idempotence.mjs (la suite jouée 3 fois dans la même page, fenêtre 412×4000) : totaux stables (5745 ×3), mais **3 → 10 → 10 échecs**, avec **7 fantômes** aux passes 2 et 3. Chronomètre hh:mm:ss, écran de célébration, « Tu as débloqué N badges », page au passage de rang, légendes post/story, charte, cyan au repos : la suite ne restaure pas ce qu'elle emprunte, alors que le README exige « fantomes2 vides ». - Le README de scripts/verif annonce encore « 4 142 / 18 » (mesure du 02/09) : ce chiffre de référence est périmé.

1. Brancher les tests existants sur la CI, et faire échouer la livraison — Priorité : Critique · Effort : S

Fusion. l'étape 4 (worker.yml) est remplacée par le workflow cloudflare.yml de l'agent Backend (point 4) ; le lint est porté par l'agent Performance (point 5).

Constat. .github/workflows/firebase.yml:107-160 lance syntaxe, actifs, worker-actifs, suppression-distante, illustrations, regles et tarifs, mais ni cloudflare/test, ni fonctions/test, ni app/tests.js. grep -n "node --test|cloudflare" .github/workflows/*.yml ne donne rien. scripts/verif/suite.mjs:87 se termine toujours par process.exit(0), même avec des échecs ou une suite interrompue. Il ne peut donc servir de barrière à personne. cloudflare/installer.ps1 et deploye.sh ne lancent aucun test non plus.

Pourquoi le mettre en place. 5 745 assertions front et 92 tests serveur existent, mais seule la discipline de celui qui livre les fait tourner. Le Worker gère PayPal, les droits et les notifications : une régression y coûte de l'argent ou des messages en double. La suite front vérifie déjà les régressions visibles (plancher typographique, menus de séance, notifications). Un lot livré sans la jouer peut tout casser sans qu'une seule ligne rouge apparaisse.

Intérêt réel. Une étape CI de 4 s (Worker) plus environ 3 min (suite front dans Chromium) attrape les régressions avant qu'un athlète ne les voie. Or il y a 87 commits depuis le 20/09, soit plusieurs lots par jour (C2 à C6 le 29/09) : sans barrière, la probabilité qu'un lot casse une fonction voisine est forte. Ce contrôle a déjà un coût visible : le chiffre de référence du README (4 142 / 18) n'a pas été relevé depuis quatre semaines. Pour le coach, c'est moins de « l'appli bug » remonté par ses athlètes, donc moins de désabonnements. Face à Trainerize ou Everfit, qui livrent derrière des pipelines, c'est la condition pour garder ce rythme de livraison sans perdre la confiance.

Prompt de mise en place.

```
Dépôt RepCore. Objectif : que la livraison échoue si un test échoue.
1. Dans scripts/verif/suite.mjs, remplacer le process.exit(0) final par : exit 1 si rap.erreur, si rap.echecs>0, ou si une ligne de rap.liste commence par «
  SUITE INTERROMPUE » ; exit 0 sinon. Ajouter une option --tolere=N (défaut 0) pour ignorer N échecs connus le temps de les corriger, et afficher
  total/echecs sur une ligne finale lisible.
2. Dans suite.mjs, avant le premier Runtime.evaluate, appeler Emulation.setDeviceMetricsOverride {width:412,height:4000,deviceScaleFactor:1,mobile:true} : la
  suite ne doit plus dépendre de la fenêtre par défaut (voir tests.js:56054, le menu de gène se referme quand sa case sort de l'écran).
3. Dans .github/workflows/firebase.yml, AVANT l'étape « Deployer », ajouter :
  - « Tests du serveur léger » : working-directory cloudflare, run: node --test "test/*.test.mjs"
  - « Tests des fonctions » : node --test fonctions/test/
  - « Suite intégrée » : lancer 'python3 -m http.server 8799 --bind 127.0.0.1 &' à la RACINE du dépôt (les règles sont lues en ../database.rules.json),
  démarrer Chrome (ubuntu-latest l'a : google-chrome --headless=new --no-sandbox --remote-debugging-port=9223 about:blank &), attendre que
  http://127.0.0.1:9223/json/version réponde, puis node scripts/verif/suite.mjs "http://127.0.0.1:8799/app/index.html".
4. Créer .github/workflows/worker.yml (déclenché sur push/PR touchant cloudflare/**) qui lance node --test dans cloudflare/.
5. Mettre à jour scripts/verif/README.md avec le nouveau chiffre de référence (total mesuré, échecs attendus = 0).
Critère de fin : sur une branche où l'on casse volontairement une assertion (ex. changer DLR_ALLEGE à 0.81), la CI est rouge ; sur main, elle est verte.
```

2. La suite front s'arrête net au premier test qui lève une exception — Priorité : Haute · Effort : M

Constat. Presque tous les tests sont écrits ok('nom', (()=>{...})) : la fonction s'exécute **avant** d'entrer dans ok. Si une assertion lève une exception au lieu de rendre _echec, tout part dans le catch global (tests.js:77993-78010) et la suite s'arrête. Reproduction : suite.mjs avec la fenêtre par défaut de Chrome sans tête. _r16Gene (tests.js:56054-56064) lève throw new Error('la liste de gène ne s'ouvre pas') parce que _woChoixPlacer (rc-core.1693.js:48906) referme la liste quand la case sort de l'écran (r.top>vh). Résultat : 3 985 assertions jouées au lieu de 5 745, soit **30 % de la suite ignorée**. Même chose si un seul test lève demain une TypeError. En plus, la suite **n'est pas rejouable** : idempotence.mjs rend 3 puis 10 puis 10 échecs, avec 7 fantômes (chronomètre, célébration, badges débloqués, page de rang, légendes, charte, cyan). Un test laisse derrière lui un état qui fait tomber les suivants.

Pourquoi le mettre en place. Une exception dans un test doit faire échouer ce test, pas masquer les 1 760 suivants. Aujourd'hui, le résultat de la suite dépend de la hauteur de la fenêtre : c'est la définition d'un test instable. Le README décrit déjà le même mal pour Ciquel (« 2 117 joués sans ce chargement, 3 729 avec »).

Intérêt réel. Chaque lot touche en moyenne plusieurs écrans. Si la moitié finale de la suite (santé, évictions, nutrition, canal, prospects...) ne tourne plus, les régressions de ces zones passent. Isoler les tests transforme « suite interrompue » en « 1 échec localisé ». La lecture du rapport tombe de plusieurs dizaines de minutes à quelques secondes. C'est aussi ce qui rend le point 1 fiable : une barrière CI qui passe au vert parce que la moitié des tests n'a pas tourné est pire que pas de barrière.

Prompt de mise en place.

```
Dépôt RepCore, fichier app/tests.js (5 Mo, ne pas le réécrire entièrement).
1. Localiser les fonctions ok/okA de testExercices (grep -n "const ok=|function ok(" app/tests.js). Faire accepter à ok un 2e argument FONCTION : si typeof v==='function', l'exécuter dans try/catch et enregistrer {n,ok:false,d:'EXCEPTION : '+message} en cas d'erreur, sans propager.
2. Script de migration (node, dans scratchpad) qui remplace mécaniquement le motif `ok('...',(()=>{ ... }));` par `ok('...',()=>{ ... })`, et `ok('...',(()=>{ ... }));` par `ok('...',()=>{ ... })`.
3. Dans _r16Gene (tests.js:56054) : appeler b.scrollToView({block:'center'}) avant b.click(), et remplacer les throw par un retour d'échec.
4. Relancer scripts/verif/suite.mjs deux fois : fenêtre par défaut ET avec Emulation.setDeviceMetricsOverride 412x4000.
5. Fantômes de rejouabilité : lancer node scripts/verif/idempotence.mjs "http://127.0.0.1:8799/app/index.html" (Chrome avec --remote-debugging-port=9223 --
```

```
window-size=412,4000). Pour chacun des 7 tests listés en fantomes2 (« Le chronometre de seance affiche hh:mm:ss », « L'écran de célébration... », « Le récapitulatif dit « Tu as débloquent N badges », « Page : proposée au passage de rang... », « Légendes : en post... », « Charte : ni turet cadratin... », « Le cyan n'est peint sur AUCUNE surface au repos », trouver le test PRÉCÉDENT qui laisse un état (localStorage rc_*, woState, overlay resté dans le DOM, classe sur body, timer) et ajouter la restauration dans son finally. Critère de fin : le total est IDENTIQUE dans les deux cas, plus aucune ligne « 🛑 SUITE INTERROMPUE », et idempotence.mjs rend trois fois le même nombre d'échecs avec fantomes2, fantomes3 et gueris vides.
```

3. Dates « AAAA-MM-JJ » affichées la veille hors de l'Europe (Antilles, Guyane, Québec, Polynésie) — Priorité : Haute · Effort : S

Constat. `new Date('2026-09-29')` est lu comme minuit **UTC**, puis formaté en heure locale. Vérifié dans Chromium : `_fmtDateCourte('2026-09-29')` (`rc-core.1693.js:71725`) rend « 29 sept. » à Paris et à La Réunion, mais « **28 sept.** » à America/Martinique, America/Montreal et Pacific/Tahiti. Même défaut pour la date de compétition (`rc-core.1693.js:67104` : samedi 03/10 affiché « vendredi 02 octobre ») et pour l'historique du sommeil (115042), dont `e.date` est une clé « AAAA-MM-JJ » (63205, 83120). Défaut inverse, `toISOString().slice(0,10)` sur un horodatage (14706, 14737, 14752, 14760, 14765, 14816, 14820, 19958, 11635) : la veille entre 0 h et 2 h à Paris.

Pourquoi le mettre en place. Un athlète martiniquais voit sa compétition annoncée un jour trop tôt, et ses nuits et constantes rangées au mauvais jour, à côté d'un compte à rebours « J-4 » calculé juste. L'incohérence saute aux yeux. La suite force Europe/Paris (`suite.mjs:31`), elle ne peut donc pas voir ce défaut.

Intérêt réel. Les DROM et le Québec sont des marchés francophones naturels pour un coach français en ligne. Everfit et Trainerize gèrent le fuseau de chaque client. La correction est locale : un seul analyseur « AAAA-MM-JJ » vers une date locale, réutilisé partout (le code en a déjà l'esprit avec `_cleSemaineISO`, `rc-core.1693.js:108760`, et `localISODate`, 11673). Un export de commissions (19958) daté de la veille fausse aussi une ligne comptable par mois de décalage.

Prompt de mise en place.

```
Dépôt RepCore, app/rc-core.1693.js.
1. Ajouter près de localISODate (ligne ~11673) : function dateLocaleDeCle(s){ const m=/^(d{4})-(d{2})-(d{2})$/.exec(String(s)); return m?new Date(+m[1],+m[2]-1,+m[3],12):new Date(s); } (midi, pour échapper aux changements d'heure).
2. L'utiliser dans _fmtDateCourte (71725), dans l'affichage de la compétition (67104), dans le libellé de l'historique du sommeil (115042), et dans tout `new Date(x.date).toLocaleDateString` dont x.date est une clé AAAA-MM-JJ (grep -nE `new Date\([a-z]+\.date\)\.toLocaleDateString`).
3. Remplacer `new Date(t).toISOString().slice(0,10)` par localISODate(new Date(t)) aux lignes 11635, 14706, 14737, 14752, 14760, 14765, 14816, 14820 et 19958 (le nom du fichier d'export, 120326, peut rester en UTC).
4. Tests dans app/tests.js : un bloc qui, SANS dépendre du fuseau du harnais, vérifie que _fmtDateCourte('2026-09-29') contient « 29 » et que la compétition du '2026-10-03' donne « samedi ». Ajouter dans scripts/verif/suite.mjs une option --tz (défaut Europe/Paris) et lancer la suite une fois en America/Martinique : ce bloc doit y rester vert.
Critère de fin : la sonde Playwright (fuseaux Europe/Paris, America/Martinique, Pacific/Tahiti) rend la même date dans les trois.
```

4. Les deux échecs actuels de la suite : un vrai défaut de lisibilité, deux faux positifs, un test pollué — Priorité : Moyenne · Effort : S

Constat. (a) « Aucun texte de l'accueil athlète ne descend sous le plancher » (`tests.js:11275`). « 0 / 1 800 V » (7,9 px) et « vers » (6,6 px) sont des **faux positifs** : ils vivent dans `.clh-tete .rg-txt{display:none!important}` (`rc-style.1693.css:11725`), et le scanner ne teste que le `display` de l'élément lui-même. « Encore 1 séance » est un **vrai défaut** : `#clh-streak .sk-reste{font-size:1.95cqw}` (`rc-style.1693.css:11756`, idem `#clh-assiette .sk-reste 11935`) donne 9,3 px et descend à environ 8 px sur un téléphone de 412 px, sous le plancher de 11 px que le produit s'est fixé. (b) « le choix est écrit dans u.pushPrefs AVANT la demande » (`tests.js:50963`) échoue par **pollution du test** : il crée un second `#wd-notif-invite` alors que le vrai (`app/index.html:4515`) a déjà été peint par un test antérieur. `getElementById('inv-notif-g-invitations')` lit alors la case du vrai écran. Reproduit : quand la vraie carte est peinte, `_invNotifChoixLus()` rend `invitations:false`, d'où `filleur:false`. Le code de l'app (`invNotifOui`, `rc-core.1693.js:75165`) est correct.

Pourquoi le mettre en place. Deux échecs permanents habituent à lire du rouge comme du bruit. C'est le chemin le plus court vers une vraie régression ignorée, et cela bloque le passage à « 0 échec attendu » du point 1.

Intérêt réel. Le compteur de série est l'élément de motivation le plus vu de l'accueil. Son sous-titre à 8 px est illisible bras tendu, surtout pour les 40 ans et plus, cœur de cible du coaching en ligne. Corriger ces trois points ramène la suite à 0 échec, et la CI devient une barrière stricte et crédible.

Prompt de mise en place.

```
Dépôt RepCore.
1. app/rc-style.1693.css lignes ~11756 et ~11935 : remplacer font-size:1.95cqw de #clh-streak .sk-reste et #clh-assiette .sk-reste par font-size:max(11px,1.95cqw) et vérifier visuellement à 360, 412 et 480 px (scripts/verif/capture.mjs) que le texte ne déborde pas du médaillon (au besoin, réduire letter-spacing ou autoriser 2 lignes).
2. app/tests.js ~11296 (scanner du plancher) : ignorer un élément non rendu avec `if(!el.checkVisibility||!el.checkVisibility({checkVisibilityCSS:true})) return;` au lieu du seul test cs.display/visibility, et garder le témoin « vus>15 ».
3. app/tests.js ~50963 : au lieu de créer un second #wd-notif-invite, sauvegarder innerHTML du vrai (document.getElementById('wd-notif-invite')), l'utiliser comme zone, et le restaurer dans le finally.
Critère de fin : scripts/verif/suite.mjs rend echecs:0.
```

5. 1 833 catch(e){} vides, dont 154 autour d'écritures du dossier — Priorité : Moyenne · Effort : M

Constat. Dans `rc-core.1693.js` : 3 167 `catch`, dont **1 833 vides**, 112 `.catch(()=>{})` et 384 `catch` qui rendent `null/false/[]` sans rien dire. `saveUser()` est appelée 311 fois : 242 appels ignorent son retour (qui signale le quota plein, 115417-115437), et **154** sont enveloppés dans `try{ saveUser(); }catch(e){}` (ex. 2227, 12492, 21448, 22585, 29043, 33325). Les plus dangereux : - `DB.get` (5667) rend `null` sur un `rc_users` illisible. `saveUser` fait alors `DB.get('users')||{} (115420)` et réécrit une carte ne contenant que le dossier courant. Par lecture du code : les autres dossiers en cache local (les athlètes d'un coach) sont écrasés sans un mot. - `saveUser` lève une `TypeError` si `currentUser` est `null` (115418) : les 154 enveloppes la font disparaître, et la saisie est perdue en silence. - `_tunnelEcrire` (28173) avale l'échec d'écriture de la session.

Pourquoi le mettre en place. Un `catch` vide ne corrige rien, il rend le bug invisible, pour l'utilisateur comme pour le développeur. Avec 1 833 occurrences, un bug réel de ce fichier a de fortes chances d'être muet. Or l'équipe a déjà l'outil qu'il faut : `toastSync`, et le motif « QUAND CA RATE, ET SEULEMENT QUAND CA RATE » (115438+).

Intérêt réel. Une perte de saisie silencieuse (séance, pesée, bilan) est la pire expérience possible dans une app de suivi : l'athlète s'en aperçoit une semaine plus tard, et le coach perd confiance dans les données. Rendre ces échecs visibles (journal local et toast rare) permet de mesurer leur fréquence réelle, au lieu de la supposer nulle. Aucun concurrent ne garantit l'absence de bugs, mais un « ta saisie n'a pas pu être enregistrée, réessaie » vaut mieux qu'une disparition. Le coût reste bas : une fonction utilitaire et un remplacement mécanique des 154 cas les plus sensibles.

Prompt de mise en place.

Dépôt RepCore, app/rc-core.1693.js. Ne pas toucher à la logique de synchronisation distante (CLOUD.*), hors périmètre.

1. Ajouter une fonction rcErreurMuette(ou,e) qui garde les 50 dernières erreurs dans un tableau en mémoire (window._rcErreurs) et fait console.warn(['RepCore'],ou,e). Aucun envoi réseau.
2. Ajouter saveUserOuDire(perdu) : appelle saveUser(), et si elle lève ou rend false, appelle rcErreurMuette et affiche UNE fois par minute au plus un toast orange « <perdu> n'a pas pu être enregistré sur cet appareil ».
3. Remplacer mécaniquement les `try{ saveUser(); }catch(e){` (grep -nE "try{ ?saveUser\\(\\); ?}\\catch\\(e\\)\\{\\}") par saveUserOuDire(...) aux endroits de saisie utilisateur (séance, pesée, bilan, sommeil, nutrition). Laisser les écritures de fond avec rcErreurMuette seul.
4. Dans saveUser (~115420) : si localStorage.getItem('rc_users') est non vide mais que DB.get('users') rend null (JSON corrompu), NE PAS réécrire la carte ; sauvegarder la chaîne brute sous rc_users_corrompu_<horodatage>, appeler rcErreurMuette, et n'écrire que 'session'.
5. Tests dans app/tests.js : (a) rc_users corrompu, saveUser ne détruit pas la clé et crée la copie ; (b) currentUser=null, saveUserOuDire ne lève pas et pousse une entrée dans _rcErreurs ; (c) quota simulé (localStorage.setItem remplacé pour lever QuotaExceededError), le toast apparaît une seule fois.

Critère de fin : suite.mjs sans nouvel échec ; grep du motif vide autour de saveUser réduit d'au moins 100 occurrences.

6. Couverture : ce qu'aucun test ne voit — Priorité : Moyenne · Effort : M

Constat. Tout est testé dans un seul fuseau (Emulation.setTimezoneOverride 'Europe/Paris', suite.mjs:31) et une seule taille de fenêtre, non maîtrisée (point 2). Les 3 bancs de règles Firebase (cloudflare/test/regles-amis.emu.mjs, regles-evenements.emu.mjs, regles-sante.emu.mjs) demandent l'émulateur Java et ne tournent nulle part. banniere-geometrie, video-allege et banc-sante (scripts/verif) ne tournent que sur Chrome lancé à la main (exit 1 : ECONNREFUSED 127.0.0.1:9223). Aucun test ne charge les pages publiques (/ , /i/ , /p/ , /c/ , /a/) pour y guetter une erreur JS : mon passage n'en trouve aucune aujourd'hui, mais rien ne le garantit demain. La saisie décimale n'est testée qu'à travers des champs type="number", jamais sur un vrai clavier iOS français à virgule, dont le comportement varie.

Pourquoi le mettre en place. Les défauts trouvés dans cet audit (dates décalées, test dépendant de la fenêtre) viennent précisément de ces angles morts : un fuseau et un format d'écran figés. Les règles de base sont la dernière ligne de défense des données, et leur banc n'est jamais joué.

Intérêt réel. Une matrice minimale (2 fuseaux × 2 largeurs) et un test de fumée des pages publiques coûtent quelques minutes de CI. Ils couvrent les deux classes de bugs les plus probables dans une PWA mobile francophone : décalages de date et mise en page sur petit écran. Le banc des règles attrape une règle qui refuse une écriture légitime, défaut déjà vécu deux fois (« dispo », « logo », [firebase.yml:145-150](#)) et que l'utilisateur ne voit jamais.

Prompt de mise en place.

Dépôt RepCore.

1. Créer scripts/verif/fumee-pages.mjs : avec Playwright (ou CDP comme suite.mjs), charger / , /app/ , /i/ , /p/ , /c/ , /a/ , /maj/ servis par python3 -m http.server à la racine, en 390×844 et 1280×800, fuseaux Europe/Paris et America/Martinique ; échouer (exit 1) sur tout événement pageerror, toute réponse 4xx/5xx d'une ressource du même hôte hors liste d'exceptions documentée (favicon, badges à repli), et tout texte visible correspondant à /NaN|undefined|Infinity|\\[object Object\\]/.
2. Ajouter dans .github/workflows/firebase.yml une étape « Fumée des pages », et une étape « Règles (émulateur) » qui installe firebase-tools, lance `firebase emulators:exec --only database --project demo-repcore "node cloudflare/test/regles-amis.emu.mjs && node cloudflare/test/regles-evenements.emu.mjs && node cloudflare/test/regles-sante.emu.mjs"` (adapter au port lu par les scripts).
3. Rendre banniere-geometrie.mjs lançable en CI avec le même Chrome que la suite.

Critère de fin : les nouvelles étapes sont vertes sur main et rouges si l'on introduit volontairement 'undefined' dans un libellé de l'accueil.

7. Petits défauts de robustesse au chargement — Priorité : Basse · Effort : S

Constat. - navigator.serviceWorker.register('./sw.js').then(reg=>{ reg.update(); ... }) (app/index.html:408-414) n'a ni .catch ni garde sur reg. Quand l'enregistrement échoue ou est bloqué (navigation privée Firefox, politique d'entreprise, Playwright), on obtient une promesse rejetée non traitée ou TypeError: Cannot read properties of undefined (reading 'update'), reproduit sur /app/ , /i/ et /maj/ . - /p/ n'a pas de <link rel="icon"> : 404 sur /favicon.ico à chaque visite. - Pendant la suite, 404 sur 23 visuels app/img/badges/*.webp et sur tout app/img/cartes/ (absent). Le repli existe (_htmlBadgeImg, rc-core.1693.js:76290 ; cadre dessiné, 80091), mais chaque affichage de collection coûte des requêtes en échec. - _lundiDe est déclarée deux fois, à l'identique (rc-core.1693.js:26753 et 62987) : c'est la seconde qui gagne, et une divergence future ne se verrait pas.

Pourquoi le mettre en place. Ce sont des erreurs de console qui masquent les vraies dans les rapports d'erreurs et dans la fumée du point 6, et des requêtes inutiles sur réseau mobile.

Intérêt réel. Gain modeste mais immédiat : une console propre rend tout futur incident lisible, et une page publique sans 404 est mieux notée par les outils d'audit (Lighthouse), ce qui compte pour la page de vente et l'acquisition. Supprimer le doublon _lundiDe évite qu'une correction de l'une des deux copies (par exemple pour le changement d'heure) reste sans effet. Chaque correctif fait moins de 10 lignes.

Prompt de mise en place.

Dépôt RepCore.

1. app/index.html ~408 : `navigator.serviceWorker.register('./sw.js').then(reg=>{ if(!reg) return; try{ reg.update(); }catch(e){ ... }).catch(e=>{ try{ console.warn(['RepCore'] service worker non enregistré,e); }catch(\_){});`
2. p/index.html (et c/ , a/ si absents) : ajouter <link rel="icon" href="/app/icons/icon-192x192.png">.
3. Dans _htmlBadgeImg (rc-core ~76290) : mémoriser dans un Set les visuels déjà tombés en erreur et servir directement le repli ensuite ; idem pour carteCadreImage (~80095) : ne pas redemander un cadre en échec.
4. Supprimer la seconde déclaration de _lundiDe (rc-core ~62987) en gardant le commentaire utile à côté de la première.
5. Ajouter dans app/tests.js une assertion qui compte les déclarations `function \_lundiDe` dans _prodSrc() et exige 1, et une qui vérifie la présence de `.catch(` après `serviceWorker.register`.

Critère de fin : chargement de /app/ et /p/ sans aucune erreur console (hors certificats externes), suite.mjs sans nouvel échec.

Agent 14 — Onboarding et activation · 5/10

Les fondations d'onboarding sont soignées : profilage progressif (nom, date de naissance et consentement santé demandés au moment où ils servent : `s-identite`, `s-naissance`), une seule touche pour poser les jours d'entraînement (`s-jours-entrainement`), la liste « Pour démarrer » en 3 étapes (`_htmlDemarrage`, `rc-core:43231`), un essai de 30 jours sans carte, et une gestion fine des navigateurs internes à Instagram (`i/index.html`, `s-install`). Mais la première valeur n'arrive pas. Mesuré avec Playwright (390 px, UA Android) : un athlète **autonome** traverse 9 écrans et fait environ 25 gestes, puis tombe sur **sept jours vides**. Pour s'entraîner, il doit construire sa séance exercice par exercice (6 à 10 min) ou payer 14,90 €. Un athlète **invité** perd son code dès qu'il installe l'app, ce que le tuto lui fait pourtant faire en premier. Enfin, un coach qui veut embarquer ses 20 athlètes n'a ni import en masse, ni moyen de préparer leurs programmes avant leur inscription.

1. L'athlète autonome n'a aucune première séance prête (pas de moment « aha ») — Priorité : Critique · Effort : S

Constat. Le parcours « 3 questions → 1 séance générée » existe toujours (`ouvrirPremiereSeance`, `genererSeanceDepart` `rc-core:42913`), mais il est coupé par `const PS_PARCOURS_ACTIV=false` (`rc-core:43307`). Du coup, « 2 · Lance ta première séance » (`pdLancerSeance`, voie `gerer`) ouvre `s-session-manager` avec 7 jours « REPOS / Active ce jour pour y mettre une séance » (`capture_shots14/13-premiere-seance-vierge.png`). « Partir d'un programme » (`ouvrirBoutique`) ne propose qu'un seul programme, Fondations, à 14,90 € (`RC_PROGRAMMES` `rc-core:118`, `prixCts:1490`, `RC_BOUTIQUE_GRATUITE=false`). Les 4 autres emplacements sont en `aCompleter`.

Pourquoi le mettre en place. L'essai de 30 jours (`essaiOuvrir`, `rc-core:1654`) sert à faire vivre une première séance réussie. Aujourd'hui, l'athlète reçoit à la place un travail de saisie ou une demande de paiement. Hevy, Fitbod et Trainerize proposent tous une séance jouable en moins de 2 minutes après l'inscription.

Intérêt réel. Le « moment aha » d'une app de musculation, c'est la première série notée avec une charge proposée. Il passerait de 6 à 10 minutes de construction manuelle (ou d'un achat) à environ 60 secondes : 3 touches, puis « Lancer ». Le code existe déjà et il est testé (`tests.js:43041` et suivantes), donc l'effort est minime. Chaque point d'activation J0 gagné remonte mécaniquement la conversion à la fin de l'essai (`s-essai-bilan`). Le bilan de fin d'essai (`essaiBilan`, qui compte `faites` et `seances`) aurait enfin quelque chose à montrer. La séance porte `_essai` : un coach ou un achat Fondations l'écrase sans conflit. On peut aussi faire de Fondations le « programme d'essai » gratuit pendant 30 jours, puis payant, ce qui crée un vrai motif de conversion.

Prompt de mise en place.

```
Contexte : RepCore, app/rc-core.1693.js. Le parcours de première séance (ouvrirPremiereSeance, _psRendre, psRepondre, _psPoserSeance, genererSeanceDepart, PS_OBJECTIFS, PS_LIEUX, PS_FREQUENCES) est désactivé par 'const PS_PARCOURS_ACTIV=false;' (vers la ligne 43307). Un athlète autonome en essai qui touche « Lance ta première séance » (_htmlDemarrage -> pdLancerSeance -> _pdSeance voie 'gerer') tombe sur s-session-manager avec 7 créneaux éteints.
```

À faire :

1. Réactiver le parcours UNIQUEMENT pour l'athlète sans coach : passer `PS_PARCOURS_ACTIV` à `true` et ajouter dans `_doitProposerPremiereSeance` la garde `'if(u.coachId|u.coachEmailKey) return false;'` (le coaché attend le programme de son coach).
2. Dans `_pdSeance(u)`, pour un athlète sans coach et sans séance active, renvoyer `{voie:'parcours', sous:'3 questions, ta séance est prête'}`. Dans `pdLancerSeance`, la voie `'parcours'` appelle `ouvrirPremiereSeance()`. Garder `'gerer'` pour les autres cas.
3. Sur l'écran final (`_htmlSeanceDepart`), ajouter sous le bouton Lancer un lien secondaire « Voir le programme complet Fondations » -> `ouvrirBoutique()`.
4. Ne rien changer à `psPasser` (il n'écrit rien dans le dossier) ni au marqueur `_essai` de la séance générée (`_configReelle` doit rester `false` pour elle). Cas limites : un compte qui a déjà `u.sessions` non vide ne voit pas le parcours ; une séance générée est écrasée par une publication du coach ou un achat ; en hors ligne, `_psPoserSeance` fait `saveUser()` en local.

Tests (`app/tests.js`, section première séance vers la ligne 43841) : mettre à jour les assertions qui figent `PS_PARCOURS_ACTIV=false` ; ajouter : (a) athlète sans coach, sans séance -> `_doitProposerPremiereSeance` vrai ; (b) athlète avec `coachId` -> faux ; (c) `_pdSeance` rend voie `'parcours'` pour (a) et `'gerer'/'selecteur'` sinon ; (d) `genererSeanceDepart({objectif:'muscle',freq:3,lieu:'salle'},0)` rend une séance active avec au moins 4 exercices et `_essai` vrai.

Lancer la suite : servir le dépôt (`python3 -m http.server`), ouvrir `/app/`, exécuter `chargerTests()` dans la console (ou via Playwright) : 0 échec.

Critère de fin : avec Playwright, un compte athlète neuf sans coach va de l'accueil à la première série en 5 touches ou moins.

2. Le code d'invitation se perd : installation de la PWA et bouton « J'ai un code coach » — Priorité : Critique · Effort : S

Constat. `?inv=` n'est gardé qu'en mémoire (`window._invitationCode=inv`, `rc-core:5892`). `window._importedCoach` (`rc-core:5869`) aussi. Rien n'est écrit dans `localStorage`. Or le tuto de `/i/` et l'écran `s-install` poussent à **installer d'abord**, et l'app installée s'ouvre sur `start_url: ./index.html` (`app/manifest.json`), sans paramètres : l'invitation disparaît. Deuxième bug, reproduit : ouvrir `/i/?c=RC-A3KZ-W9MX`, toucher « Ouvrir RepCore », « Continuer sans installer », puis « J'ai un code coach ». On arrive sur `s-athlete-entry` avec le champ `#ae-code` vide, alors que `window._invitationCode` vaut `RC-A3KZ-W9MX`. `ouvrirCodeCoach` (`rc-core:11588`) ne le relit pas (`capture_shots14/17-invite-code-non-prerempli.png`). Entre-temps, l'invité est passé par la page de vente avec les prix (`shots14/16-invite-apres-passer.png`).

Pourquoi le mettre en place. L'invité d'un coach est l'utilisateur le plus rentable : son coach paie 19 ou 39 €. C'est aussi le plus fragile, car il n'a rien choisi. Lui faire retaper un code `RC-XXXX-XXXX` qu'il n'a souvent pas en clair (le message `messageRelance` ne contient que le lien) est la première cause probable d'abandon et de message « ça marche pas » au coach.

Intérêt réel. Sur le chemin nominal, on supprime 2 écrans et une saisie de 12 caractères. L'invité touche le lien, arrive directement sur l'inscription avec « Coach Kevin reconnu », et le code est appliqué automatiquement. Chaque invitation ratée coûte au coach un échange WhatsApp de support et un athlète qui démarre en retard. Sur 20 athlètes, c'est facilement 5 à 10 relances évitées. Le compteur `code_entered / code_valid` (`RCM_EVENTS`) mesurera directement le gain.

Prompt de mise en place.

```
Contexte : app/rc-core.1693.js. importFromURL (vers la ligne 5836) lit ?inv= et ?coachpkg= mais ne les garde qu'en mémoire (window._invitationCode, window._importedCoach). Après installation de la PWA, le lancement se fait sur start_url sans paramètres (app/manifest.json) : l'invitation est perdue. ouvrirCodeCoach() (vers 11588) ouvre s-athlete-entry sans pré-remplir #ae-code.
```

À faire :

1. Dans `importFromURL`, quand `inv` est valide (`/^[A-Z0-9-]{6,20}$/`), persister `localStorage 'rc_invitation' = JSON {inv, coach:(coach validé ou null), le:Date.now() }` dans `try/catch`. Même chose quand seul `coachpkg` est présent (`inv=null`).
2. Ajouter une fonction pure `invitationEnAttente(maintenant)` qui relit `'rc_invitation'`, l'ignore au-delà de 30 jours et rend `{inv, coach}` ou `null`. Au démarrage, si `window._invitationCode` est vide, le réhydrater depuis cette fonction.
3. Dans `rcEcranDeDepart` / le routage sans session (là où `window._importedCoach` envoie sur `s-athlete-entry`, vers la ligne 6950) : si `invitationEnAttente()` existe et qu'aucun utilisateur n'est connecté, aller sur `s-athlete-entry`, zone code ouverte, `#ae-code` pré-rempli et bannière « Coach X reconnu ». On ne passe PAS par `s-welcome` (page de vente).
4. Dans `ouvrirCodeCoach` : pré-remplir `#ae-code` avec `window._invitationCode || invitationEnAttente()?.inv`.
5. Effacer `'rc_invitation'` après un rattachement réussi (après `_appliquerCodeApresInscription / doLinkCoach ok`).
6. Ajouter `'inv'` à la liste `aNettoyer` de `importFromURL` pour nettoyer l'URL.

Cas limites : `localStorage` indisponible (`try/catch`, pas d'erreur) ; code déjà consommé (le message d'erreur existant `_verifierCodeSansConsommer` s'affiche) ; utilisateur déjà connecté (`s-client-code` pré-rempli, comportement `_annoncerInvitation` existant) ; iOS, où la PWA installée ne partage pas le stockage de Safari : ajouter sous `#ae-code` le texte « Colle le lien reçu de ton coach », déjà accepté par `doAthleteCode`.

```
Tests app/tests.js : invitationEnAttente (absent, frais, périmé à 31 jours, JSON corrompu) ; ouvrirCodeCoach pré-remplit le champ quand window._invitationCode est posé ; importFromURL retire inv de l'URL. Vérifier avec Playwright : /i/?c=RC-A3KZ-W9MX -> Ouvrir -> Continuer sans installer : on arrive sur s-athlete-entry avec le code pré-rempli ; recharger /app/ sans paramètres : le code est toujours pré-rempli. chargerTests() : 0 échec.
```

3. Le lien envoyé par le coach contourne la page /i/ et son tuto vidéo — Priorité : Haute · Effort : S

Constat. lienInvitation (rc-core:120643) construit APP_BASE_URL+'?coachpkg=...&inv=...', c'est-à-dire un lien direct vers /app/. Or le tuto vidéo de 1 min 40 (« Installer / Ton compte / Ton bilan / Les photos / Me prévenir », i/index.html, tuto.mp4 de 12,5 Mo) et l'aide spécifique au navigateur d'Instagram ne s'affichent que sur /i/?inv=... (mode coach). Aucune invitation créée par _envoyerInvitation ou relancerInvitation ne passe donc par ce tuto. Le lien fait aussi plus de 200 caractères (base64 du coach).

Pourquoi le mettre en place. Le tuto a été tourné pour ce moment précis (commit b72c976, « /i : le tuto vidéo pour l'invité d'un coach »). Le lien réellement envoyé l'ignore. Un lien court en /i/ passe aussi mieux sur WhatsApp et Instagram : il est lisible, ne se coupe pas et ne ressemble pas à un lien suspect.

Intérêt réel. Chaque invité voit enfin le tuto d'installation. En pratique, ce sont les questions « comment j'installe » et « où est mon bilan » que Kevin n'a plus à traiter à la main, par athlète. Le lien passe d'environ 250 à environ 45 caractères, ce qui compte pour un SMS et pour la confiance. La page /i/ compte déjà l'arrivée (/arrivée du Worker) : on mesure enfin le taux « clic sur l'invitation → compte créé ». Dans le navigateur d'Instagram, l'invité reçoit l'explication « ouvrir dans le navigateur » au lieu d'une app qui ne s'installe pas.

Prompt de mise en place.

```
Contexte : app/rc-core.1693.js, lienInvitation(code,user) (vers la ligne 120643) rend APP_BASE_URL+'?coachpkg=<base64>&inv=<code>'. La page i/index.html gère ?inv= (mode 'coach' : tuto vidéo, chapitres, aide Instagram) puis redirige vers ../app/ en gardant les paramètres. RC_LIEN_COURT (vers la ligne 956) vaut origin+'i/' sur Firebase et APP_BASE_URL ailleurs.

À faire :
1. lienInvitation rend désormais : (RC_LIEN_COURT se terminant par /i ? RC_LIEN_COURT : APP_BASE_URL.replace(/app\/$/, '')+'i/' + '?inv=' + code, passé à lienAttribute(...,{src:'invitation'})). coachpkg n'est plus dans le lien : le coach est retrouvé par le code via _verifierCodeSansConsommer, qui lit déjà le payload du code (coachId, coachName...).
2. Vérifier que le chemin app ?inv=CODE seul, sans coachpkg, affiche la bannière du coach : après _verifierCodeSansConsommer, poser le nom du coach tiré du payload dans #ae-coach-banner. Dépend du point « invitation perdue » : pré-remplir #ae-code depuis window._invitationCode.
3. Garder la compatibilité : les anciens liens avec coachpkg continuent de fonctionner (importFromURL inchangé).
4. Mettre à jour messageRelance si le texte mentionne le lien : il doit rester une seule URL.
Cas limites : hébergement GitHub Pages (sous-dossier), où /i n'existe pas en réécriture mais i/index.html existe dans le dépôt : utiliser le chemin relatif 'i/' ; code invalide ou expiré (message existant).
Tests app/tests.js : lienInvitation rend une URL contenant '/i/' et 'inv=' et ne contenant pas 'coachpkg' ; longueur < 90 caractères ; un test de bout en bout simulé : ?inv=CODE -> s-athlete-entry pré-rempli.
Vérifier avec Playwright : générer un lien via inviterAthlete (fetch stubbé), l'ouvrir : on voit le tuto sur /i/, puis Ouvrir RepCore -> s-athlete-entry pré-rempli. chargerTests() : 0 échec.
```

4. Embarquer 20 athlètes existants : invitations une par une, sans programme préparé avant l'inscription — Priorité : Haute · Effort : L

Constat. s-coach-code n'accepte qu'un prénom et un nom à la fois (_envoyerInvitation → inviterAthlete, rc-core:120785), suivis d'une feuille de partage par personne. Il n'existe aucun import CSV ou de contacts : le seul « csv » du code est de l'export (rc-core:12645). Surtout, tant que l'athlète ne s'est pas inscrit, le coach ne peut rien lui préparer : resolveClient renvoie « Cet athlète n'a pas encore créé son compte. Le programme sera disponible après son inscription. » (rc-core:36210). À la première ouverture, l'invité voit donc « Programme en cours de création » (rc-core:47523). Estimation : 20 × (saisie, partage, choix du contact, envoi) ≈ 40 à 60 s, soit 15 à 20 minutes d'invitations. Il faut ensuite surveiller chaque inscription pour assigner un modèle.

Pourquoi le mettre en place. C'est le moment où un coach décide de migrer ou non depuis Excel, WhatsApp ou TrueCoach. Trainerize, Everfit et TrueCoach permettent tous d'ajouter des clients en lot et de leur préparer un programme avant leur premier login. Sans cela, le coach doit revenir 20 fois, au rythme de ses athlètes.

Intérêt réel. Le coach colle sa liste (« Prénom Nom » par ligne), choisit un modèle de programme par défaut et obtient 20 liens prêts à envoyer en 2 minutes au lieu de 20 plus un suivi. Chaque athlète trouve son programme dès sa première ouverture : pas d'écran vide, et la première séance est possible le jour même. Une activation plus rapide des 20 athlètes accélère le passage du palier Libre (1 athlète) à Coach à 19 € ou Pro à 39 € (COACH_PALIER, rc-core:1418). C'est l'argument commercial direct face à Trainerize.

Prompt de mise en place.

```
Contexte : app/rc-core.1693.js et app/index.html. Invitation unitaire : écran s-coach-code (champs #inv-prenom, #inv-nom, bouton _envoyerInvitation) -> inviterAthlete(prenom,nom) -> _genAccessCode(studentName,INV_MOIS_DEFAULT) écrit /codes/<code> dans Firebase, puis push dans currentUser.studentCodes. Un code porte un payload (coachId, prenom, nom, expiry...). resolveClient(cid) refuse un client _fromCode (pas encore inscrit).

À faire :
1. Invitations en lot : dans s-coach-code, ajouter un bouton « Inviter plusieurs athlètes » qui ouvre une modale : un textarea (une personne par ligne, « Prénom Nom » ou « Prénom Nom ; email »), et un select « Programme de départ » listant les modèles du coach (la même source que _assignerModele / openCoachPrograms) avec l'option « aucun ». Fonction pure parserListeInvites(texte) -> [{prenom,nom,email?}] : trim, lignes vides ignorées, doublons retirés via _clePersonne, 50 lignes maximum.
2. inviterEnLot(liste,modeleId) : appelle inviterAthlete séquentiellement (pas en parallèle, pour ménager Spark), saute les invitationExistante, collecte {prenom, lien} ou {prenom, raison}. Si modeleId est défini, stocker dans le payload du code un champ programmeModeleId (l'ajouter à _genAccessCode via un paramètre optionnel ; database.rules.json : autoriser ce champ, une chaîne d'au plus 64 caractères, sous /codes/$code).
3. À l'inscription de l'athlète (_appliquerCodeApresInscription ou l'endroit où le payload du code est appliqué), si programmeModeleId est présent et que le modèle est lisible, poser ce programme dans sessions_config sans_essai, pour que _configReelle soit vrai. Sinon, ne rien faire. Si la lecture du modèle depuis le compte athlète est impossible (règles), écrire à la place un événement /evenements (type 'assigner_modele') que le Worker cloudflare/ applique ; documenter le choix dans cloudflare/README.md.
4. Écran de résultat : la liste des 20 lignes avec « Copier » par ligne, « Tout copier » (un texte prenom : lien par ligne) et « Partager » (navigator.share, un par un). Les invitations apparaissent dans la section « En attente » existante (_htmlInvitationsEnAttente).
Cas limites : hors ligne (arrêté propre et rapport partiel) ; ligne sans nom de famille ; doublon avec une invitation en attente ; quota du palier : afficher coachQuotaDepasse en avertissement sans bloquer (règle existante).
Tests : app/tests.js pour parserListeInvites (vide, doublons, 51 lignes, séparateur ; ) et pour l'application de programmeModeleId (config posée, _configReelle vrai) ; si le Worker est touché, un test dans cloudflare/test/ (npm test dans cloudflare/).
Critère de fin : avec Playwright et fetch stubbé, coller 20 lignes crée 20 invitations en attente avec 20 liens ; un athlète inscrit avec l'un d'eux voit son programme à la première ouverture, sans « Programme en cours de création ».
```

5. « Importer une fiche existante (PDF ou photo) » est promis au coach, mais l'import est désactivé — Priorité : Moyenne · Effort : S

Fusion. complémentaire du point 4 de l'agent IA (import par Claude Vision). Faire d'abord ce point (retrait de la promesse et import par collage), l'import IA ensuite.

Constat. La carte d'accueil coach PREMIERS_PAS (rc-core:25413) affiche l'étape 3 « Importer une fiche existante : PDF ou photo ». Or l'import OCR est coupé par `const LEGACY_PDF_IMPORT=false` (rc-core:36255). Toute tentative finit sur le toast « L'import de séance par PDF ou photo a été remplacé par la banque d'exercices » (`_refusImportLegacy`). Tesseract reste embarqué (`_TESS='./vendor/tesseract/'`, rc-core:112840).

Pourquoi le mettre en place. Une promesse faite dès le premier écran puis démentie détruit la confiance à l'instant où le coach évalue l'outil. Deux choix honnêtes : retirer la promesse, ou la remplacer par un import texte fiable (copier-coller d'un tableau Excel ou Google Sheets), qui couvre l'usage réel des coachs qui migrent.

Intérêt réel. Retirer la ligne prend 10 minutes et supprime un « bug perçu » dès J0. Aller plus loin avec un import par collage (« Exercice ; séries ; reps ; charge » par ligne, rapproché de la banque par `exKey`) permet à un coach de recréer ses programmes existants en quelques minutes au lieu de les ressaisir exercice par exercice. C'est le principal frein à la migration depuis Excel, et l'argument « importez vos programmes » tenu par Everfit et Trainerize.

Prompt de mise en place.

```
Contexte : app/rc-core.1693.js. PREMIERS_PAS (vers la ligne 25413) contient une 3e action « Importer une fiche existante » (PDF ou photo) qui mène à openAddAthlete, alors que LEGACY_PDF_IMPORT=false (vers 36255) ferme tout import (_refusImportLegacy).

À faire (étape 1, obligatoire) : remplacer la 3e entrée de PREMIERS_PAS par {icone:'📄', titre:'Coller un programme existant', detail:'Depuis Excel ou Google Sheets : une ligne par exercice.', action:'ouvrirImportCollage()}.
Étape 2 : écrire ouvrirImportCollage() : une modale avec un textarea et un select de jour (Lundi à Dimanche) ou « nouveau modèle ». Fonction pure
parserCollageProgramme(texte) : séparateurs acceptés tabulation, ';' et ',' ; colonnes nom, séries, reps (plage « 8-10 » acceptée), charge ou RIR facultatifs ; ligne vide = nouvelle séance ; chaque nom rapproché de la banque d'exercices par exKey (la même normalisation que l'identité stable d'un exercice). Rendre {seances:[{nom, exercices:[{name, sets, reps, _inconnu:bool}]}]}. Aperçu avant validation : les exercices non reconnus sont surlignés, avec une liste de suggestions de la banque. Validation : création d'un modèle (même structure que la création de modèle vers la ligne 33749, sessions_H et sessions_F).
Cas limites : collage depuis Sheets (tabulations), lignes d'en-tête (« Exercice », « Séries » : ignorées), 0 exercice reconnu (message clair), 200 lignes maximum.
Tests app/tests.js : mettre à jour l'assertion vers la ligne 19745 (les 3 titres de PREMIERS_PAS) ; tests de parserCollageProgramme (tabulations, ';', plage de reps, ligne vide = nouvelle séance, en-tête ignorée, exercice inconnu marqué). chargerTests() : 0 échec.
Critère de fin : aucune chaîne « PDF ou photo » visible dans l'onboarding coach tant que LEGACY_PDF_IMPORT vaut false (ajouter une assertion sur la source).
```

6. Montres et santé : les deux portes d'entrée sont vides, et rien ne les propose au démarrage — Priorité : Moyenne · Effort : S

Fusion. la réécriture de docs/TRACKER_SOURCES.md (étape 4) est déjà demandée par l'agent Suivi des progrès (point 7) : l'ignorer ici.

Constat. La synchro automatique passe par l'APK v4 (Health Connect) et par un Raccourci iPhone (`sanSyncTirer`, rc-core:114592). Mais `const RC_APK_URL=''` (rc-core:941) masque le lien APK de `s-install` (condition `if(_apk&&RC_APK_URL)`, rc-core:118030), alors que `aide-apk.html` pointe vers une release GitHub apk-4 qui existe. Et `const RACCOURCI_SANTE_URL=''` (rc-core:114638) masque « Obtenir le Raccourci » (`raccourciSanteUrl()` rend ""). **Aucun iPhone ne peut donc activer la synchro.** Par ailleurs, docs/TRACKER_SOURCES.md affirme encore « Aucune synchronisation automatique », et ni « Pour démarrer » ni `s-jours-entrainement` ne parlent de montre.

Pourquoi le mettre en place. Le sommeil, les pas et la fréquence cardiaque qui arrivent seuls sont un argument d'engagement fort les premiers jours : l'app se remplit sans effort. Ici, tout est codé mais bloqué par deux constantes vides et une doc périmée.

Intérêt réel. Débloquer les deux URL rend la fonction utilisable par 100 % des téléphones au lieu de 0 % sur iPhone et d'environ 0 % sur Android hors APK. Une tuile « Connecte ta montre (1 min) » en 4e étape facultative de « Pour démarrer » donne une petite victoire dès J1. Le coach reçoit ensuite des données de récupération sans rien demander, un argument de vente face à Trainerize, qui met en avant ses intégrations santé. La doc corrigée évite qu'un futur lot supprime la synchro en la croyant absente.

Prompt de mise en place.

```
Contexte : app/rc-core.1693.js. RC_APK_URL='' (vers la ligne 941) masque le lien APK dans s-install (vers 118030) ; RACCOURCI_SANTE_URL='' (vers 114638) masque « Obtenir le Raccourci » dans l'assistant de synchro santé. aide-apk.html pointe vers https://github.com/Repcore131/coaching/releases/download/apk-4/RepCore-4.apk. docs/TRACKER_SOURCES.md affirme « Aucune synchronisation automatique », ce qui est faux depuis sante.js (cloudflare/src/sante.js).

À faire :
1. Poser RC_APK_URL sur l'URL de la release GitHub apk-4 (la même que aide-apk.html) ; factoriser : une seule constante lue par le code et reprise dans aide-apk.html (ou un test qui vérifie que les deux sont égales).
2. Laisser RACCOURCI_SANTE_URL vide si Kevin n'a pas encore publié le lien iCloud, MAIS dans ce cas, sur iPhone, afficher dans l'assistant « Bientôt disponible sur iPhone » au lieu d'une étape sans bouton ; ajouter un TODO visible dans la carte admin. Si le lien est fourni, il doit respecter RACCOURCI_SANTE_FORME.
3. Ajouter une 4e ligne facultative à _htmlDemarrage : « Connecte ta montre (1 min) », visible seulement si sanSyncActif() est faux et que la plateforme a un chemin (Android APK >= SAN_SYNC_APK_MIN, Android hors APK -> aide-apk.html, iPhone seulement si raccourciSanteUrl()). Elle ne compte pas dans le « x sur 3 » (etapesDemarrage reste à 3 actes).
4. Réécrire la section « Ce qui n'existe pas » de docs/TRACKER_SOURCES.md : décrire Health Connect (APK >= 4) et le Raccourci iPhone, ce qui reste manuel, et les deux constantes à remplir.
Tests app/tests.js : RC_APK_URL non vide et égal au lien de aide-apk.html ; sur UA iPhone sans Raccourci, l'assistant n'affiche pas d'étape sans bouton ; _htmlDemarrage contient la ligne montre seulement quand sanSyncActif() est faux et qu'un chemin existe. chargerTests() : 0 échec.
```

7. L'activation n'est pas mesurable : compteurs globaux, sans délai ni segment — Priorité : Moyenne · Effort : M

Constat. `rcm(nom)` (rc-core:651) incrémente un compteur par jour et par événement (`/metrics/<date>/<nom>`) : `register_completed`, `first_workout_started`, `first_workout_completed`, `first_bilan_completed`... Il n'y a ni horodatage par compte, ni distinction autonome / invité / parrainé, ni événement pour la sortie de `s-install` (installé ou non) avant l'inscription, ni pour « a vu une séance jouable ». On ne peut donc calculer ni le temps jusqu'à la première valeur, ni la conversion en fin d'essai par cohorte. Les compteurs sont aussi coupés sur localhost, ce qui est voulu.

Pourquoi le mettre en place. Sans mesure du temps entre l'inscription et la première séance, par canal d'arrivée, aucun des points 1 à 4 ne peut être évalué. La rétention J1/J7/J30 existe déjà côté Worker (`cloudflare/src/retention.js`) : il suffit d'y brancher des jalons d'activation.

Intérêt réel. On obtient un tableau de bord d'activation par cohorte hebdomadaire : % inscrit → 1re séance en moins de 24 h, délai médian, % bilan de départ fait en moins de 7 jours, % converti à J30, séparés entre autonome, invité coach, ami et ambassadeur. Kevin sait alors quel canal amène des athlètes qui s'entraînent vraiment, et chaque lot d'onboarding peut être validé par un chiffre plutôt qu'à l'intuition. Coût nul, car tout reste dans le Worker gratuit et RTDB.

Prompt de mise en place.

```
Contexte : app/rc-core.1693.js a rcm(nom) (compteur journalier global) et rcmRetention(user) (vers la ligne 898) ; cloudflare/src/retention.js calcule J1/J7/J30 (tests : cloudflare/test/retention.test.mjs). Aucune date d'activation n'est gardée par compte.

À faire :
1. Dans le dossier utilisateur, ajouter u.activation = {inscrit:ts, canal:'autonome'|'coach'|'ami'|'ambassadeur', premiereSeance:ts|null, premierBilan:ts|null, premierRepas:ts|null}. Le poser dans doRegister (canal déduit de _coachInviteCode / pendingCode / parrainage / ambassadeur), puis compléter chaque champ UNE fois : à la fin de la première séance (là où rcm('first_workout_completed') est appelé), au premier bilan de départ, au premier repas (_pdRepasNote devient vrai). Ne jamais écraser une valeur déjà posée.
2. database.rules.json : autoriser users/$u/activation (nombres ou null, canal dans une liste fermée).
3. cloudflare/src/retention.js : nouvelle fonction pure activationCohortes(dossiers, maintenant) -> par semaine d'inscription et par canal : n, % première séance < 24 h, délai médian en heures, % bilan < 7 j, % payant à J30 (palier != 'aucun'). L'exposer sur la route /stats/retention existante.
4. Ajouter à RCM_EVENTS (et à la liste blanche de database.rules.json) : 'install_passe' (lien « Continuer sans installer », rcInstallPasser) et 'premiere_seance_prete' (première séance jouable affichée).
Cas limites : comptes antérieurs sans activation (exclus des cohortes, pas de rétro-calcul) ; horloge du téléphone fausse (borner premiereSeance >= inscrit) ; comptes coach exclus.
Tests : cloudflare/test/retention.test.mjs pour activationCohortes (médiane, canaux, bornes 24 h et 7 j) ; npm test dans cloudflare/ ; app/tests.js : l'activation n'est posée qu'une fois et n'est pas écrasée par un saveUser ultérieur. chargerTests() : 0 échec.
```

8. L'athlète coaché sans programme est invité à « créer sa séance » lui-même — Priorité : Basse · Effort : S

Constat. `_pdSeance(u)` (rc-core:43196) ne regarde pas `u.coachId`. Pour un invité dont le coach n'a pas encore publié, la ligne 2 de « Pour démarrer » affiche « Crée ta séance, exercice par exercice » et ouvre `s-session-manager`. Au même moment, l'accueil affiche « Programme en cours de création. Ton coach n'a pas encore publié ton programme » (rc-core:47523). Les deux messages se contredisent.

Pourquoi le mettre en place. Un invité qui compose une séance dans son coin crée des doublons et de la confusion quand le programme du coach arrive. L'action utile à ce moment-là, c'est le questionnaire de départ, qui sert justement au coach pour écrire le programme.

Intérêt réel. Le message devient cohérent : « Ton coach prépare ton programme, fais ton questionnaire pour l'aider ». Les invités font plus vite leur bilan de départ, donc le coach reçoit plus tôt de quoi programmer, ce qui raccourcit le délai d'activation de tout le portefeuille. On évite aussi les séances « sauvages » que le coach devra ensuite écraser.

Prompt de mise en place.

```
Contexte : app/rc-core.1693.js, _pdSeance(u) (vers la ligne 43196) et _htmlDemarrage (vers 43231).

À faire : dans _pdSeance, si (u.coachId||u.coachEmailKey) et que _configReelle(u.sessions_config) est faux, rendre {voie:'attente', sous:'Ton coach prépare ton programme : il apparaîtra ici'}. Dans _htmlDemarrage, une ligne séance en voie 'attente' est rendue non cliquable (même style que pd-fait mais sans ✓, libellé « En attente »), et la ligne questionnaire passe en premier avec le sous-titre « ~12 min · ton coach s'en sert pour écrire ton programme ».
pdLancerSeance ignore la voie 'attente'.
Cas limites : le coach publie pendant que l'écran est ouvert (le rendu suivant de _rendreDemarrage bascule sur 'selecteur') ; un athlète qui quitte son coach (coachId null) retrouve la voie normale.
Tests app/tests.js : _pdSeance rend 'attente' pour {coachId:'x', sessions_config:[]} et 'selecteur' quand une config réelle est posée ; _htmlDemarrage ne contient pas 'pdLancerSeance' en voie 'attente'. chargerTests() : 0 échec.
```

Agent 15 — Suivi des progrès, mesures et analyse vidéo · 7/10

Le socle statistique est nettement au-dessus de Trainerize/TrueCoach : moyenne mobile 7 j sans interpolation (`mm7`, `app/rc-core.1693.js:108690`), régression sur dates réelles avec coupure des séries après 30 j (`vitesseHebdo / vitesseHebdoEtendue`, `:108716-108800`), projection bornée avec fourchette (`ccdProjection`, `:53921`), FC de repos/VFC comparées à la base personnelle en log (`recupCardio`, `:110341`), contrôle de pose MediaPipe sur les photos de bilan (`photoContrôle`, `:70717`), avant/après aligné sur les repères, tempo mesuré par vidéo. En revanche, plusieurs données arrivent fausses ou se perdent en route : la masse grasse synchronisée est jetée, la pesée synchronisée est celle du soir, le tempo peut être inversé, et le rapport de période affiche une vitesse qui n'est pas celle de la période. Et il manque ce qui fait l'attrait de Happy Scale : un poids objectif avec une date estimée côté athlète.

1. La masse grasse des balances connectées est reçue puis jetée, et aucune saisie de % mesuré n'existe — Priorité : Haute · Effort : M

Constat. L'APK lit `BodyFatRecord` (`android/.../sante/SyncSante.kt:123`) et le Raccourci iPhone envoie `masseGrasse` (`cloudflare/src/lignes.js:87`). Le Worker les valide et les stocke (`cloudflare/src/sante.js:38` et `:119`), mais `sanFusionSync` n'a pas de case pour eux (`const plan={pas,sommeil,poids,fc,vfc...}`, `app/rc-core.1693.js:114505`) : la valeur meurt dans `sante_sync`. Seule source de % de masse grasse côté app : la formule US Navy sur les bilans (`ccdCompositionSerie` `:53874`, `calcBF` `:81364`). `athlete.bodyFatPct` est lu (`:62015`) mais n'est écrit nulle part. Pour reproduire : synchroniser une balance Withings ou Xiaomi via Health Connect, puis ouvrir Évolution > Masse grasse. Seul le chiffre US Navy apparaît, ou rien s'il manque le tour de cou.

Pourquoi le mettre en place. La formule US Navy se trompe de ± 3 à 4 points (l'app l'écrit elle-même, `:2577`) et ne bouge qu'à chaque bilan. Une balance à impédance, même imprécise en absolu, donne une tendance quotidienne. L'app dit déjà « Regarde la tendance ». Aujourd'hui, elle jette justement la donnée qui permettrait de la suivre.

Intérêt réel. Les athlètes équipés d'une balance connectée ont une courbe de masse grasse et de masse maigre continue au lieu d'un point par mois. Pendant une sèche, le coach voit en 2 à 3 semaines si la masse maigre tient, au lieu d'attendre 2 bilans. Le moteur sodium (`fatFreeMassKg`, `:62009`) passe de « estimée » à « mesurée » sans rien changer d'autre. Côté commercial, « Ta balance Withings/Xiaomi alimente ton coach » répond à Trainerize et Everfit, qui affichent tous deux la masse grasse venue des balances. Le coût est faible : le transport existe déjà de bout en bout, il ne manque que la dernière étape.

Prompt de mise en place.

```
Contexte : RepCore, dépôt /home/user/coaching. La synchro santé (cloudflare/src/sante.js, lignes.js, android/.../sante/Jours.kt) transmet déjà 'masseGrasse' (en %, bornes 3-60) dans sante_sync/<clé>/jours/<date>. Côté app, sanFusionSync (app/rc-core.1693.js, ~l.114503) l'ignore, et le seul % de masse grasse vient de calcBF (US Navy) via ccdCompositionSerie (~l.53874).
```

A faire dans `app/rc-core.1693.js` :

```
1. Nouveau journal 'u.masseGrasseLog'=[{date:'YYYY-MM-DD', pct:Number, dataStatus:'sync'|'manual', source?:string, updatedAt}]'. Ajoute 'masseGrasseLog' à CHAMPS_SANTE (~l.6055) pour qu'il suive les mêmes règles que weightLog.
2. Dans sanFusionSync : ajoute plan.masseGrasse=[] et, pour chaque jour avec j.masseGrasse entre 3 et 60, pousse {date,pct} (arrondi à 0,1), sauf si une saisie manuelle plus récente l'emporte (sanManuelGagne) ou si la valeur est déjà la même. Dans sanAppliquerSync, écris avec _sanJournalPoser (rétention 730 j, pas 180 : c'est une donnée de composition).
3. Saisie manuelle : dans l'étape Mensurations du bilan (BIL_STEPS, ~l.73658), ajoute un champ facultatif « Masse grasse mesurée (%) » (clé bil-bf-mesure) avec une liste de méthodes (balance à impédance, pince à plis, DEXA). Stocke aussi la valeur dans masseGrasseLog, avec dataStatus 'manual'.
4. Nouvelle fonction PURE pctMasseGrasseDu(u,dateISO) : renvoie {pct, methode:'mesure'|'navy'}. Priorité : une mesure à  $\pm 3$  j (DEXA > plis > impédance), sinon calcBF. ccdCompositionSerie l'utilise et ajoute 'methode' à chaque point. La courbe et les cartes affichent « mesuré (impédance) » ou « estimé (US Navy) » sans jamais relier les deux méthodes par un même trait. Dessine deux séries distinctes, pour que le passage d'une méthode à l'autre ne crée pas un faux saut.
5. Renseigne bodyFatPct (lu par fatFreeMassKg ~l.62009) à partir de la dernière mesure de moins de 30 jours.
Cas limites : pct hors bornes rejeté ; un jour avec une mesure manuelle ET une synchro, la manuelle récente gagne ; TCA/masquerPoids : même masquage que le poids.
Tests (app/tests.js, format ok('...',( ()=>{...})) : fusion d'une masseGrasse synchronisée ; manuel récent conservé ; ccdCompositionSerie qui préfère la mesure à Navy et marque la méthode ; séries non mélangées. Lance la suite via scripts/verif/suite.mjs (voir scripts/verif/README.md) et cd cloudflare && npm test. Critère de fin : une masse grasse synchronisée apparaît dans Évolution > Masse grasse et sur la fiche coach avec le libellé de sa méthode, et 0 test en échec.
```

2. La pesée synchronisée retenue est la dernière de la journée, pas celle du matin — Priorité : Haute · Effort : S

Constat. Android garde la mesure la plus tardive du jour (`dernier(poids) → maxByOrNull { it.instant }`, `android/.../sante/Jours.kt:161-168`). iPhone fait de même (`dernier(types.poids...)`, `cloudflare/src/lignes.js:85`, commentaire « dernière valeur du jour » l.12). Pour reproduire : une pesée à 7 h (80,0 kg) et une autre à 21 h (81,4 kg) le même jour. `weightLog` reçoit 81,4, et `mm7`, `vitesseHebdo` et l'ajustement calorique partent de ce chiffre.

Pourquoi le mettre en place. Toute la méthode de l'app (`mm7`, régimes précis/étendu, alertes de phase) suppose des pesées faites dans les mêmes conditions. L'écart entre le matin et le soir (1 à 2 kg) dépasse le bruit que l'app retient pour le poids (`SYN_BRUIT_POIDS=0.3`, `:80858`). Une seule pesée du soir suffit à fausser une semaine de tendance.

Intérêt réel. La vitesse hebdomadaire pilote un ajustement calorique (`AJUST_PAS_ETENDU`, commentaire `:31103`). Une pesée du soir au mauvais moment peut déclencher une baisse de calories injustifiée ou masquer une stagnation. Les balances connectées et les montres qui notent aussi le poids produisent souvent plusieurs mesures par jour. Le correctif fait 2 lignes par plateforme, plus un test, et fiabilise tout ce qui s'appuie sur le poids pour les athlètes synchronisés.

Prompt de mise en place.

```
Dépôt /home/user/coaching. Pour le POIDS (et la masse grasse), la synchro doit retenir la PREMIÈRE mesure du jour, prise entre 4 h et 12 h (heure locale/Paris). S'il n'y a aucune mesure du matin, elle retient la première du jour. La FC de repos reste « dernière valeur » et la VFC reste une moyenne.
1. android/app/src/main/java/com/repcore/app/sante/Jours.kt : ajoute une fonction 'premierDuMatin(l, poser)'. Elle groupe par date locale et prend la mesure la plus ancienne dont l'heure est entre 4 et 12, sinon la plus ancienne du jour. Utilise-la pour poids et gras.
2. cloudflare/src/lignes.js : même logique pour types.poids et types.masseGrasse (heure via paris(t).heure). Mets à jour le commentaire d'en-tête (l.12-13).
3. Ajoute un champ facultatif 'poidsHeure:'HH:MM'' au jour transmis (nettoyerJour dans cloudflare/src/sante.js le valide avec la regex HHMM). Côté app (sanFusionSync, app/rc-core.1693.js ~l.114548), stocke-le dans l'entrée weightLog pour que le coach voie l'heure.
Tests : cloudflare/test/sante.test.mjs, avec 3 lignes poids le même jour (07:10 80,0 ; 13:00 80,9 ; 21:00 81,4) → 80,0. Sans pesée le matin (14:00 et 20:00) → la valeur de 14:00. Ajoute aussi un test unitaire Kotlin si le module a déjà des tests, sinon un commentaire de cas. Lance cd cloudflare && npm test. Critère de fin : un jour à plusieurs pesées remonte la pesée du matin sur les deux plateformes, et tous les tests passent.
```

3. Motion Lab : le tempo mesuré s'inverse quand la phase concentrique ne va pas vers le haut — Priorité : Haute · Effort : M

Constat. `mTempo` compte comme concentrique toute vitesse positive (`app/motion-lab.js:1558`). Pour un mouvement vertical, cette vitesse est la verticale vers le haut (`mSerieRelue`, `:1784`, et `mMetricsBarre`, `:646`). Sinon, c'est l'axe principal orienté « vers le point le plus éloigné du départ » (`mLaxePrincipal`, `:1472`). Rien ne relie ce sens à l'exercice. Pour reproduire : tirage vertical ou extension triceps à la poulie haute avec le suivi posé sur la poignée (verticalité $\geq$

0,75). La descente, qui est la phase concentrique, est comptée comme excentrique, et `mlTempoComparaison` (:1612) écrit « Prescrit : 3 s de descente. Mesuré : 1,0 s » en mesurant la mauvaise phase. Même inversion pour une presse à cuisses découpée depuis le verrouillage (mode « chemin », le point le plus éloigné est le bas).

Pourquoi le mettre en place. Le lot T3 vend « le tempo prescrit, et vérifié par la vidéo ». Un chiffre de tempo faux affiché avec aplomb décrédibilise tout l'outil, et le coach n'a aucun moyen de le voir.

Intérêt réel. Les tirages verticaux, les extensions à la poulie haute et la presse à cuisses font partie des exercices les plus filmés en salle. Si le coach corrige un athlète sur un excentrique « trop rapide » qui était en fait sa phase concentrique, il perd la confiance de l'athlète. Aucun concurrent grand public (Trainerize, TrueCoach, Hevy Coach) ne mesure le tempo : c'est un vrai atout, à condition qu'il soit juste. Il faut relier le sens de la phase concentrique à l'exercice (déjà connu par le lien vers la série) et laisser le coach l'inverser d'une touche. On garde ainsi l'avantage sans risquer de fausse mesure.

Prompt de mise en place.

```
Dépôt /home/user/coaching, fichier app/motion-lab.js (chargé à la demande, types JSDoc tsc --checkJs --strict).
Problème : mlTempo(serie) suppose v>0 = concentrique. v = vitesse verticale (vers le haut) si verticalite=ML_VERTICALITE, sinon projection sur
mlAxePrincipal orienté vers le point le plus éloigné du départ. Pour une poulie haute suivie à la poignée (concentrique vers le bas) ou une presse à cuisses
découpée depuis le verrouillage, excentrique et concentrique sont inversés.
À faire :
1. Ajoute `sensConc: 1|-1` au segment analysé (seg.barre.sc), stocké par mlCompacterBarre et relu par mlDecompacterBarre. Absent = 1 (les analyses existantes
ne bougent pas).
2. Valeur par défaut déduite de l'exercice lié (lien de la vidéo vers la série, voir _mlTempoPrescrit) : une table PURE mlSensConcentriqueDefault(nomExercice)
→ -1 pour les motifs /tirage vertical|lat pull|pulldown|poulie haute|pushdown|extension.*poulie|crunch.*poulie/i, 1 sinon. En mode « chemin », le sens se lit
sur le PREMIER mouvement du segment : si la première phase en mouvement s'éloigne du départ et que l'exercice commence en position courte (presse, hack,
squat machine), la phase qui s'éloigne est excentrique.
3. Multiplie v par sensConc avant mlTempo, dans mlMetriquesBarre ET dans mlSerieRelue/mlLireRepetition. Adapte les libellés « descente/remontée » de
mlTempoComparaison : utilise « excentrique/concentrique » quand sensConc=-1.
4. UI : dans le panneau tempo, un bouton « Inverser les phases » qui bascule sensConc, recalcule et enregistre.
Tests dans app/tests.js (des tests mlTempo existent, cherche « mlTempo») : série synthétique verticale qui descend 1 s puis remonte 3 s avec sensConc=-1 →
conMs=1000, excMs=3000 ; même série avec sensConc=1 → l'inverse ; analyse ancienne sans sc → résultat inchangé. Lance tsc --checkJs sur motion-lab.js si la
config tsconfig.json le permet, puis la suite (scripts/verif/suite.mjs).
Critère de fin : un tirage vertical filmé donne un excentrique égal à la remontée de la poignée, le coach peut inverser manuellement, et aucune analyse
existante ne change.
```

4. Courbe de poids : l'écart affiché est brut, et l'athlète n'a ni objectif ni date estimée — Priorité : Haute · Effort : M

Constat. La bulle de la courbe athlète/coach affiche `der.kg - pts[0].kg`, soit pesée brute moins pesée brute (`app/rc-core.1693.js:112227`), alors que la tendance `mm7` est tracée juste à côté. Avec ± 1 kg de variation d'eau, ce chiffre peut dire « +0,8 kg » sur une sèche qui avance. La tendance de début de fenêtre est calculée sur `segmentsWeight(pts)` déjà coupé à la période (:112160), donc elle démarre tard et biaisée. Il n'existe pas de poids objectif en dehors des échéances de compétition (`poidsCible`, :67274), et la projection avec date n'existe que côté coach (`ccdProjection`, :53921). Bug d'affichage en plus : pour une pente nulle, (`p.kgSem>0?'+'':'-'`) écrit « -0 kg par semaine » (:53992).

Pourquoi le mettre en place. Le chiffre le plus visible de l'écran doit être le plus fiable. Happy Scale et Libra tiennent tout entiers sur « tendance lissée + objectif + date estimée ». C'est la première raison de rouvrir l'app chaque matin.

Intérêt réel. Un athlète en sèche qui voit « +0,6 kg » un lundi alors que sa tendance baisse de 0,4 kg/sem, c'est un message d'inquiétude au coach, voire un abandon. Afficher l'écart de la tendance supprime une grande partie de ces faux signaux (l'écart brut subit tout le bruit de ± 1 kg). Un objectif intermédiaire (« -3 kg ») avec une date estimée et sa fourchette réutilise `ccdProjection`, déjà testée. On gagne un rituel de pesée quotidienne, donc des données plus denses, donc le régime « précis » plus souvent atteint. C'est aussi un argument face à Trainerize, dont la courbe ne montre que les points bruts.

Prompt de mise en place.

```
Dépôt /home/user/coaching, app/rc-core.1693.js.
1. _courbePesee (~L.112140) : calcule mm7 sur la série COMPLÈTE (paramètre `serieComplete`, par défaut serie), puis filtre les points de tendance sur la
fenêtre. Les premiers jours de la période ont alors une vraie moyenne. La bulle affiche l'écart de TENDANCE (mm7 au dernier jour moins mm7 au premier jour de
la fenêtre) quand les deux existent, avec le libellé « tendance », et l'écart brut seulement en repli (libellé « pesées »). Garde la couleur par phase
(o.couleur).
2. _htmlCcdProjection (~L.53978) : si |kgSem|<0.05, écris « stable » au lieu de « -0 kg par semaine ».
3. Objectif de poids athlète : champ `u.objectifPoids={kg, fixeLe, par:'athlete'|'coach'}`. Réutilise refusObjectifPerte(u) pour refuser un objectif de perte
(TCA, grossesse, SCOFF) et masque tout si aTCA/masquerPoids, comme renderCartePesee. Sur la courbe, trace une ligne horizontale en tirets à ce poids. Sous la
courbe, une phrase calculée à partir de ccdProjection/vitesseHebdoEtendue : « Au rythme actuel : objectif vers le 12 novembre (fourchette 3 nov. - 24 nov.) ».
Rien si la projection manque ou si la pente va dans le mauvais sens (« La tendance ne va pas vers l'objectif pour l'instant »). Jamais de date à plus de
26 semaines.
4. Le coach peut poser/modifier l'objectif depuis la fiche (même patron que coachSetPhase, avec la carte users passée à getOwnedClient).
Tests (app/tests.js) : bulle = écart de tendance sur une série synthétique bruitée ( $\pm 0,8$  kg) décroissante de 0,5 kg/sem ; tendance présente dès le 1er jour
de la fenêtre si des pesées précèdent ; « stable » pour une pente nulle ; objectif refusé sous TCA ; date estimée absente si la pente est opposée à
l'objectif. Lance la suite complète (scripts/verif/suite.mjs).
Critère de fin : la bulle ne change plus de signe à cause d'une seule pesée, l'athlète voit son objectif et sa date estimée, et 0 échec.
```

5. Le rapport de période imprime une vitesse de poids qui n'est pas celle de la période — Priorité : Moyenne · Effort : S

Constat. `rapPoids` calcule `vitesseHebdo(serie)` sur la série entière (`app/rc-core.1693.js:63376`), donc sur les 14 jours qui précèdent la DERNIÈRE pesée, quelle que soit la période choisie. Le rendu (:63759) l'affiche sous la courbe de la période sans l'avertissement « constaté le... » que porte le bloc Tendances (:63742). « Écart » est aussi calculé de pesée brute à pesée brute (`delta`, :63385). Pour reproduire : imprimer le rapport de mars d'un athlète qui a pris 0,4 kg/sem en septembre. Le PDF de mars affiche « +0,40 kg/sem ».

Pourquoi le mettre en place. Le rapport de période est LE document qu'un coach remet à un client ou garde pour la fin du cycle. Un chiffre faux dans un document daté est pire qu'un chiffre absent.

Intérêt réel. Les coaches qui facturent un suivi de 99 à 600 € s'appuient sur ce PDF pour justifier les ajustements du cycle. Montrer la vitesse réelle de mars, et l'écart de tendance début-fin, rend le document juste et défendable. Le correctif est petit : `vitesseHebdoEtendue(serie, jourISO)` accepte déjà un jour de fin, et `vitesseHebdo` n'a besoin que d'un paramètre de fin.

Prompt de mise en place.

```
Dépôt /home/user/coaching, app/rc-core.1693.js.
1. vitesseHebdo(serie, jourISO?) (~L.108716) : paramètre facultatif de fin. Tronque la série à jourISO avant segmentsWeight. Sans paramètre, le comportement
actuel ne change pas (tous les appelants existants).
2. rapPoids (~L.63364) : fin = dernière pesée dans la période. v = vitesseHebdo(serie, fin) || vitesseHebdoEtendue(serie, fin). Ajoute `regime` au rendu («
mesure précise » / « mesure étendue »). delta : écart de mm7 (mm7(serie, premier) → mm7(serie, dernier)) quand les deux existent, sinon écart brut, avec un
```

```
libellé qui dit lequel.  
3. Rendu (~1.63759) : « Vitesse sur les deux dernières semaines de la période : ... ». Si v est null : « Pas assez de pesées en fin de période pour une vitesse ».  
Tests (app/tests.js) : série avec -0,5 kg/sem en mars et +0,4 kg/sem en septembre. Le rapport de mars rend kgSem=-0,5. Sans pesées suffisantes en fin de période, pas de vitesse. Les appels existants de vitesseHebdo sans 2e argument sont inchangés (assertion sur un cas déjà testé). Suite :  
scripts/verif/suite.mjs.  
Critère de fin : chaque chiffre du bloc Poids du rapport porte sur la période imprimée.
```

6. Motion Lab : les répétitions se découpent entièrement à la main — Priorité : Moyenne · Effort : M

Constat. Le module se décrit ainsi : « Le coach isole le mouvement utile... en une ou plusieurs répétitions » (app/motion-lab.js:4). Les segments naissent de `mlNouveauSegment` (:146) et de bornes posées à la main (`mlBorne`, :165). Aucune fonction ne découpe automatiquement (recherche « auto/detect/decoup » : seulement `mlDisqueDetecter`, `mlAplombAuto`, `mlTrajectoireAuto`). Pourtant, la trajectoire suivie et sa vitesse signée existent déjà (`mlVitesses`, :1521). La perte de vitesse d'une répétition à l'autre (`mlPertesSerie`, :2087) n'apparaît donc que si le coach a découpé chaque répétition lui-même.

Pourquoi le mettre en place. Sur une série de 10 répétitions, découper à la main prend plusieurs minutes par vidéo. C'est le principal frein à l'adoption d'un outil déjà très riche (tempo, perte de vitesse, couple).

Intérêt réel. Sur un temps de retour vidéo typique, le coach passe d'environ 5 min de découpe à une validation en 10 s. Le comptage automatique vérifie aussi les répétitions saisies par l'athlète, ce qui donne un signal d'observance que personne d'autre n'offre. La perte de vitesse sur toute la série devient disponible sans effort, ce qui correspond à un vrai usage VBT (arrêter la série à 20-30 % de perte). Les outils spécialisés (Metric VBT, Qwik VBT) facturent cette fonction à part : l'intégrer au coaching est un argument de vente pour l'offre coach à 39 €.

Prompt de mise en place.

```
Dépôt /home/user/coaching, app/motion-lab.js (JSDoc strict, fonctions PURES testées dans app/tests.js).  
Ajoute mlDecouperRepetitions(serie, options) PURE : entrée {t:number[], v:number[]} (vitesse signée déjà orientée concentrique>0, voir mlSerieRelue) et {pasMs, seuil=ML_TEMPO_ARRET, amplitudeMinM=0.08, dureeMinMs=600}. Elle trouve les cycles excentrique-concentrique grâce aux changements de signe hors bande morte, fusionne les micro-cycles sous amplitudeMinM (déplacement cumulé sur l'axe) ou sous dureeMinMs, et renvoie [{debutMs, finMs, amplitudeM, vPicCon, confiance}]. Chaque borne tombe dans l'immobilité qui entoure le cycle, pour que mlTempo rende complet:true.  
Intégration : après une analyse de trajectoire sur un segment « série entière », un bouton « Découper en répétitions (N détectées) » remplace le segment par N segments nommés via mlProchainLibelle. Le coach garde la main : aperçu des bornes sur la frise avant de valider, annulable. Relance ensuite mlTableauSerie et mlPertesSerie.  
Cas limites : série tronquée au début ou à la fin (répétition partielle écartée et signalée), pause longue au milieu (rest-pause, deux groupes), confiance de suivi < ML_CONF_DOUTE sur plus de 30 % d'un cycle (cycle marqué douteux, pas de tempo).  
Tests : série synthétique de 5 sinusoïdes avec du bruit → 5 répétitions, bornes à ±1 image ; répétition partielle en fin → 4 + 1 signalée ; micro-oscillations sous 8 cm → ignorées. Suite via scripts/verif/suite.mjs.  
Critère de fin : une série de 8 répétitions analysée donne 8 segments proposés en un geste, avec le tableau de perte de vitesse rempli.
```

7. Capture de stats (OCR) : elle écrase la synchro et la saisie manuelle sans relecture, et la doc dit l'inverse de ce que fait le code — Priorité : Moyenne · Effort : S

Constat. `_appliquerCaptureStats` (app/rc-core.1693.js:113041) écrit directement, sans écran de validation, par-dessus n'importe quelle valeur existante : synchronisée (Health Connect) ou manuelle. Il ne fait que compter `remplaces` pour le toast. Pour les pas, la valeur retenue est le plus grand entier entre 100 et 99 999 de la ligne (`Math.max(...cands)`, :113027) : une ligne de capture qui porte aussi des calories ou des mètres (« 8 432 pas · 6 120 m ») peut donner le mauvais chiffre. Enfin, docs/TRACKER_SOURCES.md:65 affirme « Aucune synchronisation automatique... chaque entrée est `dataStatus:'manual'` », alors que `sante.js` et l'APK synchronisent pas, sommeil, FC, VFC, poids et masse grasse.

Pourquoi le mettre en place. La synchro est la source la plus fiable. Une capture approximative ne devrait jamais la remplacer, et un chiffre mal lu écrit sans relecture fausse les moyennes que le coach consulte (`bilanDomaineCoach`, :110463).

Intérêt réel. La règle de priorité devient la même pour tous (synchro > manuel récent > capture), ce qui évite les « mes pas ont changé tout seuls ». Une relecture en un écran (7 lignes pré-remplies, un bouton Valider) coûte 3 secondes et supprime les erreurs d'OCR silencieuses. Une doc à jour évite qu'un développeur ou un agent reconstruise ce qui existe déjà, ou pire, promette aux athlètes l'absence de synchro.

Prompt de mise en place.

```
Dépôt /home/user/coaching.  
1. app/rc-core.1693.js, _appliquerCaptureStats (~1.113041) : ne remplace JAMAIS une entrée dataStatus 'sync', ni une entrée 'manual' dont updatedAt date de moins de 24 h. Compte-les dans 'gardes', affiché dans le toast (« 2 jours déjà synchronisés conservés »).  
2. Avant l'écriture, affiche une feuille de relecture (rcConfirm ou une feuille existante) : une ligne par jour détecté, avec un champ modifiable pré-rempli, la valeur actuelle grisée si elle existe, et un bouton « Enregistrer ». On en sort avec Annuler sans rien écrire. Cette relecture revient sur une décision de la spec docs/superpowers/specs/2026-08-06-import-capture-stats-design.md : ajoute dans cette spec une section « Révision 09/2026 » qui explique pourquoi (conflit avec la synchro).  
3. Extraction des pas (~1.113026) : exclut les entiers suivis d'une unité (m, km, kcal, cal, min, bpm, %) et préfère celui qui est suivi de « pas »/« steps ».  
4. docs/TRACKER_SOURCES.md : remplace la section « Ce qui n'existe pas » par l'état réel. Synchro Android (Health Connect) : pas, sommeil+phases, FC de repos, VFC RMSSD, poids, masse grasse) et iPhone (Raccourci, SDNN), format rc-sante-1, jeton, rétention 30 j côté Worker et 180 j côté app, priorité synchro/manuel/capture.  
Tests (app/tests.js) : capture sur un jour 'sync' → conservé ; manuel de moins de 24 h → conservé ; capture « 8 432 pas 6 120 m 320 kcal » → 8432. Suite : scripts/verif/suite.mjs.  
Critère de fin : aucune capture ne remplace une donnée synchronisée, la relecture précède toute écriture, et la doc décrit le code réel.
```

8. Un historique tronqué : pesées limitées à 2 ans, pas/sommeil/FC/VFC à 6 mois, records réduits à la charge max — Priorité : Basse · Effort : S

Fusion. le volet « records par tranche de répétitions » est porté par l'agent Préparateur physique (point 4). Ne réaliser ici que le compactage de l'historique (étapes 1 et 2 du prompt).

Constat. `_recordWeight` supprime toute pesée de plus de 730 jours à chaque nouvelle saisie (`PESEE_RETENTION_JOURS=730`, app/rc-core.1693.js:108617 et :108654). Les pas et le sommeil sont purgés à 180 j (`STEPS_RETENTION_JOURS`, :108441), FC et VFC aussi (`SAN_SYNC_RETENTION_JOURS`, :114460). Le tableau Records n'affiche qu'une ligne par exercice, la « meilleure charge » (`_htmlRecords`, :81890, et `recordsExercice`), sans record par nombre de répétitions ni record de volume par série.

Pourquoi le mettre en place. Un client fidèle depuis 3 ans perd la courbe de sa première année (seuls les bilans survivent). On ne peut pas comparer son sommeil ou sa FC de repos d'un hiver à l'autre. Et « ton meilleur 8 reps » compte plus pour lui que sa charge max absolue.

Intérêt réel. Garder les clients longtemps repose sur la preuve du chemin parcouru : « il y a 3 ans tu pesais 94 kg » est un argument de renouvellement que l'app efface elle-même. Le poids des données reste minime : une pesée quotidienne fait environ 30 octets, soit environ 11 Ko par an. On peut agréger en

moyennes hebdomadaires au-delà de 2 ans plutôt que supprimer. Des records par tranche de répétitions (1, 3, 5, 8, 10, 12) alimentent aussi les cartes de partage déjà en place (`_dessinerCarteRecord`), ce que Hevy met en avant.

Prompt de mise en place.

```
Dépôt /home/user/coaching, app/rc-core.1693.js.
1. Poids : au lieu de supprimer au-delà de PESEE_RETENTION_JOURS dans _recordWeight (~1.108645), compacte. Les pesées de plus de 730 j sont remplacées par
UNE entrée par semaine ISO {date: lundi, kg: moyenne, n, agrege:true}. serieWeight garde ces entrées pour l'affichage. mm7, vitesseHebdo et les segments les
ignorent (filtre !e.agrege), car la moyenne d'une semaine n'est pas une pesée. La courbe « 1 an » et une nouvelle période « Tout » (PESEE_PERIODES) les
affichent.
2. Pas, sommeil, FC, VFC : même compactage hebdomadaire au-delà de 180 j (moyenne, n) dans des champs `stepsHebdo`, `sleepHebdo`, `fcReposHebdo`, `vfcHebdo`
(VFC : une moyenne par méthode). Ajoute-les à CHAMPS_SANTE. Le tableau de bord coach gagne une période « 1 an » qui les lit.
3. Records : recordsExercice rend aussi `parReps: {1,3,5,8,10,12: {kg, date}}` (meilleure charge à au moins N répétitions validées, done===true, non
aberrantes) et `meilleurVolumeSerie {kgxreps, date}`. _htmlRecords affiche une ligne dépliant par exercice avec ces tranches.
Cas limites : entrée agrégée jamais modifiable par la saisie du jour ; idempotence du compactage (relancer ne change rien) ; migration à la première
ouverture sans perte.
Tests (app/tests.js) : 3 ans de pesées synthétiques → 2 ans bruts + 52 semaines agrégées, mm7 identique sur les 2 dernières années ; compactage relancé =
même résultat ; parReps correct sur un jeu de séances. Suite : scripts/verif/suite.mjs.
Critère de fin : plus aucune donnée de progression n'est supprimée, seulement agrégée, et le tableau Records montre les records par tranche de répétitions.
```

Agent 16 — IA et automatisisation du coaching · 4/10

Le socle déterministe est remarquable : `signauxEntraînement` (`app/rc-core.1693.js:12787`) calcule onze signaux, `brouillonBilan` (73277) pré-écrit la réponse au bilan avec des garde-fous explicites (aucun conseil, pas de doubleur nommée, pas de chiffre pour un profil TCA), les relances serveur sont pures et journalisées (`cloudflare/src/relances.js:114`), et la propagation de modèle (Lot C4, `diffModele` 35202) est propre. Le Worker sait déjà authentifier un appel de l'app (`repondreAppel`, `cloudflare/src/appels.js`). En revanche il n'y a **aucun appel à un LLM** dans tout le dépôt : `analyzePhotoWithClaude` (116287) porte ce nom mais ne fait que du Tesseract, derrière un import désactivé (`LEGACY_PDF_IMPORT=false`, 36255). Tout le texte libre que l'athlète écrit (détail des difficultés, raisons des écarts, modifications demandées, notes de séance) n'est lu par aucune automatisisation. Face à Trainerize, Everfit et Kahunas, qui vendent tous désormais un « AI assistant », RepCore est en retard sur ce point précis, alors que ses données sont plus riches que les leurs.

Hypothèses de coût, utilisées partout : Claude Sonnet 5.5 (`claude-sonnet-5-5`) à 2 \$ / 10 \$ par million de tokens (entrée / sortie), lecture de cache à 0,20 \$; Claude Haiku 4.5 (`claude-haiku-4-5`, alias de `claude-haiku-4-5-20251001`) à 1 \$ / 5 \$; Batch API à -50 % ; 1 \$ ≈ 0,90 €.

1. Le socle : un proxy IA sur le Worker, avec quota, journal et validation du coach — Priorité : Critique · Effort : M

Constat. Aucune clé ni aucun appel IA n'existe (`grep anthropic|claude` : seuls des commentaires et le nom trompeur `analyzePhotoWithClaude`, `app/rc-core.1693.js:116287`). En revanche, tout le chemin est déjà là. `CLOUD._callFn` (3662) poste vers `SERVEUR_LEGER_URL+'fn/<nom>'` avec le jeton Firebase. `repondreAppel` (`cloudflare/src/appels.js`) vérifie ce jeton en RS256 et en tire l'e-mail. `APPELS` (`cloudflare/src/index.js`) ne contient aujourd'hui que `cloudinaryDestroy` et `santeJeton`. Enfin, `cloudflare/src/medias.js:80` contrôle déjà le lien coach-athlète dans les deux sens (`users/<athlète>/coachEmailKey` et `coachs/<coach>/clients/<athlète>`).

Pourquoi le mettre en place. Toutes les fonctions IA des points 2 à 8 passent par le même tuyau. La clé API ne doit jamais atteindre le navigateur, et chaque appel doit être compté, borné par offre et tracé. Construire ce socle une seule fois évite sept intégrations bricolées. Il permet aussi de couper toute l'IA d'un seul interrupteur si la facture dérape.

Intérêt réel. Le coût d'infrastructure est nul : le Worker gratuit suffit, car l'attente de la réponse de l'API ne compte pas dans les 10 ms de CPU, et un appel ne consomme qu'une sous-requête sur 50. Le quota mensuel par compte (par exemple 0 € en offre « libre », 3 € en « coach », 10 € en « pro ») rend la marge prévisible, même pour un coach à 30 athlètes. Le journal `ia_journal` mesure pour chaque brouillon la distance entre le texte proposé et le texte envoyé : c'est la seule preuve honnête que l'IA fait gagner du temps, et c'est ce qui guidera les réglages de prompt. Le garde-fou « rien ne part sans le coach » devient une propriété du socle : le proxy ne renvoie que des propositions et n'écrit jamais dans un dossier athlète. Le repli serveur (`fallbacks: "default"`) évite que les classifieurs de Sonnet 5.5 bloquent un brouillon légitime qui parle de santé.

Prompt de mise en place.

Dans ce dépôt (RepCore), crée le socle IA côté serveur léger, sans rien brancher encore dans l'app.

Contexte : `cloudflare/src/appels.js` expose `repondreAppel` (protocole `onCall`, jeton Firebase vérifié → `{auth:{email,uid}, data}`). `cloudflare/src/index.js` déclare `const APPELS = { cloudinaryDestroy, santeJeton }` et construit les outils avec `fetchCompte` (compteur de sous-requêtes). Le contrôle d'appartenance coach-athlète à copier est dans `cloudflare/src/medias.js` (vers la ligne 80 : `users/<athlète>/coachEmailKey === clé du coach ET coachs/<coach>/clients/<athlète> existe`). Le projet n'a aucune dépendance npm (`cloudflare/package.json`).

À faire :

- Ajoute la dépendance `@anthropic-ai/sdk` dans `cloudflare/package.json`. Crée `cloudflare/src/ia.js` qui exporte `creerIA({env, db, fetchImpl, maintenant})`. Le client est `new Anthropic({apiKey: env.ANTHROPIC_API_KEY, fetch: fetchImpl})` : le `fetch` injecté garde le compteur de sous-requêtes et permet les faux dans les tests. N'écris jamais la clé ailleurs que dans le secret.
 - Écris une fonction `PURE routeIA(tache) → {model, max_tokens, effort}`. Tâches : `'bilan'`, `'hebdo'`, `'import'`, `'programme'`, `'relance'` → `'claude-sonnet-5-5'` ; `'repas'`, `'relance_courte'` → `'claude-haiku-4-5'`. Sur Sonnet 5.5 : `thinking` adaptatif par défaut, `output_config.effort` `'low'` (bilan, hebdo, relance) ou `'medium'` (import, programme), plus betas [`'server-side-fallback-2026-07-01'`] et `fallbacks:'default'`. Sur Haiku 4.5 : ni effort ni `thinking`. Réponses structurées obligatoires via `output_config.format` (JSON schema), jamais de préfill. Vérifie toujours `stop_reason` : `'refusal'` ou `'max_tokens'` donnent `{ok:false, raison}`, jamais un texte partiel.
 - Ajoute un gestionnaire `APPELS.ia({auth, data}, ctx)` dans `index.js` qui délègue à `ia.js` :
 - `data = {tache, athlète?(clé), charge}`. Si `athlète` est fourni, contrôle d'appartenance identique à `medias.js` ; sinon 403 `ErreurAppel`.
 - Interrupteur global : si `env.IA_COUPEE === '1'` ou si la clé manque, renvoie 503 « L'assistant est en pause. »
 - Quota : `lis ia_quota/<compte>/<AAAA-MM>` (micro-dollars consommés). Plafond selon l'offre lue dans le dossier (offre coach libre 0, coach 3 000 000 µs, pro 10 000 000 µs ; athlète `Ultime` 1 000 000 µs ; mets ces valeurs dans une constante `IA_PLAFONDS`). Au-delà : 429 « Quota IA du mois atteint. »
 - Après l'appel : `coût = usage.input_tokens×prixEntrée + cache_read×prixCache + output_tokens×prixSortie` (table `PRIX` par modèle, en \$/Mtok : sonnet 2/10/0,2, haiku 1/5/0,1). Incrémente `ia_quota` par une transaction et écris `ia_journal/<compte>/<id> = {t, tache, modele, tin, tout, cout, athlète, statut:'proposé'}`. Purge au-delà de 90 jours dans le travail nocturne existant (même patron que `journalAPurger` dans `relances.js`).
 - Le gestionnaire ne renvoie QUE `{ok, proposition, journalId, coutMois, plafond}`. Il n'écrit jamais dans `users/<athlète>`.
 - Ajoute `APPELS.iaRetour({data:{journalId, statut:'valide'|'modifie'|'rejeté', distance})` : il met à jour le statut et la distance (0..1) entre la proposition et le texte finalement envoyé.
 - `database.rules.json` : `ia_quota` et `ia_journal` en lecture pour le propriétaire (`auth.token.email` normalisé comme ailleurs), écriture refusée au client (seul le Worker écrit, avec le compte de service).
 - Documentation de mise en service : ajoute `ANTHROPIC_API_KEY` et `IA_COUPEE` dans les commentaires `SECRETS` de `cloudflare/src/index.js` et de `wrangler.toml`, dans `cloudflare/secrets.ps1` et dans `cloudflare/README.md`.
 - Tests : `cloudflare/test/ia.test.mjs` avec la fausse base (`test/fausse-base.mjs`) et un faux `fetch` qui imite `/v1/messages`. Cas à couvrir : `routeIA` pour chaque tâche ; appel refusé sans jeton (401) ; athlète d'un autre coach (403) ; interrupteur (503) ; quota dépassé (429) ; `stop_reason` `'refusal'` → `ok:false` sans journal `'proposé'` ; calcul de coût exact sur un usage connu ; `iaRetour` met à jour la distance ; aucune écriture sous `users/`.
- Critère de fin : `cd cloudflare && npm test` passe entièrement (anciens tests compris), et `node --check` sur chaque fichier modifié.

2. Le brouillon de bilan lit enfin ce que l'athlète a écrit — Priorité : Haute · Effort : M

Constat. `brouillonBilan` (`app/rc-core.1693.js:73277-73366`) ne lit que le poids (`getBW`, `mm7`), le nombre de séances, le volume et au plus deux signaux. Il n'ouvre **aucun** champ libre du bilan déclaré dans `BILAN_QUESTIONS.suivi` (71285) : `bil-diff-detail`, `bil-cheat-reasons`, `bil-prog-modifs`, `bil-stress-detail`, `bil-new-goals-detail`. Il ne lit pas non plus les notes de séance (`sess.notes`, 44790), ni les réponses précédentes du coach (`reponseCoach`). La question finale est tirée d'une table fixe (`_brSignal`, 73230).

Pourquoi le mettre en place. Le bilan est le moment où l'athlète dit ce qui le bloque, et c'est justement ce que le brouillon ignore. Un athlète qui écrit « je n'arrive pas à caser la 4e séance, et les fentes me tirent sur le genou » reçoit aujourd'hui « Tu t'entraînes moins souvent que les semaines d'avant ». Un LLM qui reçoit les faits calculés par `brouillonBilan`, le texte libre et les deux dernières réponses du coach peut écrire une réponse qui montre que le coach a lu, avec le ton du coach.

Intérêt réel. Répondre à un bilan prend aujourd'hui 8 à 15 minutes à un coach. Avec un brouillon qui reprend les mots de l'athlète, on vise 3 à 5 minutes. Pour 30 athlètes à un bilan par semaine, cela fait environ 5 heures gagnées par mois. Le sentiment d'être lu est le premier levier de rétention d'un coaching à 150-600 € : l'athlète qui retrouve ses propres mots dans la réponse reste. Coût : environ 8 000 tokens en entrée et 1 200 en sortie sur Sonnet 5.5, soit environ 0,03 \$ par brouillon, donc environ **0,11 € par athlète et par mois** avec un bilan hebdomadaire. Les garde-fous existants restent en place : les faits chiffrés viennent toujours du calcul déterministe (le LLM ne calcule rien), le profil TCA garde le brouillon déterministe, et l'envoi reste le bouton « Envoyer ma réponse » (`saveReponseBilan`, 73073).

Prompt de mise en place.

```
Prérequis : le socle IA existe (APPELS.ia dans cloudflare/src/ia.js, tâche 'bilan'). Sinon, arrête-toi et dis-le.

Objectif : enrichir le brouillon C2 par Claude sans perdre ses garde-fous. Fichiers : app/rc-core.1693.js (brouillonBilan vers 73277, _brouillonPourChamp vers 73375, rendu du champ vers 73460, saveReponseBilan vers 73073), cloudflare/src/ia.js, app/tests.js, cloudflare/test/ia.test.mjs.

Côté app :
1. Nouvelle fonction PURE chargeBrouillonIA(c, bilan, signaux, coach). Elle rend l'objet envoyé au serveur :
  {faits: brouillonBilan(...).bouge, accroche: brouillonBilan(...).accroche,
  texteLibre: {difficultes, ecarts, modifs, stress, objectifs} (lus avec _texteReponse sur les clés bil-diff-detail, bil-cheat-reasons, bil-prog-modifs,
  bil-stress-detail, bil-new-goals-detail, chaque champ coupé à 1 200 caractères),
  notesSeances: les sess.notes non vides depuis le bilan précédent (au plus 8, 300 caractères chacune),
  styleCoach: les 3 dernières reponseCoach de CE coach à CET athlète (800 caractères chacune),
  formules: formulesReponse(coach), prenom: c.fname}.
EXCLUSIONS : si aTCA(c), renvoie null (on reste en déterministe). Ne transmits jamais les champs 'alerte' du bilan de départ (deb-health, deb-traitement, deb-tca), ni le sommeil chiffré, ni le stress chiffré.
2. Dans le rendu du dernier bilan sans réponse, sous « Brouillon pré-écrit », ajoute un lien « Réécrire avec l'assistant ». Au clic : appel CLOUD.callFn('ia',{tache:'bilan', athlete:c.email clé, charge}); le textarea montre un état d'attente ; en retour, la proposition REMPLACE le texte seulement si le coach n'a pas tapé entre-temps (compare avec rbrouillon) ; sinon, affiche-la dans un bloc « Proposition » avec « Utiliser ». Mémoise journalId dans ta.dataset.
3. Dans saveReponseBilan, après un envoi réussi, si ta.dataset.journalId existe, appelle CLOUD.callFn('iaRetour',{journalId, statut, distance}). distance = distance de Levenshtein normalisée entre la proposition et le texte envoyé (fonction pure _distanceTexte, bornée à 4 000 caractères). N'attends pas la réponse et avale les erreurs.
4. Échec, quota ou hors ligne : un toast calme, le brouillon déterministe reste en place.

Côté serveur (tâche 'bilan' dans ia.js) : prompt système figé (mis en cache avec cache_control) qui reprend mot pour mot les règles de l'en-tête « LOT C2 » de rc-core. Aucun conseil ni prescription (« baisse la charge », « prends un jour » sont interdits). Ne jamais nommer une douleur, une blessure, un médicament ou une donnée de santé : une gêne signalée devient « j'aimerais qu'on fasse un point ». N'inventer aucun chiffre : seuls ceux présents dans faits sont autorisés. Reprendre au moins un élément du texte libre, avec les mots de l'athlète. Une seule question ouverte. Tutoiement. Ton et longueur calés sur styleCoach. Ouverture et clôture = formules. Sortie JSON {texte, elementsRepris:[...], question}. Contrôle serveur après génération : tout nombre du texte absent de faits → rejet et repli ({ok:false, raison:'chiffre_invente'}).

Tests app/tests.js : chargeBrouillonIA rend null pour un profil TCA ; n'inclut jamais deb-health ni deb-traitement ; coupe les champs longs ; _distanceTexte(a,a)=0 et _distanceTexte('', 'x')=1. Tests cloudflare : le contrôle « chiffre inventé » rejette « tu as perdu 3 kg » quand faits ne contient pas 3 ; une réponse conforme passe.
Critère de fin : cd cloudflare && npm test vert ; la suite front (node scripts/verif/suite.mjs selon scripts/verif/README.md) sans nouvel échec ; les tests C2 existants (vers la ligne 49817 de tests.js) inchangés et verts.
```

3. Le lundi du coach : un résumé du portefeuille écrit pendant la nuit — Priorité : Haute · Effort : M

Constat. Le coach voit les signaux athlète par athlète : ccdSignaux (53448) dans l'étag « Ce qui appelle un œil », rapSignaux (63406), et un tri par urgencyScore (16759) qui renvoie un rang de 2 à 10. Rien ne lui dit en une page ce qui a changé cette semaine sur l'ensemble de ses athlètes, avec leurs propres mots. Le Worker lance déjà des travaux de nuit par lots (cloudflare/src/planif.js : retention à 4 h 30, relances à 10 h 30), mais aucun ne produit de synthèse pour le coach.

Pourquoi le mettre en place. Trainerize et Everfit envoient un digest hebdomadaire. Celui de RepCore peut être meilleur, parce qu'il s'appuie sur des signaux calculés, pas inventés. Le LLM ne décide rien : il regroupe, hiérarchise et cite. La Batch API divise le coût par deux et correspond exactement au rythme d'une nuit.

Intérêt réel. Un lundi matin à 30 athlètes, c'est aujourd'hui 30 fiches à ouvrir. On le remplace par une page : « 3 à traiter aujourd'hui, 5 qui progressent (à féliciter), 2 silencieux depuis 10 jours », chaque ligne menant à la fiche. On vise 45 minutes gagnées par semaine et par coach, et moins d'athlètes oubliés. Coût : environ 6 000 tokens en entrée et 600 en sortie par athlète, en batch, soit environ 0,009 \$ par semaine et **environ 0,035 € par athlète et par mois**. C'est aussi un argument de vente direct pour l'offre coach à 39 €.

Prompt de mise en place.

```
Prérequis : socle IA (cloudflare/src/ia.js). Objectif : un résumé hebdomadaire du portefeuille, écrit la nuit du dimanche au lundi par la Batch API et lu dans l'app par le coach.

Côté app (app/rc-core.1693.js) : l'app calcule déjà les signaux, le Worker non. Crée une fonction PURE resumeHebdoAthlete(c, maintenant) → objet compact (< 3 Ko) :
{prenom, urgence: urgencyScore(c), motifs: ccdSignaux(c).map(x=>x.motif), seances:{faites, prevues} sur 7 jours, poidsTendance (null si aTCA(c)),
bilanSansReponse: hasNewBilan(c), notes: les sess.notes de la semaine (≤ 5 × 200 caractères), texteBilan: les champs libres du dernier bilan de la semaine (mêmes clés et mêmes exclusions que chargeBrouillonIA)}.
Côté coach, une fois par jour à l'ouverture, écris hebdo_entree/<coach>/<athlète> = resumeHebdoAthlete(...) pour chaque client actif, seulement s'il a changé (compare une empreinte). Règles database.rules.json : écriture par le coach propriétaire seulement, lecture par le Worker.

Côté Worker (cloudflare/src/planif.js et un nouveau cloudflare/src/hebdo.js pur) :
1. Travail 'hebdo envoi' le dimanche après 23 h (heure de Paris, même helper apres(p,h,m)) : pour chaque coach ayant au moins une entrée, un POST /v1/messages/batches (client.messages.batches.create) avec une requête par coach (custom_id = clé du coach), tâche 'hebdo' (Sonnet 5.5, effort low). Stocke hebdo_batch/<AAAA-Www> = {id, cree}. Respecte le budget de sous-requêtes du réveil (voir compteur dans planif.minute).
2. Travail 'hebdo collecte' chaque heure du lundi : batches.retrieve ; quand processing_status === 'ended', lis les résultats en les indexant par custom_id, JAMAIS par position. Écris hebdo/<coach>/<AAAA-Www> = {texte, sections:{aTraiter:[{athlete, pourquoi}], progres:[...], silencieux:[...], t}. Impute le coût (×0,5) dans ia_quota. Ensuite, push « Ton point de la semaine est prêt » par M.envoyerPush.
3. Prompt système figé : ne rien recommander d'autre que « ouvrir la fiche » ou « répondre au bilan » ; ne jamais nommer une douleur ; ne citer que des faits présents dans l'entrée ; 3 sections au plus, 12 lignes au plus ; chaque ligne porte la clé de l'athlète. Sortie en JSON schema strict.
4. Contrôle après génération : toute clé d'athlète inconnue du coach est retirée.

Côté app : une carte « Ton point de la semaine » en tête du tableau de bord coach. Chaque ligne appelle openClientDetail(id). Un lien « Masquer » garde un choix local (try/catch sur localStorage).

Tests : cloudflare/test/hebdo.test.mjs (construction du batch, collecte par custom_id avec des résultats dans le désordre, résultat 'errored' ignoré sans planter, clé étrangère retirée, pas de second batch la même semaine). app/tests.js : resumeHebdoAthlete omet le poids si aTCA, reste sous 3 Ko avec 50 notes, est stable (même entrée, même empreinte).
Critère de fin : npm test (cloudflare) vert, suite front sans nouvel échec.
```

4. Importer les programmes existants d'un coach (PDF, photo, Excel) avec Claude Vision — Priorité : Haute · Effort : M

Fusion. s'appuie sur l'import par collage de l'agent Onboarding (point 5), à faire en premier.

Constat. L'import par OCR est fermé : LEGACY_PDF_IMPORT=false (36255), et le commentaire (36232-36236) explique pourquoi : il « perdait de l'information, se trompait de lecture ». Le code qui reste enchaîne quatre passes Tesseract par photo et des regex fragiles : parseWorkoutSheet (vers 116370),

`_buildVideoLinksArray`, et des valeurs par défaut arbitraires `series=3`, `reps='10'`, `repos='90 sec'`. Tesseract reste utilisé pour les étiquettes alimentaires (`_etiAnalyser`, 93836).

Pourquoi le mettre en place. Le premier frein d'un coach qui quitte TrueCoach, Excel ou un PDF Canva, c'est de ressaisir ses 20 modèles de programme. Un modèle multimodal lit un tableau, une fiche manuscrite ou un PDF de plusieurs pages, et rend une structure. En imposant que chaque nom soit rapproché de la banque d'exercices (`rechercherBanque`, 37314), la sortie tombe au format `sessions_config` et hérite des illustrations, de l'historique et de la revue morpho.

Intérêt réel. L'onboarding d'un coach passe de plusieurs soirées de saisie à une dizaine de minutes de relecture. C'est l'argument de conversion le plus fort pour les offres coach à 19 et 39 €, car c'est la barrière de migration n° 1 face aux concurrents. Coût : environ 15 000 tokens en entrée (6 pages d'image plus la liste des ~460 noms de la banque) et 6 000 en sortie, soit environ **0,08 € par programme importé**, en coût ponctuel. La revue passe par la fenêtre de relecture existante (`showOcrReviewModal`), donc le coach valide ligne par ligne. Le chemin Tesseract peut ensuite être retiré du chargement.

Prompt de mise en place.

```
Prérequis : socle IA (tâche 'import'). Objectif : rouvrir l'import de séance par PDF/photo avec Claude Vision à la place de Tesseract, derrière un nouveau drapeau.

Fichiers : app/rc-core.1693.js (LEGACY_PDF_IMPORT et _importLegacyOuvert vers 36255, analyzePhotoWithClaude vers 116287, analyzeProgPhotos vers 116003, parseWorkoutSheet, showOcrReviewModal, rechercherBanque vers 37314, exKey), cloudflare/src/ia.js, app/tests.js, cloudflare/test/ia.test.mjs.

1. Renomme analyzePhotoWithClaude en analyzePhotoOcr (il ne fait que du Tesseract). Mets à jour les appelants et tests.js.
2. Ajoute const IMPORT_IA=true et _importIAOuvert(). La nouvelle entrée importerSeanceIA(idx) accepte : jusqu'à 6 images (déjà réduites par resizeForOcr à 1 568 px de grand côté, en JPEG qualité 0.85) ou un PDF (base64, ≤ 10 Mo, ≤ 20 pages). Si IMPORT_IA est faux ou si l'appel échoue, retombe sur l'ancien chemin seulement si LEGACY_PDF_IMPORT est vrai ; sinon, message clair.
3. Charge envoyée : {pages:[{type:'image'|'document', media_type, data}], banque: les noms canoniques de la banque (exKey dédoublé, au plus 600)}. Côté serveur, le prompt demande une sortie JSON schema stricte :
  {seances:[{nom, jour?, echauffement?, exercices:[{nomLu, nomBanque|null, series:int|null, reps:string|null, repos:string|null, tempo:string|null, note:string|null, videoUrl:string|null, confiance:0..1}]}], nonLu:[string]}.
  Règles : ne jamais inventer une valeur absente (null plutôt qu'un défaut) ; nomBanque doit être EXACTEMENT un nom de la liste, sinon null ; une URL n'est rendue que si elle est lisible en entier.
4. Côté app, contrôle a posteriori : un nomBanque absent de la banque devient null. Les null ne sont PAS remplacés par 3x10 : la fenêtre de relecture les montre vides et en orange. Les lignes où confiance < 0.6 sont surlignées. Réutilise showOcrReviewModal en ajoutant ces états, ou écris une variante showImportIAModal si le modal existant ne s'y prête pas.
5. Ce qui est validé part en brouillon (enregistrerBrouillon, vers 36020), jamais publié directement.

Tests app/tests.js : un faux retour serveur (mock de CLOUD.callFn) avec un nomBanque inventé → null après contrôle ; un champ series null reste null dans la relecture ; l'ancien chemin reste fermé quand IMPORT_IA=false et LEGACY_PDF_IMPORT=false. Tests cloudflare : le schéma de sortie est validé ; une réponse avec stop_reason 'max_tokens' → ok:false.
Critère de fin : suites cloudflare et front vertes ; plus aucune occurrence de « WithClaude » dans rc-core.
```

5. Un premier programme proposé à partir de l'anamnèse, validé par le coach — Priorité : Moyenne · Effort : L

Constat. La seule génération automatique est `genererSeanceDepart` (42913) : une séance fixe choisie parmi 4 listes codées en dur (`PS_SEANCES`, salle ou maison, full body ou haut du corps), selon l'objectif, le lieu et la fréquence. Le bilan de départ recueille pourtant 30 réponses riches dans `BILAN_QUESTIONS.depart` (71305-71330) : blessures, jours, durée de séance, lieu, salle, objectifs, métier. Les longueurs morphologiques servent déjà à `revueMorpho` (15303). Le coach, lui, part de ses modèles (`_assignerModele`, 35114).

Pourquoi le mettre en place. Construire le premier bloc d'un nouvel athlète est la tâche la plus longue du coach (30 à 90 minutes). Un LLM peut choisir parmi SES modèles le plus proche, puis proposer les adaptations : jours, durée, matériel de la salle, exercices à éviter selon les contraintes. Chaque changement est justifié en une ligne. La sortie est ensuite passée à la revue morpho existante et arrive en brouillon.

Intérêt réel. On vise un premier programme prêt à relire en 10 minutes au lieu d'une heure. Pour un coach qui prend 5 nouveaux athlètes par mois, cela fait environ 4 heures gagnées. L'athlète reçoit son programme le jour même, et c'est la première impression de l'offre à 99-600 €. Coût : environ 25 000 tokens en entrée (anamnèse, résumés des modèles, banque) et environ 12 000 en sortie avec la réflexion, soit environ 0,17 \$ par génération, donc **environ 0,10 € par athlète et par mois** avec un bloc toutes les 6 semaines. Le coach garde la main : rien n'est publié sans lui, et chaque écart au modèle est listé et peut être décoché, comme la propagation du Lot C4 (`diffModele`, 35202).

Prompt de mise en place.

```
Prérequis : socle IA (tâche 'programme'). Objectif : depuis la fiche d'un athlète qui a rempli son bilan de départ et n'a pas de programme, un bouton « Proposer un premier programme » qui crée un BROUILLON à partir d'un modèle du coach.

Fichiers : app/rc-core.1693.js (BILAN_QUESTIONS.depart vers 71305, _assignerModele vers 35114, diffModele vers 35202, revueMorpho vers 15303, enregistrerBrouillon vers 36020, rechercherBanque, _texteReponse, aTCA), cloudflare/src/ia.js, tests.

1. Fonction PURE chargeProgrammeIA(c, modeles, banque). Contenu : les réponses de départ utiles (objectifs, jours, durée, moment, lieu, salle, niveau, rythme de travail), les contraintes déclarées (deb-health, texte libre, envoyé en tant que CONTRAINT, jamais repris dans un texte destiné à l'athlète), les modèles du coach résumés ({id, nom, jours, seances:[{nom, exercices:[{nom, series, reps}]}]}, au plus 12 modèles), la banque (noms canoniques, avec le matériel si disponible).
2. Serveur : Sonnet 5.5, effort medium, sortie JSON schema stricte :
  {modeleId, raisonChoix, ajustements:[{type:'jour'|'remplacement'|'retrait'|'series'|'duree', seance, exercice?, par?, pourquoi}], alertes:[string]}.
  Le LLM NE réécrit PAS le programme : il choisit un modèle et liste des ajustements. Remplacement : 'par' doit être un nom de la banque. Toute contrainte de santé ambiguë donne une alerte « à vérifier avec l'athlète » plutôt qu'un ajustement.
3. App : applique les ajustements de façon PURE (appliquerAjustementsIA(modele, ajustements) → sessions_config) en rejetant ceux qui ne valident pas (exercice inconnu, séance inconnue). Lance revueMorpho sur le résultat. Affiche un récapitulatif à cocher (même composant que la propagation C4) : chaque ajustement coché par défaut, avec son « pourquoi », et les alertes en tête. Valider → enregistrerBrouillon ; jamais de publication automatique.
4. Profil aTCA : aucun ajustement lié à la nutrition ou au poids.

Tests : appliquerAjustementsIA rejette un 'par' hors banque, applique un changement de jour, reste idempotent ; chargeProgrammeIA ne contient pas les clés deb-tca ni deb-traitement ; le bouton est absent si l'athlète a déjà un programme. Tests cloudflare : schéma et refus de sortie invalide.
Critère de fin : suites vertes ; un scénario manuel (athlète de test « salle, 3 jours, 45 min, douleur épaule ») produit un brouillon qui ne contient aucun exercice hors banque.
```

6. La photo du repas qui pré-remplit la saisie — Priorité : Moyenne · Effort : M

Constat. La saisie nutritionnelle est déjà rapide : récents à un geste et « Comme hier » (Lot N3), code-barres via BarcodeDetector ou ZXing puis Open Food Facts (92618-92932), lecture Tesseract de l'étiquette (`_etiAnalyser`, 93836), recherche CIQUAL (`_classerAliments`, 90952). En revanche, un plat maison ou un repas au restaurant n'a ni code-barres ni étiquette, et il n'y a aucune reconnaissance d'assiette.

Pourquoi le mettre en place. C'est précisément sur ces repas que l'observance s'effondre : personne ne pèse une assiette au restaurant. La photo analysée (MyFitnessPal, Cal AI, Trainerize Meal Scan) est devenue le standard attendu par les athlètes autonomes des offres Essentielle et Ultime. Ici, le LLM

propose seulement des noms d'aliments et des grammages estimés. La correspondance CIQUAL reste faite par la recherche locale existante, et l'athlète confirme chaque ligne.

Intérêt réel. Un repas hors maison se saisit en 20 secondes au lieu de 2 minutes, ou n'est plus abandonné. Chaque jour renseigné en plus nourrit signauxEntraînement (restriction) et l'assiette N7. Coût sur Haiku 4.5 : environ 2 700 tokens en entrée (image comprise) et 400 en sortie, soit environ 0,005 \$ par photo. Avec un plafond de 60 photos par mois, cela fait **au plus environ 0,27 € par athlète et par mois**, à réserver à Ultime (24,90 €) ou aux athlètes coachés. Garde-fou strict : la fonction est invisible pour un profil aTCA (111989), sur le même principe que le masquage des chiffres dans brouillonBilan.

Prompt de mise en place.

```
Prérequis : socle IA (tâche 'repas', Haiku 4.5). Objectif : dans l'ajout d'aliment, un bouton « Photo du repas » qui propose une liste d'aliments et de grammages à confirmer.

Fichiers : app/rc-core.1693.js (écran d'ajout d'aliment autour de 94400-94460, recherche locale _classerAliments vers 90952 et _fjNorm, _ciqualDB / _loadCiqua, aTCA vers 111989, estAutonome vers 22424), cloudflare/src/ia.js, tests.

1. Visibilité : jamais si aTCA(currentUser) ; seulement si l'offre est Ultime ou si l'athlète est suivi par un coach dont l'offre est coach ou pro. Écris cette condition dans une fonction pure repasPhotoOuvert(u).
2. Capture : input file accept="image/*" capture="environment". Réduis l'image à 1 024 px de grand côté, en JPEG 0.8, sur l'appareil. Aucune copie n'est stockée : l'image n'est ni téléversée sur Cloudinary ni écrite en base.
3. Serveur : sortie JSON schema stricte {aliments:[{nom:string (en français, générique, ex. « riz blanc cuit »), grammes:int, confiance:0..1}], remarque:string|null}. Règles du prompt : au plus 8 aliments ; ne jamais donner de calories (elles viennent de CIQUAL) ; pas de commentaire sur la qualité du repas ni sur le poids.
4. App : pour chaque aliment, lance la recherche locale existante et prends le premier résultat CIQUAL ; sinon, la ligne reste « à choisir ». Affiche la liste éditée (grammage modifiable, ligne supprimable, aliment remplaçable). Rien n'est ajouté au journal avant « Ajouter ces aliments ». Enregistre ensuite comme une saisie manuelle, avec source:'photo' sur chaque entrée pour mesurer l'usage.
5. Quota mensuel côté serveur (60 appels 'repas' par mois et par compte), en plus du plafond en euros.

Tests : repasPhotoOuvert est faux pour un profil TCA, faux pour un athlète Essentielle autonome, vrai pour Ultime ; le mapping d'un faux retour {nom:'riz blanc cuit', grammes:150} donne une entrée CIQUAL avec q=150 ; un aliment introuvable reste en « à choisir » et n'est pas enregistré ; aucune écriture tant que l'athlète n'a pas confirmé.
Critère de fin : suites vertes ; aucune requête vers Cloudinary pendant le parcours (vérifiable dans l'onglet réseau).
```

7. Risque d'abandon : un score appris sur les données RepCore, et des relances rédigées que le coach approuve — Priorité : Moyenne · Effort : M

Constat. Les relances automatiques ne réagissent qu'à 5 signaux administratifs (signauxRelance, cloudflare/src/relances.js:70 : nostart, overdue, bilan, noprog, expiring) et envoient un texte fixe (RELANCE_CORPS, relances.js:34 ; texteRelance, 140). La rétention est mesurée en agrégats (J1, J7, J30 et leviers ; cloudflare/src/retention.js) mais jamais transformée en risque individuel. chuteAssidue et décrochage existent côté app, mais sans probabilité ni priorité entre eux.

Pourquoi le mettre en place. Le Worker dispose déjà, chaque nuit, des résumés d'activité (/activite/<compte> : jours actifs, check-ins, notifications, coach, statut payant). Une régression logistique à 8 variables, entraînée sur ces résumés avec pour cible « inactif à J+14 » ou « non renouvelé », est de l'IA classique qui ne coûte rien. Elle classe les athlètes par risque réel. Le LLM n'intervient qu'ensuite, pour personnaliser le message, que le coach approuve en un geste.

Intérêt réel. Un point de churn évité vaut cher. Avec 100 athlètes Essentielle à 9,50 €, 5 abonnés retenus par mois représentent 570 € par an ; pour un coaching à 150 € par mois, un seul athlète retenu rembourse des années d'API. Le score est calculé sans aucun coût LLM. La relance personnalisée coûte environ 0,003 \$ sur Haiku 4.5 (moins de 0,01 € par athlète et par mois). Le garde-fou « le coach valide » est respecté : les relances rédigées ne partent jamais seules, elles sont proposées dans l'écran « Relances automatiques » avec « Envoyer » et « Modifier ». Les relances fixes existantes restent inchangées.

Prompt de mise en place.

```
Objectif : (A) un score de risque d'abandon par athlète, appris chaque nuit sur les résumés d'activité, sans LLM ; (B) une relance rédigée par Haiku 4.5, proposée au coach, jamais envoyée seule.

(A) Fichiers : cloudflare/src/retention.js (format du résumé r = {inscrit, sem, src, debut, jour, j30, seance1, parcours, finEssai, payant, lev}), nouveau cloudflare/src/risque.js (PUR), cloudflare/src/planif.js (travail 'retention' à 4 h 30), app/rc-core.1693.js (activiteResume vers 19682, urgencyScore vers 16759).
1. risque.js : variables(r, t) → 8 variables numériques (jours actifs sur 7, sur 14 et sur 30 depuis j30 ; pente 14 contre 14 ; jours depuis le dernier actif ; check-ins ; notif ; coach ; ancienneté plafonnée). La cible pour l'entraînement est : actif au moins une fois dans les 14 jours qui SUIVENT une date d'observation située 14 jours avant aujourd'hui (on la reconstruit depuis j30, sans fuite du futur). Régression logistique par descente de gradient (L2, 200 itérations, déterministe), entraînée seulement si le jeu contient ≥ 200 exemples et ≥ 30 positifs et ≥ 30 négatifs ; sinon, poids par défaut documentés. Publie risque_model = {poids, n, auc, t} et risque/<compte> = {p (0..1), t}. L'AUC est calculée sur 20 % d'exemples tenus à l'écart (tirage par hachage de la clé, stable).
2. App : sur la fiche athlète et dans la liste coach, une puce « risque » quand p ≥ 0,6, avec les 2 variables qui pèsent le plus (poids × valeur), en mots. urgencyScore n'est PAS modifié : c'est une information de plus, pas un nouveau rang.
(B) Dans l'écran « Relances automatiques » (côté app) : pour chaque athlète avec p ≥ 0,6 sans relance depuis 7 jours, une ligne « Proposer un message ». Appel CLOUD.callFn('ia',{tache:'relance_courte', athlete, charge:{prenom, joursInactif, derniereSeance (nom), styleCoach (2 derniers messages du coach), formules})). Sortie JSON {texte ≤ 280 caractères}. Règles : aucune culpabilisation, aucune donnée de santé, une seule question simple, tutoiement. Le coach voit le texte, peut le modifier, puis « Envoyer » passe par le moyen existant (message privé ou notification de type « relance », comme dans relances.js). Journalise dans relances_auto/<coach>/<athlète> avec moyen:'ia_valide'.

Tests : cloudflare/test/risque.test.mjs (variables sur un résumé connu ; entraînement déterministe sur un jeu synthétique séparable → AUC > 0,9 ; jeu trop petit → poids par défaut ; aucune clé de compte dans risque_model). app/tests.js : la puce n'apparaît pas sous 0,6, et urgencyScore est inchangé sur les six cas de non-régression existants.
Critère de fin : suites vertes ; aucun envoi de relance IA sans clic du coach (vérifié par un test sur le questionnaire côté app).
```

8. Un assistant pour l'athlète autonome, limité à ses propres données — Priorité : Basse · Effort : L

Constat. L'athlète sans coach (estAutonome, 22424) n'a qu'une séance de départ figée (genererSeanceDepart, 42913) et les règles automatiques de l'app. Il n'a personne à qui demander « pourquoi ma charge ne monte plus ? » ou « par quoi remplacer le hip thrust, la machine est prise ? ». Les réponses existent pourtant dans ses données : signaux, volume par muscle, reperesEffectifs, banque d'exercices.

Pourquoi le mettre en place. C'est la différence visible entre Essentielle et Ultime face à Hevy ou Fitbod, et une passerelle naturelle vers le coaching payant, puisque l'assistant peut proposer « en parler à un coach » quand la question dépasse ses données. Grâce au tool use, le modèle interroge les fonctions pures existantes au lieu de deviner.

Intérêt réel. L'assistant donne une raison concrète de passer de 9,50 € à 24,90 €. Même une conversion de quelques pourcents couvre largement le coût. Coût : Sonnet 5.5 en effort low, avec environ 7 000 tokens de contexte mis en cache et 500 en sortie, soit environ 0,012 \$ par message. Avec un plafond de

60 messages par mois, cela fait **au plus environ 0,65 € par athlète Ultime et par mois**, soit moins de 3 % du prix. Garde-fous : aucune réponse médicale ; un drapeau rouge (`drapeauQuelconqueActif`) ou une douleur déclarée coupe l'assistant sur ce sujet et renvoie vers un professionnel ; aucune modification du programme sans le bouton « Appliquer », et seulement pour les remplacements proposés par la banque ; profil TCA exclu des questions de nutrition.

Prompt de mise en place.

```
Prérequis : socle IA. Objectif : un écran « Demander à RepCore » pour les athlètes autonomes en offre Ultime, dont les réponses s'appuient UNIQUEMENT sur leurs données via des outils.

Architecture : la boucle d'outils tourne DANS L'APP, pas dans le Worker. Le Worker reste un proxy sans état (tâche 'assistant' : il relaie messages et tools à Sonnet 5.5, effort low, et renvoie la réponse brute : contenu et stop_reason). L'app exécute les appels d'outils localement, avec les fonctions pures existantes, puis renvoie les tool_result. Au plus 4 tours d'outils par question.

Outils (définis en JSON schema, strict:true, dans une constante ASSISTANT_OUTILS de app/rc-core.1693.js) :
- signaux() → signauxEntraînement(currentUser,{complet:true}) sans détails de douleur ;
- volumeMuscle(semaines:int≤8) → volume par muscle et repères (volumeSemaine, repèresEffectifs) ;
- progression(exercice:string) → les 8 dernières séances de cet exercice (charge, reps) ;
- remplacements(exercice:string, materiel?:string) → au plus 5 exercices de la banque (rechercherBanque) qui ciblent le même muscle principal ;
- proposerRemplacement(seance, exercice, par) → NE MODIFIE RIEN : rend une carte avec un bouton « Appliquer », que l'athlète doit toucher.

Prompt système figé et mis en cache : périmètre = entraînement et habitudes ; aucun diagnostic ni conseil médical ; si l'athlète parle de douleur, de blessure, de médicament ou de trouble alimentaire, réponse fixe qui oriente vers un professionnel de santé ; pas de nutrition chiffrée si aTCA ; citer les chiffres venus des outils, n'en inventer aucun ; proposer « en parler à un coach » (lien vers l'offre coaching) quand la question dépasse les données.

Côté app : l'écran est visible seulement si estAutonome(currentUser) && offre Ultime. Historique local (20 derniers échanges, try/catch sur le stockage) ; le compteur du mois est lu dans la réponse du proxy (coutMois, plafond). Si drapeauQuelconqueActif(currentUser) est vrai, un bandeau fixe renvoie vers un professionnel et l'assistant refuse les questions d'entraînement liées à la zone concernée.

Tests app/tests.js : chaque outil rend un objet sérialisable de moins de 4 Ko ; proposerRemplacement ne modifie pas currentUser.sessions_config ; « Appliquer » modifie exactement une ligne et passe par le chemin d'écriture habituel ; l'écran est absent pour Essentielle et pour un athlète coaché ; la boucle s'arrête à 4 tours. Tests cloudflare : la tâche 'assistant' refuse un appel sans offre Ultime (403) et respecte le quota.

Critère de fin : suites vertes ; démonstration manuelle « par quoi remplacer le hip thrust ? » → 3 à 5 exercices de la banque, aucune modification sans « Appliquer ».
```

Récapitulatif des coûts (Claude API, par mois) : - **Athlète coaché** : brouillon de bilan 0,11 €, résumé hebdo 0,035 €, programme 0,10 €, import ponctuel amorti 0,02 €, relances moins de 0,01 €. Total : **environ 0,28 € par athlète et par mois**, soit environ 8 € pour un coach à 30 athlètes. C'est à couvrir par les quotas de l'offre pro à 39 €. - **Athlète Ultime autonome** : photos de repas 0,27 € et assistant 0,65 € au plafond, soit **au plus environ 0,92 €**, pour 24,90 € encaissés.

L'analyse vidéo du motion-lab (`mlSynthese`, `mlPhrases`, `app/motion-lab.js:1711` et `2374`) reste volontairement déterministe : elle mesure des angles et des vitesses, et un LLM n'y apporterait rien de fiable.

Feuille de route

Les 129 prompts se déroulent en quatre vagues : d'abord ce qui fait perdre des données ou de l'argent, ensuite la justesse des calculs, puis l'expérience, enfin la croissance. Lance un prompt par session Claude Code, dans cet ordre.

Vague 1 — Fiabilité et argent (semaine 1)

- ☐ QA 1 : les tests bloquent la mise en ligne (à faire en tout premier : chaque correctif suivant sera protégé)
- ☐ Performance 5 (étapes 1 à 4) : les trois bugs trouvés par le lint, lint bloquant
- ☐ Sécurité 1, Backend 7, Business 1 : un seul chantier « droits calculés par le serveur »
- ☐ Sécurité 2 : faille XSS athlète → coach
- ☐ Synchronisation 1, 2, 3, 4 : plus aucune perte de séance
- ☐ Business 2, 3, 4 : double abonnement, résiliation, remboursement
- ☐ Backend 2, 3, 4, 5 : alerte, sauvegarde, déploiement du Worker, budget du webhook PayPal

Vague 2 — Justesse des calculs (semaine 2)

- ☐ Préparateur physique 1 à 5 : e1RM, charge suggérée, poids du corps, modèle unique, cartographie musculaire
- ☐ Nutrition 1 à 5 : garde-fous, objectif, formule, macros, aliments à 0 kcal
- ☐ Suivi des progrès 1 à 3 : masse grasse, pesée du matin, tempo vidéo
- ☐ Engagement 3 : une seule définition de la semaine validée
- ☐ QA 3 : dates décalées hors d'Europe

Vague 3 — Expérience coach et athlète (semaines 3 et 4)

- ☐ Séance 1 à 5 : repos, écran, saisie en un tap, confirmation de fin, virgule
- ☐ Onboarding 1, 2, 3, 4, 8 : première séance, invitation, embarquement en lot
- ☐ Coach 1 à 5 : tableau de bord, messagerie, cadence des bilans, réponse vocale
- ☐ Performance 1 à 4 : service worker, minification, écran noir, CSS
- ☐ Design 1 à 6 : thème clair, pages publiques, couleurs, typo, cibles tactiles
- ☐ Engagement 1, 2, 4, 7 : priorité des notifications, scores serveur, mission du jour, relances

Vague 4 — Croissance et différenciation (mois 2)

- ☐ IA 1 puis 2, 3, 4 : proxy Claude, brouillon de bilan, résumé du lundi, import de programmes
- ☐ Marketing 1 à 7 : promesse invité, page coachs, preuve sociale, SEO, Play Store
- ☐ Business 5 à 7 : grille tarifaire, boutique, tableau de bord MRR
- ☐ Engagement 5, 6, 8 : ligues, équilibrage des rangs, saisons automatiques
- ☐ Benchmark : paiement direct au coach, recettes, montres, marque du coach, réservation
- ☐ Tous les points de priorité Moyenne et Basse restants

Décisions qui te reviennent avant de lancer certains prompts : le modèle économique (Business 1), l'engagement de 12 mois (Business 3), les nouveaux prix annuels (Business 5), le compte Instagram officiel (Marketing 6), et l'activation de l'IA avec une clé API (IA 1).